

Certified Kubernetes Administrator

Bootcamp

RX-M Cloud Native Advisory, Consulting and Training

2

Copyright 2013-2020, RX-M LLC

- **Microservice Oriented**

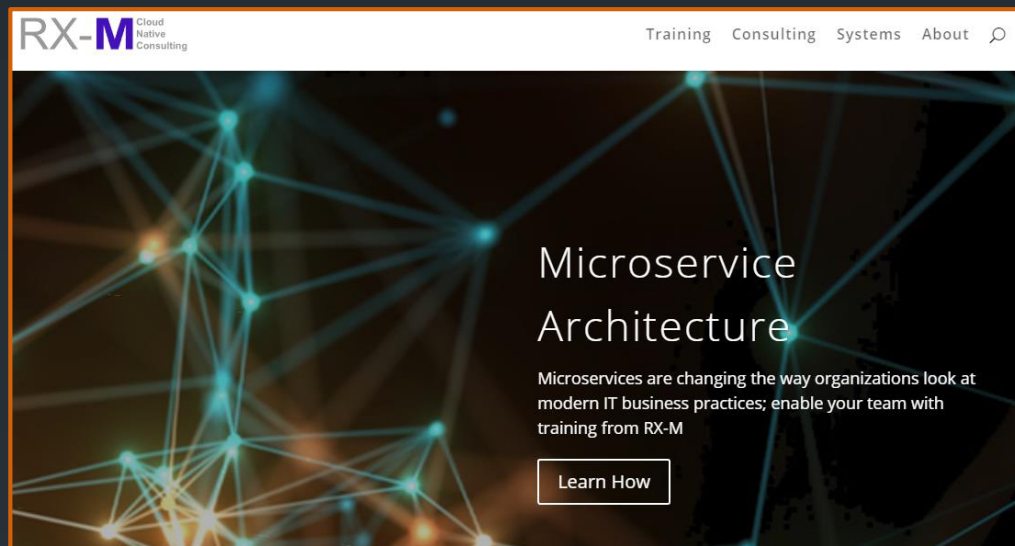
- Microservices Foundation [3 Day]
- Building Microservices on Kubernetes [3 Day]
- Building Microservices on AWS [3 Day]
- Building Microservices on GCP [3 Day]
- Building Microservices on Azure [3 Day]
- Building Microservices with Go [3 Day]
- Building Microservices with Thrift [2 Day]
- Building Microservices with gRPC [2 Day]

- **Container Packaged**

- Docker Foundation [3 Day]
- Docker Advanced [2 Day]
- Containerd [2 Day]
- Cri-O [2 Day]

- **Dynamically Managed**

- Docker Orchestration (Compose/Swarm) [2 Day]
- Kubernetes Foundation [2 Day]
- Kubernetes Advanced [2 Day]
- Kubernetes for Developers [4 Day]
- Kubernetes for Operators [5 Day]
- Kubernetes CKA boot camp [4 Day]
- Kubernetes CKAD boot camp [5 Day]
- Kubernetes Day 2 operations [2 Day]
- Securing Kubernetes [2 Day]
- Monitoring Kubernetes with Prometheus [2 Day]
- Mesos Foundation [2 Day]



RX-M Cloud Native Consulting

Overview

3

Copyright 2013-2020, RX-M LLC

Day One

1. Containers & Orchestration
2. Kubernetes Architecture
3. Pods & Configs

Day Two

4. Controllers
5. Services & KubeProxy
6. Managing State

Day Three

7. Security
8. Metrics
9. Ingress

Day Four

10. Networking
11. etcd
12. Test Prep I

Day Five

13. Test Prep II
14. CKA Exam

Prerequisites:

RX-M Docker Foundation
or similar container experience

Equivalent experience:

- Familiarity with container-based microservice packaging
- Understanding of container isolation and constraints
- Basic familiarity with Docker, RunC, Rocket, Garden or similar container engine

Administrative Info

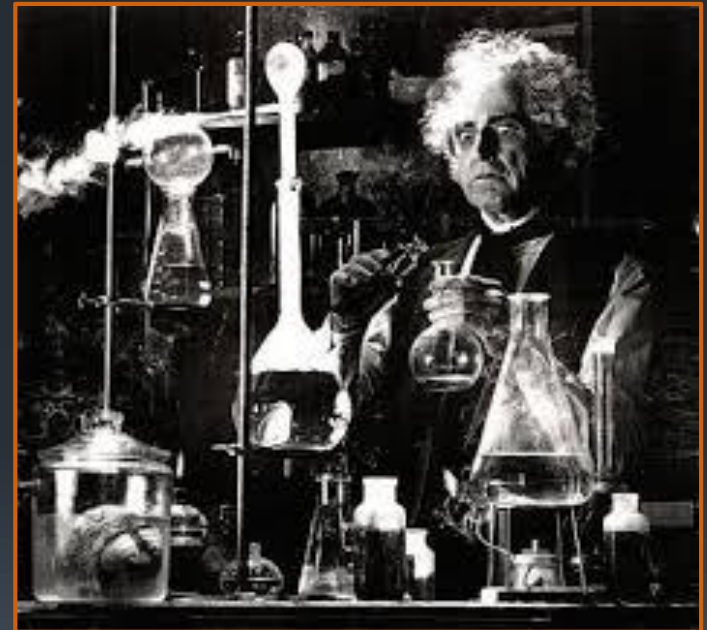
- Course: Kubernetes CKA Boot Camp
- Length: 5 Days
- Format: Lecture/Labs/Discussion
- Schedule: 8:30AM – 5:30PM
15 minute break, AM & PM
1 hour lunch at noon
Lab time at the end of each AM and PM session
- Location: Fire exits, Restrooms, Security, other matters
- Attendees: Name/Role/Experience/Goals for the Course

Lecture and Lab

5

Copyright 2013-2020, RX-M LLC

- Our Goals in this class are two fold:
 1. **Introduce concepts and ecosystems**
 - Covering concepts and where things fit in the world is the primary purpose of the lecture/discussion sessions
 - The instructor will take you on a **tour of the museum**
 - Like a museum tour, you should interact with the instructor (tour guide), ask questions, discuss
 - Like a museum tour, you will not have time to read the slides during the tour, instead, the instructor will discuss and point out the **highlights** of the slides (exhibits) which will be waiting for you to read in depth later should you like to dig deeper
 2. **Impart practical experience**
 - This is the primary purpose of the labs
 - Classes rarely have time for complete real world projects so think of the labs as thought experiments
 - Like **hands on exhibits** at the museum



Day 1

1. Containers & Orchestration
2. Kubernetes Architecture
3. Pods & Configs

1: Containers & Orchestration

Objectives

- Review container technology
- Discuss basic container architecture
- Explore the container ecosystem
- Review installation methods & requirements for Docker
- Describe the basic features supplied by container orchestration systems
- Explore the container orchestration ecosystem
- Gain a high level understanding of the Kubernetes container orchestration platform
- Consider the history and track record of Kubernetes

What is a container?

- **Lightweight Linux environment**
 - More recently, lightweight Windows environments as well
- **Encapsulated and deployable**
- **Runnable**
- A way to package applications for **reliable deployment**
 - Particularly popular for **packaging microservices**
- Microservice centric (one atomic service per container)
- Made widely popular by **Docker**
 - Docker, Inc. provides a container platform including systems for publishing and sharing containers
 - Container technology predates and enables Docker (LXC, OpenVZ, ...)
 - Docker has been the dominant mover in this space but competition is growing and standards are evolving
- Relies on integral features of the **Linux Kernel**
 - CGroups
 - Namespaces
 - Linux Bridge/IPTables/Capabilities/etc.



```
$ time docker container run ubuntu echo "hello world"
hello world
```

```
real    0m0.319s
user    0m0.005s
sys      0m0.013s
```

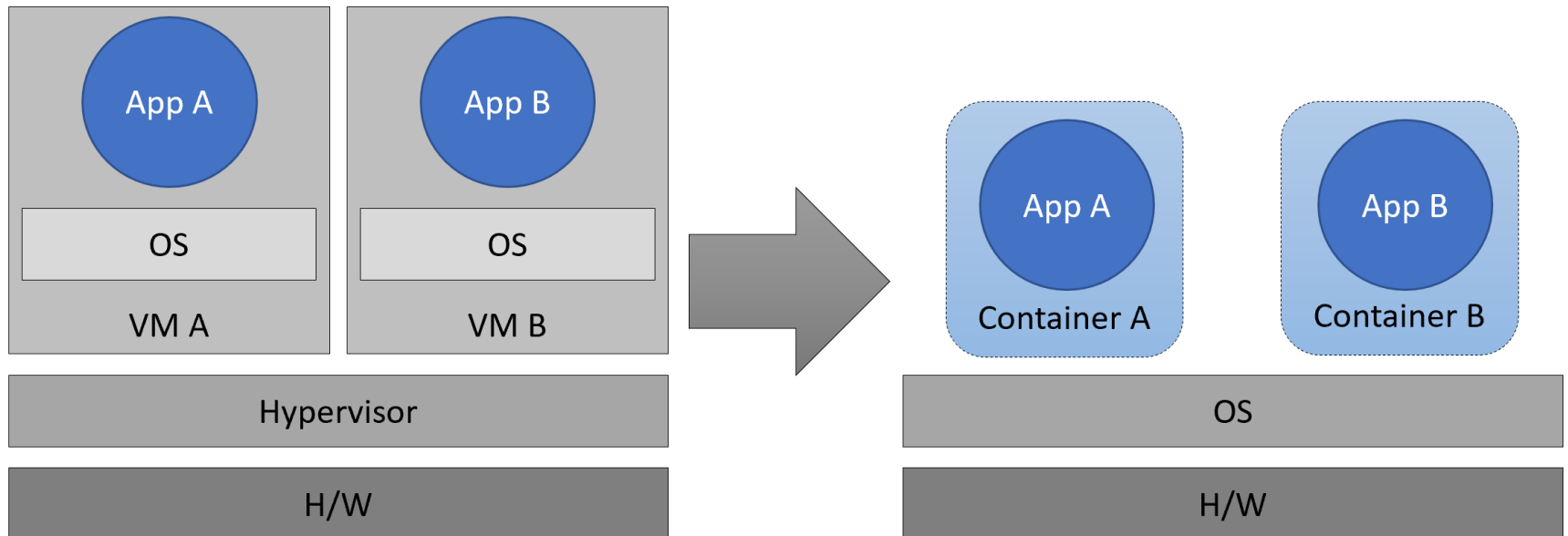
Disk usage: less than 100 kB
Memory usage: less than 1.5 MB

Contrasting Containers with VMs

10

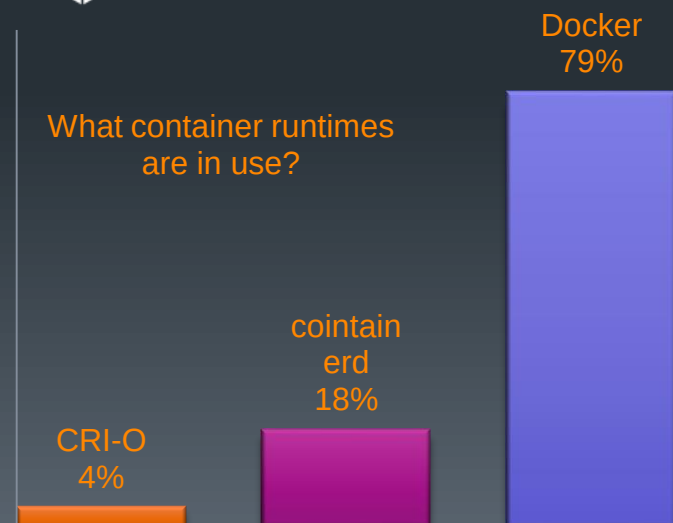
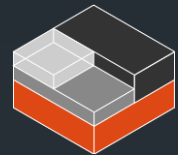
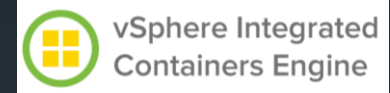
Copyright 2013-2020, RX-M LLC

- VM
 - A **virtual machine** for operating systems
- Container
 - A **virtual operating system** for applications
- These are not mutually exclusive and many environments become optimal when properly combining both



Container Runtimes

- **Docker**
 - Container platform for publishing and sharing containers
- **containerd**
 - Daemon for Linux and Windows with an emphasis on simplicity
- **Rocket (rkt)** [**defunct**, first achieved CNCF project]
 - Simple and lightweight Docker-like container platform
 - Used in the Tectonic platform - CoreOS/**rkt**/Kubernetes integration
- **CRI-O**
 - Implementation of the K8s CRI (Container Runtime Interface) to enable using OCI (Open Container Initiative) compatible runtimes
 - Formerly OCID (OCI daemon)
 - Top contributors from Red Hat - used by OpenShift Online 3.9+
- **vSphere Integrated Containers (VIC)**
 - Docker API-compatible engine for instantiating container images that are run as VMs
- **Virtual Machines**
 - **Kata Containers** – CRI/OCI compatible lightweight VMs w/ individual kernel instances
 - **gVisor** – OCI runtime (runsc) with user-space kernel
 - "merged guest kernel and VMM, or seccomp on steroids"
- **Mesos**
 - Lightweight containerization and resource isolation of executors using control cgroups and namespaces
- **LXC**
 - Original Linux container library (now at v2)
- **LXD/OpenVZ**
 - Systems in containers (lightvisors)
- **BSD Jails**
 - Isolation features of BSD Unix
- **OpenSolaris Container**
 - Zones-based isolation model



Container Standardization

- Open Container Initiative (OCI) [circa 6/2015]
 - A standard that ensures any container can run on any machine or cloud computing service
- Run by the Linux Foundation
- Three specifications:
 - **Runtime Specification (runtime-spec)**
 - Specifies the configuration, execution environment, and lifecycle of a container
 - Released v1.0 July 19, 2017
 - **Image Specification (image-spec)**
 - An OCI implementation downloads an OCI Image, unpacks it, then runs it
 - Released v1.0 July 19, 2017
 - **Distribution specification (distribution-spec)**
 - Standardize container image distribution based on the specification for the Docker Registry HTTP API V2 protocol
 - Released v1.0 RC 0 February 14, 2019
- Docker donated the container format, runtime code (**libcontainer**) and specifications
 - **runc** was created as a libcontainer client, representing the lowest level tool for spawning a containerized process

The Open Container Initiative (OCI) is a lightweight, open governance structure (project), formed under the auspices of the Linux Foundation, for the express purpose of creating open industry standards around container formats and runtime. The OCI was launched on June 22nd 2015 by Docker, CoreOS and other leaders in the container industry



Core Docker Components

13

Copyright 2013-2020, RX-M LLC

- Docker client and server executables

- \$ **dockerd**

- Runs Docker as a daemon on the Linux host supporting docker container execution (/usr/bin/dockerd)
 - Docker manages containers, the linux kernel runs containers

- \$ **docker**

- The Docker client sends requests to local and remote Docker daemons (/usr/bin/docker)

- **Docker Remote API**

- The Docker client talks to the Docker daemon through the Docker Remote RESTful API

- Docker **Images**

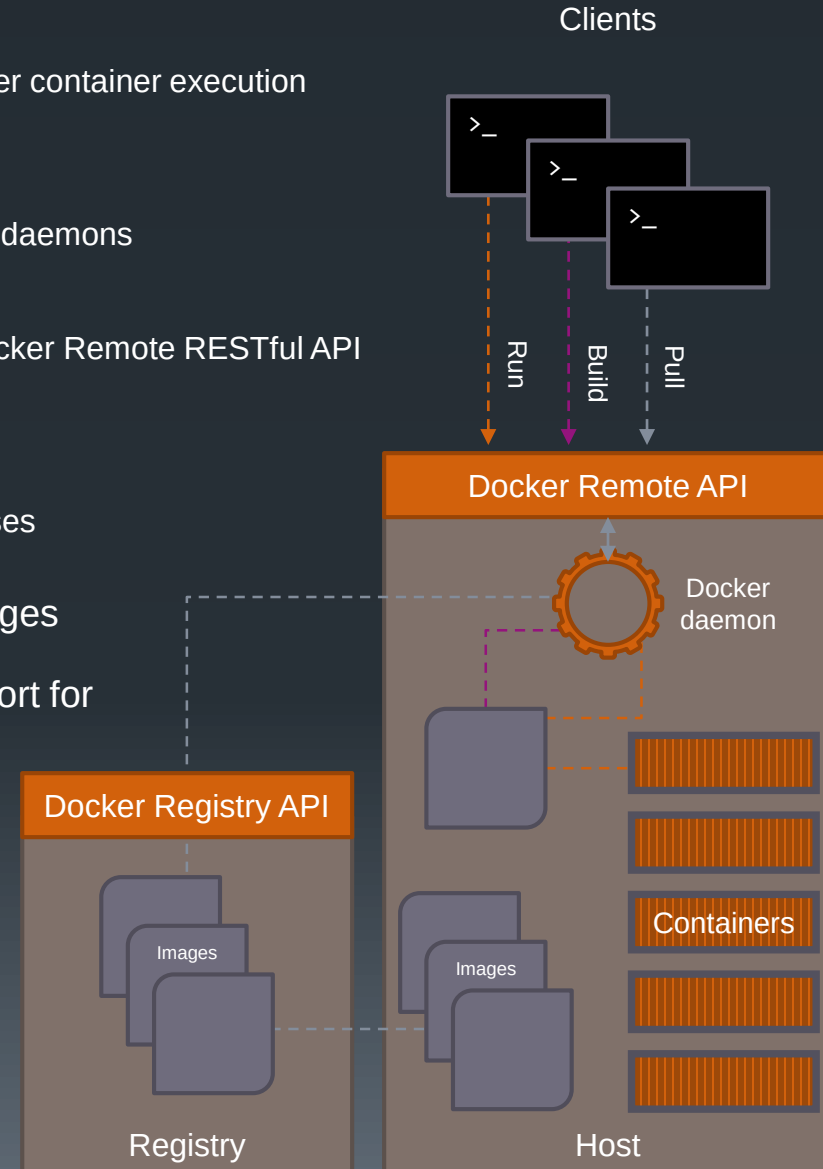
- Images are used to generate containers
 - Just as a VM image can create multiple VM instances
 - Just as an executable (/usr/bin/vi) can create multiple processes

- **Registries**

- Registries are network services from which Docker Images can be saved and retrieved
 - The Docker Hub is an internet based registry with support for public and private images
 - Private registry servers can be created for an organization's internal use

- Docker **Containers**

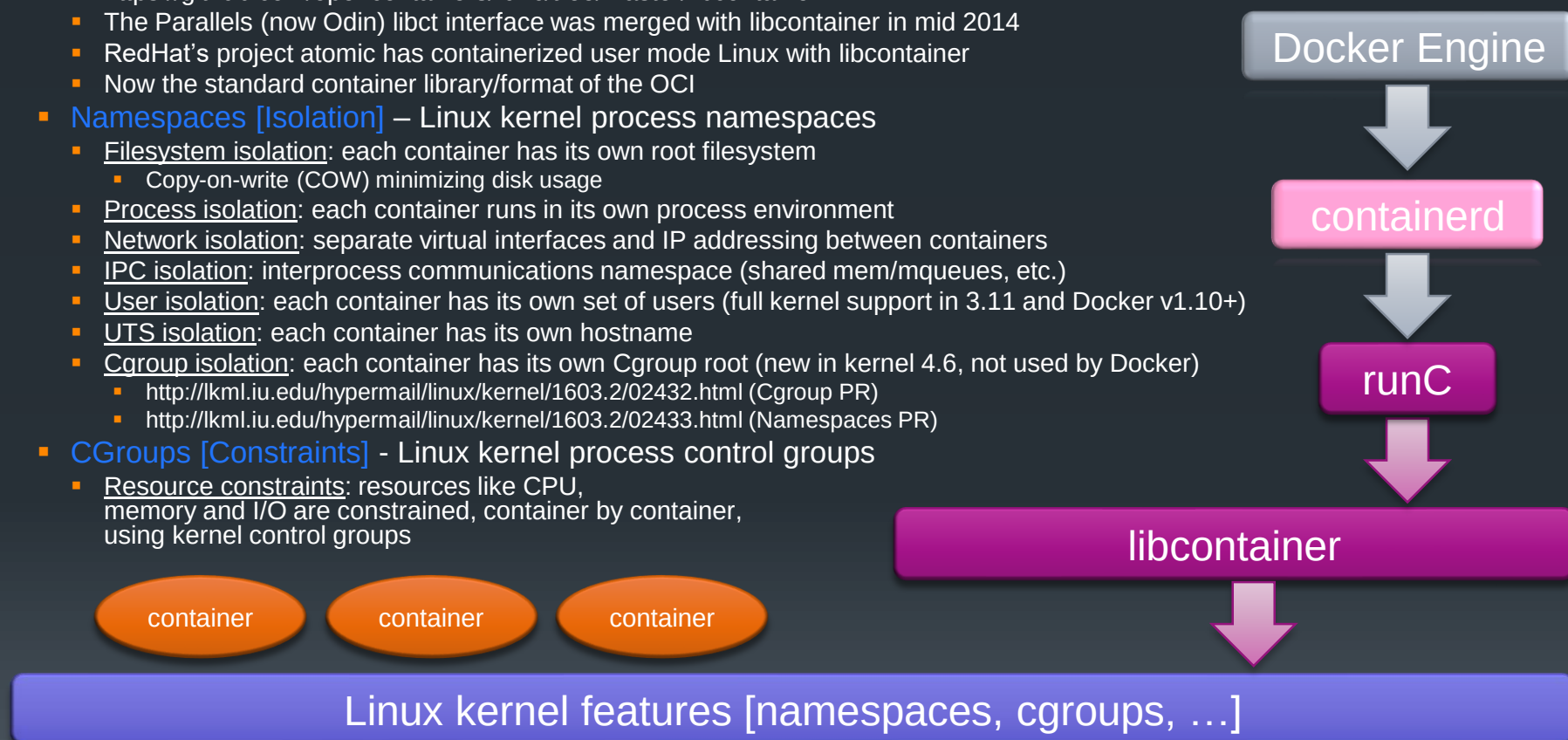
- A container is a software package generated from an image
 - Said to be running when processes are executing within it
 - Can be stopped and started



- Docker installations include the following elements:

- **dockerd** [Docker daemon] – service performing high level container management
 - <https://github.com/docker/docker-ce>
- **containerd** [runc manager] – daemon directly managing OCI containers (used by Docker v1.11+)
 - <https://github.com/containerd/containerd>
- **runc** [launcher] – OCI container launcher (used by Docker v1.11+)
 - <https://github.com/opencontainers/runc>
- **libcontainer** [system interface] – OCI kernel container interface (used by Docker v0.9+)
 - <https://github.com/opencontainers/runc/tree/master/libcontainer>
 - The Parallels (now Odin) libct interface was merged with libcontainer in mid 2014
 - RedHat's project atomic has containerized user mode Linux with libcontainer
 - Now the standard container library/format of the OCI
- **Namespaces** [Isolation] – Linux kernel process namespaces
 - Filesystem isolation: each container has its own root filesystem
 - Copy-on-write (COW) minimizing disk usage
 - Process isolation: each container runs in its own process environment
 - Network isolation: separate virtual interfaces and IP addressing between containers
 - IPC isolation: interprocess communications namespace (shared mem/mqueues, etc.)
 - User isolation: each container has its own set of users (full kernel support in 3.11 and Docker v1.10+)
 - UTS isolation: each container has its own hostname
 - Cgroup isolation: each container has its own Cgroup root (new in kernel 4.6, not used by Docker)
 - <http://lkml.iu.edu/hypermail/linux/kernel/1603.2/02432.html> (Cgroup PR)
 - <http://lkml.iu.edu/hypermail/linux/kernel/1603.2/02433.html> (Namespaces PR)
- **CGroups** [Constraints] - Linux kernel process control groups
 - Resource constraints: resources like CPU, memory and I/O are constrained, container by container, using kernel control groups

Container Enablers



```

$ ps -f $(pgrep docker)
UID      PID    PPID    C  STIME TTY      STAT   TIME CMD
root      1388      1    0 Feb28 ?        Ss1    5:06 /usr/bin/dockerd -H fd://
$ ps -f $(pgrep containerd)
UID      PID    PPID    C  STIME TTY      STAT   TIME CMD
root      9145      1    0 15:27 ?        Ss1    0:31 /usr/bin/containerd
  
```

```

user@ubuntu:~$ ls -l $(which docker runc)
-rwxr-xr-x 1 root root 68549616 Jan  9 19:37 /usr/bin/docker
-rwxr-xr-x 1 root root 12316488 Jan  8 00:43 /usr/sbin/runc
  
```

Docker Installation

15

Copyright 2013-2020, RX-M LLC

- Docker is easy to install on a range of platforms and cloud systems:

- OSes with predefined Docker packages

- Debian/Ubuntu
 - Red Hat EL/CentOS/Oracle EL/Fedora
 - Gentoo
 - Arch Linux
 - FrugalWare
 - OpenSUSE
 - CRUX Linux
 - Mac OS X
 - Microsoft Windows

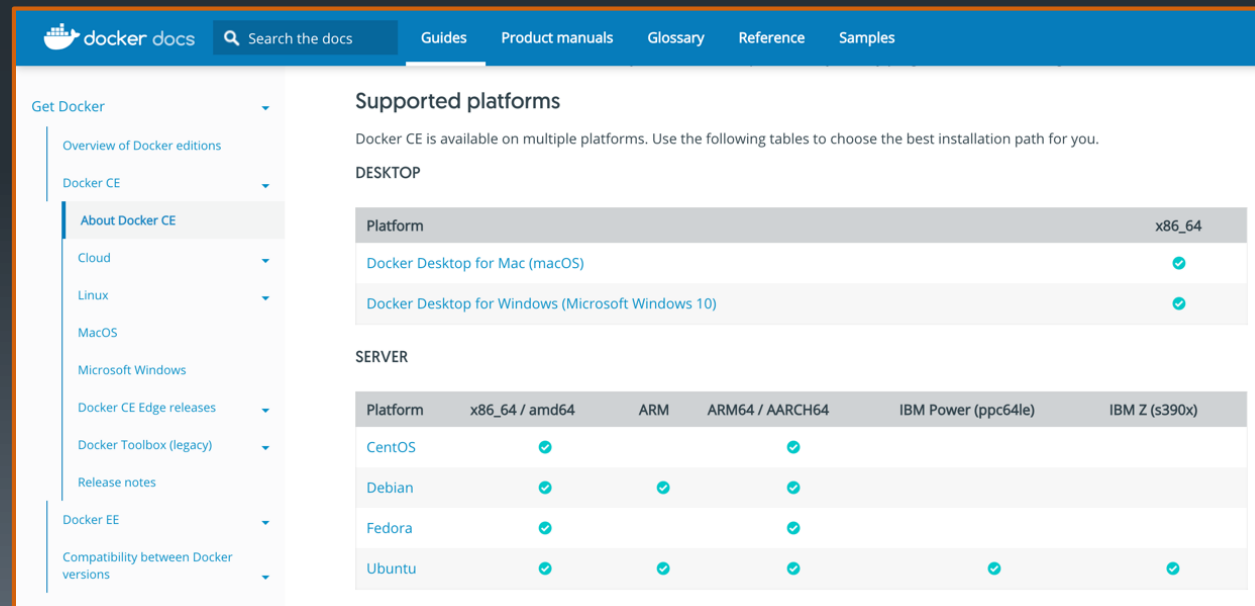
- Cloud Docker packages

- Google Cloud Platform
 - DigitalOcean
 - Microsoft Azure
 - Packet
 - Amazon Web Services
 - IBM Softlayer

- Others (e.g. CoreOS) provide their own direct support

- Binaries are also supplied for custom installations

RHEL and Ubuntu are the recommended platforms



The screenshot shows the Docker documentation website. The left sidebar contains a navigation menu with links to 'Get Docker', 'Overview of Docker editions', 'Docker CE', 'About Docker CE', 'Cloud', 'Linux', 'MacOS', 'Microsoft Windows', 'Docker CE Edge releases', 'Docker Toolbox (legacy)', 'Release notes', 'Docker EE', and 'Compatibility between Docker versions'. The main content area is titled 'Supported platforms' and includes a search bar and navigation tabs for 'Guides', 'Product manuals', 'Glossary', 'Reference', and 'Samples'. Below the title, it states 'Docker CE is available on multiple platforms. Use the following tables to choose the best installation path for you.' There are two tables: 'DESKTOP' and 'SERVER'. The 'DESKTOP' table lists 'Docker Desktop for Mac (macOS)' and 'Docker Desktop for Windows (Microsoft Windows 10)' with checkmarks. The 'SERVER' table lists various operating systems and their compatibility with different architectures.

Platform	x86_64 / amd64	ARM	ARM64 / AARCH64	IBM Power (ppc64le)	IBM Z (s390x)
CentOS	✓		✓		
Debian	✓	✓	✓		
Fedora	✓		✓		
Ubuntu	✓	✓	✓	✓	✓

Docker Dependencies

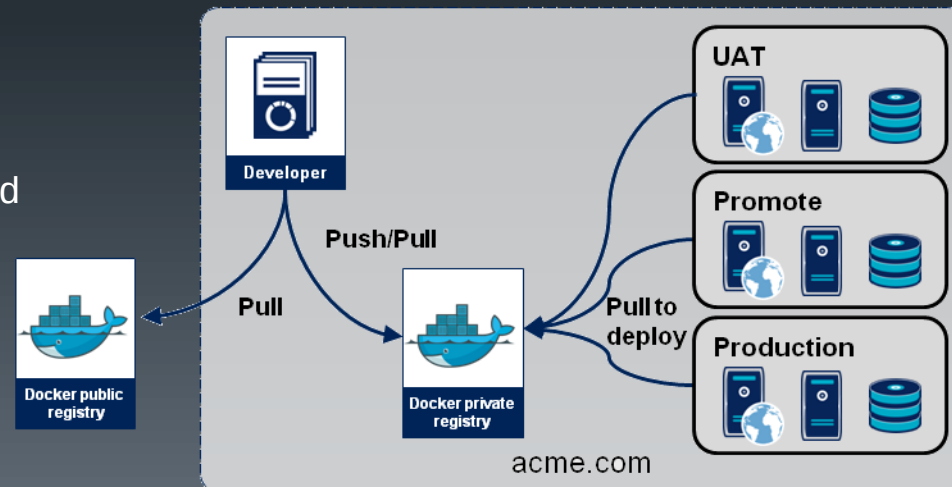
16

Copyright 2013-2020, RX-M LLC

- The Docker daemon requires **Linux kernel 3.10+**
 - Kernels older than 3.10 have bugs which can cause data loss and panic under certain conditions
 - The latest minor version (3.x.y) of the 3.10 or newer Linux kernel is recommended
 - Keeping kernels up to date will ensure critical kernel bugs get fixed
 - Key Distros:
 - RHEL/Centos 7: Kernel 3.10+
 - Ubuntu 14.04: Kernel 3.13+, 16.04: Kernel 4.4+
 - Warning:
 - Custom kernels and kernel packages are not usually supported by Linux distribution vendors
 - Installing a newer kernel may fail with distributions which provide packages which are too old or incompatible with newer kernels
- Docker is supported on **x86_64** (or **amd64**), **ARM**, **IBM Power** and **Z** architectures
 - The Docker goal is to run everywhere, some vendors support Docker on other platforms
 - FreeBSD offers a 64 bit Linux compatibility layer supporting Docker
 - Free BSD info: <https://wiki.freebsd.org/Docker>
 - The Docker Engine API is supported on Joyent SmartOS by Triton
 - Windows Server 2016 and later – enables native Docker containers on Windows Server
- The Docker client can run on virtually any *nix system
 - The client also builds on OS X and Windows

```
user@ubuntu:~$ uname -a
Linux ubuntu 4.4.0-31-generic #50-Ubuntu SMP Wed Jul 13 00:07:12 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux
user@ubuntu:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="16.04.1 LTS (Xenial Xerus)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 16.04.1 LTS"
VERSION_ID="16.04"
HOME_URL="http://www.ubuntu.com/"
SUPPORT_URL="http://help.ubuntu.com/"
BUG_REPORT_URL="http://bugs.launchpad.net/ubuntu/"
UBUNTU_CODENAME=xenial
```

- Image signing/scanning and authentication/authorization support should be considered when choosing a registry**



After Containers, what's left?

18

Copyright 2013-2020, RX-M LLC

■ Orchestration!

- Real cloud native applications are delivered in 10s of images and require 100s - 1000s of running containers
- **Registries** manage image distribution

■ Orchestration features:

- **Container Scheduling**
 - Distributing containers to appropriate hosts
 - Host resource leveling
 - Availability zone diversity
 - Related container packaging and co-deployment
 - Affinity/Anti-affinity
- **Container Management**
 - Monitoring and recovery
 - Image upgrade rollout
 - Scaling
 - Logging
- **Service Endpoints**
 - Discovery/Name-Resolution
 - High Availability
 - Load Balancing
 - Traffic Routing
- **External Services**
 - Container Runtimes
 - CRI <--> OCI
 - Network configuration and management
 - CNI
 - Durable volume management
 - CSI



Container Orchestration

19

Copyright 2013-2020, RX-M LLC

Orchestration Tools

Kubernetes

- Google authored open-source platform for automating deployment, scaling, and operations of application containers across clusters of hosts
- AWS **EKS**, Azure **AKS**, IBM **IKS**, Google **GKE**, RedHat **OpenShift** PaaS platform and RH/CoreOS **Tectonic** are all based on Kubernetes

Docker Swarm

- Docker-authored native clustering for Docker containers, now Docker Data Center, built on Docker Universal Control Plane (UCP)

Apache Mesos (DC/OS)

- A distributed system kernel which abstracts CPU, memory, storage, and other compute resources away from machines

Cloud Foundry Diego

- Elastic runtime written in Go supporting the Garden container manager with runC as the backend

Cloud Support

Google Kubernetes Engine [GKE]

- Optimized VMs with Kubernetes preinstalled

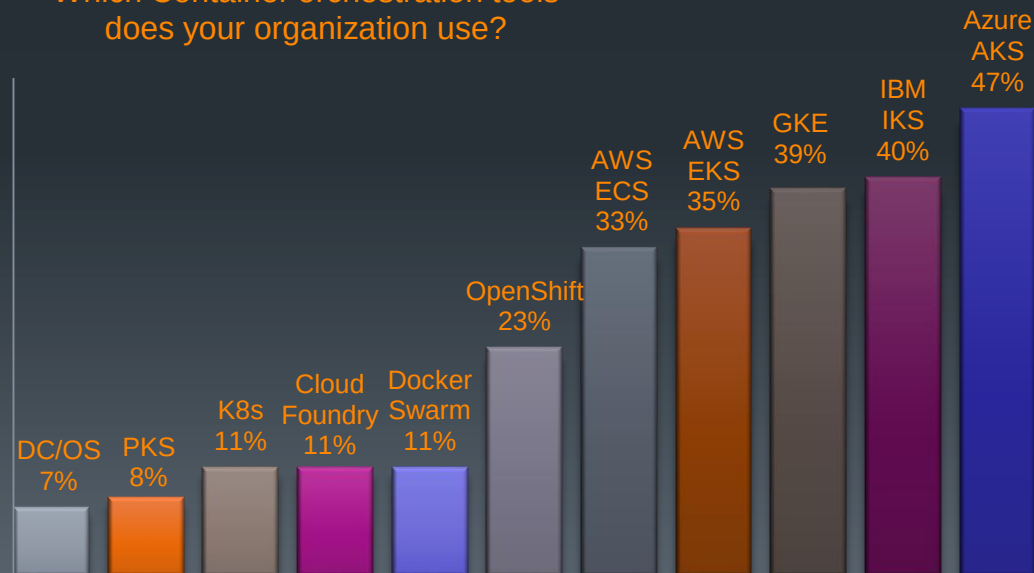
Amazon

- EC2 Container Service [ECS]** non-Kubernetes container scheduler
- Elastic Container Service for Kubernetes [EKS]** Kubernetes-as-a-service (not released at the time of the Portworx survey)

Microsoft

- Azure Container Service [ACS]** – supports Mesos (DC/OS), Docker Swarm and K8s
- Azure Kubernetes Service [AKS]** – hosted K8s

Which Container orchestration tools does your organization use?



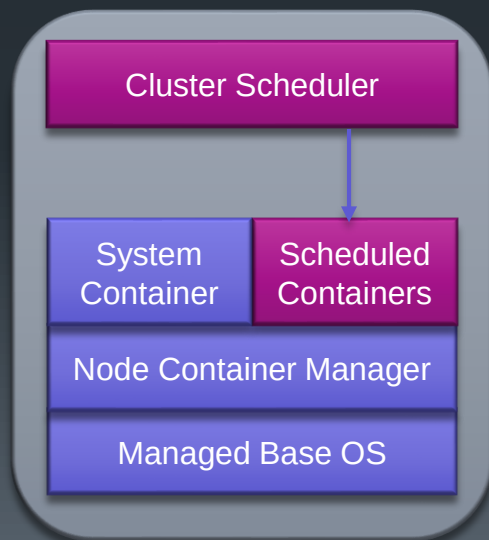
Containers at scale

20

Copyright 2013-2020, RX-M LLC

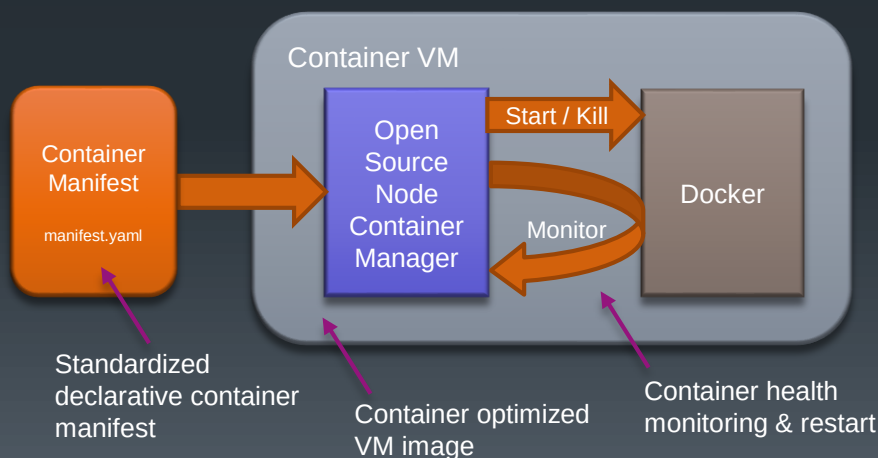
- Containers - a key enabler of PaaS cloud environments
 - Google App Engine is one of the most visible container based cloud PaaS systems
 - Google's new **Google Cloud Platform** is a native container as a service based cloud: CaaS
- Everything at Google, from Search to Gmail, is packaged and run in a Linux container
 - Google's **Borg** system (now replaced by its successor **Omega**) is a cluster manager that runs hundreds of thousands of jobs, from many thousands of different containerized applications, across a number of clusters each with up to tens of thousands of machines (Borg is the basis for Kubernetes)
 - Over 2 billion containers are started per week at Google (over 3,000 per second on average)
- Imctfy** ("Let Me Contain That For You") open source version of Google's container stack
 - Now collaborating with Docker and porting the core Imctfy concepts and abstractions to **libcontainer**,
 - libcontainer is the go forward **OCI** solution

Google Container Infrastructure



Google Cloud Platform

Docker containers in Google Compute Engine with Kubernetes



Containers At Scale by
Joe Beda
Published May 22, 2014
in Technology

What is Kubernetes

21

Copyright 2013-2020, RX-M LLC

- Kubernetes **builds upon a decade and a half of experience** that Google has with running production workloads at scale (**Borg**), combined with best-of-breed ideas and practices from the community
 - The 2015 Borg white paper discusses the architecture and usage of Borg within Google
 - “8.3 Conclusion: Virtually all of Google’s cluster workloads have switched to use Borg **over the past decade**. We continue to evolve it, and **have applied the lessons we learned from it to Kubernetes.**”
- Kubernetes is an open-source platform for automating deployment, scaling, and operations of application containers across clusters of hosts
- Responds quickly to user demand:
 - **Scaling** applications on the fly
 - Seamlessly **rollout** new features
 - **Optimized** use of hardware using only the resources needed



Kubernetes

- Greek word (κυβερνήτης) meaning helmsman or captain
- Derived from the Greek word kubernan
- Derivations in other languages include the English cybernetics

Cloud Native Landscape

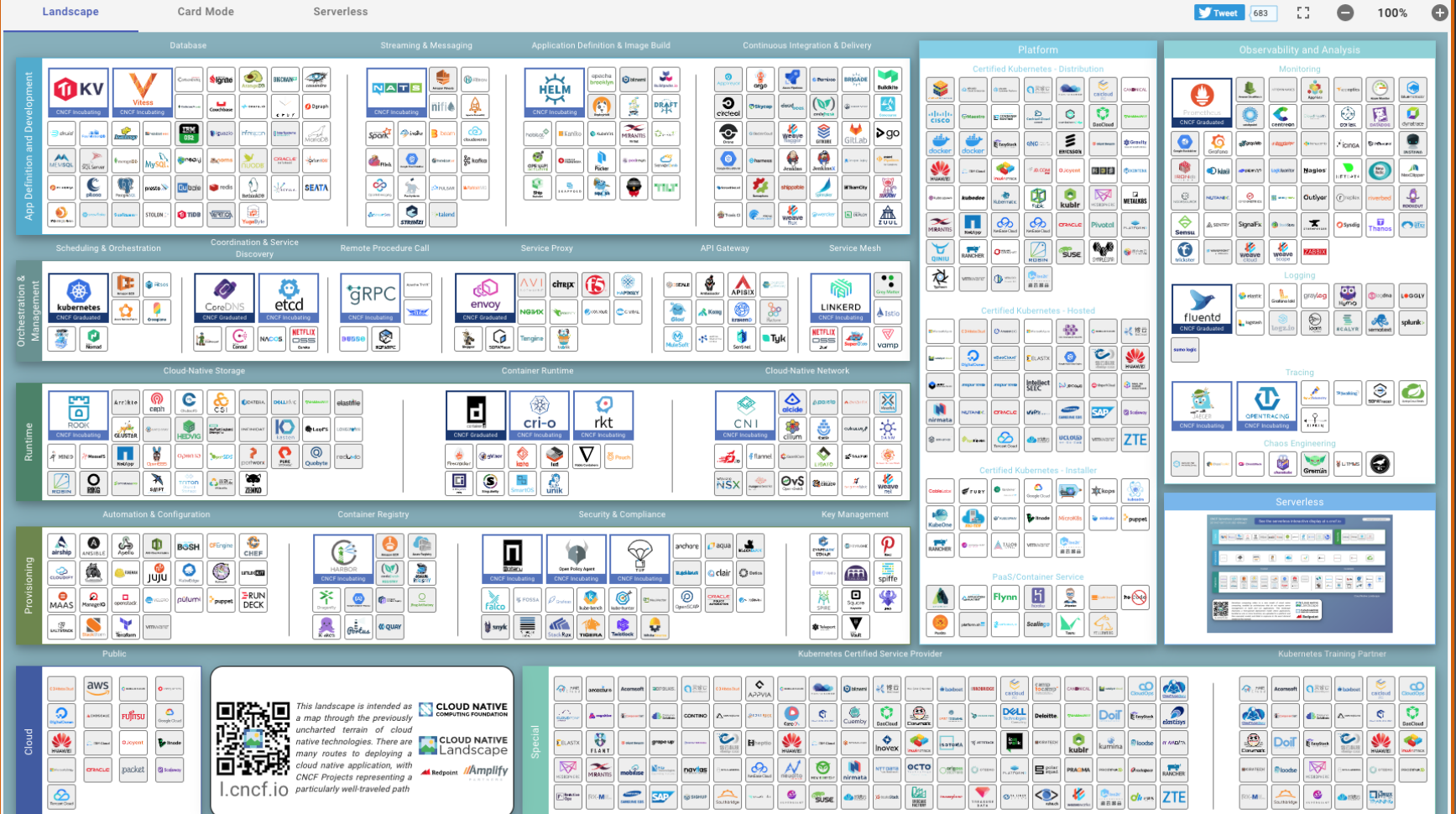
22

Copyright 2013-2020, RX-M LLC

CNCF Cloud Native Interactive Landscape

The Cloud Native Trail Map ([png](#), [pdf](#)) is CNCF's recommended path through the cloud native landscape. The cloud native landscape ([png](#), [pdf](#)) and serverless landscape ([png](#), [pdf](#)) are dynamically generated below. Please [open](#) a pull request to correct any issues. Greyed logos are not open source. Last Updated: 2019-07-08 11:59:37Z.

You are viewing 715 cards with a total of 1,703,782 stars, market cap of \$7.8T and funding of \$12.86B.



K8S

23

Copyright 2013-2020, RX-M LLC

- A high end road bike
- A Chinese multimedia gizmo
- A modular microphone
- An abbreviation for Kubernetes



Supported Installations

Copyright 2013-2020, RX-M LLC

24

- Local machine
 - **kubeadm** – the reference installer for Kubernetes
 - **Minikube** – the recommended method for creating a single node cluster for testing & development
 - **Docker Desktop** – single-node K8s cluster for development
 - **Minishift** – community version of OpenShift for Windows, macOS, Linux
 - **MicroK8s** – single-command fast install (~30 seconds) w/ plugin support
 - **kind** – Kubernetes-in-Docker multi-node container-based cluster
 - **Ubuntu on LXD** – supports a 9-instance deployment on localhost via LXC
 - **IBM Cloud private community edition** (free version of the enterprise solution) – uses VirtualBox to deploy Kubernetes to one or more VMs for dev and test scenarios
- VMs & bare metal
 - **CloudStack** – Ansible playbooks deploy K8s on a CloudStack-based cloud using CoreOS images
 - **CoreOS** – Vagrant single and multimode installs and a HA cloud guide
 - coreos-libvirt creates a single master & 3 nodes
 - **Fedora** – Ansible playbooks which setup a master and 1+ nodes on Fedora with flannel
 - Manual setup is also possible in single and multi node configurations
 - **kops** – supported solution for AWS deployment (GCE on beta and VMware vSphere in alpha)
 - **kubicorn** – simple, cloud native infrastructure for Kubernetes: <http://kubicorn.io>
 - **kubespray** – Ansible playbooks & tools to install a Kubernetes cluster hosted on GCE, Azure, OpenStack, AWS, or baremetal
 - **oVirt** – import or install Project Atomic or Fedora to generate a VM template for K8s
 - Currently no community-supported or pre-loaded VM images
 - **Vagrant/VirtualBox** – single & multi node deployments
 - Setup script creates a Master and a Node using CoreOS & flannel
 - **VMware**
 - vSphere
 - Kubernetes-Anywhere will deploy and configure a cluster from scratch on Photon OS
 - Photon Controller
 - Runs one Master & 3+ Debian Nodes
- Integrations
 - Kubernetes on **Mesos**
 - Deploys the Kubernetes-Mesos service on a Mesos cluster
 - Some known issues with centralized logging and monitoring
 - Kubernetes on **Mesosphere DCOS**
 - Installs Kubernetes-Mesos on Datacenter Operating System (DCOS) with the DCOS CLI

Two layer methodology

- Layer 1 – Infrastructure Management
 - Mesos (Marathon)
 - OpenStack (Magnum)
 - AWS/Gcloud/etc.
- Layer 2 – Application Management
 - Kubernetes
 - Swarm

K8s as a Service Solutions

25

Copyright 2013-2020, RX-M LLC

- All major clouds presently offer a fairly mature **Kubernetes as a Service** solution
 - Open cloud console, click some buttons, use Kubernetes
- **Certified Kubernetes**
 - <https://www.cncf.io/certification/software-conformance/>
 - Software conformance ensures that every vendor's version of Kubernetes supports the required APIs, as do open source community versions
 - For organizations using Kubernetes, conformance enables interoperability from one Kubernetes installation to the next
 - CNCF runs the Certified Kubernetes Conformance Program
 - Most of the world's leading enterprise software vendors and cloud computing providers have Certified Kubernetes offerings
- KaaS certified offerings:
 - **EKS** – Amazon EKS - Managed Kubernetes Service
 - **AKS** – Microsoft Azure Kubernetes Service
 - **GKE** – Google Kubernetes Engine
 - **IKE** – IBM Cloud Kubernetes Service
 - **OCE** – Oracle Container Engine
 - **RKS** – Rackspace Kubernetes as a Service
 - **PKS** – VMware Cloud PKS
 - **RHOSD** – RedHat OpenShift Dedicated
 - Many others



- Works with VMs, physical servers and cloud servers
- Designed to be part of a large provisioning system (used by kops for example)
- Also for easy manual provisioning
- `kubeadm init` bootstraps a K8s cluster:
 - Runs a series of pre-flight checks to **validate the system** state prior to making changes
 - Generates warnings or errors that cause kubeadm to exit (can be bypassed with `--skip-preflight-checks`)
 - **Generates a token** that additional nodes can use to register themselves with the master
 - User supplied token supported
 - Generates a **self-signed CA** using openssl to provision identities for each node in the cluster
 - Also used by the API server to secure communication with clients
 - Outputs a **kubeconfig file for the kubelet** to use to connect to the API server
 - Creates an additional kubeconfig file for administration
 - Generates Kubernetes resource manifests on the master in `/etc/kubernetes/manifests` for **etcd**, **api-server**, **controller manager** and **scheduler**
 - Installs any add-on components, such as DNS and kube-proxy
- Configuring kubeadm with a configuration file instead of command line flags is *recommended*
 - Some advanced features (such as using an external etcd cluster) only available as configuration file options

kubeadm join

- Uses the token to talk to the API server and securely get the root CA certificate
- Creates a local key pair; prepares a certificate signing request (CSR) and sends that off to the API server for signing
- Configures the local kubelet to connect to the API server
- Using the `--control-plane` flag enables joining the node as a Master (Beta as of 1.15)

```
# kubeadm init --help
Usage:
  kubeadm init [flags]
  kubeadm init [command]
```

Flags:

<code>--apiserver-advertise-address string</code>	The IP address the API Server will advertise it's listening on. Specify '0.0.0.0' to use the address
<code>--apiserver-bind-port int32</code>	Port for the API Server to bind to. (default 6443)
<code>--apiserver-cert-extra-sans strings</code>	Optional extra Subject Alternative Names (SANs) to use for the API Server serving certificate. Can be
<code>--cert-dir string</code>	The path where to save and store the certificates. (default "/etc/kubernetes/pki")
<code>--config string</code>	Path to kubeadm config file. WARNING: Usage of a configuration file is experimental.
<code>--cri-socket string</code>	Specify the CRI socket to connect to. (default "/var/run/dockerhim.sock")
<code>--dry-run</code>	Don't apply any changes; just output what would be done.
<code>--feature-gates string</code>	A set of key-value pairs that describe feature gates for various features.
<code>--ignore-preflight-errors strings</code>	A list of checks whose errors will be shown as warnings. Example: 'IsPrivilegedUser,Swap'. Value 'all' ignores errors from all checks.
<code>--image-repository string</code>	Choose a container registry to pull control plane images from (default "k8s.gcr.io")
<code>--kubernetes-version string</code>	Choose a specific Kubernetes version for the control plane. (default "stable-1")
<code>--node-name string</code>	Specify the node name.
<code>--pod-network-cidr string</code>	Specify range of IP addresses for the pod network. If set, the control plane will automatically allocate CIDRs for every node.
<code>--service-cidr string</code>	Use alternative range of IP address for service VIPs. (default "10.96.0.0/12")
<code>--service-dns-domain string</code>	Use alternative domain for services, e.g. "myorg.internal". (default "cluster.local")
<code>--skip-phases strings</code>	List of phases to be skipped

Summary

- Containers enable multiple deterministic application environments to run on a single kernel instance
- OCI containers can run anywhere there is a 64 bit Linux kernel api
- Docker can run on many platforms
 - Typically runs on x64 Linux and requires a 3.10 or later kernel
- Containers can be orchestrated to empower microservice style applications
- Many burgeoning container orchestration solutions are rapidly evolving
- Kubernetes is a system for managing containerized applications across multiple hosts
 - Kubernetes is production ready and evolved from a long line of proprietary Google container systems
 - Kubernetes is the leading container orchestration platform
- Kubernetes can be deployed in a wide range of environments including bare metal OS, VM and cloud

Lab 1

- Kubernetes cluster setup with kubeadm on your lab VM

Lab 1 Documentation Refs

- Installing Kubeadm
 - <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>
- Bootstrapping a Cluster with Kubeadm
 - <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/>
- Installing a network plugin
 - <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/#pod-network>

2: Kubernetes Architecture

Objectives

- Describe the goals of the Kubernetes system
- Examine the design principles of Kubernetes
- Explore the Kubernetes architecture
- List the Kubernetes service components

K8s Design Overview

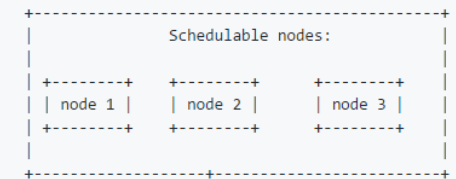
32

Copyright 2013-2020, RX-M LLC

- Kubernetes establishes **robust declarative primitives** for maintaining the desired state requested by the user
 - Main value add of Kubernetes: **declarative target state**
- Also designed to facilitate **migration of non-containerized application stacks** to Kubernetes
 - Includes abstractions for grouping containers in both **loosely coupled and tightly coupled formations**
 - Provides ways for containers to find and communicate with each other in relatively familiar ways
- Kubernetes scheduler automatically chooses hosts to run those containers on
 - The scheduler takes into account:
 - Individual and collective **resource requirements**
 - Quality of service requirements
 - Hardware/software/policy **constraints**
 - Affinity and anti-affinity specifications
 - Data locality
 - Inter-workload interference
 - Deadlines

The scheduling algorithm

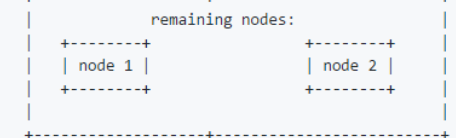
For given pod:



v

Pred. filters: node 3 doesn't have enough resource

v



v

Priority function: node 1: p=2
node 2: p=5

v

select max{node priority} = node 2

K8s Design Overview

33

Copyright 2013-2020, RX-M LLC

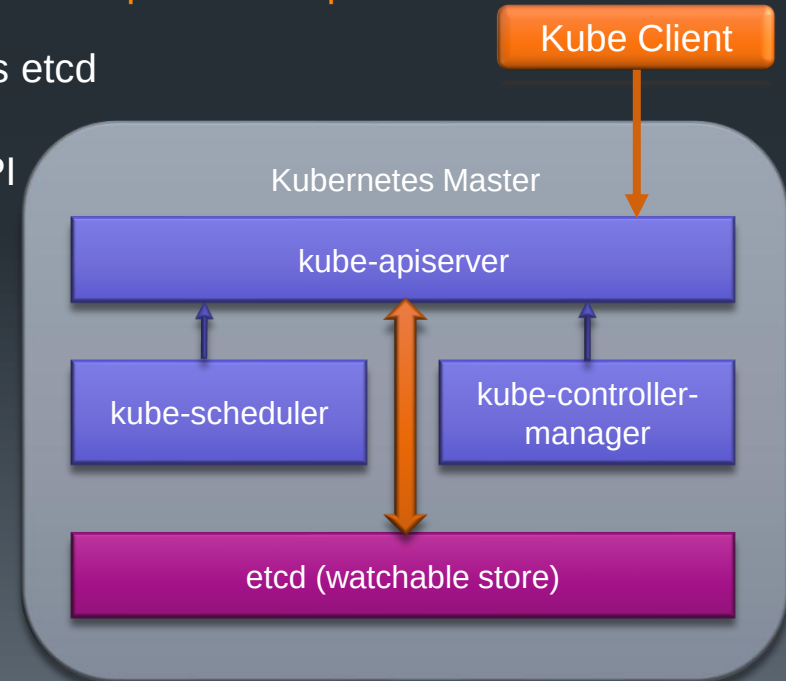
- Self-healing mechanisms
 - Auto-restart
 - Re-schedule
 - Replication
- Kubernetes runs on a number of cloud providers, as well as on physical hosts
- A single Kubernetes cluster is not intended to span multiple regions
 - Recommend building a higher-level layer to replicate complete deployments of highly available applications across multiple zones
- Kubernetes aspires to be an extensible, pluggable, building-block OSS platform and toolkit
 - Kubernetes is built as a collection of pluggable components and layers
 - Use alternative schedulers, controllers, storage systems, and distribution mechanisms
 - Others provide higher-level PaaS functionality or multi-cluster layers, without modification of core Kubernetes source (Ex: OpenShift)
 - API isn't (or even necessarily mainly) targeted at end users, but at tool and extension developers
 - All APIs are visible and available, including the APIs used by the scheduler, the node controller, the replication-controller manager, etc.

Master Components

34

Copyright 2013-2020, RX-M LLC

- The Kubernetes Control Plane
 - The Kubernetes control plane is split into a set of microservices
 - These components work together to provide a unified view of the cluster
- **etcd**
 - All **persistent master state** is stored in an etcd cluster
 - **Watch support** allows coordinating components to be notified of changes
- **API Server**
 - Serves the Kubernetes API
 - A CRUD-y server, most/all business logic **implemented in separate components or plug-ins**
 - Processes REST operations, validates them, updates etcd
- **Scheduler**
 - Binds unscheduled pods to nodes via the /binding API
 - The **scheduler is customizable** and pluggable
- **Controller Manager**
 - Other cluster-level functions are performed by the Controller Manager
 - Endpoints sync with services; Node discovery and management; Controllers (RCs, RSs, Autoscalers, etc.)
 - **Split into separate components to make them independently pluggable**



The scheduling process

35

Copyright 2013-2020, RX-M LLC

- **Node**: a physical or virtual machine running Kubernetes, onto which pods can be **scheduled**.
- **Pod**: collocated group of application containers with shared volumes
 - **Smallest deployable units** that can be created, scheduled, and managed with Kubernetes
 - Pods can be created individually, but it's recommended that you use a Controller (RC, RS, Deployment) even if creating a single pod
- The **scheduler** tries to find a **Node** for each **Pod**, one at a time, as it notices these **Pods** via **watch**, in three steps:
 - First it applies a set of "**predicates**" that filter out inappropriate nodes
 - Ex, if the PodSpec specifies resource requests, then the scheduler will filter out nodes that don't have at least that much resources available (computed as the capacity of the Node minus the sum of the resource requests of the containers that are already running on the Node)
 - Second, it applies a set of "**priority** functions" that rank the nodes that weren't filtered out by the predicate check
 - EX, it tries to spread Pods across nodes and zones while at the same time favoring the least-loaded nodes (where "load" here is sum of the resource requests of the containers running on the node, divided by the node's capacity)
 - Finally, the Node with the highest priority is chosen (or, if there are multiple such nodes, then one of them is chosen at random)
 - The code for this main scheduling loop is in the function Schedule() in
 - plugin/pkg/scheduler/generic_scheduler.go

Filtering Nodes

- The purpose of filtering the nodes is to filter out the nodes that do not meet certain requirements of the Pod
- **NoDiskConflict**: Evaluates if a pod can fit due to the volumes it requests and those that are already mounted
 - Currently supported volumes are: AWS EBS, GCE PD, ISCSI and Ceph RBD
- **NewMaxPDVolumeCountPredicate**: Ensures that the number of attached disk volumes does not exceed a maximum value
 - **AzureDiskVolumeFilter** is a VolumeFilter for filtering Azure Disk Volumes
 - **GCEPDVolumeFilter** is a VolumeFilter for filtering GCE PersistentDisk Volumes
 - Default, 16, which is the maximum GCE allows
 - **EBSVolumeFilter** is a VolumeFilter for filtering AWS ElasticBlockStore Volumes
 - Default, 39, since Amazon recommends a maximum of 40 with one of those 40 reserved for the root volume
 - The maximum value can be controlled by setting the KUBE_MAX_PD_VOLS environment variable
- **NewVolumeZonePredicate**: Evaluates if the volumes a pod requests are available on the node, given the Zone restrictions
 - Currently only supported with PersistentVolumeClaims, and looks to the labels only on the bound PersistentVolume
- **PodFitsResources**: Check if the free resource (CPU and Memory) meets the requirement of the Pod
 - The free resource is measured by the capacity minus the sum of requests of all Pods on the node
- **PodFitsHost**: Filter out all nodes except the one specified in the PodSpec's **nodeName** field
- **NewNodeLabelPredicate**: evaluates whether a pod can fit based on the Node labels which match a filter that it requests
- **NewServiceAffinityPredicate**: matches Nodes in such a way to force a ServiceAffinity
 - If the first pod of a service was scheduled to a node with label "region=foo", all the other subsequent pods belong to the same service will be schedule on Nodes with the same "region=foo" label

Filtering Nodes 2

- **PodFitsHostPorts**: Check if any HostPort required by the Pod is already occupied on the node
- **NewPodAffinityPredicate**: Checks if a pod can be scheduled on the specified node with pod affinity/anti-affinity configuration
- **PodMatchNodeSelector**: Check if the labels of the node match the labels specified in the Pod's **nodeSelector** field
- **CheckNodeUnschedulablePredicate**: Checks if a pod can be scheduled on a node with Unschedulable spec
- **CheckNodeMemoryPressure**: Check if a pod can be scheduled on a node reporting memory pressure condition
 - Currently, no BestEffort should be placed on a node under memory pressure as it gets automatically evicted by kubelet
- **CheckNodeDiskPressure**: Check if a pod can be scheduled on a node reporting disk pressure condition
 - Currently, no pods should be placed on a node under disk pressure as it gets automatically evicted by kubelet
- **CheckNodePIDPressurePredicate**: Checks if a pod can be scheduled on a node reporting pid pressure condition
- **CheckNodeConditionPredicate**: checks if a pod can be scheduled on a node reporting out of disk, network unavailable and not ready condition
 - Only node conditions are accounted in this predicate

Ranking Nodes

- The filtered nodes are considered suitable to host the Pod, and it is often that there are more than one nodes remaining
- Kubernetes prioritizes the remaining nodes to find the "best" one for the Pod
- A priority function gives a score from 0-10 with 10 representing for "most preferred"
- Each priority function is weighted by a positive number and the final score of each node is calculated by adding up all the weighted scores
 - $\text{finalScoreNodeA} = (\text{weight1} * \text{priorityFunc1}) + (\text{weight2} * \text{priorityFunc2})$
- If there is more than one node with an equal highest score, a random one among them is chosen
- Currently, Kubernetes scheduler provides some practical priority functions, including:
 - **LeastRequestedPriority/MostRequestedPriority**: The node is prioritized based on the fraction of the node's resources that are free (used)
 - Least: $\text{cpu}((\text{capacity} - \text{sum}(\text{requested})) * 10 / \text{capacity}) + \text{memory}((\text{capacity} - \text{sum}(\text{requested})) * 10 / \text{capacity}) / 2$
 - Most: $(\text{cpu}(10 * \text{sum}(\text{requested}) / \text{capacity}) + \text{memory}(10 * \text{sum}(\text{requested}) / \text{capacity})) / 2$
 - CPU and memory are equally weighted
 - The node with the highest free (used) fraction is the most preferred
 - Note that this priority function has the effect of spreading Pods across the nodes (or packing them) with respect to resource consumption
 - **BalancedResourceAllocation**: This priority function tries to put the Pod on a node such that the CPU and Memory utilization rate is balanced after the Pod is deployed
 - MUST be used together with LeastRequestedPriority
 - **SelectorSpreadPriority**: Spread Pods by minimizing the number of Pods belonging to the same Service, Replication Controller, StatefulSets, or Replica Set on the same node
 - If zone information is present on the nodes, the priority will be adjusted so that pods are spread across zones and nodes
 - **ImageLocalityPriority**: Nodes are prioritized based on locality of images requested by a pod
 - If none of the images are present, the node will be given the lowest priority
 - If some of the images are present on a node, the larger their sizes' sum, the higher the node's priority
 - **NewNodeLabelPriority**: Prioritizes nodes that have the specified label
 - **InterpodAffinity & NodeAffinity**: compute a sum if the corresponding PodAffinity is satisfied for a node

Scheduler Policies

39

Copyright 2013-2020, RX-M LLC

- K8s < v1.6:
 - The Kubernetes Scheduler policies overridden by passing the flag `--policy-config-file` to the scheduler pointing to a JSON file specifying which scheduling policies to use
- K8s >= v1.6
 - New approach is to use a ConfigMap to supply the same information to the scheduler
 - `--policy-configmap`
 - Supplies the ConfigMap name that contains scheduler's policy configuration
 - `--policy-configmap-namespace`
 - The namespace where the policy ConfigMap is located
 - The system namespace will be used if this is not provided or is empty (default kube-system)
 - `--policy-config-file`
 - Only used if policy ConfigMap is not provided or
`--use-legacy-policy-config==true`

Policy Config Files were deprecated in K8s 1.6 but still work

```
{
  "kind" : "Policy",
  "apiVersion" : "v1",
  "predicates" : [
    {"name" : "PodFitsPorts", "order": 1},
    {"name" : "PodFitsResources", "order": 2},
    {"name" : "NoDiskConflict", "order": 5},
    {"name" : "NoVolumeZoneConflict", "order": 4},
    {"name" : "MatchNodeSelector", "order": 6},
    {"name" : "HostName", "order": 3}
  ],
  "priorities" : [
    {"name" : "LeastRequestedPriority", "weight" : 1},
    {"name" : "BalancedResourceAllocation", "weight" : 1},
    {"name" : "ServiceSpreadingPriority", "weight" : 1},
    {"name" : "EqualPriority", "weight" : 1}
  ]
}
```

Custom Schedulers

40

Copyright 2013-2020, RX-M LLC

- Users can create custom schedulers
 - You can use the normal K8s scheduler with a **custom configuration**
 - You can write a **completely new scheduler** of your own delegating responsibility for scheduling arbitrary subsets of pods to your own custom scheduler
 - Can be written in any language and **can be as simple or complex as you need**
 - One example in the K8s docs is a bash script that assigns nodes randomly!
 - A custom scheduler or the K8s scheduler with a custom config **can run alongside, or instead of**, the default Kubernetes scheduler
- Running a scheduler with a unique name string makes it identifiable as a non-default scheduler:
 - **--scheduler-name=my-scheduler**
 - Names the scheduler
- Pods can select a non-default scheduler using the "**spec.SchedulerName**" pod spec setting
 - This value defaults to "default-scheduler", the name of the scheduler registered with no **--scheduler-name**

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  schedulerName: my-scheduler
  containers:
  - name: nginx
    image: nginx:1.10
```

Well-Known Labels

- Kubernetes reserves all labels and annotations in the `kubernetes.io` namespace
- The following node labels are predefined in K8s 1.14:
 - `kubernetes.io/arch`
 - `beta.kubernetes.io/arch` (deprecated)
 - `kubernetes.io/os`
 - `beta.kubernetes.io/os` (deprecated)
 - `kubernetes.io/hostname`
 - `beta.kubernetes.io/instance-type`
 - `failure-domain.beta.kubernetes.io/region`
 - e.g. `failure-domain.beta.kubernetes.io/region=us-east-1`
 - `failure-domain.beta.kubernetes.io/zone`
 - e.g. `failure-domain.beta.kubernetes.io/zone=us-east-1c`
- These labels can be used by load balancers (like Istio), the K8s scheduler and other tools that need to make informed decisions having to do with failure domains and host architecture

Node Components

42

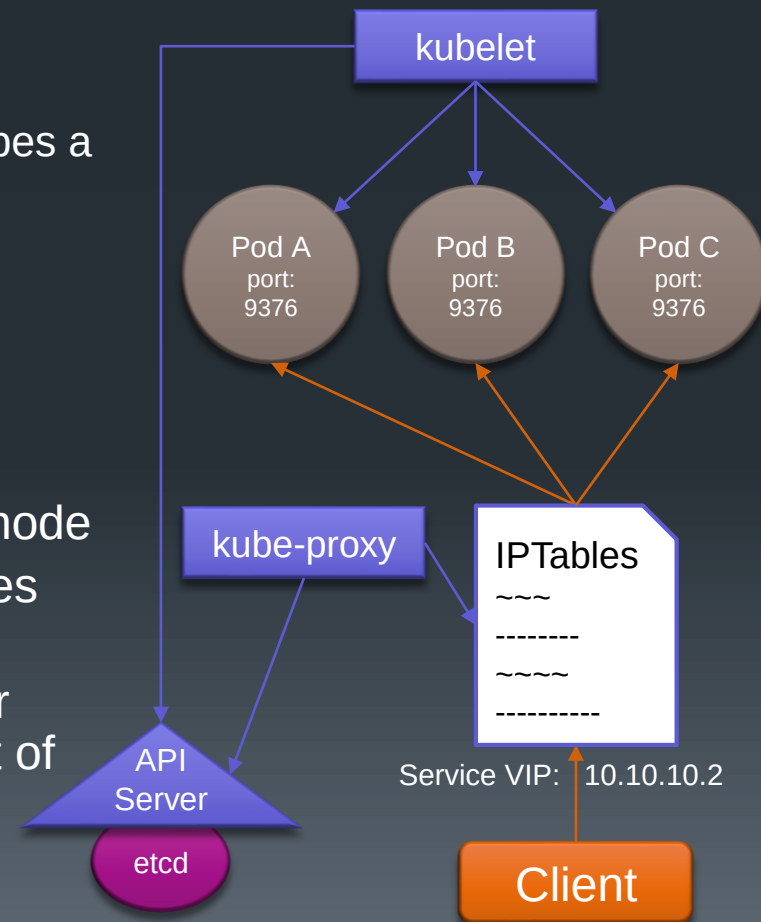
Copyright 2013-2020, RX-M LLC

■ kubelet

- The kubelet is the primary “node agent” that runs on each node
- The kubelet **manages pods** and their containers, their images, their volumes, etc.
- The kubelet works in terms of a PodSpec
 - A PodSpec is a YAML or JSON object that describes a pod
 - kubelet PodSpecs are provided through various mechanisms (primarily through the apiserver)
 - Kubelet ensures that the containers described in PodSpecs are running and healthy

■ kube-proxy (optional)

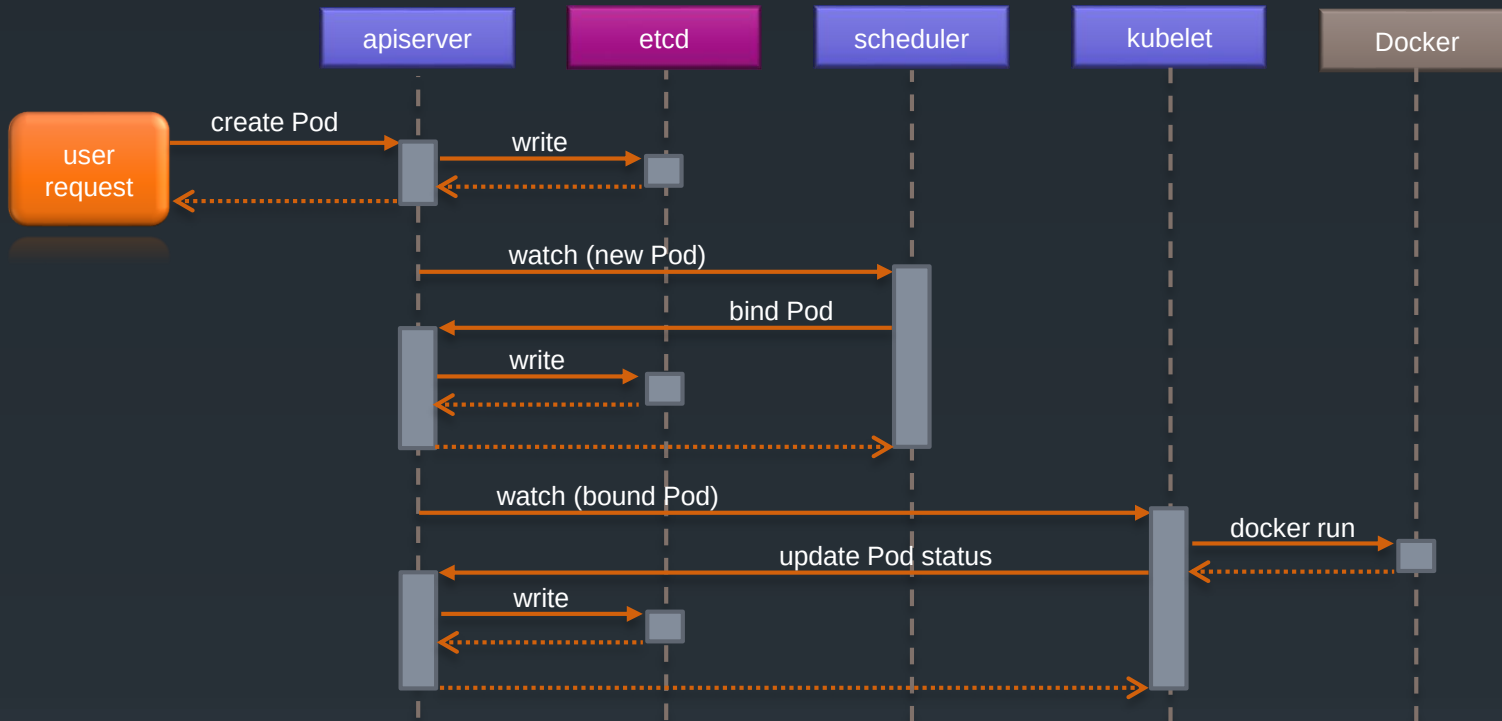
- The Kubernetes network proxy runs on each node
- Manages the **iptables service mesh** for services defined in the Kubernetes API
- Can do simple TCP/UDP stream forwarding or round robin TCP/UDP forwarding across a set of backends in proxy mode



Pod Creation Flow

43

Copyright 2013-2020, RX-M LLC



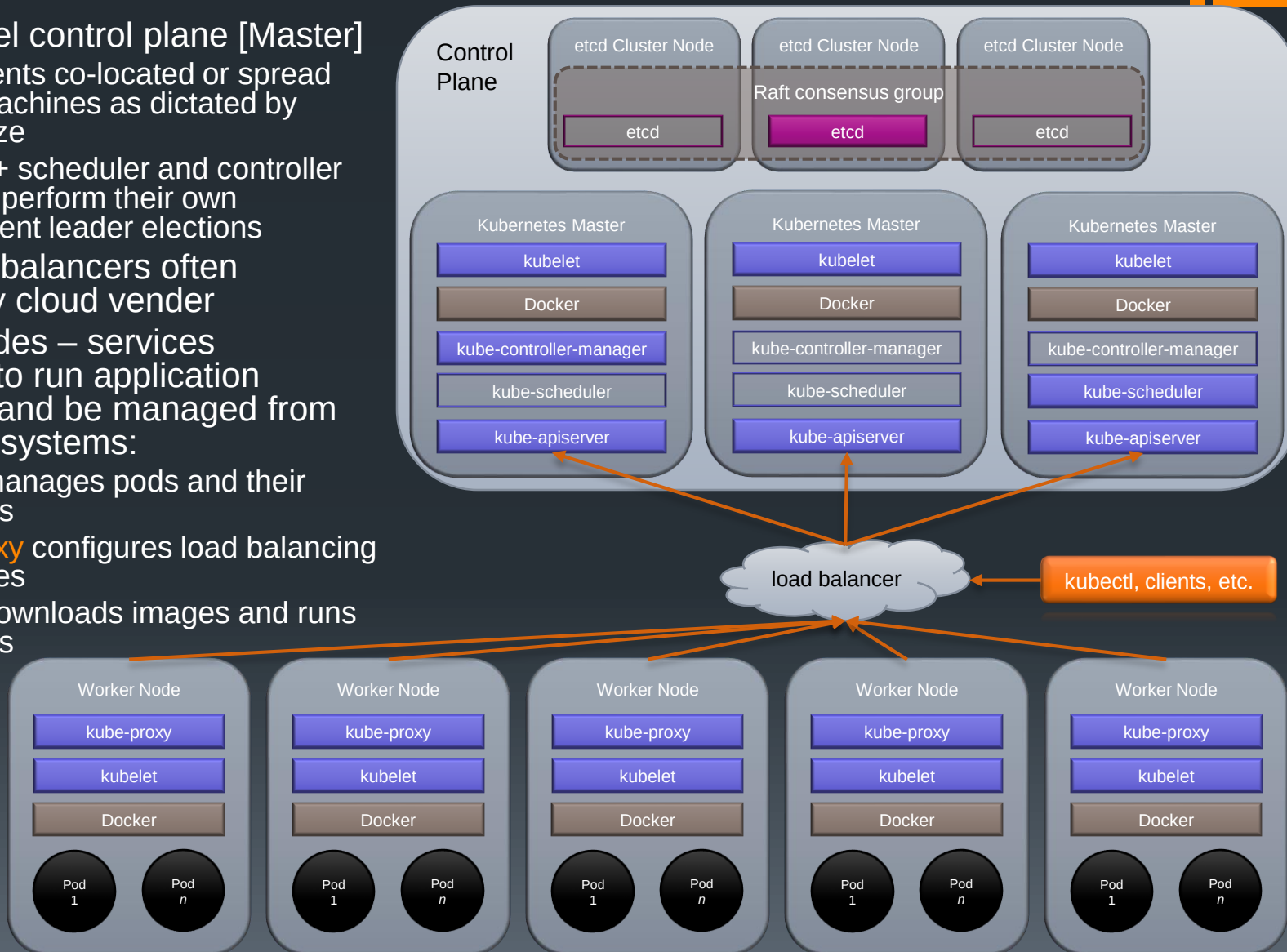
- The user creates a Pod via the API Server and the API server writes it to etcd
- The scheduler notices an “unbound” Pod and decides which node to run that Pod on
 - It writes that binding back to the API Server
- The Kubelet notices a change in the set of Pods that are bound to its node
 - It, in turn, runs the container via the container runtime (Docker)
- The Kubelet monitors the status of the Pod via the container runtime
 - As things change, the Kubelet will reflect the current status back to the API Server

Scaled Architecture & HA Model

44

Copyright 2013-2020, RX-M LLC

- Distributed watchable storage via etcd & Raft
- Cluster-level control plane [Master]
 - Components co-located or spread across machines as dictated by cluster size
 - K8s v1.2+ scheduler and controller manager perform their own independent leader elections
- Proxy load balancers often provided by cloud vendor
- Worker Nodes – services necessary to run application containers and be managed from the master systems:
 - **kubelet** manages pods and their containers
 - **kube-proxy** configures load balancing via iptables
 - **Docker** downloads images and runs containers

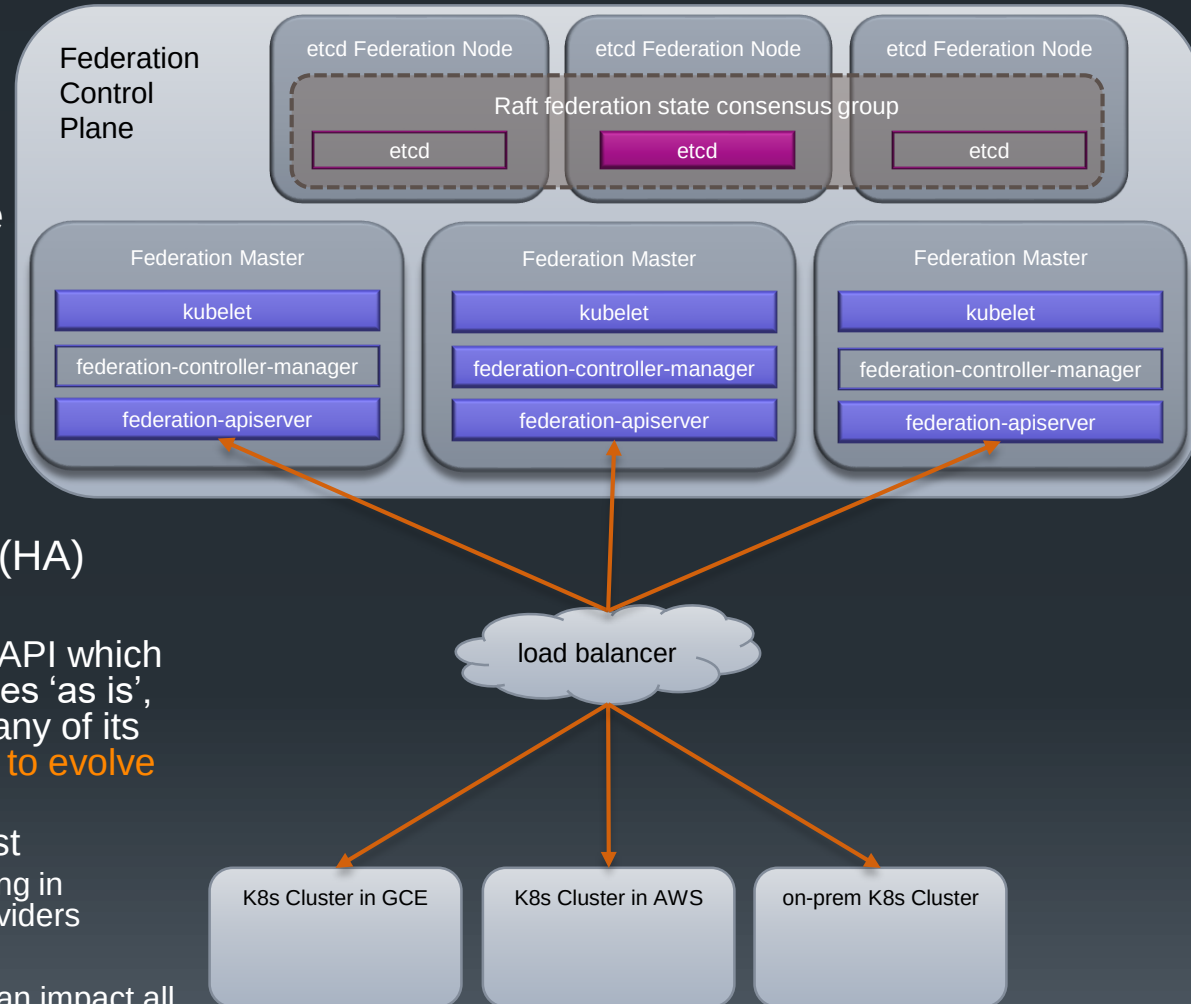


Federation v1 (abandoned)

45

Copyright 2013-2020, RX-M LLC

- Federation makes it possible to manage multiple K8s clusters
 - Hybrid cloud – have multiple clusters with different cloud providers and on-prem
 - Prevents cloud/cluster provider lock-in
- Resources in multiple clusters are **kept in sync**
 - Same Deployment resource can exist in multiple clusters
- **Cross-cluster service discovery**
 - Auto-configure DNS servers & load balancers with backends from all clusters
- Mitigates impact of cluster failure (HA)
- Limitations
 - The current Kubernetes federation API which reuses the Kubernetes API resources 'as is', is currently **considered alpha** for many of its features, and there is **no clear path to evolve the API to GA**
 - Increased network bandwidth & cost
 - May be significant if clusters are running in different regions and with different providers
 - Reduced cross-cluster isolation
 - A bug in the federation control plane can impact all clusters
 - **Maturity** (federation project is relatively new)
 - Many resources are alpha or unavailable



Federation v2

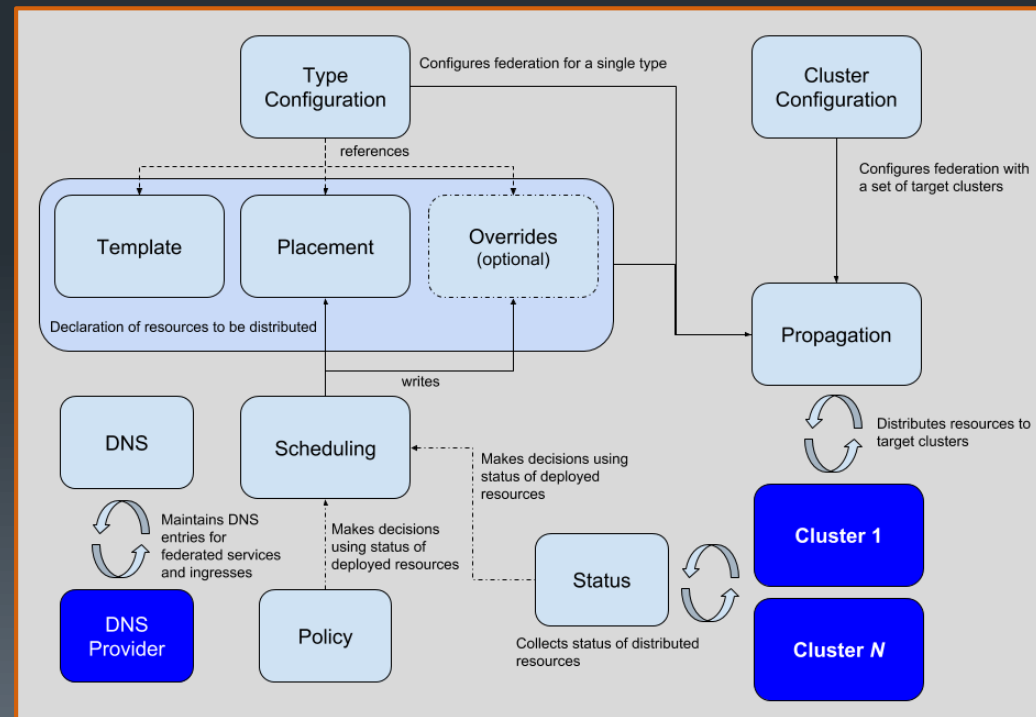
46

Copyright 2013-2020, RX-M LLC

- WIP Prototype on GitHub
 - Builds on sync controller concepts from Federation v1
- Configured with:
 - **Cluster configuration** declares which clusters federation should target
 - **Type configuration** declares which API types federation should handle and has 3 fundamental concepts:
 - **Templates** – definition of the resource that will have common settings across clusters
 - **Placement** – how resources are propagated in each cluster
 - **Overrides** – how resources are differentiated in each cluster
 - Credential schemes, replica counts, how rolling updates are handled, etc.

Features

Feature	Maturity
Push propagation of arbitrary types to remote clusters	Alpha
CLI utility (<code>kubefedctl</code>)	Alpha
Generate Federation APIs without writing code	Alpha
Multicluster Service DNS via <code>external-dns</code>	Alpha
Multicluster Ingress DNS via <code>external-dns</code>	Alpha
Replica Scheduling Preferences	Alpha

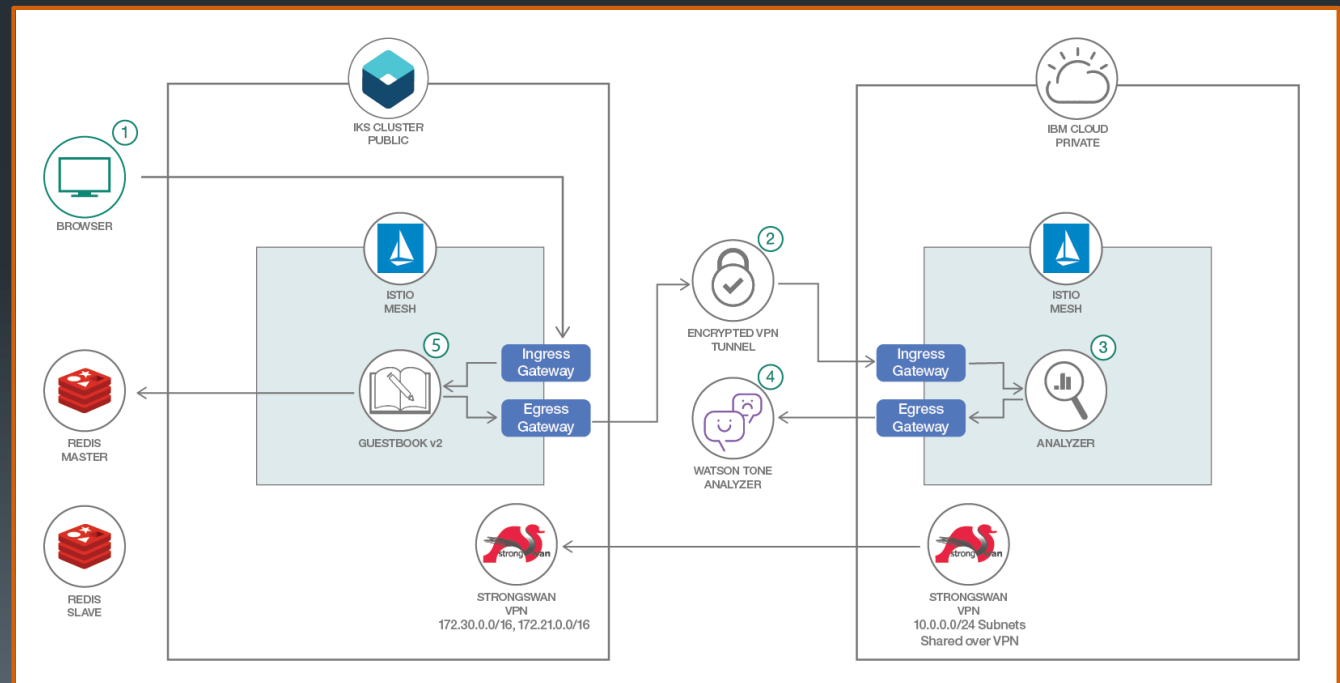


Istio

47

Copyright 2013-2020, RX-M LLC

- Istio is a **multi cloud capable service mesh** for Kubernetes that support hybrid and multicloud application development
- Originally developed by Google, IBM and Lyft
- Uses the CNCF Envoy project as the Pod proxy



kubectl Config: Cluster and Context

- A kubectl **cluster** is a Kubernetes API endpoint for a given Kubernetes cluster
 - kubectl can be configured with multiple clusters
 - Only one context is active at a time and each context points to precisely one cluster
 - `kubectl config set-cluster` sets a cluster entry in kubeconfig [`~/.kube/config`]
 - `kubectl config set-cluster NAME [--server=server] [--api-version=apiversion] [--certificate-authority=path/to/certificate/authority] [--insecure-skip-tls-verify=true]`
 - Example: `$ kubectl config set-cluster k8s \ --server=https://15.22.31.4:6443`
- A kubectl **context** defines the cluster connection metadata to be used with kubectl commands
- Set with: `kubectl config set-context`
 - Sets a context entry in kubeconfig [`~/.kube/config`]
 - Specifying a name that already exists will merge new fields on top of existing values for those fields
 - `kubectl config set-context NAME \ [--cluster=cluster_nickname] \ [--user=user_nickname] \ [--namespace=namespace]`
 - Example: `$ kubectl config set-context \ admin@k8s --user=cluster-admin`

```
$ kubectl config view
```

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: REDACTED
    server: https://192.168.225.236:6443
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
    name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

kubectl Use

49

Copyright 2013-2020, RX-M LLC

■ Syntax

- `kubectl [command] [TYPE] [NAME] [flags]`
 - `command` – operation to perform on the named resource(s)
 - `TYPE` – specifies the resource type
 - Specify singular, plural or abbreviated forms (deployment, deployments, deploy)
 - `NAME` – specifies the name of the resource
 - `flags` – optional flags: (-o name for just names, -o yaml for yaml output, etc.)

■ Basic Resource Commands:

- `get` – display one or many resources
- `describe` – show details of a specific resource or group of resources
- `create` – create a resource by filename or stdin
- `apply` – apply a configuration change to a resource from a file or stdin
- `delete` – delete resources by filenames, stdin, resources and names, or by resources and label selector
- `logs` – print the logs for a container in a pod, (-p) for previous container
- `attach` – attach to a running container
- `exec` – execute a command in a container
- `explain` – get documentation of resources
- `api-resources` – list of all the supported resource types and their abbreviated aliases

■ Autocompletion support

- Add it to your current shell
 - `source <(kubectl completion bash)`
- To add to your profile so it is automatically loaded in future shells :
 - `echo "source <(kubectl completion bash)" >> ~/.bashrc`

Summary

- Kubernetes is a system for managing containerized applications across multiple hosts
- Kubernetes is itself a microservice based system and some or all of it may be executed via containers
- Kubernetes relies on etcd to maintain distributed watchable state
- kubectl offers several commands to work with resources
 - create
 - delete
 - describe
 - get
 - logs

Lab 2

- Using Kubernetes with kubectl

Lab 2 Documentation Refs

- Kubectrl reference
 - <https://kubernetes.io/docs/reference/kubectrl/overview/>
- Kubectrl Usage Conventions
 - <https://kubernetes.io/docs/reference/kubectrl/conventions/>
- Kubectrl cheat sheet
 - <https://kubernetes.io/docs/reference/kubectrl/cheatsheet/>
- Viewing and Finding resources
 - <https://kubernetes.io/docs/reference/kubectrl/cheatsheet/#viewing-finding-resources>
- Updating resources:
 - <https://kubernetes.io/docs/reference/kubectrl/cheatsheet/#updating-resources>
- Interacting with Pods (getting logs, viewing details, etc):
 - <https://kubernetes.io/docs/reference/kubectrl/cheatsheet/#interacting-with-running-pods>

3: Pods and Configs

Objectives

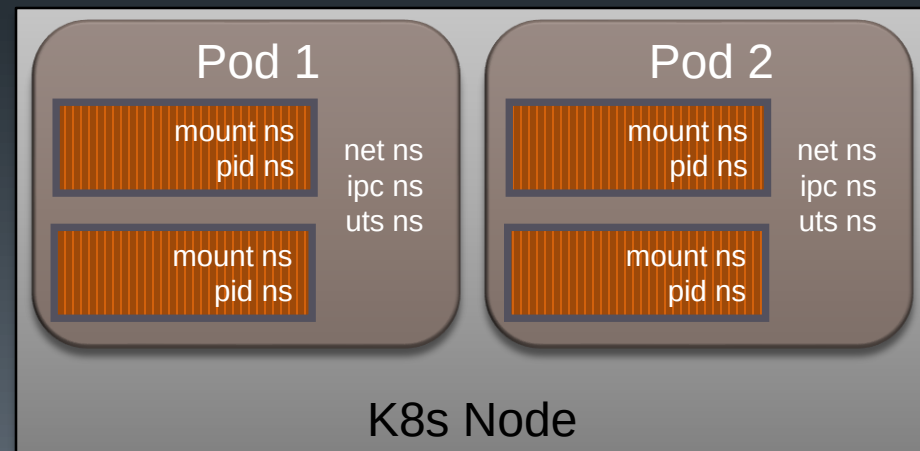
- Understand Kubernetes resource types
- Learn how to define resources with YAML files
- Define Pod and the operation of Pods on Kubernetes
- Understand the relationship between images, containers and Pods
- Start, Stop and monitor Pods

Pods

55

Copyright 2013-2020, RX-M LLC

- Many times a set of containers run as a team on the same host
- These containers usually need to be located together
 - You want to ensure that they do not become separated during dynamic placement
- A pod is a set of containers that are placed and scheduled together as a unit on a Kubernetes Node
 - A pod is the atomic unit of scheduling
 - When the pod is replicated, all the containers are replicated by the same factor
 - Ex: replicating a 2-container pod 3 times results in 6 total containers
- By working to place a group of pods, Kubernetes can pack lots of work onto a node in a reliable way
- Pods are best defined using YAML (or JSON) files



Pod Patterns

56

Copyright 2013-2020, RX-M LLC

■ Patterns

■ Sidecar

- Sidecars extend and enhance the "main" container in the Pod
- Example: Nginx web server container; add a container that syncs the file system with a git repository, share the file system between the containers and you have built Git push-to-deploy

■ Ambassador

- Ambassadors proxy a Pod local connection to the world outside
- Example: Redis cluster with read-replicas and a single write master; create a Pod that groups the main application with a Redis ambassador container which splits reads and writes, sending them on to the appropriate servers

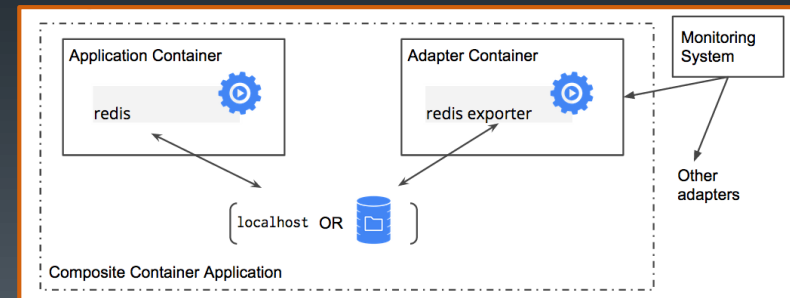
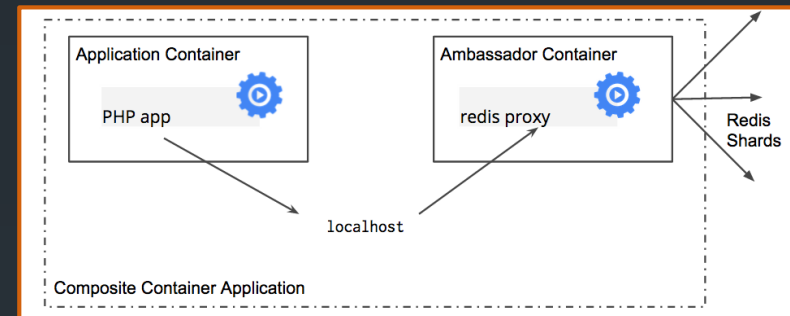
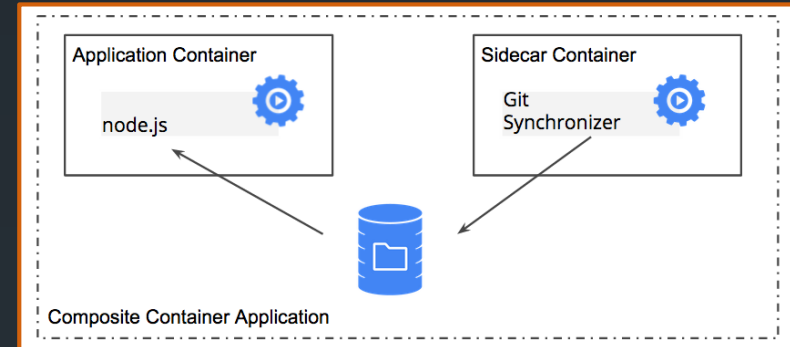
■ Adapter

- Adapter containers standardize and normalize output
- Example: A task monitoring N different applications where each application has a different way of exporting monitoring data (e.g. JMX, StatsD, application specific statistics) but every monitoring system expects a consistent and uniform data model for the monitoring data it collects

■ Benefits

■ Containers are units of:

- **Resource accounting and allocation** – "main" container given priority over resources, "helper" container can be given strict restraints
- **Reuse** – single version of "helper" used among many apps
- **Deployment** – roll out/back independently
- **Failure boundaries** – "main" container can continue even if "helper" fails
- **Divide responsibility** for development between two separate programming teams



- Brendan Burns, Distinguished Engineer at Microsoft and former Software Engineer at Google
<https://research.google.com/pubs/pub45406.html>

Config File Commonalities

57

Copyright 2013-2020, RX-M LLC

- YAML formatted
- Main parts
 - **# Comment**
 - **apiVersion**
 - Kubernetes API version to build with
 - Pods are v1 but other objects will use use API groups (like apps/v1 for Deployments)
 - **kind**
 - Kubernetes object type
 - Pod, Deployment, Service, Namespace, etc...
 - **metadata**
 - Name, labels and other data describing the object
 - **spec**
 - Instructions for building the object
 - The desired state
- Construction
 - **kubectl apply -f filename**

```
# mypod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: web-container
      image: bitnami/apache:latest
      ports:
        - containerPort: 80
```

Pod Configuration Files

- Configured using YAML or JSON files
 - Much like Docker Compose
 - `kubectl apply -f mynewpod.yaml`
- Fields for Pod configuration include:
 - **spec**: the pod specification
 - **volumes**[]): List of volumes that can be mounted by containers in the pod
 - **restartPolicy**: applies to all containers in the pod
 - **terminationGracePeriodSeconds**: number of seconds to shutdown before kill
 - **hostNetwork**: run pod in host net namespace
 - **imagePullSecrets**: secrets to use for pulling images (e.g. DockerConfig for private reg access)
 - **containers**[]): A list of containers belonging to the pod
 - **name**: Name of the container
 - **image**: Docker image name
 - **command**[]): command line (like Docker ENTRYPOINT)
 - **args**[]): entrypoint arguments (like Docker CMD)
 - **env**[]): environment vars
 - **ports**[]): port mappings
 - **containerPort**: port to expose
 - **protocol**: port protocol
 - **hostIP**: host IP to bind to (deprecated)
 - **hostPort**: host port to bind to (deprecated)
 - **volumeMounts**[]): paths to mount volumes

```
$ cat etcd-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: etcd
  name: etcd
spec:
  containers:
  - command:
    - etcd
    - --advertise-client-urls=https://etcdnode:2379
    - --client-cert-auth=true
    - --snapshot-count=10000
    image: gcr.io/etcd:3.3.10
    imagePullPolicy: IfNotPresent
    name: etcd
    volumeMounts:
    - mountPath: /var/lib/etcd
      name: etcd-data
  hostNetwork: true
  volumes:
  - hostPath:
      path: /var/lib/etcd
      name: etcd-data
```

```
$ kubectl apply -f etcd-pod.yaml
pod/etcd created
```

```
$ kubectl get pod
NAME      READY   STATUS    RESTARTS   AGE
etcd      0/1     Pending   0           15s
```

```
$ kubectl logs etcd
...
```

```
$ kubectl get pod
NAME      READY   STATUS    RESTARTS   AGE
etcd      1/1     Running   0           30s
```

Init Containers

59

Copyright 2013-2020, RX-M LLC

- **Init Containers run before App Containers**
 - Contain utilities that are not desirable to include in the app container for security reasons
 - Execute custom code for setup that is not present in an app image
 - Place values into a config file or run a template tool to dynamically generate a config file for the main app container
- Easy way to block or delay the startup of containers until some set of preconditions are met
- Init Containers are **started in order**
 - **After** network and volumes are initialized
 - Each Init Container **must exit successfully** before the next is started
 - Retried according to the Pod restartPolicy
 - If Always Init Containers use OnFailure
- A Pod is not Ready until all Init Containers have succeeded
- Ports on an Init Container are not aggregated under a service

```
spec:
  initContainers:
  - name: "init-chown-data"
    image: "busybox:latest"
    # 65534 is the nobody user that prometheus uses.
    command: ["chown", "-R", "65534:65534", "/data"]
    securityContext:
      capabilities:
        add: ["CAP_CHOWN"]
        drop: ["ALL"]
    volumeMounts:
    - name: storage-volume
      mountPath: /data

  containers:
  - name: prometheus-server
    securityContext:
      runAsUser: 65534
    image: prom/prometheus:latest
    args:
    - --config.file=/etc/config/prometheus.yml
    - --storage.tsdb.path=/data
    volumeMounts:
    - name: storage-volume
      mountPath: /data
    securityContext:
      capabilities:
        drop: ["ALL"]
  volumes:
  - name: storage-volume
    persistentVolumeClaim:
      name: storage-volume
  ...
```


Infrastructure Container

60

Copyright 2013-2020, RX-M LLC

- The Infrastructure Container is often referred to as the pod "pause" container and is used to set up and hold the networking namespace and resource limits for each pod

```
user@ubuntu:~/pods$ cat long.yaml
apiVersion: v1
kind: Pod
metadata:
  name: long-running
spec: # specification of the pod's contents
  containers:
  - name: long
    image: "ubuntu:14.04"
    command: ["/usr/bin/tail", "-f", "/dev/null"]
```

```
user@ubuntu:~/pods$ kubectl create -f long.yaml
pod "long-running" created
```

```
user@ubuntu:~/pods$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
long-running	0/1	Running	0	3s

```
user@ubuntu:~/pods$ sudo docker container ls -f "name=long-running"
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
8fe590c80a12	ubuntu:14.04	"/usr/bin/tail -f /de"	2m ago	Up 2m	k8s_long.94251326_long...
8bf3dbb786cc	gcr.io/google_containers/pause-amd64:3.0	"/ pause "	2m ago	Up 2m	k8s_POD.6d00e006_long...

Resource Limits

61

Copyright 2013-2020, RX-M LLC

- You can specify the **desired** CPU and memory for containers in specs to ensure the scheduler chooses a node with the appropriate resources
- CPU and memory are each a resource type
 - **CPU** is specified in units of cores
 - `spec.container[].resources.requests.cpu`
 - **Memory** is specified in units of bytes
 - `spec.container[].resources.requests.memory`
- Each container of a Pod can optionally specify **constraints**
 - `spec.container[].resources.limits.cpu`
 - `spec.container[].resources.limits.memory`
- Default values are cluster configured
 - If value of requests is not specified, they are set to be equal to limits by default (when limits are defined)
 - Limits must be greater than or equal to resource requests
- Requests/limits can only be specified on individual containers
 - Pod request/limits are the sum of their container request/limits
- If the scheduler cannot find any node where a pod can fit, then the pod will remain unscheduled until a place can be found

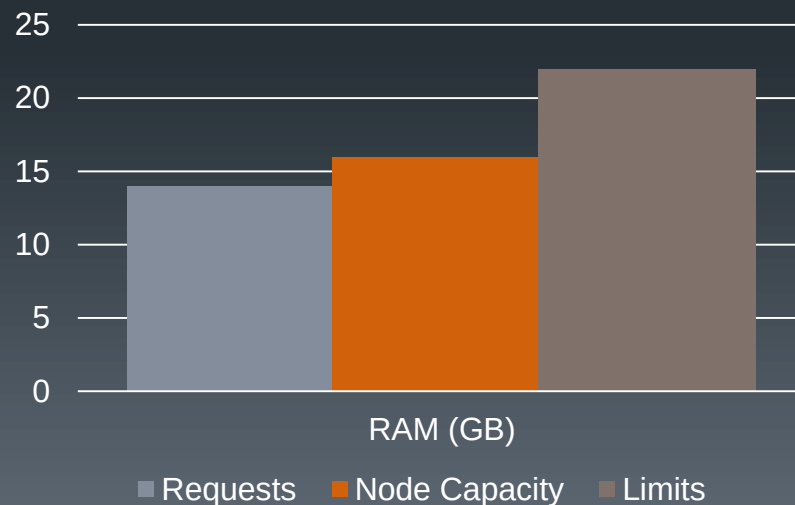
```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
    - name: app
      image: app:v2.2
      resources:
        requests:
          memory: "128Mi"
          cpu: "0.25"
        limits:
          memory: "256Mi"
          cpu: "0.5"
    - name: sidecar
      image: sidecar:v3.4
      resources:
        requests:
          memory: "64Mi"
          cpu: "0.1"
        limits:
          memory: "128Mi"
          cpu: "0.25"
```

Ephemeral Storage

- Kubelet's root directory (`/var/lib/kubelet` by default) is also shared and consumed by Pods via `emptyDir` volumes
 - This partition is ephemeral and applications cannot expect any performance SLAs (Disk IOPS)
- Requests and limits for local ephemeral storage
 - Each Container of a Pod can specify:
 - `spec.containers[].resources.limits.ephemeral-storage`
 - `spec.containers[].resources.requests.ephemeral-storage`
- The scheduler ensures that the sum of the resource requests of the scheduled Containers is less than the capacity of the node
- A pod will be evicted if:
 - A container's writable layer and logs usage exceeds its storage limit
 - The sum of the local ephemeral storage usage from all containers plus the pod's `emptyDir` volumes exceeds the limit

- Kubernetes uses QoS classes to make decisions about scheduling and evicting Pods
- When Kubernetes creates a Pod it assigns one of these QoS classes to the Pod:
 - **Guaranteed** – Pods are considered top-priority and are guaranteed to not be killed until they exceed their limits
 - For a Pod to be given a QoS class of Guaranteed:
 - Every Container in the Pod must have a memory limit and a memory request, and they **must be the same**
 - Every Container in the Pod must have a cpu limit and a cpu request, and they **must be the same**
 - **Burstable** – Pods have some form of minimal resource guarantee, but can use more resources when available
 - Under system memory pressure, these containers are more likely to be killed once they exceed their requests and no Best-Effort pods exist
 - A Pod is given a QoS class of Burstable if:
 - **At least one Container** in the Pod **has a memory or cpu request**
 - **BestEffort** – Pods will be treated as lowest priority
- Processes in these pods are the first to get killed if the system runs out of memory but can use any amount of free memory on the node
 - For a Pod to be given a QoS class of BestEffort, the Containers in the Pod **must not have any** memory or cpu limits or requests

Node Over-commitment



- Eviction is the removal of a tenant from rental property by the landlord
 - What the Kubelet does to certain pods when resource levels become critical
- Types of eviction:
 - Soft** – eviction that occurs only after a threshold is exceeded for a certain period of time
 - Hard** – eviction that occurs immediately when a threshold is exceeded
- Kubelet eviction configuration:
 - evictionSoft** – set of eviction thresholds (e.g. `memory.available<1.5Gi`) that if met over a corresponding grace period would trigger a pod eviction
 - evictionSoftGracePeriod** – set of eviction grace periods (e.g. `memory.available=1m30s`) that correspond to how long a soft eviction threshold must hold before triggering a pod eviction
 - evictionHard** – set of eviction thresholds (e.g. `memory.available<1Gi`) that if met would trigger a pod eviction. (default "`memory.available<100Mi`")
 - evictionMaxPodGracePeriod** – maximum allowed grace period (in seconds) to use when terminating pods in response to a soft eviction threshold being met
 - If negative, defers to pod specified value
 - evictionMinimumReclaim** – set of minimum reclaims (e.g. `imagefs.available=2Gi`) that describes the minimum amount of resource the kubelet will reclaim when performing a pod eviction if that resource is under pressure
 - evictionPressureTransitionPeriod** – duration for which the kubelet has to wait before transitioning out of an eviction pressure condition (default 5m0s)
 - Protects against oscillation in and out of a threshold

Kubelet Eviction

```
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
evictionHard:
  imagefs.available: 15%
  memory.available: 100Mi
  nodefs.available: 10%
  nodefs.inodesFree: 5%
```



Reserve Resources for System Daemons

65

Copyright 2013-2020, RX-M LLC

- System daemons are expected to be treated similar to Guaranteed pods
- Kube reserved
 - Meant to capture resource reservations for kubernetes system daemons (kubelet, container runtime, node problem detector, etc.)
 - Not meant to reserve resources for system daemons that are run as pods
 - Kubelet flags:
 - `--kube-reserved=[cpu=100m],[memory=100Mi],[ephemeral-storage=1Gi],[pid=1000]`
 - Specify the parent control group: `--kube-reserved-cgroup=`
- System reserved
 - Meant to capture resource reservation for OS system daemons (sshd, udev, etc.)
 - Should reserve memory for kernel because kernel memory is not accounted to pods in Kubernetes currently
 - Reserving resources for user login sessions is also recommended (user.slice in systemd world)
 - only use if nodes have been profiled exhaustively to come up with precise estimates and is confident in their ability to recover if any process in that group is oom_killed
 - Kubelet flags:
 - `--system-reserved=[cpu=100m],[memory=100Mi],[ephemeral-storage=1Gi],[pid=1000]`
 - Specify the parent control group: `--system-reserved-cgroup=`
- Node Allocatable
 - Scheduler treats *Allocatable* as the available capacity for pods
 - Will use instead of *Capacity*, thereby not overcommitting the node
 - Kubelet flags:
 - `--enforce-node-allocatable=pods,[kube-reserved],[system-reserved]`

Guidelines in the K8s docs:

- Enforce Allocatable on pods
- With adequate monitoring and alerting, enforce kube-reserved based on usage heuristics
- If necessary, enforce system-reserved over time

<https://kubernetes.io/docs/tasks/administer-cluster/reserve-compute-resources/>

Assigning Pods to Nodes

- **nodeName** – simplest form of node selection constraint using the node's name
- **nodeSelector** – specifies a map of key-value pairs
 - For a pod to be eligible to run on a node, the node *must have* each of the indicated key-value pairs as labels
 - The most common usage is one key-value pair
- Built-in node labels – in addition to user-created labels, nodes come pre-populated with a standard set of labels:
 - `kubernetes.io/hostname`
 - `failure-domain.beta.kubernetes.io/zone`
 - `failure-domain.beta.kubernetes.io/region`
 - `beta.kubernetes.io/instance-type`
 - `kubernetes.io/os`
 - `kubernetes.io/arch`

```
apiVersion: v1
kind: Pod
metadata:
  name: nodename-pod
  labels:
    app: someapp
spec:
  nodeName: kube-node-01
  containers:
  - name: someapp-container
    image: app-image:01
...
---
```

```
apiVersion: v1
kind: Pod
metadata:
  name: selective-pod
  labels:
    app: myapp
spec:
  nodeSelector:
    disk-type: local-storage
  containers:
  - name: myapp-container
    image: myapp:1.5
...
```

Node Affinity

- **NodeAffinity** – generalization of the nodeSelector feature but using a more expressive syntax
 - Constrain which nodes your pod is eligible to schedule on, based on **labels on the node**
- Two types:
 - **requiredDuringSchedulingIgnoredDuringExecution**: Specifies rules that **must be met** for a pod to schedule
 - If no node matches the criteria (plus all of the other normal criteria, such as having enough free resources for the pod's resource request), then the pod won't be scheduled
 - **preferredDuringSchedulingIgnoredDuringExecution**: Specifies **preferences** that the scheduler will try to enforce but will not guarantee
 - If nodes match the rules, they will be chosen first, and only if no preferred nodes are available will non-preferred nodes be chosen
- “IgnoredDuringExecution” part of the names means that, if labels on a node change at runtime such that the affinity rules on a Pod are no longer met, the Pod will *still continue to run on the node*
- Node anti-affinity can be achieved by using negative operators (eg: NotIn)

```
...
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/k8s-az-name
                operator: In
                values:
                  - k8s-az1
                  - k8s-az2
...

```


Pod Affinity & Anti-Affinity

- **Inter-pod affinity and anti-affinity** – specify rules about how pods should be placed relative to one another
 - Based on **labels on pods** that are already running on nodes
- Spread or pack pods within a service or relative to pods in other services
- Requires substantial processing which can slow down scheduling in large clusters significantly
 - Not recommended in clusters larger than several hundred nodes
- For `requiredDuringSchedulingIgnoredDuringExecution` pod affinity and anti-affinity, **`topologyKey` must not be empty**
- For `preferredDuringSchedulingIgnoredDuringExecution` pod anti-affinity, an empty `topologyKey` is interpreted as “all topologies” (hostname, zone, and region)

```
...
affinity:
  podAntiAffinity:
    ## Pods will not be scheduled on the same node as
    ##   another pod (by hostname)
    ##
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values:
          - etcd-app
        topologyKey: kubernetes.io/hostname
    ## Pods will not be scheduled in the same AZ as
    ##   another pod unless all AZs already have pods
    ##
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 1
      podAffinityTerm:
        labelSelector:
          matchExpressions:
          - key: app
            operator: In
            values:
            - etcd-app
          topologyKey: failure-domain.beta.kubernetes.io/zone
    ...
```

Taints & Tolerations

69

Copyright 2013-2020, RX-M LLC

- A “Taint” is a condition assigned to a node
- The `kubectl` command can update taints on one or more nodes
 - `$ kubectl taint nodes kube-node nodetype=master:NoSchedule`
 - This places a taint on kube-node
 - Taints have an arbitrary **key** (“nodetype” in this example)
 - Taints have an *optional* arbitrary **value** (“master” in this example)
 - Taints also have a defined **effect** (“NoSchedule” in this example)
- Taint effects:
 - **NoSchedule**: pods can not be scheduled to the node unless they can tolerate the taint (pods already on the node are unaffected)
 - **PreferNoSchedule**: the scheduler “tries” not to run pods on the node unless they can tolerate the taint
 - **NoExecute**: like NoSchedule but also evicts pods not tolerating the taint even if they are already scheduled and running on the node
- Pods can specify taints that they can ignore (tolerate)
 - **If a pod does not explicitly tolerate a taint it will be affected by the taint’s effect**
 - In the example above, a pod can not be scheduled on kube-node unless it tolerates the: “nodetype=master:NoSchedule” taint
 - *Nothing prevents a pod from tolerating a taint!*

```
apiVersion: v1
kind: Pod
metadata:
  name: toleration-pod
  labels:
    heritage: static
spec:
  tolerations:
    - effect: NoSchedule
      key: disk-type
      operator: Equal
      value: local
  containers:
    - name: container
      image: appimage:2.3
```

Pod Phases

70

Copyright 2013-2020, RX-M LLC

- Pod Phase are **macro states** in the lifecycle of a Kubernetes pod
 - Pending** - Pod is accepted by the system but ≥ 1 of the containers has not been created (Includes time before scheduling and time spent downloading images)
 - Running** - The pod has been bound to a node, and all of the containers have been created, at least one container is still running, or is in the process of starting or restarting
 - Succeeded** - All containers in the pod have terminated in success, and will not be restarted
 - Failed** - All containers in the pod have terminated, ≥ 1 container has terminated in failure (non-zero exit status or terminated by the system)
 - Unknown** - For some reason the state of the pod could not be obtained, typically due to an error in communicating with the host of the pod
- Pod Conditions:
 - PodScheduled** - the Pod has been scheduled to a node
 - Ready** - able to serve requests, should be added to service mesh
 - Initialized** - all init containers have started successfully
 - Unschedulable** - scheduler cannot schedule the Pod right now (e.g. due to lack of resources)
 - ContainersReady** - all containers in the Pod are ready
- Container States
 - Waiting** - default state of container
 - Running** - executing without issues
 - Terminated** - container completed its execution and has stopped running
- Exit Statuses
 - Completed**
 - The pod has run to completion; there's nothing to keep it running
 - CrashLoopBackOff**
 - One of the containers in the pod has exited unexpectedly, likely with a non-zero error code even after restarting due to restart policy

Status	Meaning
Init:x/y	The Pod has x Init Containers, and y have completed so far
Init:Error	An Init Container has failed to execute
Init:CrashLoopBackOff	An Init Container has failed repeatedly
Pending	The Pod has not yet begun executing Init Containers
PodInitializing or Running	The Pod has already finished executing Init Containers

Summary

- Pods are the unit of application deployment in Kubernetes
 - Pods are atomic and scheduled together on a single node as a unit
 - A Pod may contain one or more user defined containers
 - Pods share network, UTS, and IPC namespaces
- YAML files are used to specify resources and the metadata associated with them
- A Pod has an infrastructure container (Pause) used as a place holder for the Pods namespaces and CGroups
- nodeName, nodeSelector, (anti-)affinities, and taints/tolerations can be used to influence Pod scheduling

Lab 3

- Pods and Config Files

Lab 3 Documentation References

- Kubectl reference
 - <https://kubernetes.io/docs/reference/kubectl/overview/>
- Kubectl Usage Conventions
 - <https://kubernetes.io/docs/reference/kubectl/conventions/>
- Pod Overview & Template Examples
 - <https://kubernetes.io/docs/concepts/workloads/pods/pod-overview/>

Day 2

- 4. Controllers
- 5. Services & KubeProxy
- 6. Managing State

4: Controllers

Objectives

- Understand labels and selectors and how they are used to identify resources in a Kubernetes cluster
- Explore the types, roles, and workings of Controllers
 - Deployments, Replica Sets, Replication Controllers, Daemon Sets, Jobs, CronJobs, Stateful Sets, and Autoscalers
 - Understand the differences between RSs & RCs
- Understand how to perform application rollouts with Controllers
- Explore additional spec config options
 - Health and readiness
 - Pre/post actions

Controller Types

77

Copyright 2013-2020, RX-M LLC

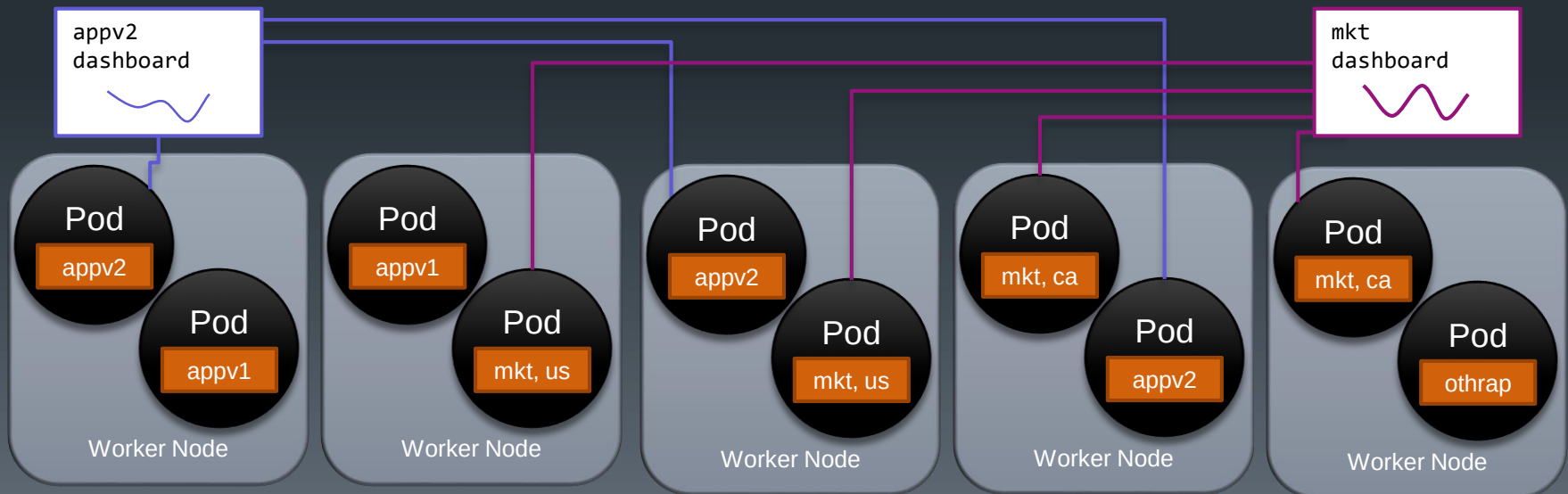
- While pods can be created directly, in a cluster they are almost always created through a Controller
- **Deployments** – for pods that back long running services with support for upgrades and scaling
 - Deployment object describes a desired state
 - Deployment controller changes the actual state to the desired state at a controlled rate
- **Replica Set (RS)** – the next-generation Replication Controller
 - Deployments recommended instead of directly using ReplicaSets, unless you require custom update orchestration or don't require updates
 - You may never need to manipulate ReplicaSet objects
 - Only difference between a RS and a RC right now is the selector support
- **Replication Controller (RC)** [deprecated] – ensures that a specified number of Pod replicas are running at any one time
- **Daemon Set** – ensures that all (or some) Nodes run a copy of a Pod
- **Job** – for one shot pods used to complete a start to finish task
- **CronJob** – manages time-based Jobs
- **StatefulSets** – for pods with an identity tied to state storage in volumes or other backends
 - **PetSets** – renamed StatefulSet starting in Kubernetes 1.5
- **HorizontalPodAutoscaler** – automatically scales the number of pods in a RC, Deployment or RS based on observed metrics



Controllers use Labels & Selectors to manage Pods

Labels

- In Kubernetes sets of pods are managed using two concepts:
 - labels
 - Controllers
- Every pod in Kubernetes has sets of key/value pairs associated with it called labels
- You can select a set or sets of pods by constructing a query based on labels
- Kubernetes has no opinion on the "correct" way to organize pods
 - It is up to you to organize your pods in a way that makes sense to you
 - You can organize by application tiers, geographic location, development environment etc.
 - Labels are non-hierarchical, you can organize your pods in multiple ways simultaneously



Selectors

79

Copyright 2013-2020, RX-M LLC

- There are several options you can use to select on labels:
 - `=` or `==`
 - You can use either style to select keys with values equal to the string on the right
 - `name = apache`
 - `!=`
 - Select keys with values that do not equal the string on the right
 - `environment != test`
 - `in`
 - Select resources whose labels have keys with values in this set
 - `tier in (web,app)`
 - `notin`
 - Select resources whose labels have keys with values not in this set
 - `tier not in (lb,app)`
 - `<Key name>`
 - Use a key name only to select resources whose labels contain this key
 - `tier`
- Multiple selectors can be separated by commas and are logically “AND”ed
- Many kubectl subcommands offer the `-l` switch to specify label selectors (get, delete, etc.)

```
$ kubectl describe deployment logger
```

```
...
```

```
Labels:      deployment=demo
            app=logger
```

```
...
```

```
$ kubectl get pod -l "deployment"
```

NAME	READY	STATUS	RESTARTS	AGE
logger-84ocd	1/1	Running	0	4m
logger-tc068	1/1	Running	0	4m
logger-zgtj9	1/1	Running	0	4m

```
$ kubectl get pod -l "deployment=demo"
```

NAME	READY	STATUS	RESTARTS	AGE
logger-84ocd	1/1	Running	0	4m
logger-tc068	1/1	Running	0	4m
logger-zgtj9	1/1	Running	0	4m

```
$ kubectl get pod -l "deployment=prod"
```

```
No resources found.
```

```
$ kubectl get pod -l "deployment!=prod"
```

NAME	READY	STATUS	RESTARTS	AGE
logger-84ocd	1/1	Running	0	5m
logger-tc068	1/1	Running	0	5m
logger-zgtj9	1/1	Running	0	5m

```
$ kubectl get pod -l "deployment in (prod,demo)"
```

NAME	READY	STATUS	RESTARTS	AGE
logger-84ocd	1/1	Running	0	4m
logger-tc068	1/1	Running	0	4m
logger-zgtj9	1/1	Running	0	4m

```
$ kubectl get pod -l "deployment in (prod,demo), app=logger"
```

NAME	READY	STATUS	RESTARTS	AGE
logger-84ocd	1/1	Running	0	9m
logger-tc068	1/1	Running	0	9m
logger-zgtj9	1/1	Running	0	9m

Deployment Config

80

Copyright 2013-2020, RX-M LLC

- A Deployment controller provides declarative updates for Pods and ReplicaSets
- You can define Deployments to create new ReplicaSets, or to remove existing Deployments and adopt all their resources with new Deployments
- Keys related to the Deployment kind
 - **replicas** - defines the desired number of pods
 - **selector** - specifies a label selector for the Pods targeted by this deployment
 - Tells the Replica Set which pods to watch
 - Assigning selectors that cause more than one RS to attempt to control a Pod is a **user error**
 - K8s tries to avoid oscillation but can not always
 - **strategy**
 - **type**: Recreate or RollingUpdate (default)
 - **maxSurge**: max number of pods that can be scheduled above the desired number of pods
 - **maxUnavailable**: max number of pods that can be unavailable during the update
 - **minReadySeconds** - optional field that specifies the minimum number of seconds for which a newly created Pod should be ready without any of its containers crashing, for it to be considered available
 - **revisionHistoryLimit** - optional field that specifies the number of old ReplicaSets to retain to allow rollback
 - **template** - defines a template to launch a new pod
 - Contains the same elements defined for pod configs
- Selector values need to match the labels values specified in the Pod template
 - In API version apps/v1, a Deployment's label selector is **immutable** after it gets created

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: k8sapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: k8sapp
      deployment: development
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: k8sapp
        deployment: development
    spec:
      containers:
        - name: k8sapp-container
          image: k8sapp:v2
          ports:
            - containerPort: 80
```

```
$ kubectl apply -f mydep.yaml
deployment.apps/myapp created
```

```
$ kubectl get deploy,rs,po
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/myapp	3/3	3	76s	

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/myapp-1072152842	3	3	76s	

NAME	READY	STATUS	RESTARTS	AGE
pod/myapp-1072152842-fn5cg	1/1	Running	0	72s
pod/myapp-1072152842-k4pwl	1/1	Running	0	72s
pod/myapp-1072152842-v5wm2	1/1	Running	0	72s

```
$ kubectl describe deployment myapp
```

```
Name: myapp
Namespace: default
CreationTimestamp: Wed, 01 Aug 2017 12:48:13 -0800
Labels: org=ops
Annotations: deployment.kubernetes.io/revision=1
Selector: app=myapp,deployment=dev
Replicas: 3 updated | 3 total | 3 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=myapp
          deployment=dev
  Containers:
    myapp-container :
      Image: logagg
      Port: 80/TCP
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
Conditions:
  Type           Status      Reason
  ----           -
  Available      True        MinimumReplicasAvailable
  Progressing    True        NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet: myapp-1072152842 (3/3 replicas created)
Events:
  FirstSeen    LastSeen    Count    From              SubObjectPath    Type    Reason              Message
  -----
  3m           3m          1        {deployment-controller}              Normal    ScalingReplicaSet    Scaled up replica set myapp-1072152842 to 3
```

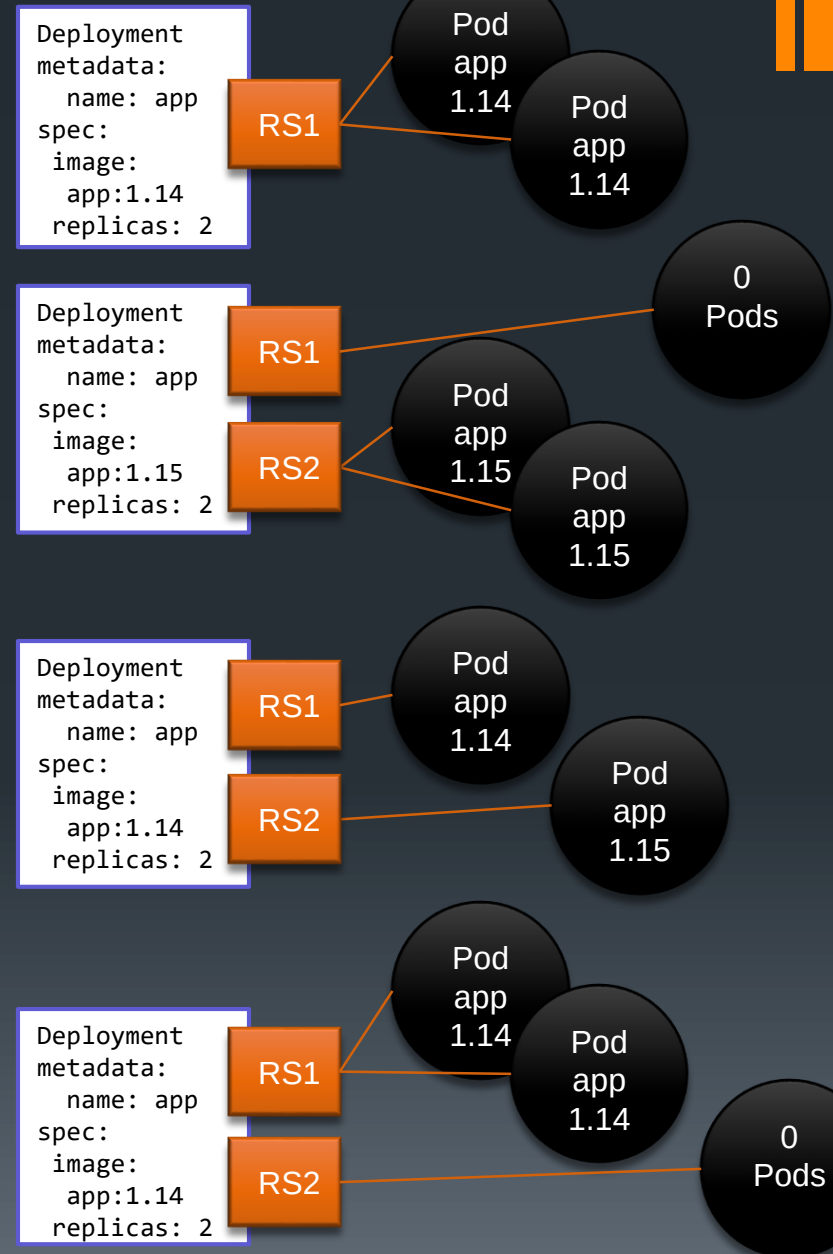
Simple Deployment / RS Walk Through

```
$ cat mydep.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  labels:
    org: ops
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
      deployment: dev
  template:
    metadata:
      labels:
        app: myapp
        deployment: dev
    spec:
      containers:
        - name: myapp-container
          image: myapp:v2.2
          ports:
            - containerPort: 80
```

Rollouts

- `kubectl apply -f appdeploy.yaml`
- `kubectl set image deploy/app \`
`mypod=app:1.15 --record`
(can also use `edit`, `patch`, or `apply`)
- `kubectl rollout undo deploy/app`
- `kubectl rollout pause deploy/app`
- `kubectl rollout resume deploy/app`
- `kubectl rollout history deploy/app`
- `kubectl rollout status deploy/app`



Daemon Sets

- Ensures that **all nodes run a copy of a pod**
 - As nodes are added to the cluster, pods are added to them
- Good for running:
 - Storage daemons (e.g. glusterd, ceph, ...)
 - Log collectors (e.g. fluentd, logstash, ...)
 - Node monitoring (Prometheus Node Exporter, cAdvisor, collectd, New Relic agent, Ganglia gmond, ...)
- Marking a node unschedulable **does not affect DaemonSet** controller
- Can make pods even when the scheduler has not been started
 - Also great for cluster bootstrap
- You can perform a rolling update on a DaemonSet

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: volume-provisioner
  labels:
    app: volume-provisioner
spec:
  selector:
    matchLabels:
      app: volume-provisioner
  template:
    metadata:
      labels:
        app: volume-provisioner
    spec:
      nodeSelector:
        storage: local
      tolerations:
        - effect: NoSchedule
          key: storage
          operator: Equal
          value: local
      containers:
        - image: local-volume-provisioner:v2.2.0
          name: provisioner
          volumeMounts:
            - mountPath: /mnt/
              name: local-storage
      volumes:
        - name: local-storage
          hostPath:
            path: /mnt/

```


■ Job

- Creates one or more pods and ensures that a specified number of them successfully terminate
 - As pods successfully complete, the job tracks the successful completions; when a specified number of successful completions is reached, the job itself is complete
 - Will start a new Pod if the first pod fails or is deleted
- 3 main types:
 - **Non-parallel** – one pod is started, job is complete as soon as Pod terminates successfully
 - **Parallel with fixed completion count**
 - Specify a non-zero positive value for `.spec.completions`
 - Job is complete when there is one successful pod for each value in the range: 1 to `.spec.completions`
 - **Parallel Jobs with a work queue** – support parallel processing of a set of independent but related work items
 - Examples: emails to be sent, frames to be rendered, files to be transcoded, ranges of keys in a NoSQL database to scan
 - Not designed to support closely-communicating parallel processes, as commonly found in scientific computing

■ Cron Job

- Time-based Job resource: it **runs a job periodically on a given schedule**, written in Cron format
 - Point in time
 - Repeatedly at point in time
- Required **schedule** key - takes a Cron format string

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox
              args:
                - /bin/sh
                - -c
                - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

Division of Labor in a Cloud Native World

85

Copyright 2013-2020, RX-M LLC

Applications

Operators: DevOps
fn/Containers/Configs
>> Deployments /
Services / Jobs >>

Applications

>> StatefulSets >>

Messaging

KV/Column/Doc/

Application Platform

Operators: SREs

Kubernetes

>> DaemonSets >>

SDN

CR

SDS

Cloud Platform

Operators: CREs

IaaS

Horizontal Pod Autoscaler

86

Copyright 2013-2020, RX-M LLC

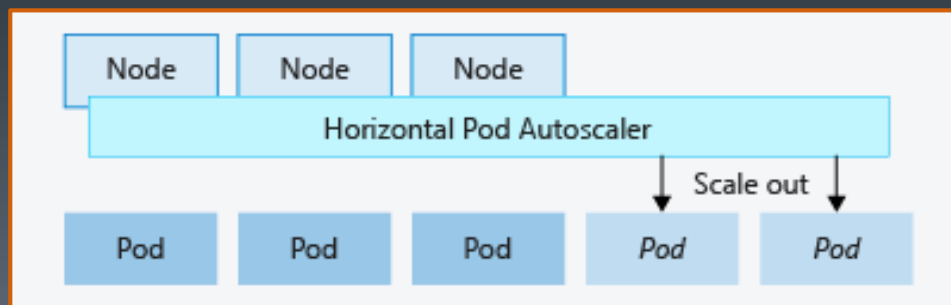
- Allows the number of pods in a replication controller or deployment to **scale automatically based on metrics**
- Metrics include:
 - **Observed CPU utilization** – scales the number of pods in a Deployment, Replica Set or Replication Controller based on observed CPU utilization
 - Autoscaler periodically queries CPU utilization for the pods it targets
 - Threshold expressed as: `targetCPUUtilizationPercentage`
 - Requires metrics aggregation in your cluster for autoscaling to work
 - **Custom metrics** (beta)
 - You can add custom metrics for the Horizontal Pod Autoscaler to use in the `autoscaling/v2beta1` API
 - `targetCPUUtilizationPercentage` field replaced with an array called `metrics`
 - For example, queries per second or average request latency

```
apiVersion: autoscaling/v2beta1
kind: HorizontalPodAutoscaler
metadata:
  name: example-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: mydep
  maxReplicas: 6
  minReplicas: 3
  metrics:
  - type: Resource
    resource:
      name: cpu
      targetAverageUtilization: 80
  - type: Pods
    pods:
      metricName: total_disk_usage
      targetAverageValue: 11520
```

Imperative command equivalent example:
`kubectl autoscale deploy mydep \`
`--min=3 --max=6 --cpu-percent=80`

HPA Algorithm

- HPA operates on the ratio between desired metric value and current metric value
 - $\text{desiredReplicas} = \text{ceil}[\text{currentReplicas} * (\text{currentMetricValue} / \text{desiredMetricValue})]$
 - Examples:
 - If current metric value is 200m and desired value is 100m, number of replicas will be doubled, since $200.0 / 100.0 == 2.0$
 - If the current value is 50m, will halve the number of replicas, since $50.0 / 100.0 == 0.5$
- $\text{currentMetricValue}$ is computed by taking the **average of the given metric across all Pods in the HPA's target**
- In the case of missing metrics for pod(s):
 - HPA assumes missing pods were consuming 0% in case of a scale up and 100% of the desired value in case of a scale down
 - Dampens the magnitude of a potential scale



HPA Algorithm Tuning

- Global flags for tuning the HPA algorithm passed to the kube-controller-manager component:
 - `--horizontal-pod-autoscaler-sync-period` – period for syncing the number of pods (15 seconds)
 - `--horizontal-pod-autoscaler-downscale-stabilization` – period for which autoscaler will look backwards and not scale down below any recommendation it made during that period (5 minutes)
 - `--horizontal-pod-autoscaler-tolerance` – minimum change (from 1.0) in the desired-to-actual metrics ratio for the horizontal pod autoscaler to consider scaling (0.1)
 - `--horizontal-pod-autoscaler-initial-readiness-delay` – period after pod start during which readiness changes will be treated as initial readiness (30 seconds)
 - `--horizontal-pod-autoscaler-cpu-initialization-period` – period after pod start when CPU samples might be skipped (5 minutes)

Health Checks

89

Copyright 2013-2020, RX-M LLC

- Kubernetes provides two layers of health checking
 - HTTP or TCP checks
 - K8s can attempt to connect to a particular endpoint and give a status of healthy on a successful connection
 - Application-specific health checks
 - K8s can be configured to use a command line scripts
- An RS/RC will restart a container failing a health check
- **livenessProbe**
 - Specifies a health check
 - Available options (typically only one of):
 - httpGet
 - Takes a path and port
 - Status codes between 200 and 399 are all considered healthy by the probe
 - tcpSocket
 - Takes a port
 - Any successful connect passes
 - exec
 - Takes a command
 - Exit code of 0 is success
- **readinessProbe**
 - Uses the same settings but uses test to assess container readiness

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
          livenessProbe: # don't use all at once!
            httpGet:
              path: /status/
              port: 80
            tcpSocket:
              port: 80
            exec:
              command: /usr/bin/health/ck.sh
          initialDelaySeconds: 30
          timeoutSeconds: 1
          periodSeconds: 10
```

Pre/Post Actions

90

Copyright 2013-2020, RX-M LLC

- Containers can be assigned lifecycle actions to run **after the container starts** and **before the container stops**
 - Convenient for initializing the started container or cleaning up before the container shutdown
- Management of the container blocks until lifecycle actions are complete
 - Unless the container process fails, in which case the action is aborted
- These hooks are called at least once (be prepared for more than one call!)
 - It is up to the hook implementation to handle this correctly
- postStart** – called immediately after a container is created
 - If the handler fails, the container is terminated and restarted according to its restart policy
 - Other management of the container blocks until the action completes
- preStop** – called immediately before a container is terminated
 - Blocking, so it must complete before the call to delete the container can be sent
 - If the hook hangs during execution, the Pod phase stays in a Terminating state and is killed after `terminationGracePeriodSeconds` of pod ends
 - Otherwise, the container is terminated after the handler completes
 - The reason for termination is passed to the handler
 - “reason” values:
 - Delete - Delete command was issued via kubectl or the API
 - Health - Health check fails
 - Dependency - Dependency failure (e.g. disk mount failure)
- Two types of hook handlers:
 - Exec** - Executes a command inside the cgroups and namespaces of the container
 - HTTP** - request against a specific endpoint on the container

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-hook
spec:
  replicas: 3
  selector:
    matchLabels:
      app: apache-hook
  template:
    metadata:
      labels:
        app: apache-hook
    spec:
      containers:
        - name: apache-hook
          image: bitnami/apache:latest
          ports:
            - containerPort: 80
          lifecycle:
            postStart:
              httpGet:
                path: http://my.regsvr.com/register/
                port: 80
            preStop:
              exec:
                command: ["/usr/bin/apachectl", "-k", "graceful- stop"]
```

Summary

- Controllers are designed to:
 - Run with a replication factor: Deployments & RSs or RCs
 - Run everywhere: DaemonSets
 - Run once: Jobs
 - Run on a schedule: CronJobs
 - Run stateful apps: StatefulSets (covered later)
- Deployments/ReplicaSets are key components in the orchestration of horizontally-scaled microservices
- RSs/RCs use kubelets to perform on-node monitoring and restart tasks
- Pods can be monitored for health and readiness
- Pods can be configured to perform post-start and pre-shutdown tasks

Lab 4

- Deployments and ReplicaSets

Lab 4 Documentation

References

93

Copyright 2013-2020, RX-M LLC

- Deployment Spec & Examples
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Rollouts & Controller Updates
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#updating-a-deployment>
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#scaling-a-deployment>
- Kubectl rollout
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-back-a-deployment>
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#checking-rollout-history-of-a-deployment>
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#pausing-and-resuming-a-deployment>
- Health Checks
 - <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes>
 - <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/#define-a-tcp-liveness-probe>
- Job Spec & Examples
 - <https://kubernetes.io/docs/concepts/workloads/controllers/jobs-run-to-completion/>
 - (Covers Cron Jobs) <https://kubernetes.io/docs/tasks/job/>

5: Services & Kube-Proxy

Objectives

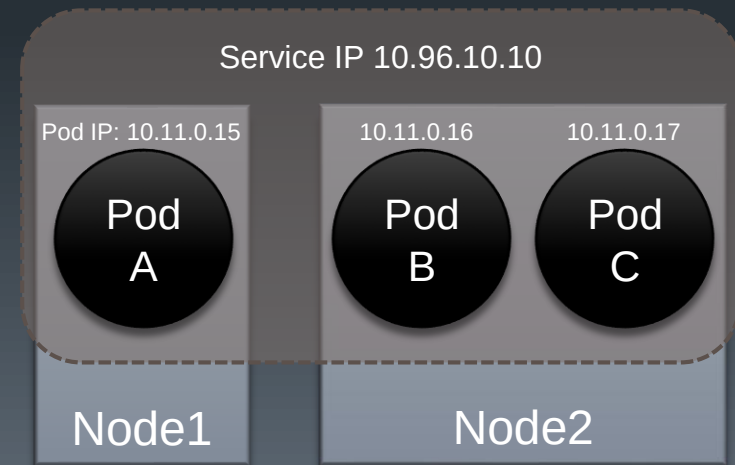
- Explore the nature of Kubernetes services
- Understand the roll of kube-proxy
- Explain the use of external load balancers with K8s
- Discuss manual networking and SDN networking options
- Review the IPTables manipulations made by the proxy

Services

96

Copyright 2013-2020, RX-M LLC

- A Kubernetes Service is an **abstraction** which defines a logical set of Pods and a policy by which to access them
 - The set of **Pods targeted are identified by a Selector**
 - Although each Pod gets its own IP address, Pod IP addresses cannot be relied upon to be stable over time
 - Controllers create and destroy Pods dynamically
 - e.g. when scaling up or down or when doing rolling updates
- Services can be exposed in different ways by specifying a type in the ServiceSpec:
 - **ClusterIP** – [Default] exposes the Service on an internal IP in the cluster
 - **NodePort** – exposes the Service on the same port of each selected Node in the cluster using NAT
 - **LoadBalancer** – Creates an external load balancer with a cloud provider and assigns an external IP to the Service
 - **ExternalName** – exposes the Service using an arbitrary name



Service Config spec

- **type** - type of exposed service
 - **ClusterIP** – [Default] uses kube-proxy and cluster based virtual IP (available only inside the cluster)
 - **NodePort** – exposes the service on the same port on each node of the cluster (for external access)
 - **LoadBalancer** – uses an external load balancer, typically with a public IP and port (AWS ELB, etc.)
 - **ExternalName** – forwards traffic to the specified external DNS name
- **ports** - The list of ports that are exposed by this service
 - Need not be the same as the port of the actual service
- **targetPort** – The port exposed by the backend Pods
 - By default the targetPort will be set to the same value as the port field
 - Can be a string, referring to the name of a port in the backend Pods
- **selector** - This service will route traffic to pods having labels matching this selector
 - If empty, all pods are selected (!)
- **clusterIP** - IP is usually assigned by the master and is the IP address of the service
 - If specified, it will be allocated to the service if it is unused or else creation of the service will fail
 - Valid values are None (headless service), empty string (""), or a valid IP address
 - Cannot be updated
- **externalIPs** - externalIPs is a list of IP addresses for which nodes in the cluster will also accept traffic for this service
 - These IPs are not managed by Kubernetes
 - The user is responsible for ensuring that traffic arrives at a node with this IP
 - A common example is external load-balancers that are not part of the Kubernetes system
- **sessionAffinity** - Supports "ClientIP" and "None"
 - Used to maintain session affinity
 - Defaults to None
- **loadBalancerIP** - Only applies to Service type: LoadBalancer
 - LoadBalancer will get created with the IP specified in this field
 - This feature depends on whether the underlying cloud-provider supports specifying the loadBalancerIP when a load balancer is created
 - This field will be ignored if the cloud-provider does not support the feature

```

apiVersion: v1
kind: Service
metadata:
  name: testweb
  labels:
    name: testweb
spec:
  type: ClusterIP
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 80
  selector:
    name: testweb

```

```

$ kubectl get endpoints
NAME           ENDPOINTS                     AGE
kubernetes     192.168.131.134:6443         5d
testweb        172.17.0.9:80                1h

```

Multiport Services

- Multiport services can be useful when working with legacy systems or applications running on another cluster
 - Begin to manage the service endpoints in Kubernetes while stitching the stack together using the K8s orchestration concepts
 - We can even start bringing over pieces of the stack, as the frontend, one at a time as the organization refactors applications for microservices and/or containerization

```
apiVersion: v1
kind: Service
metadata:
  labels:
    svc: prometheus
  name: prometheus-svc
spec:
  type: LoadBalancer
  selector:
    app: prometheus
  ports:
    - name: grafana-port
      protocol: TCP
      port: 80
      targetPort: 3000
    - name: prometheus-port
      protocol: TCP
      port: 81
      targetPort: 9090
```

Node Port Details

99

Copyright 2013-2020, RX-M LLC

- The Kubernetes master allocates a port from a flag-configured range
 - Default: 30000-32767
- Each Node will proxy that port (the same port number on every Node) into your Service
 - That port will be reported in your Service's spec.ports[*].nodePort field
- If you want a specific port number, you can specify a value in the nodePort field
 - The system will allocate you that port or else the API transaction will fail
 - The value you specify must be in the configured range for node ports
 - The port may not conflict with existing ports
- This gives developers the freedom to set up their own load balancers, to configure cloud environments that are not fully supported by Kubernetes
 - Or even to just expose one or more nodes' IPs directly
- Node Port Services also have all of the features of Cluster IP
 - Visible as both:
 - NodeIP: spec.ports[*].nodePort
 - ClusterIP: spec.clusterIp:spec.ports[*].port

```
user@ubuntu:~/svc$ cat nodeport.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: testweb-np
spec:
  type: NodePort
  ports:
    - port: 80
      protocol: TCP
  selector:
    run: testweb
```

```
user@ubuntu:~/svc$ kubectl create -f nodeport.yaml
```

```
user@ubuntu:~/svc$ kubectl get service
```

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	10.96.0.1	<none>	443/TCP	1d
testweb-np	10.111.101.166	<nodes>	80:30143/TCP	32s

```
user@ubuntu:~/svc$ kubectl describe service testweb-np
```

```
Name: testweb-np
Namespace: default
Labels: <none>
Annotations: <none>
Selector: run=testweb
Type: NodePort
IP: 10.111.101.166
Port: <unset> 80/TCP
NodePort: <unset> 30143/TCP
Endpoints: 10.32.0.4:80,10.32.0.5:80,10.32.0.6:80
Session Affinity: None
Events: <none>
```

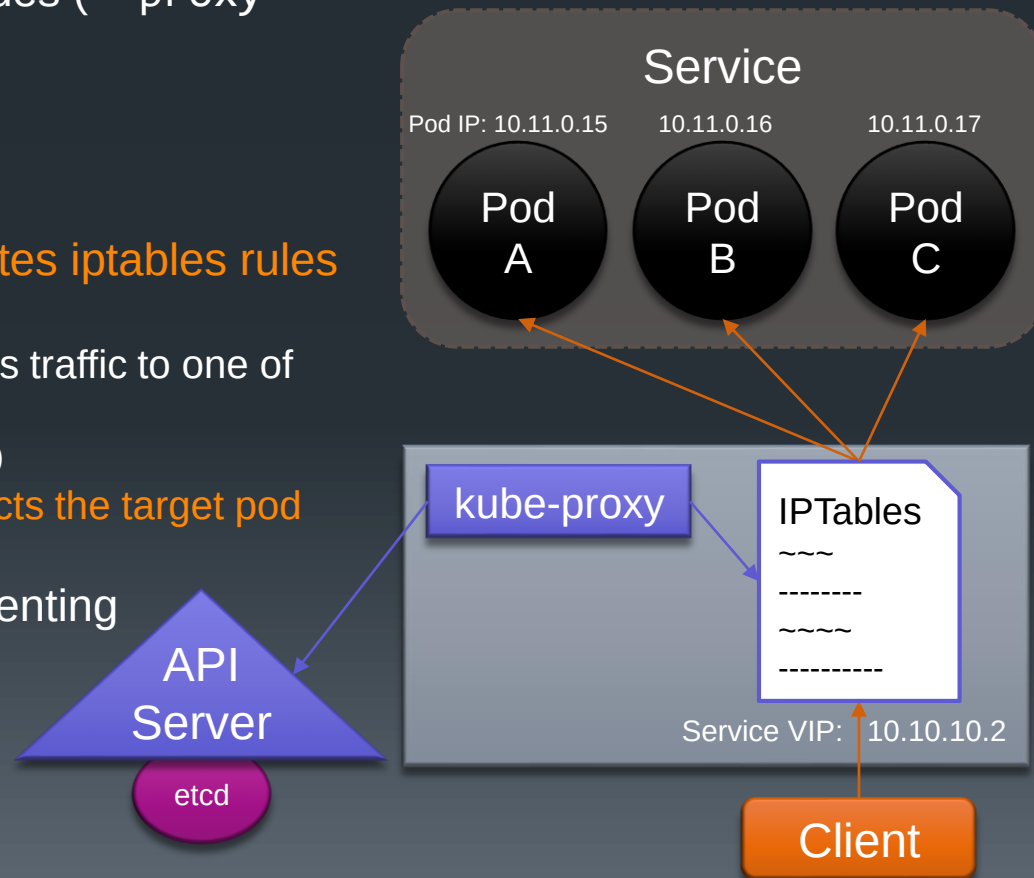

Kube-Proxy

100

Copyright 2013-2020, RX-M LLC

■ Routing Mesh

- Pods frequently need to communicate with other services
- The Kube-Proxy runs on each node in the cluster providing a routing mesh to map services to implementation pods
- The proxy operates in one of 3 modes (--proxy-mode):
 - User Mode Proxy [old]
 - IPTables [default]
 - IPVS [new, eventual default]
- In IPTables mode Kube-Proxy **creates iptables rules for every service**
 - Produces a **virtual endpoint** that routes traffic to one of the service's target pods
 - Typically a ClusterIP, aka Virtual IP (VIP)
 - IPtables loadbalancing **randomly selects the target pod** to forward to
- Kube-Proxy identifies pods implementing the service via a selector
- All **Kube-Proxies watch the API Server** for service changes, updating their iptables as needed



iptables

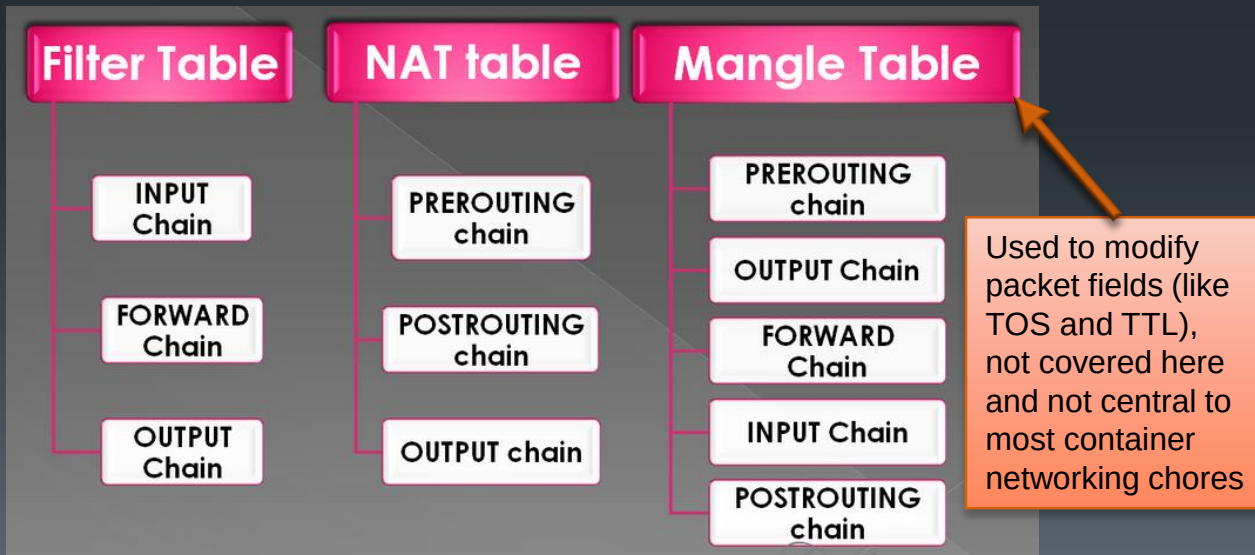
<http://www.iptables.info/en/>

101

- iptables is a program that **configures rules housed in tables** used by the Linux kernel when processing network packets
- The term iptables is also commonly used to refer to the kernel packet processing implementation, Netfilter
 - Different Netfilter kernel modules are currently used for different protocols
 - iptables applies to IPv4
 - ip6tables to IPv6
 - arptables to ARP
 - ebtables to Ethernet frames
 - x_tables is the kernel module providing the shared code used by all four modules
 - The successor of iptables is nftables
 - Backward compatible with iptables
 - Merged into Linux kernel version 3.13 (19 January 2014)
- Running the iptables command requires elevated privileges (root)

Definitions

- **Tables** house collections of related packet processing rules
 - **Filter** table
 - **Nat** table
 - **Mangle** table
- **Rules** have a matching part and an action part (target)
 - Rules are organized into chains
- **Targets** describe what to do with a packet when the rule matches
- **Chains** are sequences of rules applied at different phases of packet processing
 - **Prerouting**
 - **Input**
 - **Forward**
 - **Output**
 - **Postrouting**

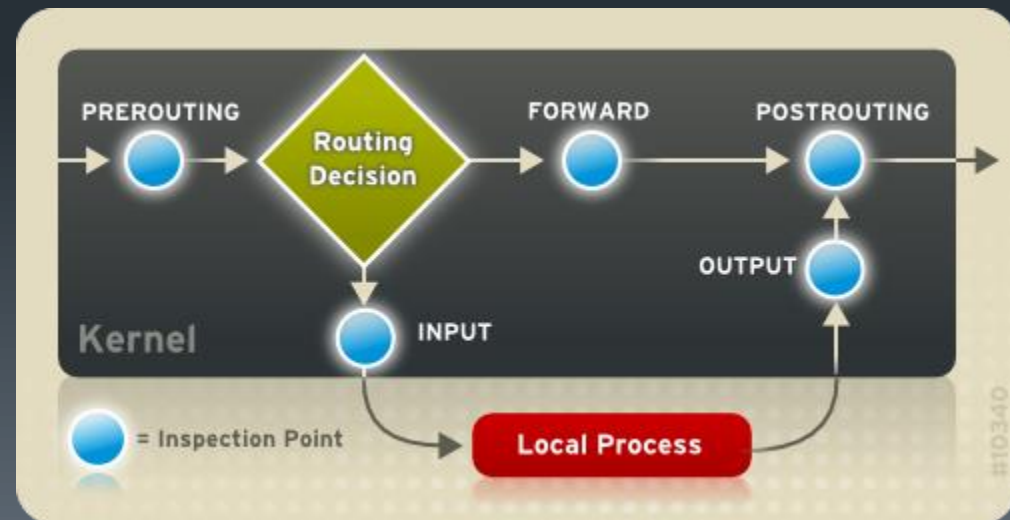


Chains

102

Copyright 2013-2020, RX-M LLC

- *xtables* allows the definition of tables containing chains of rules for the treatment of packets
 - Each table is associated with a different kind of packet processing
 - Packets are **processed by sequentially** traversing the rules in chains
 - A rule in a chain can cause a **goto or jump to another chain**, and this **can be repeated** to whatever level of nesting is desired
 - **-j** Jump – like a “call”, i.e. the point that was jumped from is remembered and returned to
 - **-g** Goto – like, well a “goto”, i.e. the point of departure is forgotten, returns will resume at the last jump point
 - Every network packet arriving at or leaving from the computer traverses at least one chain
- Packet flow paths
 - Packets start at a given box and will flow along a certain path, depending on the circumstances
 - The origin of the packet determines which chain it traverses initially
 - There are **five predefined chains** (mapping to the five available Netfilter hooks), though a table need not have all chains
 - Predefined **chains have a policy**, for example DROP, which is applied to the packet if it reaches the end of the chain
 - The system administrator can create **as many chains as desired**
 - These chains have no policy; if a packet reaches the end of a user defined chain it is returned to the prior table
 - A chain may be empty
- Chains
 - **PREROUTING**: Packets enter this chain before a routing decision is made
 - **INPUT**: Packet is going to be locally delivered
 - **FORWARD**: All packets that have been routed and were not for local delivery will traverse this chain
 - Inbound traffic bound for containers takes this path
 - **OUTPUT**: Packets sent from the machine itself visit this chain
 - **POSTROUTING**: Routing decision has been made, packets enter this chain just before handing them off to the hardware

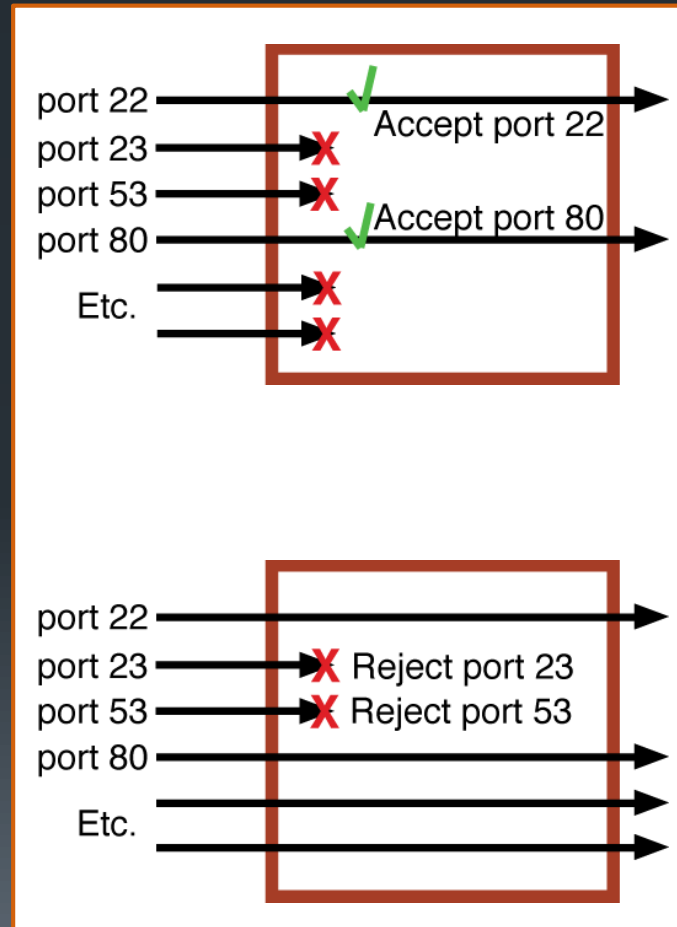


Filter table

103

Copyright 2013-2020, RX-M LLC

- Used to **allow** or **disallow** packet transmission
 - Packets can be matched and filtered in many ways
 - Provides **basic firewalling**
 - The default table
 - No need for the `-t` switch, but if you must: `-t filter`
- Most targets are legal in this table:
 - ACCEPT**
 - `iptables -A INPUT -p tcp --dport 22 -j ACCEPT`
 - Packets matching this rule are transmitted on immediately and will not continue traversing the current chain or any other chains in the same table
 - A packet accepted in one chain might still travel through chains within other tables which may drop it
 - DROP**
 - `iptables -A INPUT -p tcp --dport 22 -j DROP`
 - Change source of outbound traffic to 15.45.23.67
 - Discards packets with no further processing
 - This action may leave sockets pending response on hosts
 - If a packet is DROPEd in a sub chain it will not be processed further in any of the main chains or in any other table
 - The target will not send any kind of information in either direction
 - REJECT**
 - `iptables -A FORWARD -p tcp --dport 22 -j REJECT --reject-with tcp-reset`
 - The same as the DROP target but sends back an error message to the source host
 - Only valid in the INPUT, FORWARD and OUTPUT chains or their sub chains
 - All chains that use the REJECT target may only be called by the INPUT, FORWARD, and OUTPUT chains
 - Contrary to popular belief, DROP does not give better security than REJECT, rather it inconveniences legitimate users without additional protection from malicious ones
 - <http://www.chiark.greenend.org.uk/~peterb/network/drop-vs-reject>

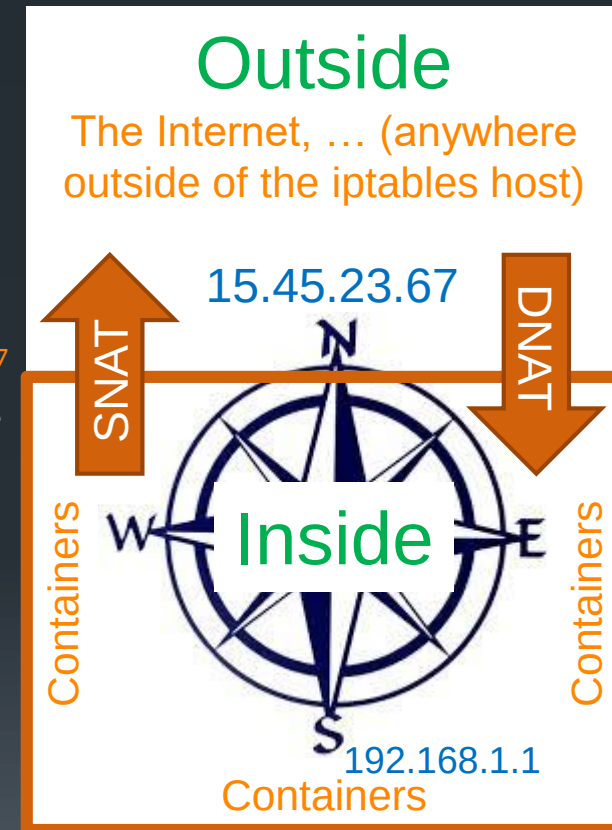


NAT Table

104

Copyright 2013-2020, RX-M LLC

- Used to translate a packet's source IP field or destination IP field
- Compass Model
 - Traffic from inside to outside: S -> N SNAT/MASQ
 - Traffic from outside to inside: N -> S DNAT
 - Traffic between containers: E -> W (or if you prefer W -> E)
- There are 4 NAT Targets:
 - DNAT** – Destination NAT : North to South
 - `iptables -t nat -A PREROUTING -p tcp -d 15.45.23.67 --dport 80 -j DNAT --to-destination 192.168.1.1:8089`
 - Change destination of inbound traffic hitting 15.45.23.67:80 to 192.168.1.1:8089
 - Changes the destination address (and optionally port) of the packet and reroutes it
 - Allows an interface with a public IP to redirect traffic to an inside host
 - To a machine on a physical (e.g. DMZ) network behind the firewall
 - To a container on a virtual (e.g. Docker) network behind the firewall
 - SNAT** – Source NAT : South to North
 - `iptables -t nat -A POSTROUTING -p tcp -o eth0 -j SNAT --to-source 15.45.23.67`
 - Change source of outbound traffic to 15.45.23.67
 - Changes the source address of packets from inside destined for outside using a defined outside IP address
 - Makes it possible to connect to the Internet from inside
 - MASQUERADE** – South to North
 - `iptables -t nat -A POSTROUTING -p tcp -o eth0 -j MASQUERADE`
 - Like SNAT but translates based on an interface not a preconfigured IP
 - The IP of the outside interface is used allowing this to work when the interface receives its address dynamically (DHCP, etc.)
 - REDIRECT**
 - `iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-ports 8080`
 - Used to redirect packets to the machine itself
 - Only valid within the PREROUTING and OUTPUT chains of the nat table or within user-defined chains that are only called from those chains
- Only the first packet in a stream will hit this table
 - After initial inspection all other packets sent over the connection will automatically have the same action taken on them



```
$ kubectl run testweb --image=nginx
deployment.apps/testweb created
```

```
$ kubectl create -f svc.yaml
service/testweb created
```

```
$ kubectl scale deployment --replicas=3 testweb
deployment.apps/testweb scaled
```

```
$ kubectl get deploy,pods
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
deploy/testweb	1	1	1	1	5m

NAME	READY	STATUS	RESTARTS	AGE
po/testweb-3248343053-d74h5	1/1	Running	0	4m
po/testweb-3248343053-ftfz8	1/1	Running	0	4m
po/testweb-3248343053-m8htj	1/1	Running	0	5m

NAME	DESIRED	CURRENT	READY	AGE
rs/testweb-3248343053	1	1	1	5m

```
$ kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.0.0.1	<none>	443/TCP	2d
testweb	NodePort	10.96.0.149	<none>	80:30143/TCP	3m

```
$ kubectl get endpoints testweb
```

NAME	ENDPOINTS	AGE
testweb	10.32.0.4:80,10.32.0.5:80,10.32.0.6:80	3m

```
$ curl -s http://10.96.0.149 | head -4
```

```
...
<title>Welcome to nginx!</title>
```

```
$ curl -s http://10.32.0.4 | head -4
```

```
...
<title>Welcome to nginx!</title>
```

```
$ vim svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: testweb
  labels:
    svc: testweb
spec:
  type: NodePort
  sessionAffinity: ClientIP
  sessionAffinityConfig:
    ClientIP:
      timeoutSeconds: 600
  ports:
    - port: 80
  selector:
    run: testweb
```

NodePort & ClusterIP Service


```
user@ubuntu:~/svc$ sudo iptables -nL -t nat
```

```
Chain PREROUTING (policy ACCEPT)
target     prot opt source                destination
KUBE-SERVICES all  --  0.0.0.0/0             0.0.0.0/0          /* kubernetes service portals */
DOCKER     all  --  0.0.0.0/0             0.0.0.0/0          ADDRTYPE match dst-type LOCAL
```

```
Chain KUBE-NODEPORTS (1 references)
target     prot opt source                destination
KUBE-MARK-MASQ tcp  --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ tcp dpt:30143
KUBE-SVC-EG66NKXDU7X2QZTA tcp --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ tcp dpt:30143
```

```
Chain KUBE-SERVICES (2 references)
target     prot opt source                destination
KUBE-SVC-XGLOHA7QRQ3V22RZ tcp --  0.0.0.0/0             10.106.125.172     /* kube-system/kubernetes-dashboard: cluster IP */ tcp dpt:80
KUBE-SVC-NPX46M4PTMTKR6Y tcp --  0.0.0.0/0             10.96.0.1          /* default/kubernetes:https cluster IP */ tcp dpt:443
KUBE-SVC-TCOU7JCQXEZGVUNU udp --  0.0.0.0/0             10.96.0.10         /* kube-system/kube-dns:dns cluster IP */ udp dpt:53
KUBE-SVC-ERIFXISQEP7F70F4 tcp --  0.0.0.0/0             10.96.0.10         /* kube-system/kube-dns:dns-tcp cluster IP */ tcp dpt:53
KUBE-SVC-EG66NKXDU7X2QZTA tcp --  0.0.0.0/0             10.96.0.149        /* default/testweb: cluster IP */ tcp dpt:80
KUBE-NODEPORTS all  --  0.0.0.0/0             0.0.0.0/0          /* kubernetes service nodeports; NOTE: this must be the last rule in this chain */
ADDRTYPE match dst-type LOCAL
```

```
Chain KUBE-SVC-EG66NKXDU7X2QZTA (1 references)
target     prot opt source                destination
KUBE-SEP-7KX2LIG5LIB5UVVO all --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ recent: CHECK seconds: 600 reap name: KUBE-SEP-7KX2LIG5LIB5UVVO side: source mask: 255.255.255.255
KUBE-SEP-TSJSFJG2GUXROSTI all --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ recent: CHECK seconds: 600 reap name: KUBE-SEP-TSJSFJG2GUXROSTI side: source mask: 255.255.255.255
KUBE-SEP-RY3VYSR3AYX2BB02 all --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ recent: CHECK seconds: 600 reap name: KUBE-SEP-RY3VYSR3AYX2BB02 side: source mask: 255.255.255.255
KUBE-SEP-7KX2LIG5LIB5UVVO all --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ statistic mode random probability 0.33332999982
KUBE-SEP-TSJSFJG2GUXROSTI all --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ statistic mode random probability 0.50000000000
KUBE-SEP-RY3VYSR3AYX2BB02 all --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */
```

Statistic IPTables module:
apply a rule with a probability

```
Chain KUBE-SEP-7KX2LIG5LIB5UVVO (1 references)
target     prot opt source                destination
KUBE-MARK-MASQ all  --  10.32.0.4             0.0.0.0/0          /* default/testweb: */
DNAT       tcp  --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ recent: SET name: KUBE-SEP-7KX2LIG5LIB5UVVO side: source mask: 255.255.255.255 tcp to:10.32.0.4:80
```

IPTables module "recent"
enables session affinity

```
Chain KUBE-SEP-TSJSFJG2GUXROSTI (1 references)
target     prot opt source                destination
KUBE-MARK-MASQ all  --  10.32.0.5             0.0.0.0/0          /* default/testweb: */
DNAT       tcp  --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ recent: SET name: KUBE-SEP-TSJSFJG2GUXROSTI side: source mask: 255.255.255.255 tcp to:10.32.0.5:80
```

Reverse NAT for hairpin traffic

```
Chain KUBE-SEP-RY3VYSR3AYX2BB02 (1 references)
target     prot opt source                destination
KUBE-MARK-MASQ all  --  10.32.0.6             0.0.0.0/0          /* default/testweb: */
DNAT       tcp  --  0.0.0.0/0             0.0.0.0/0          /* default/testweb: */ recent: SET name: KUBE-SEP-RY3VYSR3AYX2BB02 side: source mask: 255.255.255.255 tcp to:10.32.0.6:80
```

NodePort & ClusterIP Service Nat

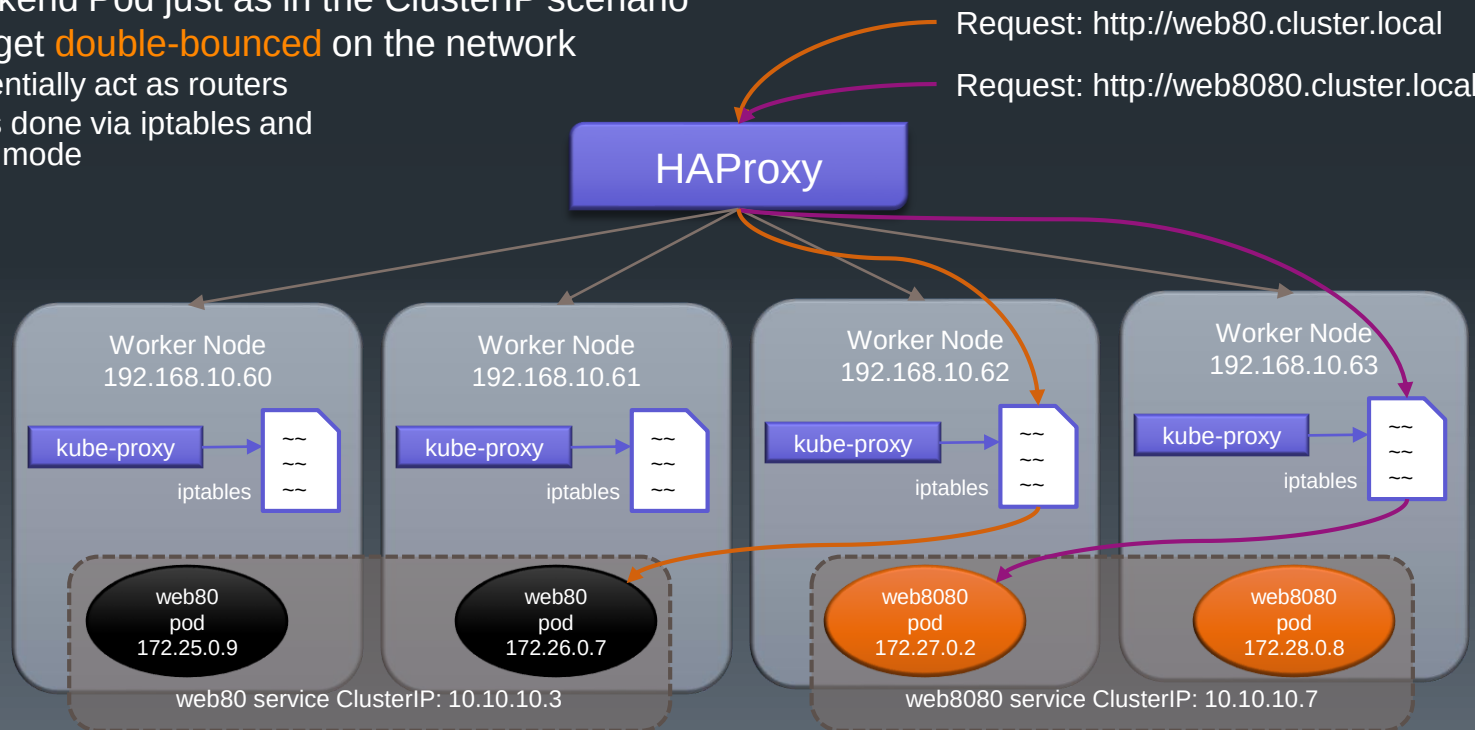
- IPVS (IP Virtual Server)
 - ipvs 1.2.1 is the latest stable version
- Transport-layer load balancing inside the Linux kernel
 - Aka Layer-4 switching
- Function:
 - IPVS running on a host can act as a load balancer at the front of a cluster of real servers, pods or containers
 - Directs requests for TCP/UDP based services to the real servers, and makes services of the real servers to appear as a virtual service on a single IP address
- Added to the official kernel 2.6.10 released on December 24, 2004
 - IPv6 support for IPVS was included in the Linux kernel 2.6.28-rc3 on November 2, 2008
- IPVS offers several connection scheduling algorithms:
 - **RR (Round-Robin)** – assigns connections sequentially
 - **WRR (Weighted Round-Robin)** – servers receive an integer weight which defines how many times they appear in a single RR sequence (A-2,B-3 == BABAB)
 - **LC (Least-Connection)** – directs connections to the server with the least connections
 - **WLC (Weighted Least-Connection)** – servers receive an integer weight, connections go to servers with the lowest ratio of connections/weight
 - **Default** K8s algorithm with weight of 1 per server unless set
 - **LBLC (Locality-Based Least-Connection)** – target IP traffic is sent to the same server (locality) until it has more connections than its weight, after which traffic is sent to the server with least connections (which must be at $\leq 50\%$ of its weight)
 - **LBLCR (LBLC with Replication)** – allows multiple servers to act as the locality target, adding new servers to the target set as the existing servers become over weighted
 - **DH (Destination Hashing)** – destination selected by destination hash key
 - **SH (Source Hashing)** – destination selected by source hash key (**ClientIP affinity**)
 - **SED (Shortest Expected Delay)** – like WLC except $(con+1)/weight$
 - **Never Queue** – destination with no connections selected, if none exist then SED
- ipvsadm is used to set up, maintain or inspect the IP virtual server table in the Linux kernel

External Load Balancers

108

Copyright 2013-2020, RX-M LLC

- Access to a pod or service from within the cluster is clean and simple, using the Pod IP or the kube-proxy established service ClusterIP (VIP)
- Accessing a pod from outside the cluster requires an alternate approach
- **LoadBalancer** services wire up cloud load balancer
 - GCE's ForwardingRules
 - AWS's ELB
 - Azure Load Balancer
- **NodePort** services expose the service on a single port on every node's external interface
 - When traffic arrives at a node on the node port it is routed to an appropriate backend Pod just as in the ClusterIP scenario
 - Most traffic will get **double-bounced** on the network
 - The nodes essentially act as routers
 - All forwarding is done via iptables and purely in kernel mode



Services & DNS

- Every Service defined in the cluster (including the DNS server itself) is assigned a DNS name
- Pod's DNS search lists include the Pod's own namespace and the cluster's default domain
 - e.g. `my-web-service.my-kube-namespace`
- Kubernetes Service DNS names can be resolved using standard methods (e.g. `gethostbyname`) inside any pod, except pods which have the `hostNetwork` field set to true
- SRV Records are created for named ports in a service
 - For each named port, the SRV record would have the form:
 - `my-port-name.my-port-protocol.my-svc.my-namespace.svc.cluster.local`
 - For a regular service, this resolves to the port number and the CNAME:
 - `my-svc.my-namespace.svc.cluster.local`
 - For a headless service, this resolves to multiple answers, one for each pod that is backing the service, and contains the port number and a CNAME of the pod of the form:
 - `auto-generated-name.my-svc.my-namespace.svc.cluster.local`
- Pod DNS
 - When enabled, pods are assigned a DNS A record in the form:
 - `pod-ip-address.my-namespace.pod.cluster.local`
 - For example, a pod with ip 1.2.3.4 in the namespace default with a dns name of cluster.local would have an entry:
 - `1-2-3-4.default.pod.cluster.local`

```
$ cat /var/lib/kubelet/config.yaml
address: 0.0.0.0
apiVersion: kubelet.config.k8s.io/v1beta1
authentication:
  anonymous:
    enabled: false
  webhook:
    cacheTTL: 2m0s
    enabled: true
x509:
  clientCAFile: /etc/kubernetes/pki/ca.crt
authorization:
  mode: Webhook
  webhook:
    cacheAuthorizedTTL: 5m0s
    cacheUnauthorizedTTL: 30s
cgroupDriver: cgroupfs
cgroupsPerQOS: true
clusterDNS:
- 10.96.0.10
clusterDomain: cluster.local
```

Pod DNS Configs

110

Copyright 2013-2020, RX-M LLC

- Allows more control on the DNS settings for a Pod
- Pod hostnames are the Pod's `metadata.name` value
 - Users can override this with `hostname` and optional `subdomain`
 - The example on the right would result in:
`mynginx.mysub.default.svc.cluster.local`
- `dnsPolicy`:
 - "Default" – inherits config from the Node where it is running
 - "ClusterFirst" – uses Cluster DNS
 - Queries not matching cluster domain suffix are forwarded to the upstream nameserver inherited from the Node
 - "ClusterFirstWithHostNet" – use when running Pods in the host network namespace
 - "None" – ignore DNS from K8s environment in favor of `dnsConfig` settings
- `dnsConfig`:
 - `nameservers` – a list of IP addresses that will be used as DNS servers
 - `searches` – a list of DNS search domains for hostname lookup
 - `options` – an optional list of objects where each object may have a `name` property (required) and a `value` property (optional)
- The `dnsConfig` field is optional and it can work with any `dnsPolicy` settings
 - When `dnsPolicy` is set to `None`, the `dnsConfig` field **must** be specified

```
apiVersion: v1
kind: Pod
metadata:
  namespace: default
  name: dns-example
spec:
  hostname: mynginx
  subdomain: mysub
  containers:
  - name: test
    image: nginx
  dnsPolicy: "None"
  dnsConfig:
    nameservers:
    - 1.2.3.4
    searches:
    - ns1.svc.cluster.local
    - my.dns.search.suffix
    options:
    - name: ndots
      value: "2"
    - name: edns0
```

Resultant Pod's /etc/resolv.conf:

```
nameserver 1.2.3.4
search ns1.svc.cluster.local my.dns.search.suffix
options ndots:2 edns0
```

Summary

- Kubernetes implements a simple Linux native networking model using standards-driven technologies
 - Virtual IPs
 - iptables
 - DNS
 - Etc.

Lab 5

- Services

Lab 5 Documentation References

113

Copyright 2013-2020, RX-M LLC

- Services
 - <https://kubernetes.io/docs/concepts/services-networking/service/>
- Label Selectors and Services
 - <https://kubernetes.io/docs/concepts/services-networking/service/#defining-a-service>

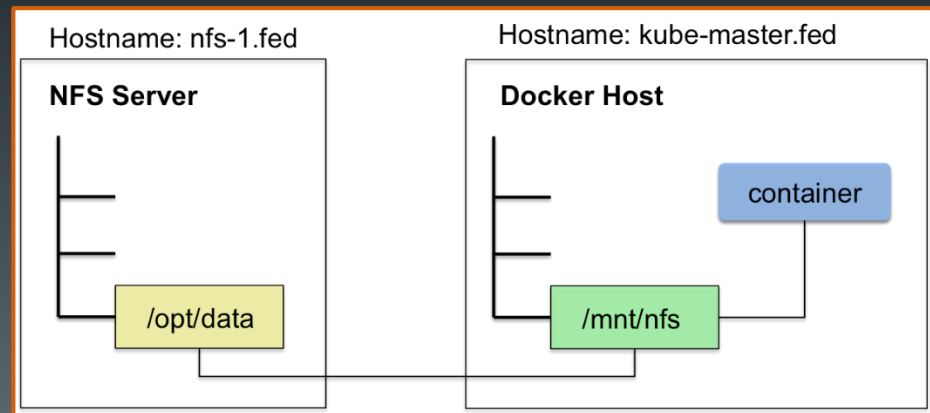
6: Managing State

Objectives

- Explore the lifecycle of volumes in Kubernetes
- Discover the different types of volumes available to Pods and containers in a Kubernetes cluster
- Examine ways to abstract storage providers
- Discuss creating and leveraging secrets in configs
 - Learn about encrypting secrets at rest
- Explore the use of ConfigMaps for configuration files & data
- Examine using pod metadata in volumes and environment variables

Kubernetes Volumes

- A Volume is a container mountable filesystem feature with storage and persistence rules **outside of the scope of the container**
- Cases for volumes:
 - **Persist data across container failures** because files in a container are **ephemeral**
 - When a container crashes, the kubelet will replace it by rerunning the original image, the **files from the dead container will be lost**
 - When running containers together in a pod it is often necessary to **share files** between those containers
 - Storage outside of the container mount namespace can be shared across containers
 - **Map parts of the host's filesystem** into the pod's containers for monitoring, storage daemons, etc.
 - **Provide configuration and metadata dynamically** to a pod
- A standard Kubernetes volume has a **lifetime the same as the pod** that encloses it
 - Different from the Docker volume model, wherein volumes remain until explicitly deleted, regardless of whether there are any running containers using it
- Pods are anchored by the infrastructure container which cannot crash
 - Thus a **pod volume outlives any containers** that run within the Pod except the pause container preserving volume data across container restarts
 - Only when a Pod is deleted or the node the pod runs on crashes does the volume cease to exist



Volume Types

Copyright 2013-2020, RX-M LLC

117

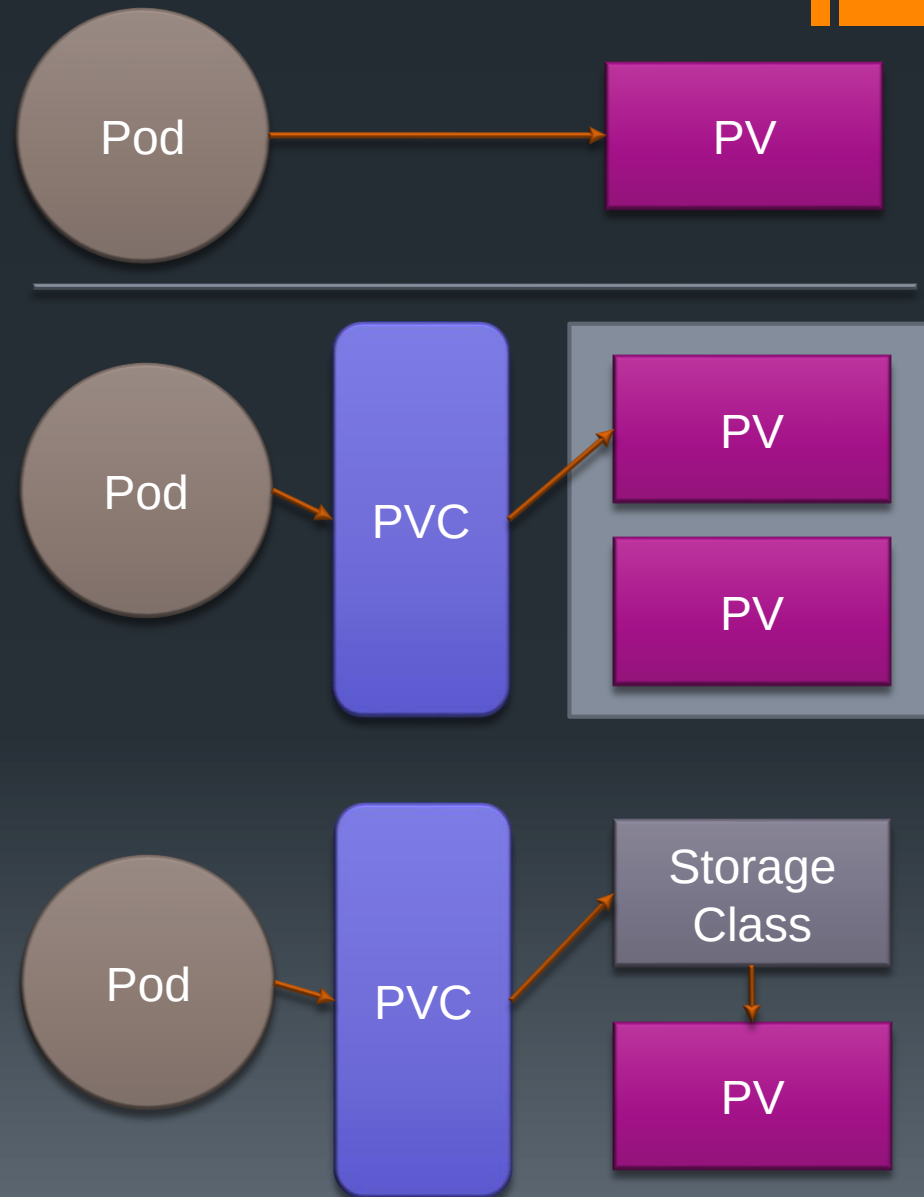
- Kubernetes supports many type of volumes, and a Pod can use any number of them simultaneously
- Cloud volumes
 - `awsElasticBlockStore` – mounts an Amazon Web Service EBS volume
 - `azureDisk` – mounts a Microsoft Azure Data Disk
 - `azureFile` – mounts an Azure File Volume
 - `gcePersistentDisk` – mounts a Google Compute Engine block storage volume
- Network storage
 - `cephfs` – mounts an existing CephFS volume
 - `fc (fibre channel)` – allows an existing fibre channel volume to be mounted in a pod
 - `glusterfs` – mounts a glusterfs volume (Glusterfs is an open source networked filesystem)
 - `iscsi` – allows an existing iSCSI (SCSI over IP) volume to be mounted
 - `nfs` – mounts an NFS share
 - `PortworxVolume` – elastic block storage layer that runs hyperconverged with Kubernetes
 - `Quobyte` – allows an existing Quobyte volume to be mounted
 - `rbd` – mounts a Rados Block Device volume
 - `ScaleIO` – software-based storage platform of scalable shared block networked storage
 - `StorageOS` – allows an existing StorageOS volume to be mounted into your pod
 - `vsphereVolume` – mounts a vSphere VMDK Volume
- Kubernetes volume types
 - `PersistentVolume` (PV) – storage provisioned by an administrator aimed at abstracting storage drivers
 - `Local` – storage device such as a disk, partition or directory which can only be used as a statically created PersistentVolume
 - `configMap` – provides a way to inject configuration data into Pods
 - `downwardAPI` – mounts pod metadata
 - `emptyDir` – provides the pod with an empty directory (data is deleted when the Pod terminates)
 - `gitRepo` – mounts an empty directory and clones a git repository into it for your pod to use
 - `hostPath` – mounts a given host path into containers using the volume
 - Example: containerized kubelet that needs `/var/lib/docker` mounted to talk to `docker.sock`
 - `projected` – maps several existing volume sources into the same directory
 - Following types of volume sources can be projected: `secret`, `downwardAPI`, `configMap`
 - `secret` – mounts a volume as an in-memory tmpfs (never touches the host disk)

Decoupling Pods from Volumes

118

Copyright 2013-2020, RX-M LLC

- **Static pod volume linkage**
 - Pods can be bound to specific preexisting volumes
 - This static relationship is simple, useful but often not flexible enough
- **Dynamic pod volume linkage**
 - In dynamic environments it may be more convenient to bind a Pod to a volume abstraction
 - **Persistent Volume Claims [PVC]** describe abstract volumes
 - Pods bind to PVCs and PVCs select an appropriate volume at Pod creation time
 - **Persistent Volumes [PV]**
 - PVCs can be used to select one of a number of suitable admin created Physical Volumes
 - **Storage Classes [SC]**
 - PVCs can also be used to dynamically allocate volumes using a Storage Class



Persistent Volumes

119

Copyright 2013-2020, RX-M LLC

- Provisioning
 - **Static** – cluster admin creates a number of PVs
 - Include the details of the real storage
 - Exist as resources in the API
 - **Dynamic** – PersistentVolume Provisioner may try to provision a PV when none of the static PVs matches a user's PVC
 - Based on Storage Classes and volume plugin feature availability
- Capacity – PVs will have a storage capacity set using the PV's capacity attribute
- Access Modes – can be set to modes supported by a given provider
 - **ReadWriteOnce** (RWO) – can be mounted as read-write by a single node
 - **ReadOnlyMany** (ROX) – can be mounted read-only by many nodes
 - **ReadWriteMany** (RWX) – can be mounted as read-write by many nodes
 - PV can only be mounted using one access mode at a time, even if it supports many
 - ex: GCE-PD can be mounted as ReadWriteOnce by a single node or ReadOnlyMany by many nodes but not at the same time
- Reclaim policy for a PV tells the cluster what to do with the volume after it has been released of its claim, can be:
 - **Retained** – requires manual reclamation
 - Previous claimant's data remains on the volume until cleaned or the PV is deleted
 - **Recycled** - performs a scrub (`rm -rf /thevolume/*`) on the volume and makes it available again for a new claim
 - Must use a volume plugin that supports the recycled feature
 - **Deleted** – removes both the PV object as well as deleting the associated external storage, such as an AWS EBS, GCE PD, Azure Disk, or Cinder volume
 - Dynamically provisioned PVs are always deleted
- Class (optional) – ensures that the PV will only be bound to PVCs requesting the given Storage Class

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: fiftygigpv
spec:
  capacity:
    storage: 50Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: bronze
  nfs:
    path: /tmp
    server: 172.17.0.2
```

Persistent Volume Claims

120

Copyright 2013-2020, RX-M LLC

- Created with a specific amount of storage and an access mode
 - **resources** – PVCs request quantities of storage
 - **accessModes** – PVCs use the same conventions as PVs
 - Mounting claims with "Many" modes (ROX, RWX) is only possible within one namespace
- **storageClassName** (optional) – selects only PVs of the given class
- **selector** (optional) – enables granular filtering of available PVs
 - Only volumes with labels that match the selector can be bound to the claim
 - **matchLabels** – volume must have a label with this value
 - **matchExpressions** – list of requirements made by specifying key, a list of values, and an operator that relates the key and values
 - Valid operators: In, NotIn, Exists, DoesNotExist
 - Requirements are ANDed together—all must be satisfied to match
- **Control loop matches new PVCs with PVs**
 - A dynamically-provisioned PV will always bind to the PVC that requested it
- Will remain unbound indefinitely if a matching PV does not exist
 - Claims are bound as matching volumes become available
 - Cluster with many 50Gi PVs would not match any to a 100Gi PVC
- PVCs have *beta* support for online resizing
- PVCs have *alpha* support for cloning when PVs are backed by CSI drivers as of v1.15

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: fiftygigpvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi
  storageClassName: bronze
  selector:
    matchLabels:
      type: "cloudvol"
    matchExpressions:
      - {key: environment, operator: In, values: [dev,test]}
```

Pods and PVCs

- Pods bind to PVs through PVCs in the same way that they use normal volumes
 - A volume is identified as a PVC using the **persistentVolumeClaim** key
 - PVCs are identified by **claimName**
- PVCs
 - Can request storage sizes and access modes
 - Have a lifecycle independent of any pod that uses them
 - Capture all of the required storage details
 - NFS, iSCSI, Cloud Provider storage, etc

```
apiVersion: v1
kind: Pod
metadata:
  name: frontendpod
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
        - mountPath: "/var/www/html"
          name: abstractedstorage
  volumes:
    - name: abstractedstorage
      persistentVolumeClaim:
        claimName: fiftygigpvc
```

Storage Classes

122

Copyright 2013-2020, RX-M LLC

- Admins set the name and other parameters of a class when first creating StorageClass objects
 - Cannot be updated once they are created
- provisioner** – determines what volume plugin is used for provisioning PVs
 - ex: GCEPersistentDisk, AWSElasticBlockStore, VsphereVolume, etc.
 - Internal** provisioners prefixed with `kubernetes.io` and shipped with Kubernetes
 - External** provisioners are independent programs that follow a specification defined by Kubernetes
 - ex: NFS doesn't provide an internal provisioner, but an external provisioner can be used
- parameters** – describe attributes of volumes belonging to the storage class
 - Are provisioner specific though some may be similar
 - Best to examine docs of a given provisioner for details on parameters
- Default StorageClass can be specified for PVCs that don't request any particular class to bind to

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gcecloud
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  zones: us-central1-a, us-central1-b

---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: awscloud
provisioner: kubernetes.io/aws-efs
parameters:
  type: gp2
```

StatefulSets

- StatefulSet Pods have a **unique ordinal index** (thus stable net identity)
 - Each Pod in a StatefulSet derives its hostname from the name of the StatefulSet and the ordinal of the Pod
 - StatefulSet “web” would have pods: **web-0,web-1,web-2**, ...
 - Works with headless services
 - The CNAME of the headless service points to SRV records (one for each Pod)
 - The IP can change so clients should use the SRV
 - kube-proxy never acts on headless services
 - Ordered deployment, scaling, deletion and termination
 - K8s v1.7+ uses **podManagementPolicy** to determine ordering
 - OrderedReady** – the first pod must be “running and ready” prior to the following pods start (Also terminated in reverse order)
 - Parallel** – launch or terminate all Pods in parallel
- StatefulSet Pods have **Stable persistent storage** – one PV for each VolumeClaimTemplate
 - PVs associated with the Pods’ PVCs are not deleted when the Pods, or StatefulSet are deleted & must be removed manually
- Update strategies
 - OnDelete** (default) – users manually delete Pods to cause the controller to create new Pods reflecting modifications made to the template
 - RollingUpdate** – deletes and recreates each Pod in the same order as Pod termination
 - Waits until an updated Pod is Running and Ready prior to updating its predecessor
 - Can be partitioned, by specifying **.spec.updateStrategy.rollingUpdate.partition**
 - If a partition is specified, all Pods with an ordinal that is greater than or equal to the partition will be updated
 - All Pods with an ordinal that is less than the partition will not be updated
 - Useful if you want to stage an update, roll out a canary, or perform a phased roll out

```
# Headless service
apiVersion: v1
kind: Service
metadata:
  name: cassandra-service
  labels:
    app: cassandra
spec:
  ports:
    - name: cql
      port: 9042
  clusterIP: None
  selector:
    app: cassandra
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: cassandra
spec:
  serviceName: cassandra-service
  replicas: 6
  selector:
    matchLabels:
      app: cassandra
  template:
    metadata:
      labels:
        app: cassandra
    spec:
      containers:
        - name: cassandra
          image: cassandra
          ports:
            - name: cql
              containerPort: 9042
          volumeMounts:
            - name: data
              mountPath: /cassandra/data/
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ "ReadWriteOnce" ]
        storageClassName: local
        resources:
          requests:
            storage: 50Gi
```


emptyDir

- One of the easiest ways to achieve improved persistence amid container crashes and **data sharing** within a pod
- Can be mounted at the **same or different paths** in each container
- Created when a Pod is assigned to a Node, **erased when a Pod is removed**
 - Persists beyond container crashes
- Stored on the medium backing the Node (SSD, network storage, etc.)
 - Specifying Memory creates a RAM-backed filesystem
 - Cleared on machine reboot
 - Counts against container memory limits

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    name: grafana
    name: grafana
spec:
  replicas: 1
  selector:
    matchLabels:
      name: grafana
  template:
    metadata:
      labels:
        name: grafana
    spec:
      containers:
        - name: grafana
          image: grafana/grafana:latest
          ports:
            - name: grafana-port
              containerPort: 3000
          volumeMounts:
            - name: grafana-dashboard-shared-volume
              mountPath: /var/lib/grafana/dashboards/
            ## sidecar - watches all configMaps and places ones
            ## with a defined label into a shared emptyDir
            - name: grafana-dashboard-sidecar
              env:
                - name: LABEL
                  value: grafana-dashboard
                - name: FOLDER
                  value: /tmp/dashboards
              image: kiwigrid/k8s-sidecar:latest
              volumeMounts:
                - name: grafana-dashboard-shared-volume
                  mountPath: /tmp/dashboards
      volumes:
        - name: grafana-dashboard-shared-volume
          emptyDir: {}
```

hostPath

- Mounts a file or directory from the **Node's filesystem** into a Pod
- Pods with identical configs may behave differently on different Nodes due to differences in files on the Nodes
- Resource-aware scheduling will not account for resources used by hostPath
- Created directories only writeable by root
 - Config options:
 - Run process as root in **privileged container**
 - **Modify file perms on the host** to write to the volume
- Unlike emptyDir, which is erased when a Pod is removed, the contents of hostPath volumes are preserved and the volume is merely unmounted

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  labels:
    app: cadvisor
spec:
  template:
    metadata:
      labels:
        app: cadvisor
    spec:
      containers:
        - name: cadvisor
          image: google/cadvisor
          securityContext:
            privileged: true
          volumeMounts:
            - name: var-run
              mountPath: /var/run
              readOnly: false
            - name: sys
              mountPath: /sys
              readOnly: true
            - name: docker
              mountPath: /var/lib/docker
              readOnly: true
      volumes:
        - name: var-run
          hostPath:
            path: /var/run
        - name: sys
          hostPath:
            path: /sys
        - name: docker
          hostPath:
            path: /var/lib/docker
```

Secrets

126

Copyright 2013-2020, RX-M LLC

- Kubernetes objects used to hold sensitive information
 - Passwords
 - OAuth tokens
 - ssh keys
- Safer and more flexible than putting it verbatim in a pod definition or in a Docker image
- Users can create secrets
 - The system also creates some secrets
- Can only be referenced by pods in the **same namespace** where created
- To use a secret, a pod needs to reference the secret
- Each item must be **base64 encoded**
- Limited to 1MB in size
 - Discourages large secrets which would exhaust apiserver and kubelet memory
- Stored as **plaintext** in etcd
 - Secret data is at rest on the disk that etcd uses

```
$ echo -n "admin" | base64
YWRtaW4=
```

```
$ echo -n "1f2d1e2e67df" | base64
MWYyZDFlMmU2N2Rm
```

```
apiVersion: v1
kind: Secret
metadata:
  name: prod-db-secret
type: Opaque
data:
  password: MWYyZDFlMmU2N2Rm
  username: YWRtaW4=
```

Encrypted Secret Data

127

Copyright 2013-2020, RX-M LLC

- Encryption at rest
 - Introduced as an alpha feature in 1.7; advanced to GA in 1.14 (irregular advancement)
 - Set `--encryption-provider-config` flag on the apiserver to point to the location of the EncryptionConfig file
 - After restarting apiserver, newly created or updated secrets should be **encrypted when stored**
- EncryptionConfig file contains keys that can decrypt content in etcd
 - Restrict permissions on masters so only the user who runs apiserver can read it
- The **providers** array is an ordered list of the possible encryption providers:
 - **aescbc** – recommended, strongest but slowest
 - AES-CBC PKCS#7 padding
 - **secretbox** – faster but not as strong as aescbc
 - XSalsa20 and Poly 1305
 - **aesgcm** – fastest, not recommended if not using automated key rotation as must be rotated every 200k writes
 - AES-GCM with random nonce
 - **identity** – no encryption
 - When set as first provider, will disable encryption at rest
- Provider key secrets generated with a 32 byte random key and base64 encoding
 - ex: `head -c 32 /dev/urandom | base64`

```
kind: EncryptionConfig
apiVersion: v1
resources:
- resources:
  - secrets
  providers:
  - aesgcm:
    keys:
    - name: key1
      secret: c2Vjc mV0IGl zIHNl Y3VyZ Q==
    - name: key2
      secret: dGhpc yBpc yBwYXN zd29yZ A==
  - aescbc:
    keys:
    - name: key1
      secret: c2Vjc mV0IGl zIHNl Y3VyZ Q==
    - name: key2
      secret: dGhpc yBpc yBwYXN zd29yZ A==
  - secretbox:
    keys:
    - name: key1
      secret: YWJ jZGV mZ2hp amts bW5vc HFyc 3R1d nd4eX oxMj M0NTY=
  - identity: {}
```

- Many applications require configuration via some combination of config files, command line arguments, and environment variables
- These **configuration artifacts should be decoupled** from image content in order to keep containerized applications portable
- The ConfigMap resource provides mechanisms to **inject containers with configuration data** while keeping containers agnostic of Kubernetes
- ConfigMap can be used to store fine-grained information like individual properties or coarse-grained information like entire config files or JSON blobs

ConfigMap of prometheus.yml config file

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data: prometheus.yml: |
  global:
    scrape_interval: 30s
    scrape_timeout: 30s
  scrape_configs:
  - job_name: 'kubernetes-apisservers'
    kubernetes_sd_configs:
    - role: endpoints
    scheme: https
    tls_config:
      ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
    bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
    relabel_configs:
    - source_labels: [__meta_kubernetes_namespace, __meta_kubernetes_service_name]
      action: keep
      regex: default;kubernetes;https
  - job_name: 'kubernetes-nodes'
  ...
```

prometheus.yml

*# Example scrape configuration for running Prometheus on a Kubernetes cluster.
Kubernetes Labels will be added as Prometheus Labels on metrics via the
'Labelmap' relabeling action.*

```
scrape_configs:
- job_name: 'kubernetes-apisservers'
  kubernetes_sd_configs:
  - role: endpoints
  scheme: https
  tls_config:
    ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
    # insecure_skip_verify: true
  bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
  relabel_configs:
  - source_labels: [__meta_kubernetes_namespace, __meta_kubernetes_service_name]
    action: keep
    regex: default;kubernetes;https
  - job_name: 'kubernetes-nodes'
  ...
```

Creating ConfigMaps

- ConfigMaps can be created from:
 - Directories
 - Files
 - Literal values
- **--from-file**
 - If a directory, combines the contents of the directory into a single ConfigMap object
 - If a file, creates the ConfigMap from the contents of the file
 - **--from-file=<my-key-name>=<path-to-file>**
 - Defines a key other than the file name to use as the data key
- **--from-literal=**
 - Define a literal value from the command line
- Multiple **--from-file** or **--from-literal** arguments can be passed, combining files from different paths into a single ConfigMap object
- As a yaml file (prev slide) using **kubectl apply -f**

```
$ cat app.conf
key=value1
property=config
another.property=configValue

$ cat ui.conf
color.one=blue
color.two=white
font.headings=sans-serif
font.body=serif

$ kubectl create configmap config \
--from-file=/home/user/configmap/

configmap/config created

$ kubectl describe configmap config

Name:         config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
app.conf:
----
key=value1
property=config
another.property=configValue

ui.conf:
----
color.one=blue
color.two=white
font.headings=sans-serif
font.body=serif

Events:  <none>
```

Mounting Secrets/ConfigMaps

- When a pod is created Secret/ConfigMap existence is not checked
- If the kubelet can not fetch the Secret/ConfigMap:
 - It will report a pod event
 - The kubelet will periodically retry
 - Once fetched the kubelet will create and mount the volume
 - None of the pod's containers will start until all the pod's volumes are mounted
- **Files**
 - You can specify the permission mode bits a Secret/ConfigMap or files of part of a Secret/ConfigMap will have
 - Otherwise 0644 is used by default
 - When a Secret/ConfigMap being consumed in a volume is updated, projected keys are **eventually updated** as well
- An **imagePullSecret** is a way to pass a secret that contains a Docker (or other) image registry password to Kubelet so it can pull a private image on behalf of your Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-file-pod
  labels:
    secretpod: db-client
spec:
  volumes:
    - name: secret-volume
      secret:
        secretName: prod-db-secret
        defaultMode: 400
    - name: config-file
      configMap:
        name: config
  containers:
    - name: secret-file-container
      image: redis
      imagePullSecrets:
        - name: myregistrykey
      volumeMounts:
        - name: secret-volume
          readOnly: true
          mountPath: "/etc/secret-volume"
        - name: config-file
          mountPath: "/etc/app/config"
```

Secrets/ConfigMaps as Environment Variables

131

Copyright 2013-2020, RX-M LLC

- Secrets appear as normal environment variables containing the **base-64 decoded values** of the secret data
- Add an environment variable **to each container for each key** you wish to consume
 - The environment variable that consumes the key should populate the name and key in `env[].valueFrom.secretKeyRef` | `configMapKeyRef`
 - Modify your image or command so that the program sees the values in the environment variables
- Keys that are considered invalid environment variable names will be skipped
 - The pod will be allowed to start
 - An `InvalidVariableNames` event will contain the list of invalid keys that were skipped
- Use **envFrom** to define all of a Secret/ConfigMap's data as container environment variables

```
apiVersion: v1
kind: Pod
metadata:
  name: env-pod
spec:
  containers:
    - name: env-container
      image: redis
      env:
        - name: SECRET_USERNAME
          valueFrom:
            secretKeyRef:
              name: prod-db-secret
              key: username
        - name: SECRET_PASSWORD
          valueFrom:
            secretKeyRef:
              name: prod-db-secret
              key: password
        - name: CONFIG_VAR
          valueFrom:
            configMapKeyRef:
              name: config
              key: property
      envFrom:
        - configMapRef:
            name: multi-config
```


ConfigMaps vs Secrets

Paul Morie (author of both resources):

"

I'm the author of both of these features. The idea is that you should:

1. Use secrets for things which are actually secret like API keys, credentials, etc
2. Use config map for not-secret configuration data

In the future there will likely be some differentiators for secrets like rotation or support for backing the secret API w/ HSMs, etc. In general we like intent-based APIs, and the intent is definitely different for secret data vs. plain old configs.

Hope that helps.

"

<http://stackoverflow.com/questions/36912372/kubernetes-secrets-vs-configmaps>



Downward API

133

Copyright 2013-2020, RX-M LLC

- Allows containers to **consume information about themselves** or the cluster without using the kubectl client or API server
- Ways of exposing Pod and Container fields through:
 - Environment variables
 - DownwardAPIVolumeFiles
- Available via **fieldRef**:
 - `spec.nodeName`
 - `status.hostIP`
 - `metadata.name`
 - `metadata.namespace`
 - `status.podIP`
 - `spec.serviceAccountName`
 - `metadata.uid`
 - `metadata.labels`
 - When appended with ['<key>'] a single value of a pod's label defined by <key>
 - `metadata.annotations`
 - When appended with ['<key>'] a single value of a pod's annotation defined by <key>
- Information available via **resourceFieldRef**:
 - Container CPU limit, CPU request, memory limit, memory request

```
apiVersion: v1
kind: Pod
metadata:
  name: downward-api-pod
  labels:
    cluster: test-cluster1
    rack: rack-22
spec:
  containers:
  - name: downward-api-container
    image: busybox
    env:
    - name: MY_POD_IP
      valueFrom:
        fieldRef:
          fieldPath: status.podIP
    volumeMounts:
    - name: dapinfo
      mountPath: /etc/dapinfo
  volumes:
  - name: dapinfo
    downwardAPI:
      items:
      - path: "labels"
        fieldRef:
          fieldPath: metadata.labels
      - path: "cpu_limit"
        resourceFieldRef:
          containerName: downward-api-container
          resource: limits.cpu
          divisor: 1m
```

Summary

- Kubernetes volumes have the same lifetime as the pods that declare them
 - Kubernetes volumes are not exactly the same as Docker volumes
- Kubernetes supports many volume types & providers
- Volumes are enabled via plugins
- Kubernetes provides powerful storage abstraction using PVs, PVCs & StorageClasses
- StatefulSets leverage abstracted storage for stateful applications
- Secrets can be used to pass sensitive data to Pods
 - kubelet can use secrets for logging into registries
 - Secrets can be encrypted by using EncryptionConfigs

Lab 6

- Volumes

Lab 6 Documentation

References

136

Copyright 2013-2020, RX-M LLC

- Volumes
 - <https://kubernetes.io/docs/concepts/storage/volumes/>
 - <https://kubernetes.io/docs/tasks/configure-pod-container/configure-volume-storage/>
- Annotations
 - <https://kubernetes.io/docs/concepts/overview/working-with-objects/annotations/>
- Downward API
 - <https://kubernetes.io/docs/tasks/inject-data-application/downward-api-volume-expose-pod-information/>
 - <https://kubernetes.io/docs/tasks/inject-data-application/environment-variable-expose-pod-information/>
- Secrets
 - <https://kubernetes.io/docs/concepts/configuration/secret/>
 - <https://kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/>
- ConfigMaps
 - <https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap>

Day 3

- 7. Security
- 8. Observability
- 9. Ingress

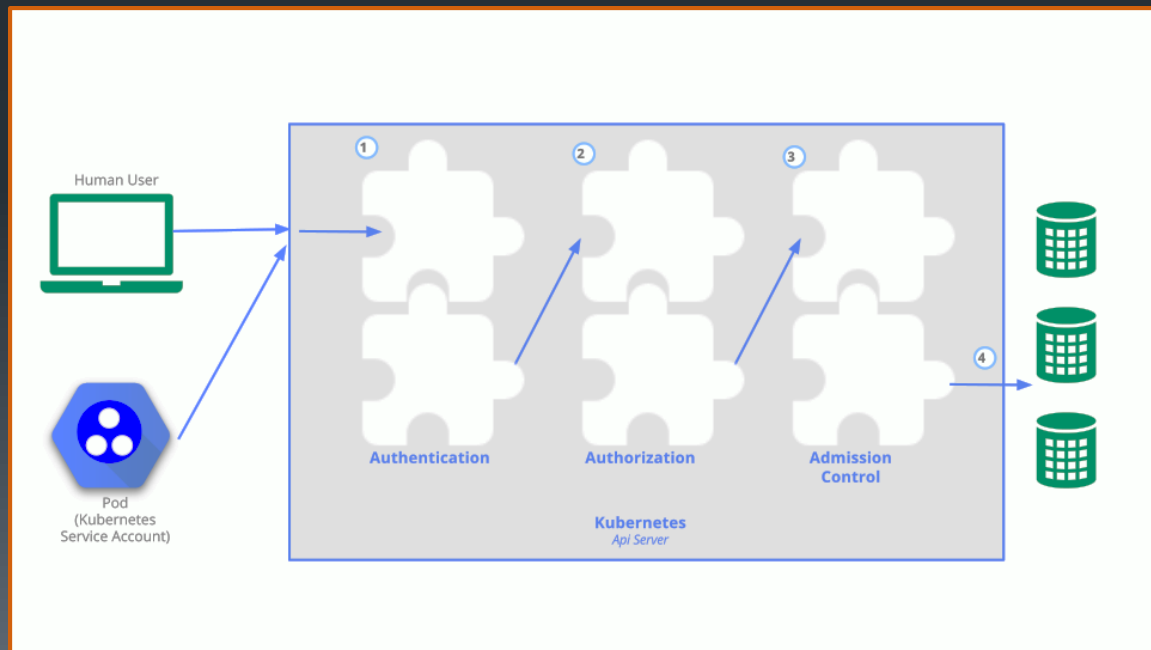
7. API and Pod Security

Objectives

- Discuss authN & authZ mechanisms of the K8s API
- Examine RBAC concepts and RBAC resources
- Understand the role and use of Admission Controllers
- Review security configurations in K8s:
 - Capabilities & privileged Pods/containers
 - Discretionary Access Control
 - AppArmor
 - SecComp
 - Analyze security settings with the K8s Security Benchmark

Kubernetes API Access

- Users access the API using kubectl, client libraries, or by making REST requests
- Cluster components, human users, and Kubernetes Service Accounts can be authorized for API access
- API requests go through a security pipeline (**authentication**, **authorization**, and **admission control**)



Kubernetes Authentication

141

Copyright 2013-2020, RX-M LLC

- Kubernetes clusters have two categories of users:
 - Service Accounts managed by Kubernetes
 - Normal users
- If a request cannot be authenticated, it is treated as anonymous and rejected with HTTP status code 401; otherwise, the user is authenticated as a **specific username**, **UID**, and **group**
 - The user name is **available to subsequent steps** to use in their decisions
- Authentication modules include:
 - **Bootstrap Tokens** (beta) – stored as Secrets in the kube-system namespace
 - **X509 Client Certificates** – primarily used for Kubernetes system components
 - **OpenID Connect Tokens** – supported by some OAuth2 providers, notably Azure Active Directory, Salesforce, and Google
 - This token is a JSON Web Token (JWT) with well known fields, such as a user's email, signed by the server
 - **Plain Tokens** – apiserver reads bearer tokens from a **csv file** with a minimum of 3 columns: token, user name, user uid, followed by optional group names
 - Tokens last indefinitely, and the token list cannot be changed without restarting API server
 - **Service Account Tokens**
 - **Static Password File** – basic auth credentials last indefinitely, and the password cannot be changed without restarting API server
 - Supported for convenience
 - **Webhook Token Authentication** – when a client attempts to authenticate with the API server using a bearer token, the authentication webhook POSTs a JSON-serialized TokenReview object containing the token to the remote service
- Multiple authentication modules can be specified, in which case **each one is tried in sequence**, until one of them succeeds
- **Client-go credential plugins** – intended for client side integrations with authentication protocols not supported by k8s.io/client-go (LDAP, Kerberos, OAuth2, SAML, etc.)
 - Implements the protocol specific logic, then returns opaque credentials to use
 - Almost all credential plugins require a server side component with support for the **webhook token authenticator** to interpret the credential format produced by the client plugin

Certificates

- Certificates allow parties in a conversation to authenticate each other
- In client-server scenarios:
 - Client can ensure that it is talking to who it thinks it is talking to, server can authenticate the client
 - Using a third party server that both client and server trust
 - **Certificate Authority** – given that both client & server trust the CA, they can then trust each other

Certificates in Kubernetes

- Cluster CA is the trusted root throughout the cluster – all certs used throughout the cluster, will be signed by the CA so that all Kubernetes components can trust one another
- Each Kubernetes core component (kubelets/schedulers/controller-managers/proxys) has its own client certificate
 - Each will have different access levels to the cluster and should have a unique identity
 - Kubelet's Common Name should include the hostname where the kubelet is running
 - Ensures each kubelet has its own identity and enables features:
 - Node Authorizer & Node Restriction Admission Plugin – combined will limit kubelet's influence to objects that are assigned to its Node only
 - ex. it would only have access to Secrets for Pods bound to its Node, not *all* Secret objects
- Certificate generation
 - Manual – admin produces all of the certificates required
 - Certificate Generation API - `certificates.k8s.io/v1beta1` – original intent of this API to enable provisioning of TLS client certificates for kubelets
 - Client creates a certificate signing request (CSR) and submits to the apiserver
 - Stays in a pending state until approved by a cluster admin
 - Certificate issued once CSR request approved
- Kubernetes does not currently support Certification Revocation Lists (CRLs)
 - Use RBAC to "revoke" access

TLS Bootstrapping & Certificate Rotation

144

Copyright 2013-2020, RX-M LLC

(Many of these features are in beta or experimental state)

■ Bootstrapping

- To request a client certificate from kube-apiserver, the kubelet first needs a path to a **kubeconfig file that contains the bootstrap authentication token**
- The bootstrap kubeconfig is **used to request a client certificate** from the API server
- On approval of the certificate request and receipt back by the kubelet, a kubeconfig file referencing the generated key and certificate is written
 - The certificate and key file will be placed in the directory specified by `--cert-dir`
 - Certificates are issued with **one year expiration**
 - The controller manager accepts the argument `--experimental-cluster-signing-duration` that controls how long certificates will be issued for

■ Rotation

- A bootstrapped kubelet will use its initial certificate to connect to the Kubernetes API and **issue a CSR**
- If the CSR meets specific criteria, it will be auto approved by the CSRApprovingController in controller manager
- The **CSRSignerController in controller manager will sign a certificate** and attach it to the certificate signing request
- **kubelet will retrieve the signed certificate** from the API and write that to disk, in the location specified by `--cert-dir`
 - **kubelet will use the new certificate** to connect to the API
- As the expiration of the signed certificate approaches, the kubelet issues another CSR and **the process repeats**

Kubernetes Users

- Kubernetes **users do not have an explicit API object** that you can create, list or modify
 - The **Common Name** (CN=tim) field of a certificate is as the user name for a request and used for identifying the user for RBAC
 - The **Organization** (O=mygroup) field of a certificate is used to determine user group membership for RBAC
 - Include multiple organization fields in the certificate to include multiple group memberships for a user
- Users are also a **parameter of a context in a kubeconfig** that also defines a cluster nickname and a namespace
 - The current context of a kubeconfig assigns `client-certificate-data` and `client-key-data` attributes to a user
- Creating a new Kubernetes user requires a number of steps:
 - Create a new private key
 - Requires access to the Kubernetes root certification authority
 - Create a certificate signing request containing the public key and other subject information
 - Sign the CSR using the root Kubernetes CA
 - Register the new credentials and config context:
 - `$ kubectl config set-credentials tim \`
`--client-certificate=/home/newusers/tim.crt \`
`--client-key=/home/newusers/tim.key`
 - `$ kubectl config set-context tim@kubernetes \`
`--cluster=kubernetes --user=tim`
 - `$ kubectl config use-context tim@kubernetes`
- Requires RBAC permissions to act upon Kubernetes API objects

```
$ kubectl config view

apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: REDACTED
    server: https://192.168.225.236:6443
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
    name: kubernetes-admin@kubernetes
- context:
    cluster: kubernetes
    user: tim
    name: tim@kubernetes
current-context: tim@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
- name: tim
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

A user can act as another user through impersonation headers (requires the "impersonate" verb in RBAC):

```
$ kubectl drain mynode \
--as=admin \
--as-group=system:admins
```

Service Accounts

146

Copyright 2013-2020, RX-M LLC

- A service account provides an **identity for processes that run in a Pod**
- When you create a Pod, a service account is automatically assigned
 - The default service account of the same namespace—unless one is explicitly specified
 - When processes in containers inside Pods contact the API server, they are authenticated as a particular Service Account
- In version 1.6+, **you can opt out** of auto-mounting API credentials for a service account by setting `automountServiceAccountToken: false` on the service account object or a given Pod spec
- Creating additional ServiceAccount objects will **automatically create tokens** used by the named service account
- Set the `serviceAccountName` field of a pod to the name of the service account to use other than the default
 - The service account has to exist at the time the Pod is created, or the Pod will be rejected
- You cannot update the service account of an already created Pod

```
$ cat special-sa.yaml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: special-sa
```

```
$ kubectl create -f special-sa.yaml
serviceaccount "special-sa" created
```

```
$ kubectl get serviceaccounts/special-sa -o yaml
```

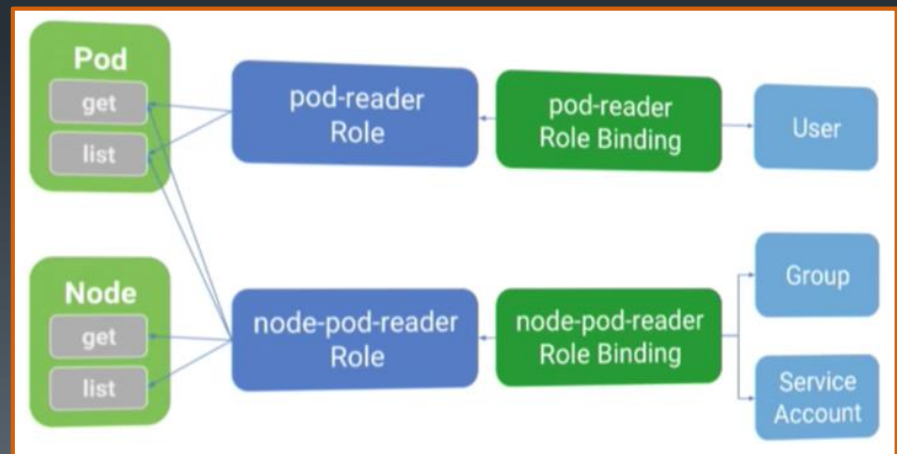
```
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: 2017-06-16T00:12:59Z
  name: special-sa
  namespace: default
  resourceVersion: "272500"
  selfLink: /api/v1/namespaces/default/serviceaccounts/special-sa
  uid: 721ab723-13bc-11e5-aec2-42010af0021e
secrets:
- name: special-sa-token-bvbk5
```

Kubernetes Authorization

- The input to the Authorization step are attributes of the REST request, including:
 - The username determined by the Authentication step
 - A verb associated with the API request
 - Most objects support these common operations: list, watch, create, update, patch, delete
- There are multiple supported Authorization Modules including:
 - **AlwaysAllow** – allows all requests (use only if you do not require authorization)
 - **AlwaysDeny** – blocks all requests (only for testing)
 - **ABAC** – rights are granted to users through the use of policies which combine attributes together
 - **Node** – grants permissions to kubelets based on the pods they are scheduled to run
 - Requires that the kubelet identify itself uniquely in the request
 - **RBAC** – controlling the ability of an individual user to perform a specific task, such as view, create, or modify an API object
 - **Webhook** – causes Kubernetes to query an outside REST service when determining user privileges
 - When multiple Authorization Modules are configured, each is checked in sequence, and if any Module authorizes the request, then the request can proceed
 - If all deny the request, then the request is denied (HTTP status code 403)

Role-based Access Control (RBAC)

- RBAC permission policies are configured using kubectl or the Kubernetes API
- Users can be authorized to make authorization policy changes using RBAC
 - Delegate resource management without giving away ssh access to the cluster master (req. w/ ABAC)
- Connection between user and resources is defined in RBAC using two objects:
 - **Roles**
 - **Role Bindings**



RBAC Roles

- RBAC API declares two top-level types:
 - **Role** and **ClusterRole**
 - Role contains rules that represent a set of permissions on API objects
 - Purely additive, no “deny” rules
 - Defined within a namespace (Role) or cluster wide (ClusterRole)
- Roles and ClusterRoles define:
 - **apiGroups** – limit to resource kinds defined in specific API group(s)
 - Empty ("") means the core API group
 - Use `kubectl api-resources` to list current groups
 - **resources** – limit to specific resource kinds
 - Empty ("") means all kinds
 - **verbs** – actions allowed against the identified resource kinds
 - **resourceNames** – restrict requests to individual resource instances

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: svc-reader
rules:
- apiGroups: [""]
  resources: [""]
  resourceNames: ["my-svc"]
  verbs: ["get", "watch", "list"]

---
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: system:coredns
rules:
- apiGroups:
  - ""
  resources:
  - endpoints
  - services
  - pods
  - namespaces
  verbs:
  - list
  - watch
```

Imperative command equivalent example:

```
kubectl create role svc-reader \
--verb=get,list,watch \
--resource=service \
--resource-name=my-service
```

Verbs

150

Copyright 2013-2020, RX-M LLC

▪ ApiResource verbs

(used to auth resource requests to /api/v1/... or /apis/<group>/<version>/.):

- create
- get
- list
- watch
- update
- patch
- delete
- deletecollection

▪ HTTP Request Verbs (used to auth non-resource requests):

- post
- get
- head
- put
- patch
- delete

Comprehensive examples here:

<https://github.com/kubernetes/kubernetes/blob/master/plugin/pkg/auth/authorizer/rbac/bootstrappolicy/testdata/cluster-roles.yaml>

Special Verbs:

- PodSecurityPolicy
 - use
- RBAC
 - bind
 - escalate
- Authentication
 - impersonate (on users, groups, and serviceaccounts)

HTTP verb	request verb
POST	create
GET, HEAD	get (for individual resources), list (for collections, including full object content), watch (for watching an individual resource or collection of resources)
PUT	update
PATCH	patch
DELETE	delete (for individual resources), deletecollection (for collections)

Default Roles

Default ClusterRole	Default ClusterRoleBinding	Description
cluster-admin	system:masters group	Allows super-user access to perform any action on any resource. When used in a ClusterRoleBinding , it gives full control over every resource in the cluster and in all namespaces. When used in a RoleBinding , it gives full control over every resource in the rolebinding's namespace, including the namespace itself.
admin	None	Allows admin access, intended to be granted within a namespace using a RoleBinding . If used in a RoleBinding , allows read/write access to most resources in a namespace, including the ability to create roles and rolebindings within the namespace. It does not allow write access to resource quota or to the namespace itself.
edit	None	Allows read/write access to most objects in a namespace. It does not allow viewing or modifying roles or rolebindings.
view	None	Allows read-only access to see most objects in a namespace. It does not allow viewing roles or rolebindings. It does not allow viewing secrets, since those are escalating.

RBAC Bindings

- RBAC API declares two top-level types:
 - **RoleBinding** and **ClusterRoleBinding**
 - Grants permissions defined in a Role/ClusterRole to a user or set of users, a group, or ServiceAccount
 - RoleBinding may only reference a Role in the same namespace
 - May also reference a ClusterRole to grant the permissions to namespaced resources
 - Allows administrators to define a set of **common roles for the entire cluster**, then reuse them within multiple namespaces
 - If the ClusterRole grants permissions to list endpoints then the user would have the ability to list endpoints in their assigned namespace
 - ClusterRoleBinding may be used to grant permission at cluster level in all namespaces

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-svc
  namespace: default
subjects:
  - kind: User
    name: usera
roleRef:
  kind: Role
  name: svc-reader
  apiGroup: rbac.authorization.k8s.io

```

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: system:coredns
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: system:coredns
subjects:
  - kind: ServiceAccount
    name: coredns
    namespace: kube-system

```

Imperative command equivalent example:

```

kubectl create rolebinding read-svc \
--role=svc-reader --user=usera \
-n default

```

Admission Control

- Admission Control Modules are **software modules that can modify or reject requests** to the Kubernetes API
 - Only after the request is authenticated and authorized
- Two phases:
 - Mutating admission controllers modify the objects they admit
 - Then, validating admission controllers filter requests
- If any admission controller module rejects the request, then the request is immediately rejected
- Once a request passes all admission controllers, it is validated using the validation routines for the corresponding API object, and then written to the object store
- **Advanced features in Kubernetes require an admission controller** to be enabled in order to properly support the feature

Admission Controllers

- **AlwaysPullImages** - modifies every new Pod to force the image pull policy to Always
- **DefaultStorageClass** – observes creation of PersistentVolumeClaim objects that do not request any specific storage class and automatically adds a default storage class to them
- **DefaultTolerationSeconds** – sets the default forgiveness toleration for pods to tolerate the taints `notready:NoExecute` and `unreachable:NoExecute` for 5 minutes if the pods don't already have toleration for taints
- **DenyEscalatingExec** – will deny exec and attach commands to pods that run with escalated privileges that allow host access
- **EventRateLimit (alpha)** – mitigates the problem where the API server gets flooded by event requests by setting event rate limits
- **ExtendedResourceToleration** – facilitates creation of dedicated nodes with extended resources (like GPUs, FPGAs etc.)
- **ImagePolicyWebhook** – allows a backend webhook to make admission decisions
- **Initializers (alpha)** – used for dynamic admission control experimental feature
- **LimitPodHardAntiAffinityTopology** – denies any pod that defines AntiAffinity topology key other than `kubernetes.io/hostname`
- **LimitRanger** – ensures that a request does not violate any of the constraints enumerated in the `LimitRange` object in a Namespace
- **NamespaceAutoProvision** – creates a namespace if an object names a namespace that doesn't already exist
- **NamespaceExists** – checks all requests on namespaced resources; if the namespace from a request doesn't exist, the request is rejected
- **NamespaceLifecycle** – enforces that a Namespace that is undergoing termination cannot have new objects created in it
- **NodeRestriction** – limits the Node and Pod objects a kubelet can modify
- **OwnerReferencesPermissionEnforcement** – protects the access to the `metadata.ownerReferences` of an object so that only users with “delete” permission to the object can change it
- **PodNodeSelector** – defaults and limits what node selectors may be used within a namespace
- **PersistentVolumeClaimResize** – prevents resizing of all claims by default unless a claim's `StorageClass` explicitly enables resizing by setting `allowVolumeExpansion` to true
- **PodPreset** – injects a pod with the fields specified in a matching `PodPreset`
- **PodSecurityPolicy** – determines if a pod should be admitted based on the requested security context and the available Pod Security Policies
- **PodTolerationRestriction** – rejects a pod if there is any conflict between a pod's tolerations and its namespace's tolerations
- **Priority**– uses the `priorityClassName` field and populates the integer value of the priority; if the priority class is not found, the Pod is rejected
- **ResourceQuota** – ensures that a request does not violate any of the constraints enumerated in the `ResourceQuota` object in a Namespace
- **SecurityContextDeny** – deny any pod that attempts to set certain escalating `SecurityContext` fields
- **ServiceAccount** – implements automation for serviceAccounts
- **MutatingAdmissionWebhook (beta in 1.9)** – calls any mutating webhooks which match the request
- **ValidatingAdmissionWebhook (alpha in 1.8; beta in 1.9)** – calls any validating webhooks which match the request
- Current (K8s >= 1.10.0) recommendation for Admission Controller is:
 - `--admission-control=NamespaceLifecycle,LimitRanger,ServiceAccount,DefaultStorageClass, DefaultTolerationSeconds, MutatingAdmissionWebhook,ValidatingAdmissionWebhook,ResourceQuota`

Namespaces

- Intended for environments with users spread across multiple teams or projects
- Provide scope for names of objects
 - Naming need only be unique within a namespace
- Cannot be nested
- An single instance of a Kubernetes resource can only exist in a single namespace
 - Most, but not all, API objects are namespaced
 - To see what is/is not namespaced:
\$ kubectl api-resources --namespaced=true|false
- Divide cluster resources equally between teams/projects/namespaces via quotas and limits
- kubectl requests can be scoped by a namespace with the --namespace (-n) flag, ex:
\$ kubectl -n storage get pods
 - Permanently save the namespace for subsequent commands by setting it as part of the context:
\$ kubectl config set-context \
\$(kubectl config current-context) \
--namespace=storage

```
$ kubectl get namespaces
```

NAME	STATUS	AGE
default	Active	16h
kube-node-lease	Active	16h
kube-public	Active	16h
kube-system	Active	16h

```
$ kubectl create namespace marketing
```

```
namespace/marketing created
```

```
$ kubectl get ns
```

NAME	STATUS	AGE
default	Active	16h
kube-node-lease	Active	16h
kube-public	Active	16h
kube-system	Active	16h
marketing	Active	110s

Quotas & LimitRanges

156

Copyright 2013-2020, RX-M LLC

- Quotas provide constraints that **limit aggregate resource consumption** for namespaces in 2 ways:
 - Quantity of objects** by type
 - Counts of objects are charged against a given quota when the object exists in etcd (whether or not is actually deployed)
 - The 1.9 release added support to quota all standard namespaced resource types in the format: `count/<resource>.<api-group>`
 - Total amount of resources:**
 - CPU
 - Memory
 - Persistent and ephemeral storage
- Quotas may be scoped** to only apply to a resource that matches the scope enumeration
 - Ex. `NotBestEffort` – matches pods that are not "best effort" quality of service
- Static and independent of cluster capacity**
 - Manual adjustments will be required in autoscaling clusters
- LimitRange configures default requests and limits** enforced for a given namespace
 - When container(s) in a podspec do not specify their own values for requests and limits, the values held by the LimitRange setting for a namespace are applied

Imperative command equivalent example:

```
kubectl create quota myquota \
--hard=count/pods=20,count/service.loadbalancer=2
```

```
$ kubectl describe ns marketing
```

```
Name:          marketing
Labels:        <none>
Annotations:   <none>
Status:        Active
```

Resource Quotas

Name:	pod-count
Resource	Used Hard
limits.cpu	0 1500m
limits.memory	0 2Gi
pods	2 5
requests.cpu	0 1
requests.memory	0 1Gi

Resource Limits

Type	Resource	Min	Max	Default Request	Default Limit	Max Limit/Request Ratio
Container	cpu	-	-	250m	500m	-
Container	memory	-	-	128Mi	256Mi	-

Pod Security Policies

- Cluster-level resource to define conditions that a pod must run with in order to be accepted
- Enforced by an admission controller in the Kubernetes API
 - Also requires that the requesting user is authorized to use the policy, otherwise the policy itself does nothing
- privileged** – controls using privileged in securityContext
- allowPrivilegeEscalation** – gates whether or not a user is allowed to set this key in securityContext to true
- Capabilities** – define what capabilities can be used in securityContext
- volumes** – whitelist of volume types
- Host PID/IPC/Network** – disallows running pods in host namespaces
- runAsUser** – controls the what user ID containers run as
- seLinux** – requires seLinuxOptions to be configured
- supplementalGroups** – controls the which group IDs containers add
- fsGroup** – controls the supplemental group applied to volumes
- Others...

```

apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: restricted
  annotations:
    seccomp.security.alpha.kubernetes.io/allowedProfileNames: 'docker/default'
    apparmor.security.beta.kubernetes.io/allowedProfileNames: 'runtime/default'
spec:
  privileged: false
  # Required to prevent escalations to root.
  allowPrivilegeEscalation: false
  # Redundant with non-root + disallow privilege escalation
  requiredDropCapabilities:
    - ALL
  volumes:
    - 'configMap'
    - 'emptyDir'
    - 'projected'
    - 'secret'
    - 'downwardAPI'
    - 'persistentVolumeClaim'
  hostNetwork: false
  hostIPC: false
  hostPID: false
  runAsUser:
    rule: 'MustRunAsNonRoot'
  seLinux:
    # Assumes nodes are using AppArmor rather than SELinux
    rule: 'RunAsAny'
  supplementalGroups:
    rule: 'MustRunAs'
    ranges:
      # Forbid adding the root group
      - min: 1
        max: 65535
  fsGroup:
    rule: 'MustRunAs'
    ranges:
      # Forbid adding the root group
      - min: 1
        max: 65535

```

Security Context

- Defines privilege and access control settings for a Pod or Container
 - Security settings that you specify for a Container override settings made at the Pod level when there is overlap
- Security context settings include:
 - **Discretionary Access Control** – permission to access an object (like a file) is based on user ID (UID) and group ID (GID)
 - **runAsUser** field specifies that for any Containers in the Pod, the first process runs with a given user ID (1000 in the example)
 - **fsGroup** field specifies that a given group ID is associated with all Containers in the Pod (2000 in the example)
 - The group ID is also associated with the mounted volume (/data in the example) and any files created in that volume
 - **Security Enhanced Linux (SELinux)** – objects are assigned security labels
 - SELinux security module must be loaded on the host OS
 - Running as **privileged** or unprivileged
 - **Linux Capabilities** – give a process some privileges, but not all the privileges of the root user
 - **runAsNonRoot** – set containers to only run as non root user
 - **readOnlyRootFilesystem** – set container filesystem as read only

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-nginx
spec:
  securityContext:      # pod level security configs
    runAsUser: 1000
    fsGroup: 2000
  volumes:
    - name: sharedvol
      emptyDir: {}
  containers:
    - name: nginx
      image: nginx
      command: ["nginx", "-g", "daemon off;"]
      volumeMounts:
        - name: sharedvol
          mountPath: /data
      securityContext:  # container level security configs
        capabilities:
          add: ["NET_ADMIN", "SYS_TIME"]
        selinuxOptions:
          level: "s0:c123,c456"
        readOnlyRootFilesystem: true
        runAsNonRoot: true
```

Capabilities

- 1FFFFFFFFF
 - Mask with 37 positions
 - One for each of the 37 capabilities
 - See: <https://github.com/torvalds/linux/blob/master/include/linux/capability.h>
- Capability Sets
 - CapInh – **Inheritable**: capabilities preserved across an execve
 - CapPrm – **Permitted**: limiting superset of capabilities
 - CapEff – **Effective**: capabilities used for permission checks
 - CapBnd – **Capability Bounding**: limiting superset for the capabilities that a thread can add to its inheritable set, constraining the capabilities that may be granted by an executable file
- pscap
 - Used to get information about the enabled capabilities of running processes
 - '+' indicates open ended bounding set
- capsh
 - Offers many capability-related features, including decoding the status fields

```
root@ubuntu:~# grep Cap /proc/self/status
CapInh: 0000000000000000
CapPrm: 0000001fffffffff
CapEff: 0000001fffffffff
CapBnd: 0000001fffffffff
```

```
root@ubuntu:~# exit
```

```
logout
```

```
user@ubuntu:~$ grep Cap /proc/self/status
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 0000001fffffffff
```

```
user@ubuntu:~$ capsh --print
```

```
Current: =
```

```
Bounding set = cap_chown,cap_dac_override,cap_setuid,cap_setpcap,cap_linux_immutable,w,cap_ipc_lock,cap_ipc_owner,cap_sys_module,sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_ap_audit_write,cap_audit_control,cap_setfcablock_suspend
```

```
Securebits: 00/0x0/1'b0
```

```
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
```

```
uid=1000(user)
```

```
gid=1000(user)
```

```
groups=4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),109(lpadmin),110(sambashare),999(docker),1000(user)
```

```
user@ubuntu:~$ kubectl run -it mycapod --image=ubuntu
```

```
root@mycapod-568f8c6946-2tpk8:/# grep Cap /proc/self/status
```

```
CapInh: 00000000a80425fb
```

```
CapPrm: 00000000a80425fb
```

```
CapEff: 00000000a80425fb
```

```
CapBnd: 00000000a80425fb
```

```
root@mycapod-568f8c6946-2tpk8:/# capsh --print
```

```
Current: = cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_mknod,cap_audit_write,cap_setfcap+eip
```

```
Bounding set = cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_mknod,cap_audit_write,cap_setfcap
```

```
Securebits: 00/0x0/1'b0
```

```
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
```

```
uid=0(root)
```

```
gid=0(root)
```

```
groups=
```

```
root@mycapod-568f8c6946-2tpk8:/# capsh --decode=00000000a80425fb
```

```
0x00000000a80425fb=cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_mknod,cap_audit_write,cap_setfcap
```

```
user@ubuntu:~$ pscap
```

ppid	pid	name	command	capabilities
1	445	root	upstart-udev-br	full
1	451	root	systemd-udev	full
1	588	root	upstart-socket-	full
1	777	root	upstart-file-br	full
1	832	root	systemd-logind	full
1	834	root	bluetoothd	full
1	873	root	dhclient	dac_override, net_bind_service, net_admin, net_raw, sys_module +

1	946	root	docker	full
1	1062	root	getty	full
1	1236	root	cron	full
1415	1426	root	Xorg	full
1	1429	root	accounts-daemon	full
1	1434	root	polkitd	full
1415	1491	root	lightdm	full
1	1510	root	upowerd	full
1570	1670	user	gnome-keyring-d	ipc_lock +
1	1967	root	udisksd	full
946	11117	root	bash	chown, dac_override, fowner, fsetid, kill, setgid, setuid, setpcap, net_bind_service, net_raw, sys_chroot, mknod, audit_write, setfcap
1	14049	root	packagekitd	full

Ubuntu container

AppArmor (beta as of v1.4)

- Currently the only Kubernetes-supported container runtime that also supports AppArmor is Docker
- AppArmor is applied to a Pod by specifying an AppArmor profile that each container should be run with
 - If a specified profile is not already loaded in the kernel, Kubelet will reject the Pod
- Currently in beta, so options are specified as annotations
 - Valid annotation values:
 - runtime/default** – the default profile for the container runtime
 - unconfined** – no profiles will be loaded
 - localhost/<profile-name>** - apply the profile loaded on the host
 - AppArmor profile generator for Docker containers:
<https://github.com/jessfraz/bane>

```
// +build linux

package apparmor

// baseTemplate defines the default apparmor profile for containers.

const baseTemplate = `
{{range $value := .Imports}}
{{$value}}
{{end}}
profile {{.Name}} flags=(attach_disconnected,mediate_deleted) {
{{range $value := .InnerImports}}
  {{$value}}
{{end}}

  network,
  capability,
  file,
  umount,

  deny @{PROC}/* w, # deny write for all files directly in /proc (not in a subdir)
  # deny write to files not in /proc/<number>/** or /proc/sys/**
  deny @{PROC}/{[^1-9],[^1-9][^0-9],[^1-9s][^0-9y][^0-9s],[^1-9][^0-9][^0-9]*}/** w,
  deny @{PROC}/sys/[^k]** w, # deny /proc/sys except /proc/sys/k* (effectively /proc/sys/kernel)
  deny @{PROC}/sys/kernel/{?,?,[^s][^h][^m]**} w, # deny everything except shm* in /proc/sys/kernel/
  deny @{PROC}/sysrq-trigger rwklx,
  deny @{PROC}/mem rwklx,
  deny @{PROC}/kmem rwklx,
  deny @{PROC}/kcore rwklx,

  deny mount,

  deny /sys/[^f]** wklx,
  deny /sys/f[^s]** wklx,
  deny /sys/fs/[^c]** wklx,
  deny /sys/fs/c[^g]** wklx,
  deny /sys/fs/cg[^r]** wklx,
  deny /sys/firmware/** rwklx,
  deny /sys/kernel/security/** rwklx,

  {{if ge .Version 208095}}
    # suppress ptrace denials when using 'docker ps' or using 'ps' inside a container
    ptrace (trace,read) peer={{.Name}},
  {{end}}
}
`
```

The Docker binary installs a default AppArmor profile generated from the template at:
<https://github.com/moby/moby/blob/master/profiles/apparmor/template.go>

Moderately protective while providing wide application compatibility

```
apiVersion: v1
kind: Pod
metadata:
  name: apparmor-nginx
  annotations:
    container.apparmor.security.beta.kubernetes.io/nginx: runtime/default
spec:
  containers:
  - name: nginx
    image: nginx
    command: ["nginx", "-g", "daemon off;"]
  ...
```

SecComp (alpha as of v1.3)

- Secure computing mode (seccomp)
- Linux kernel feature to **restrict syscalls** available to a process
 - Originally merged in 2005 kernel 2.6.12
- Since version 1.10 Docker supports Seccomp Profiles
- When enabled, Docker provides a default seccomp profile
 - A whitelist which specifies the calls that are allowed
- Specify a Seccomp profile for all containers of a **Pod**:
 - `seccomp.security.alpha.kubernetes.io/pod`
- Specify a Seccomp profile for an individual **container**:
 - `container.seccomp.security.alpha.kubernetes.io/<container_name>`
- Currently in alpha, so options are specified as annotations
 - Valid annotation values:
 - runtime/default** – the default profile for the container runtime
 - unconfined** – disable Seccomp sandboxing
 - localhost/<profile-name>** - the profile installed to the node's local seccomp profile root
 - The default folder for profiles is `/var/lib/kubelet/seccomp`

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-nginx
  annotations:
    container.seccomp.security.alpha.kubernetes.io/nginx: localhost/myscpol
spec:
  containers:
  - name: nginx
    image: nginx
    command: ["nginx", "-g", "daemon off;"]
  ...
```

GitHub, Inc. (US) | https://github.com/moby/moby/blob/master/profiles/seccomp/seccomp_default.go

Full whitelist in Docker source

```
40 }
41 }
42
43 // DefaultProfile defines the whitelist for the default seccomp profile.
44 func DefaultProfile() *types.Seccomp {
45     syscalls := []types.Syscall{
46         {
47             Names: []string{
48                 "accept",
49                 "accept4",
50                 "access",
51                 "adjtimex",
52                 "alarm",
53                 "bind",
54                 "brk",
55                 "capget",
56                 "capset",
57                 "chdir",
58                 "chmod",
59                 "chown",
60                 "chown32",
61                 "clock_getres",
62                 "clock_gettime",
63                 "clock_nanosleep",
64                 "close",
65                 "clone",
66                 "clone3",
67                 "connect",
68                 "copy_file_range",
69                 "creat",
70                 "creat64",
71                 "dup",
72                 "dup2",
73                 "dup3",
74                 "execve",
75                 "execveat",
76                 "fcntl",
77                 "fcntl64",
78                 "fchmod",
79                 "fchmodat",
80                 "fchown",
81                 "fchownat",
82                 "fdopen",
83                 "fdopen64",
84                 "fexecve",
85                 "fexecveat",
86                 "fgetxattr",
87                 "finit_module",
88                 "fork",
89                 "forkpty",
90                 "fsopen",
91                 "fstat",
92                 "fstat64",
93                 "fstatfs",
94                 "fstatfs64",
95                 "fsync",
96                 "fsync_range",
97                 "ftruncate",
98                 "ftruncate64",
99                 "futex",
100                futex64,
101                futexat,
102                futexat64,
103                getcwd,
104                getdents,
105                getdents64,
106                getegid,
107                geteuid,
108                getgid,
109                getgroups,
110                getitimer,
111                getpeername,
112                getpgid,
113                getpid,
114                getpid64,
115                getppid,
116                getpriority,
117                getrandom,
118                getrlimit,
119                getrlimit64,
120                getrusage,
121                getsockname,
122                getsockopt,
123                gettimeofday,
124                getxattr,
125                init_module,
126                inotify_add_watch,
127                inotify_rm_watch,
128                ioctl,
129                ioctl64,
130                ioeventfd,
131                ioeventfd4,
132                kill,
133                killpg,
134                killproc,
135                killproc2,
136                killproc3,
137                lchown,
138                lchown32,
139                linkat,
140                listen,
141                listen64,
142                listxattr,
143                lseek,
144                lseek64,
145                lstat,
146                lstat64,
147                lutimes,
148                lutimes64,
149                madvise,
150                madvise64,
151                malloc,
152                malloc2,
153                memfd_create,
154                memfd_create2,
155                memfd_secret,
156                mincore,
157                mincore64,
158                mkdirat,
159                mkdirat64,
160                mknodat,
161                mknodat64,
162                mlock,
163                mlock2,
164                mlock2all,
165                mlockall,
166                mlockall2,
167                mlockall3,
168                mlockall4,
169                mlockall5,
170                mlockall6,
171                mlockall7,
172                mlockall8,
173                mlockall9,
174                mlockall10,
175                mlockall11,
176                mlockall12,
177                mlockall13,
178                mlockall14,
179                mlockall15,
180                mlockall16,
181                mlockall17,
182                mlockall18,
183                mlockall19,
184                mlockall20,
185                mlockall21,
186                mlockall22,
187                mlockall23,
188                mlockall24,
189                mlockall25,
190                mlockall26,
191                mlockall27,
192                mlockall28,
193                mlockall29,
194                mlockall30,
195                mlockall31,
196                mlockall32,
197                mlockall33,
198                mlockall34,
199                mlockall35,
200                mlockall36,
201                mlockall37,
202                mlockall38,
203                mlockall39,
204                mlockall40,
205                mlockall41,
206                mlockall42,
207                mlockall43,
208                mlockall44,
209                mlockall45,
210                mlockall46,
211                mlockall47,
212                mlockall48,
213                mlockall49,
214                mlockall50,
215                mlockall51,
216                mlockall52,
217                mlockall53,
218                mlockall54,
219                mlockall55,
220                mlockall56,
221                mlockall57,
222                mlockall58,
223                mlockall59,
224                mlockall60,
225                mlockall61,
226                mlockall62,
227                mlockall63,
228                mlockall64,
229                mlockall65,
230                mlockall66,
231                mlockall67,
232                mlockall68,
233                mlockall69,
234                mlockall70,
235                mlockall71,
236                mlockall72,
237                mlockall73,
238                mlockall74,
239                mlockall75,
240                mlockall76,
241                mlockall77,
242                mlockall78,
243                mlockall79,
244                mlockall80,
245                mlockall81,
246                mlockall82,
247                mlockall83,
248                mlockall84,
249                mlockall85,
250                mlockall86,
251                mlockall87,
252                mlockall88,
253                mlockall89,
254                mlockall90,
255                mlockall91,
256                mlockall92,
257                mlockall93,
258                mlockall94,
259                mlockall95,
260                mlockall96,
261                mlockall97,
262                mlockall98,
263                mlockall99,
264                mlockall100,
265                mlockall101,
266                mlockall102,
267                mlockall103,
268                mlockall104,
269                mlockall105,
270                mlockall106,
271                mlockall107,
272                mlockall108,
273                mlockall109,
274                mlockall110,
275                mlockall111,
276                mlockall112,
277                mlockall113,
278                mlockall114,
279                mlockall115,
280                mlockall116,
281                mlockall117,
282                mlockall118,
283                mlockall119,
284                mlockall120,
285                mlockall121,
286                mlockall122,
287                mlockall123,
288                mlockall124,
289                mlockall125,
290                mlockall126,
291                mlockall127,
292                mlockall128,
293                mlockall129,
294                mlockall130,
295                mlockall131,
296                mlockall132,
297                mlockall133,
298                mlockall134,
299                mlockall135,
300                mlockall136,
301                mlockall137,
302                mlockall138,
303                mlockall139,
304                mlockall140,
305                mlockall141,
306                mlockall142,
307                mlockall143,
308                mlockall144,
309                mlockall145,
310                mlockall146,
311                mlockall147,
312                mlockall148,
313                mlockall149,
314                mlockall150,
315                mlockall151,
316                mlockall152,
317                mlockall153,
318                mlockall154,
319                mlockall155,
320                mlockall156,
321                mlockall157,
322                mlockall158,
323                mlockall159,
324                mlockall160,
325                mlockall161,
326                mlockall162,
327                mlockall163,
328                mlockall164,
329                mlockall165,
330                mlockall166,
331                mlockall167,
332                mlockall168,
333                mlockall169,
334                mlockall170,
335                mlockall171,
336                mlockall172,
337                mlockall173,
338                mlockall174,
339                mlockall175,
340                mlockall176,
341                mlockall177,
342                mlockall178,
343                mlockall179,
344                mlockall180,
345                mlockall181,
346                mlockall182,
347                mlockall183,
348                mlockall184,
349                mlockall185,
350                mlockall186,
351                mlockall187,
352                mlockall188,
353                mlockall189,
354                mlockall190,
355                mlockall191,
356                mlockall192,
357                mlockall193,
358                mlockall194,
359                mlockall195,
360                mlockall196,
361                mlockall197,
362                mlockall198,
363                mlockall199,
364                mlockall200,
365                mlockall201,
366                mlockall202,
367                mlockall203,
368                mlockall204,
369                mlockall205,
370                mlockall206,
371                mlockall207,
372                mlockall208,
373                mlockall209,
374                mlockall210,
375                mlockall211,
376                mlockall212,
377                mlockall213,
378                mlockall214,
379                mlockall215,
380                mlockall216,
381                mlockall217,
382                mlockall218,
383                mlockall219,
384                mlockall220,
385                mlockall221,
386                mlockall222,
387                mlockall223,
388                mlockall224,
389                mlockall225,
390                mlockall226,
391                mlockall227,
392                mlockall228,
393                mlockall229,
394                mlockall230,
395                mlockall231,
396                mlockall232,
397                mlockall233,
398                mlockall234,
399                mlockall235,
400                mlockall236,
401                mlockall237,
402                mlockall238,
403                mlockall239,
404                mlockall240,
405                mlockall241,
406                mlockall242,
407                mlockall243,
408                mlockall244,
409                mlockall245,
410                mlockall246,
411                mlockall247,
412                mlockall248,
413                mlockall249,
414                mlockall250,
415                mlockall251,
416                mlockall252,
417                mlockall253,
418                mlockall254,
419                mlockall255,
420                mlockall256,
421                mlockall257,
422                mlockall258,
423                mlockall259,
424                mlockall260,
425                mlockall261,
426                mlockall262,
427                mlockall263,
428                mlockall264,
429                mlockall265,
430                mlockall266,
431                mlockall267,
432                mlockall268,
433                mlockall269,
434                mlockall270,
435                mlockall271,
436                mlockall272,
437                mlockall273,
438                mlockall274,
439                mlockall275,
440                mlockall276,
441                mlockall277,
442                mlockall278,
443                mlockall279,
444                mlockall280,
445                mlockall281,
446                mlockall282,
447                mlockall283,
448                mlockall284,
449                mlockall285,
450                mlockall286,
451                mlockall287,
452                mlockall288,
453                mlockall289,
454                mlockall290,
455                mlockall291,
456                mlockall292,
457                mlockall293,
458                mlockall294,
459                mlockall295,
460                mlockall296,
461                mlockall297,
462                mlockall298,
463                mlockall299,
464                mlockall300,
465                mlockall301,
466                mlockall302,
467                mlockall303,
468                mlockall304,
469                mlockall305,
470                mlockall306,
471                mlockall307,
472                mlockall308,
473                mlockall309,
474                mlockall310,
475                mlockall311,
476                mlockall312,
477                mlockall313,
478                mlockall314,
479                mlockall315,
480                mlockall316,
481                mlockall317,
482                mlockall318,
483                mlockall319,
484                mlockall320,
485                mlockall321,
486                mlockall322,
487                mlockall323,
488                mlockall324,
489                mlockall325,
490                mlockall326,
491                mlockall327,
492                mlockall328,
493                mlockall329,
494                mlockall330,
495                mlockall331,
496                mlockall332,
497                mlockall333,
498                mlockall334,
499                mlockall335,
500                mlockall336,
501                mlockall337,
502                mlockall338,
503                mlockall339,
504                mlockall340,
505                mlockall341,
506                mlockall342,
507                mlockall343,
508                mlockall344,
509                mlockall345,
510                mlockall346,
511                mlockall347,
512                mlockall348,
513                mlockall349,
514                mlockall350,
515                mlockall351,
516                mlockall352,
517                mlockall353,
518                mlockall354,
519                mlockall355,
520                mlockall356,
521                mlockall357,
522                mlockall358,
523                mlockall359,
524                mlockall360,
525                mlockall361,
526                mlockall362,
527                mlockall363,
528                mlockall364,
529                mlockall365,
530                mlockall366,
531                mlockall367,
532                mlockall368,
533                mlockall369,
534                mlockall370,
535                mlockall371,
536                mlockall372,
537                mlockall373,
538                mlockall374,
539                mlockall375,
540                mlockall376,
541                mlockall377,
542                mlockall378,
543                mlockall379,
544                mlockall380,
545                mlockall381,
546                mlockall382,
547                mlockall383,
548                mlockall384,
549                mlockall385,
550                mlockall386,
551                mlockall387,
552                mlockall388,
553                mlockall389,
554                mlockall390,
555                mlockall391,
556                mlockall392,
557                mlockall393,
558                mlockall394,
559                mlockall395,
560                mlockall396,
561                mlockall397,
562                mlockall398,
563                mlockall399,
564                mlockall400,
565                mlockall401,
566                mlockall402,
567                mlockall403,
568                mlockall404,
569                mlockall405,
570                mlockall406,
571                mlockall407,
572                mlockall408,
573                mlockall409,
574                mlockall410,
575                mlockall411,
576                mlockall412,
577                mlockall413,
578                mlockall414,
579                mlockall415,
580                mlockall416,
581                mlockall417,
582                mlockall418,
583                mlockall419,
584                mlockall420,
585                mlockall421,
586                mlockall422,
587                mlockall423,
588                mlockall424,
589                mlockall425,
590                mlockall426,
591                mlockall427,
592                mlockall428,
593                mlockall429,
594                mlockall430,
595                mlockall431,
596                mlockall432,
597                mlockall433,
598                mlockall434,
599                mlockall435,
600                mlockall436,
601                mlockall437,
602                mlockall438,
603                mlockall439,
604                mlockall440,
605                mlockall441,
606                mlockall442,
607                mlockall443,
608                mlockall444,
609                mlockall445,
610                mlockall446,
611                mlockall447,
612                mlockall448,
613                mlockall449,
614                mlockall450,
615                mlockall451,
616                mlockall452,
617                mlockall453,
618                mlockall454,
619                mlockall455,
620                mlockall456,
621                mlockall457,
622                mlockall458,
623                mlockall459,
624                mlockall460,
625                mlockall461,
626                mlockall462,
627                mlockall463,
628                mlockall464,
629                mlockall465,
630                mlockall466,
631                mlockall467,
632                mlockall468,
633                mlockall469,
634                mlockall470,
635                mlockall471,
636                mlockall472,
637                mlockall473,
638                mlockall474,
639                mlockall475,
640                mlockall476,
641                mlockall477,
642                mlockall478,
643                mlockall479,
644                mlockall480,
645                mlockall481,
646                mlockall482,
647                mlockall483,
648                mlockall484,
649                mlockall485,
650                mlockall486,
651                mlockall487,
652                mlockall488,
653                mlockall489,
654                mlockall490,
655                mlockall491,
656                mlockall492,
657                mlockall493,
658                mlockall494,
659                mlockall495,
660                mlockall496,
661                mlockall497,
662                mlockall498,
663                mlockall499,
664                mlockall500,
665                mlockall501,
666                mlockall502,
667                mlockall503,
668                mlockall504,
669                mlockall505,
670                mlockall506,
671                mlockall507,
672                mlockall508,
673                mlockall509,
674                mlockall510,
675                mlockall511,
676                mlockall512,
677                mlockall513,
678                mlockall514,
679                mlockall515,
680                mlockall516,
681                mlockall517,
682                mlockall518,
683                mlockall519,
684                mlockall520,
685                mlockall521,
686                mlockall522,
687                mlockall523,
688                mlockall524,
689                mlockall525,
690                mlockall526,
691                mlockall527,
692                mlockall528,
693                mlockall529,
694                mlockall530,
695                mlockall531,
696                mlockall532,
697                mlockall533,
698                mlockall534,
699                mlockall535,
700                mlockall536,
701                mlockall537,
702                mlockall538,
703                mlockall539,
704                mlockall540,
705                mlockall541,
706                mlockall542,
707                mlockall543,
708                mlockall544,
709                mlockall545,
710                mlockall546,
711                mlockall547,
712                mlockall548,
713                mlockall549,
714                mlockall550,
715                mlockall551,
716                mlockall552,
717                mlockall553,
718                mlockall554,
719                mlockall555,
720                mlockall556,
721                mlockall557,
722                mlockall558,
723                mlockall559,
724                mlockall560,
725                mlockall561,
726                mlockall562,
727                mlockall563,
728                mlockall564,
729                mlockall565,
730                mlockall566,
731                mlockall567,
732                mlockall568,
733                mlockall569,
734                mlockall570,
735                mlockall571,
736                mlockall572,
737                mlockall573,
738                mlockall574,
739                mlockall575,
740                mlockall576,
741                mlockall577,
742                mlockall578,
743                mlockall579,
744                mlockall580,
745                mlockall581,
746                mlockall582,
747                mlockall583,
748                mlockall584,
749                mlockall585,
750                mlockall586,
751                mlockall587,
752                mlockall588,
753                mlockall589,
754                mlockall590,
755                mlockall591,
756                mlockall592,
757                mlockall593,
758                mlockall594,
759                mlockall595,
760                mlockall596,
761                mlockall597,
762                mlockall598,
763                mlockall599,
764                mlockall600,
765                mlockall601,
766                mlockall602,
767                mlockall603,
768                mlockall604,
769                mlockall605,
770                mlockall606,
771                mlockall607,
772                mlockall608,
773                mlockall609,
774                mlockall610,
775                mlockall611,
776                mlockall612,
777                mlockall613,
778                mlockall614,
779                mlockall615,
780                mlockall616,
781                mlockall617,
782                mlockall618,
783                mlockall619,
784                mlockall620,
785                mlockall621,
786                mlockall622,
787                mlockall623,
788                mlockall624,
789                mlockall625,
790                mlockall626,
791                mlockall627,
792                mlockall628,
793                mlockall629,
794                mlockall630,
795                mlockall631,
796                mlockall632,
797                mlockall633,
798                mlockall634,
799                mlockall635,
800                mlockall636,
801                mlockall637,
802                mlockall638,
803                mlockall639,
804                mlockall640,
805                mlockall641,
806                mlockall642,
807                mlockall643,
808                mlockall644,
809                mlockall645,
810                mlockall646,
811                mlockall647,
812                mlockall648,
813                mlockall649,
814                mlockall650,
815                mlockall651,
816                mlockall652,
817                mlockall653,
818                mlockall654,
819                mlockall655,
820                mlockall656,
821                mlockall657,
822                mlockall658,
823                mlockall659,
824                mlockall660,
825                mlockall661,
826                mlockall662,
827                mlockall663,
828                mlockall664,
829                mlockall665,
830                mlockall666,
831                mlockall667,
832                mlockall668,
833                mlockall669,
834                mlockall670,
835                mlockall671,
836                mlockall672,
837                mlockall673,
838                mlockall674,
839                mlockall675,
840                mlockall676,
841                mlockall677,
842                mlockall678,
843                mlockall679,
844                mlockall680,
845                mlockall681,
846                mlockall682,
847                mlockall683,
848                mlockall684,
849                mlockall685,
850                mlockall686,
851                mlockall687,
852                mlockall688,
853                mlockall689,
854                mlockall690,
855                mlockall691,
856                mlockall692,
857                mlockall693,
858                mlockall694,
859                mlockall695,
860                mlockall696,
861                mlockall697,
862                mlockall698,
863                mlockall699,
864                mlockall700,
865                mlockall701,
866                mlockall702,
867                mlockall703,
868                mlockall704,
869                mlockall705,
870                mlockall706,
871                mlockall707,
872                mlockall708,
873                mlockall709,
874                mlockall710,
875                mlockall711,
876                mlockall712,
877                mlockall713,
878                mlockall714,
879                mlockall715,
880                mlockall716,
881                mlockall717,
882                mlockall718,
883                mlockall719,
884                mlockall720,
885                mlockall721,
886                mlockall722,
887                mlockall723,
888                mlockall724,
889                mlockall725,
890                mlockall726,
891                mlockall727,
892                mlockall728,
893                mlockall729,
894                mlockall730,
895                mlockall731,
896                mlockall732,
897                mlockall733,
898                mlockall734,
899                mlockall735,
900                mlockall736,
901                mlockall737,
902                mlockall738,
903                mlockall739,
904                mlockall740,
905                mlockall741,
906                mlockall742,
907                mlockall743,
908                mlockall744,
909                mlockall745,
910                mlockall746,
911                mlockall747,
912                mlockall748,
913                mlockall749,
914                mlockall750,
915                mlockall751,
916                mlockall752,
917                mlockall753,
918                mlockall754,
919                mlockall755,
920                mlockall756,
921                mlockall757,
922                mlockall758,
923                mlockall759,
924                mlockall760,
925                mlockall761,
926                mlockall762,
927                mlockall763,
928                mlockall764,
929                mlockall765,
930                mlockall766,
931                mlockall767,
932                mlockall768,
933                mlockall769,
934                mlockall770,
935                mlockall771,
936                mlockall772,
937                mlockall773,
938                mlockall774,
939                mlockall775,
940                mlockall776,
941                mlockall777,
942                mlockall778,
943                mlockall779,
944                mlockall780,
945                mlockall781,
946                mlockall782,
947                mlockall783,
948                mlockall784,
949                mlockall785,
950                mlockall786,
951                mlockall787,
952                mlockall788,
953                mlockall789,
954                mlockall790,
955                mlockall791,
956                mlockall792,
957                mlockall793,
958                mlockall794,
959                mlockall795,
960                mlockall796,
961                mlockall797,
962                mlockall798,
963                mlockall799,
964                mlockall800,
965                mlockall801,
966                mlockall802,
967                mlockall803,
968                mlockall804,
969                mlockall805,
970                mlockall806,
971                mlockall807,
972                mlockall808,
973                mlockall809,
974                mlockall810,
975                mlockall811,
976                mlockall812,
977                mlockall813,
978                mlockall814,
979                mlockall815,
980                mlockall816,
981                mlockall817,
982                mlockall818,
983                mlockall819,
984                mlockall820,
985                mlockall821,
986                mlockall822,
987                mlockall823,
988                mlockall824,
989                mlockall825,
990                mlockall826,
991                mlockall827,
992                mlockall828,
993                mlockall829,
994                mlockall830,
995                mlockall831,
996                mlockall832,
997                mlockall833,
998                mlockall834,
999                mlockall835,
1000               mlockall836,
1001               mlockall837,
1002               mlockall838,
1003               mlockall839,
1004               mlockall840,
1005               mlockall841,
1006               mlockall842,
1007               mlockall843,
1008               mlockall844,
1009               mlockall845,
1010               mlockall846,
1011               mlockall847,
1012               mlockall848,
1013               mlockall849,
1014               mlockall850,
1015               mlockall851,
1016               mlockall852,
1017               mlockall853,
1018               mlockall854,
1019               mlockall855,
1020               mlockall856,
1021               mlockall857,
1022               mlockall858,
1023               mlockall859,
1024               mlockall860,
1025               mlockall861,
1026               mlockall862,
1027               mlockall863,
1028               mlockall864,
1029               mlockall865,
1030               mlockall866,
1031               mlockall867,
1032               mlockall868,
1033               mlockall869,
1034               mlockall870,
1035               mlockall871,
1036               mlockall872,
1037               mlockall873,
1038               mlockall874,
1039               mlockall875,
1040               mlockall876,
1041               mlockall877,
1042               mlockall878,
1043               mlockall879,
1044               mlockall880,
1045               mlockall881,
1046               mlockall882,
1047               mlockall883,
1048               mlockall884,
1049               mlockall885,
1050               mlockall886,
1051               mlockall887,
1052               mlockall888,
1053               mlockall889,
1054               mlockall890,
1055               mlockall891,
1056               mlockall892,
1057               mlockall893,
1058               mlockall894,
1059               mlockall895,
106
```


K8s Security Benchmark

kube-bench by Aqua Security

- The Kubernetes Bench for Security is a Go application that checks whether Kubernetes is deployed securely by running the checks documented in the CIS Kubernetes Benchmark
- Tests are configured with YAML files
- kube-bench supports the tests for multiple versions of Kubernetes (1.6, 1.7 and 1.8) as defined in the CIS Benchmarks 1.0.0, 1.1.0 and 1.2.0 respectively
- <https://github.com/aquasecurity/kube-bench>

docker-bench-security

- The Docker Bench for Security is a containerized script that checks for dozens of common best-practices around deploying Docker containers in production
- Available for Docker 1.6, 1.11, 1.12 and 17.xx as 1.1.0
- <https://dockerbench.com>

aquasecurity / kube-bench

Code

The Kubernetes best practices

185 commits

Branch: master

lizrice Merge

cfg

check

cmd

hooks

images

.gitignore

.travis.yml

Dockerfile

LICENSE

OWNERS

README

entrypoint

main.go

docker / docker-bench-security

Code

Issues 11

Pull requests 7

Pulse

Graphs

The Docker Bench for Security is a script that checks for dozens of common best-practices a production. <https://dockerbench.com>

268 commits

3 branches

0 releases

Branch: master

New pull request

Create new

konstruktoid committed on GitHub Merge pull request #165 from konstruktoid/version

distros	use debian default docker.io
tests	1.12.1
.dockerignore	add .dockerignore
.gitignore	First version of the CIS Docker Benchmark v1.0.0
CONTRIBUTING.md	mention 1.11 benchmark instead of 1.6



[INFO] 1 Master Node Security Configuration

[INFO] 1.1 API Server

[FAIL] 1.1.1 Ensure th

[FAIL] 1.1.2 Ensure th

[PASS] 1.1.3 Ensure th

[PASS] 1.1.4 Ensure th

[FAIL] 1.1.5 Ensure th

[PASS] 1.1.6 Ensure th

[PASS] 1.1.7 Ensure th

[PASS] 1.1.8 Ensure th

[FAIL] 1.1.9 Ensure th

[FAIL] 1.1.10 Ensure t

[PASS] 1.1.11 Ensure t

[FAIL] 1.1.12 Ensure t

[FAIL] 1.1.13 Ensure t

[FAIL] 1.1.14 Ensure t

[PASS] 1.1.15 Ensure t

[FAIL] 1.1.16 Ensure t

[FAIL] 1.1.17 Ensure t

[FAIL] 1.1.18 Ensure t

[FAIL] 1.1.19 Ensure t

[PASS] 1.1.20 Ensure t

[PASS] 1.1.21 Ensure t

[FAIL] 1.1.22 Ensure t

```
# Docker Bench for Security v1.0.0
# Docker, Inc. (c) 2015-
# Checks for dozens of common best-
# Inspired by the CIS Docker 1.11 B
# https://benchmarks.cisecurity.org
```

Initializing Sat Apr 30 23:04:50 CE

```
[INFO] 1 - Host Configuration
[WARN] 1.1 - Create a separate par
[PASS] 1.2 - Use an updated Linux
[PASS] 1.4 - Remove all non-essent
[PASS] 1.5 - Keep Docker up to dat
[INFO] * Using 1.12.0 which i
[INFO] * Check with your oper
[INFO] 1.6 - Only allow trusted us
[INFO] * docker:x:999:tsj
[PASS] 1.7 - Audit docker daemon -
[PASS] 1.8 - Audit Docker files an
[PASS] 1.9 - Audit Docker files and
[PASS] 1.10 - Audit Docker files and
[PASS] 1.11 - Audit Docker files and
[PASS] 1.12 - Audit Docker files and
[INFO] 1.13 - Audit Docker files and
[INFO] * File not found
[PASS] 1.14 - Audit Docker files and
[PASS] 1.15 - Audit Docker files and
```

CIS Kubernetes Benchmark

v1.2.0 - 10-04-2017

CIS Docker Community Edition Benchmark

v1.1.0 - 07-06-2017

Summary

- Access to apiserver & API resources performed in several steps:
 - Authentication, authorization, admission control
 - The go-forward authorization mechanism is RBAC
- Workloads can use service accounts to access cluster resources
- Security leverages several sets of tooling in K8s
 - Linux capabilities
 - AppArmor
 - SELinux
 - Seccomp

Lab 7

- Namespaces, quotas, limitRanges and admission controllers

Lab 7 Documentation

References

165

Copyright 2013-2020, RX-M LLC

- Namespaces
 - <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>
 - <https://kubernetes.io/docs/tasks/administer-cluster/namespaces-walkthrough/>
- Resource Quotas
 - <https://kubernetes.io/docs/concepts/policy/resource-quotas/>
 - <https://kubernetes.io/docs/tasks/administer-cluster/quota-api-object/>
- Admission control
 - <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>
 - <https://kubernetes.io/blog/2019/03/21/a-guide-to-kubernetes-admission-controllers/>
- PodSecurityPolicy
 - <https://kubernetes.io/docs/reference/access-authn-authz/authorization/>
 - <https://kubernetes.io/docs/concepts/policy/pod-security-policy/>

8. Observability

Objectives

- Examine methods to monitor and aggregate metrics about Nodes and Pods
- Discuss the Metrics APIs
- Understand Docker logging and logging drivers
- Explore strategies for application logging
- Learn about common debugging methods for Pods, Services and cluster components

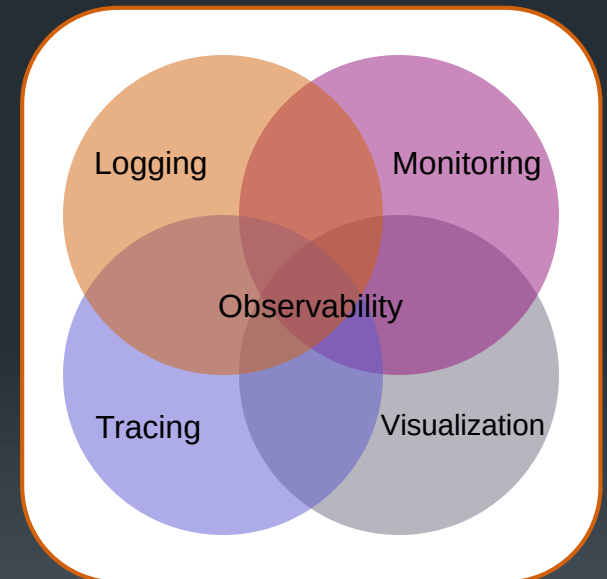
Observability

168

Copyright 2013-2020, RX-M LLC

- **Observability**
 - What is required in order to “see” the behavior of applications and infrastructure
- **Distributed Systems**
 - In distributed systems, to “see” your application running you need data from many places
 - “Seeing” behavior in distributed systems requires the collection of data from many physical systems and logical layers
 - It hard to “see” what is happening in distributed systems with traditional, single host oriented solutions
- **Cloud native observability** is based on three independent sets of data:
 - Metrics
 - Logs
 - Traces
- **Visualization**
 - Tools that display data with charts, graphs, tables and other means are called visualization systems

Metrics, logging and tracing are all cross cutting concerns (best implemented in one way cluster wide)



Weaveworks
<https://www.weave.works/technologies/monitoring-kubernetes-with-prometheus/>

Tracing

- A trace tracks an **execution path through a system** from component to component
 - Tracing is critical in distributed systems (**distributed tracing**) as application features used by clients typically require multiple services interacting with each other
- Tracing can be considered a **specialized use of logging to record information about a program's execution**
 - There is not always a clear distinction between tracing and other forms of logging
 - However, specific “tracing” systems have become prevalent in cloud native systems
- This information is used by operators, developers, SREs and devops staff for:
 - **Introspection** (examining internal processes for the purpose of understanding)
 - **Diagnostics** (understanding system problems)
 - **Debugging** (repairing application errors)
 - **Activity Monitoring** (tracking customer activity)
 - ...
- Tracing is a cross-cutting concern
 - All cloud native applications need tracing, so best to build once and implement everywhere
- Tracing can be implemented in many ways:
 - **Custom** application logging code or custom code written to a specific tracer tools
 - **Pluggable Libraries** allow developers to code tracing abstractly: **OpenTracing** with plug ins:
 - Jaeger
 - Open Zipkin
 - Lightstep
 - **Proxies** can be used to intercept, report on, and instrument network interactions without app support
 - Linkerd
 - Envoy
 - Conduit

```
// 2) Demonstrate simple OpenTracing instrumentation
parent := tracer.StartSpan("Parent")
for i := 0; i < 20; i++ {
    parent.LogEvent(fmt.Sprintf("Starting child #%d", i))
    child := tracer.StartSpan("Child", opentracing.ChildOf(parent.Context()))
    time.Sleep(50 * time.Millisecond)
    child.Finish()
}
parent.LogEvent("A Log")
parent.Finish()
```

Logging

170

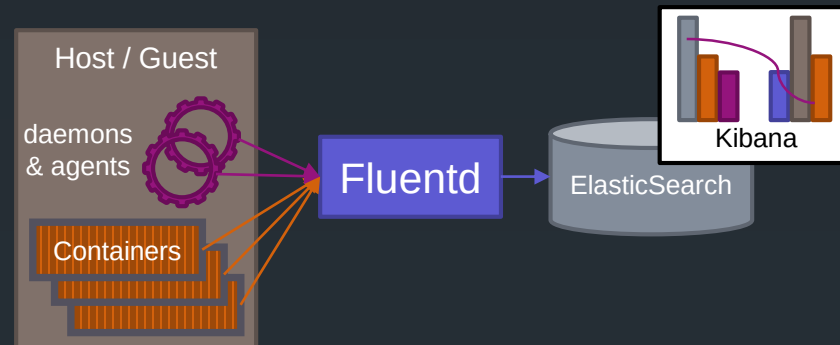
Copyright 2013-2020, RX-M LLC

- A log is a record of discrete events reported by a system component
- Software systems may write log data in a structured or informal way
 - User place a new order
 - NewOrder, UID 56749, BUY 800 GSCO MKT
 - 7,56749,1,800,GSCO,8
 - 0x26 0xFA 0xA4 0x11 0x3D ...
- Log data allows applications and other system elements to report rich information associated with a wide range of problems and activities
- Log data is typically timestamped

```
-- Logs begin at Fri 2018-08-24 21:50:18 PDT, end at Wed 2018-08-29 12:06:41 PDT. --
Aug 24 21:50:18 ubuntu systemd-journald[65140]: Runtime journal (/run/log/journal/) is 22.3M, max 19.8M, 0B free.
Aug 24 21:50:18 ubuntu systemd-journald[65140]: Journal started
Aug 24 21:50:17 ubuntu systemd[1]: systemd-logind.service: Watchdog timeout (limit 3min)!
Aug 24 21:50:17 ubuntu dhclient[4288]: DHCPREQUEST of 192.168.91.132 on ens33 to 192.168.91.254 port 67 (xid=0x4ed41778)
Aug 24 21:50:17 ubuntu systemd[1]: systemd-journald.service: Watchdog timeout (limit 3min)!
Aug 24 21:50:17 ubuntu dhclient[4288]: DHCPACK of 192.168.91.132 from 192.168.91.254
Aug 24 21:50:17 ubuntu systemd[4854]: Time has been changed
Aug 24 21:50:17 ubuntu dhclient[4288]: bound to 192.168.91.132 -- renewal in 893 seconds.
Aug 24 21:50:18 ubuntu systemd[1]: systemd-logind.service: Service has no hold-off time, scheduling restart.
Aug 24 21:50:18 ubuntu systemd[1]: systemd-timesyncd.service: Service has no hold-off time, scheduling restart.
Aug 24 21:50:18 ubuntu systemd[1]: Stopped Network Time Synchronization.
Aug 24 21:50:18 ubuntu systemd[1]: Starting Network Time Synchronization...
Aug 24 21:50:18 ubuntu systemd[1]: Stopped Login Service.
Aug 24 21:50:18 ubuntu systemd[1]: Starting Login Service...
Aug 24 21:50:18 ubuntu systemd[1]: Starting Flush Journal to Persistent Storage...
Aug 24 21:50:18 ubuntu systemd[1]: Started udev Kernel Device Manager.
Aug 24 21:50:18 ubuntu systemd[1]: Started Network Time Synchronization.
Aug 24 21:50:18 ubuntu systemd[1]: Started Flush Journal to Persistent Storage.
Aug 24 21:50:18 ubuntu systemd-logind[65153]: New seat seat0.
Aug 24 21:50:18 ubuntu systemd[1]: Started Login Service.
Aug 24 21:50:18 ubuntu systemd-logind[65153]: Watching system buttons on /dev/input/event0 (Power Button)
Aug 24 21:50:18 ubuntu systemd[1]: Started User Manager for UID 1000.
Aug 24 21:50:18 ubuntu systemd-logind[65153]: New session c2 of user user.
Aug 24 21:50:16 ubuntu systemd-timesyncd[65150]: Synchronized to time server 91.189.94.4:123 (ntp.ubuntu.com).
Aug 24 21:50:16 ubuntu systemd[1]: Time has been changed
Aug 24 21:50:16 ubuntu systemd[4854]: Time has been changed
```

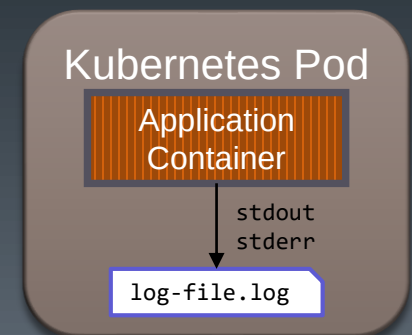
Logging in Distributed Systems

- In cloud native environments there are many logs coming from many machines:
 - **Application Logs**
 - Modern cloud native applications
 - **Platform Logs**
 - Databases, messaging platforms, ...
 - **Orchestrator Logs**
 - Orchestration systems: k8s, Mesos, Swarm, ...
 - **System Logs**
 - Host operating systems and hypervisors
- **Host Logging**
 - Linux captures operating system log data through **syslog**
 - The **logger** command in scripts
 - The libc/syslog.h **openlog()**, **syslog()** and **closelog()** function in C
 - Etc.
 - The syslog daemon (rsyslogd, syslog-ng, ...) can forward logs off machine
 - If a node dies or is broken into, log is preserved
 - **journald** is an integral (can't be removed) part of systemd for local log viewing on desktops
 - Syslog solutions can read the journal if it exists and forward it
 - In Cloud Native Systems host logging is used for: **OS, Docker, Kubelet**
- **Container Engine Logging**
 - Containerized applications generally write to **standard output and error** to generate log data
 - Container engines can also manage the cluster control plane logs (if containerized)
 - Most container engines are designed to capture, and optionally forward, log data
 - In Cloud Native Systems used for: **Everything else**
- **Cluster Logging**
 - Logs from every level of a cluster should be captured (system, orchestrator, platform systems, apps)
 - Logs should have a cluster independent lifecycle and storage solution
 - **Kubernetes provides no native storage solution for log data**
 - Many different cluster logging solutions can be integrated



Container Logging

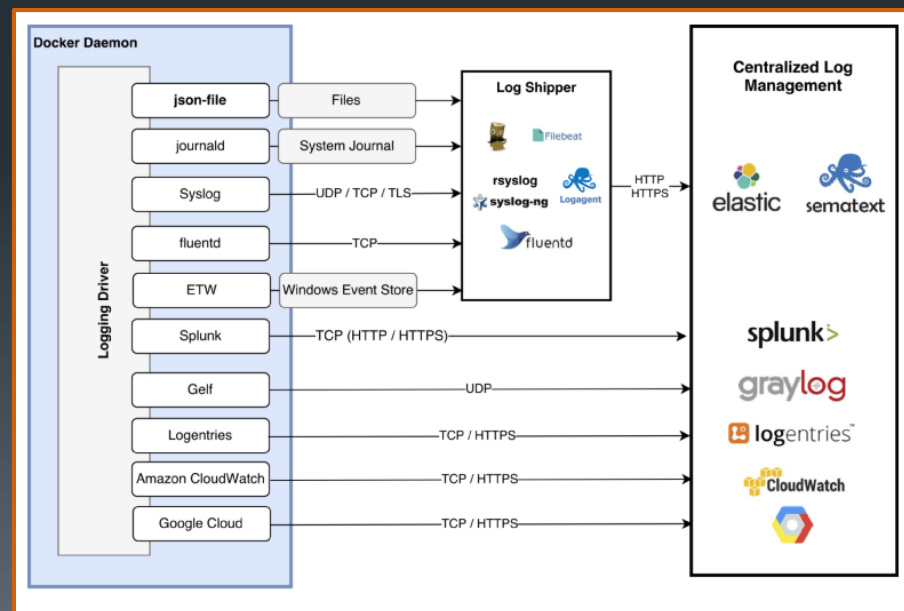
- Everything a containerized application writes to stdout and stderr is handled and redirected somewhere by a container engine
 - Typically sent to the **container's ephemeral file system layer**
- If a container restarts, the **kubelet keeps one terminated container with its logs**
- If a pod is evicted from a Node, all corresponding **containers are also evicted, along with their logs**
- Implementing **log rotation is critical** as well, so that logs don't consume all available storage on the Node
 - **Kubernetes currently is not responsible for rotating logs**
 - in Kubernetes clusters, deployed by the `kube-up.sh` script, there is a **logrotate** tool configured to run each hour
 - You can also set up a container runtime to rotate application's logs automatically
 - Docker's `log-opt`



Docker Logging Drivers

- Use `--log-driver=VALUE` with `dockerd` to configure the default logging driver
 - Can also be used with `docker container run` to configure a container's logging driver, but this functionality is not exposed through Kubernetes
- The following logging drivers are supported:
 - `none` Disables any logging for the container
 - `json-file` Default logging driver writes JSON messages to file
 - `syslog` Writes log messages to syslog
 - `journald` Writes log messages to journald
 - `gelf` Writes to Graylog Extd Log Format (GELF) endpoint (e.g. Graylog, Logstash)
 - `fluentd` Writes log messages to fluentd
 - `awslogs` Writes messages to Amazon CloudWatch Logs
 - `splunk` Splunk driver
 - `etwlogs` Event tracing for Windows
 - `gcplogs` Google Cloud logging driver
 - `logentries` Rapid7 Logentries
- `json-file` options
 - `--log-opt max-size=[0-9+][k|m|g]`
 - Logs that reach max-size are rolled over (size in kilobytes(k), megabytes(m), or gigabytes(g))
 - `--log-opt max-size=50m`
 - Logs are not rolled over if max-size is not set
 - `--log-opt max-file=[0-9+]`
 - Specifies the maximum number of files before files are discarded
 - `--log-opt max-file=10`
 - If max-size is not set max-file is not honored
 - Add data to log via options (labels/env)
 - `--log-opt labels=label1,label2`
 - `--log-opt env=env1,env2`
 - ex. "attrs":{"fizz":"buzz","foo":"bar"}

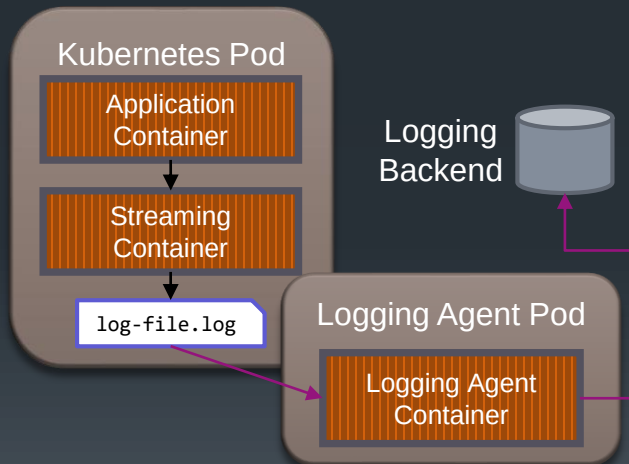
<https://docs.docker.com/config/containers/logging/configure/>



Logging Strategies

Node logging agent – dedicated tool that exposes logs or pushes logs to a backend

- Container that has access to a directory with log files from all of the application containers on that Node
- Common to implement it as either a DaemonSet or a dedicated native process on the node
 - DaemonSet is the **most common and encouraged** approach for a Kubernetes cluster
 - Creates only one agent per node
 - Does not require any changes to the apps running on the node
 - Only works for applications' standard output and standard error

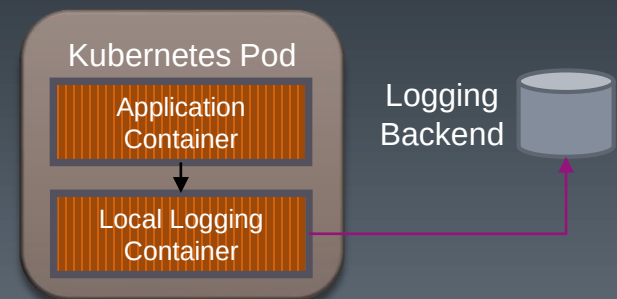
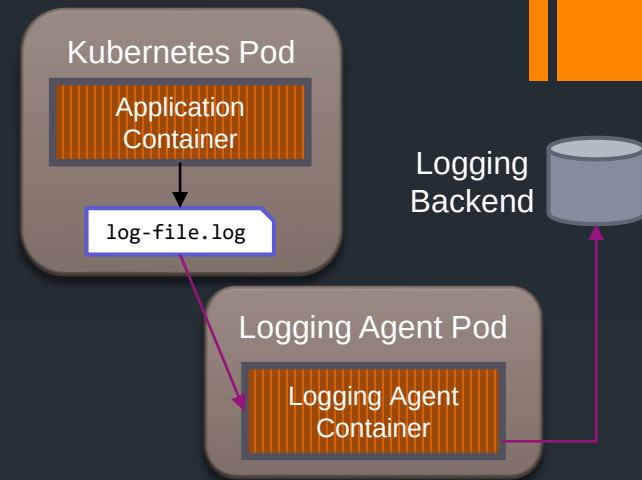


Streaming sidecar container – read logs from a file, a socket, or the journald

- Each individual sidecar container prints log to its own stdout or stderr stream
 - Takes advantage of the kubelet and the logging agent that run on each node

Sidecar with a logging agent – sidecar container with a separate logging agent configured specifically to run with your application

- If the node-level logging agent is not flexible enough



Metrics

175

Copyright 2013-2020, RX-M LLC

- Metrics are **sets of numbers** that give information about a particular process or activity
- Generally metrics come in the form of a **timeseries**
 - Meter** – the thing being measured
 - Meter Type – percentage (% cpu), rate (packets per second), count (# of files), ...
- Each event/**sample** in a timeseries has
 - Timestamp (2018-04-22,21:08:15.261)
 - Number (57.9%)
- Timeseries are usually sampled at regular intervals
 - Useful for predicting trends which may lead to outages, resource exhaustion, ...
- Timeseries are **easy to compress, store, plot, process, and retrieve**
 - Extremely long periods can be maintained by removing intermediate metrics
 - e.g. replacing a days 86,400 per second samples with 4: begin, high, low, end
- Prometheus**
 - The leading metric monitoring solution for Kubernetes
 - Prometheus metrics add an **arbitrary set of labels** to the sample data enriching the information provided with each sample
 - Adds multidimensionality
 - Search by http REQ or http REQ w/ status 404 or http REQ w/ status 404 and method POST
 - Prometheus metrics format spun out to form **Open Metrics Project**

Resource ID	Name	Type	Volume	Unit	Timestamp
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	2156.0	B	2014-05-12T19:59:15
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	2433.0	B	2014-05-12T16:47:59
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	2498.0	B	2014-05-12T17:41:19
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	2498.0	B	2014-05-12T17:51:19
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	2498.0	B	2014-05-12T18:01:19
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	2924.0	B	2014-05-12T18:11:19
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	3243.0	B	2014-05-12T16:57:59
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	3257.0	B	2014-05-23T18:58:30
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	3322.0	B	2014-05-14T11:06:31
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	3358.0	B	2014-05-12T18:21:19
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	3421.0	B	2014-05-12T16:03:12
instance-00000002-bf8cf0d1-e4b5-46f1-8df2-83ad603bf9d2-tap76c94c76-43	network.outgoing.bytes	cumulative	3669.0	B	2014-05-12T17:08:00

KEY			VALUE
Metric name	Labels	Timestamp	Sample Value
...			
http_requests_total	{status="200",method="GET"}	@1434317560938	94355
http_requests_total	{status="200",method="GET"}	@1434317561287	94934
http_requests_total	{status="200",method="GET"}	@1434317562344	96483
http_requests_total	{status="404",method="GET"}	@1434317560938	38473
http_requests_total	{status="404",method="GET"}	@1434317561249	38544
http_requests_total	{status="404",method="GET"}	@1434317562588	38663
http_requests_total	{status="200",method="POST"}	@1434317560885	4748
http_requests_total	{status="200",method="POST"}	@1434317561483	4795
http_requests_total	{status="200",method="POST"}	@1434317562589	4833
http_requests_total	{status="404",method="POST"}	@1434317560939	122
...			

cAdvisor

176

Copyright 2013-2020, RX-M LLC

- An open source container resource usage and performance analysis agent
- Purpose-built for containers and supports Docker containers natively
- In Kubernetes cAdvisor is integrated into the Kubelet binary
 - cAdvisor auto-discovers all containers in the machine and collects CPU, memory, filesystem, and network usage statistics
 - cAdvisor also provides the overall machine usage by analyzing the 'root' container on the machine
- Kubelet exposed the cAdvisor UI up to version v1.11
 - Starting from v1.13, you can deploy cAdvisor as a DaemonSet to reenale the cAdvisor UI



Kubernetes Metrics API

- Kubernetes 1.8 adds the **Metrics API**
 - `/apis/metrics.k8s.io/`
 - Exposes resource usage metrics
 - Container/Pod/Node CPU usage
 - Container/Pod/Node memory usage
 - ...
 - The Metrics API offers **only the current sample** (not a storage interface)
 - The Metrics API requires the Kubernetes Metrics Server to be deployed in the cluster
- Kubernetes **Metrics Server**
 - Metrics Server is a cluster-wide aggregator of resource usage data
 - Essentially a replacement for Heapster
 - Tracks Container/Pod/Node CPU and Memory metrics
 - Supported in Kubernetes 1.7+
 - Starting from Kubernetes 1.8 the Metrics Server is deployed by default with `kube-up.sh`
 - It can also be deployed using deployment yamls here:
<https://github.com/kubernetes/kubernetes/blob/master/cluster/addons/metrics-server/metrics-server-deployment.yaml>
 - Metric server collects metrics from the Summary API exposed by the Kubelet on each node
 - In the heapster era a separate "CAdvisor" container provided this data
 - The **Metrics Server stores data in memory** and only holds the most recent sample
 - Crash/restart eliminates all data
 - Metrics Server supports 30 pods per cluster node (running more pods may cause an OOM failure)
- Metrics API clients
 - `kubectl top`
 - Horizontal Pod Autoscaler

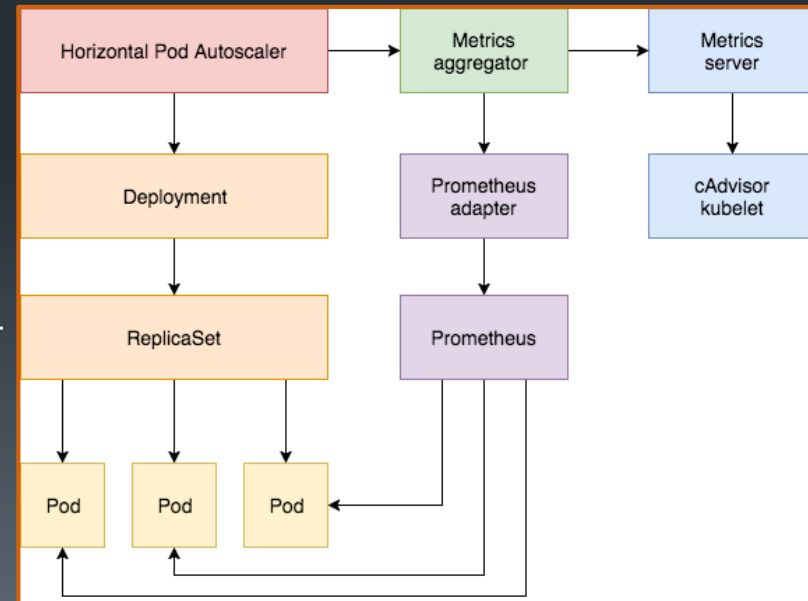
```
Floreks [master|+4...1]$ kubectl top pod
NAME                                CPU(cores)   MEMORY(bytes)
grafana-3867175031-pgcsn            0m           7Mi
nginx-2371676037-ngw6v              0m           2Mi
nginx-2371676037-b16bp              0m           2Mi
nginx-deployment-3646295028-z3rd7   0m           2Mi
Frontend-51sbd                      0m           10Mi
Frontend-rx5z7                      0m           10Mi
nginx-2371676037-q4gk9              0m           2Mi
nginx-deployment-3646295028-qcv76   0m           2Mi
```


Custom Metrics API

178

Copyright 2013-2020, RX-M LLC

- Defined to allow any system to implement custom Kubernetes metrics
 - **Intentionally just an API definition and not an implementation**
 - The metrics aggregator allows multiple systems to supply metrics through the K8s metrics API
- Implementations are installed into a Kubernetes cluster as **Aggregated APIs**
 - Aggregated APIs were introduced in K8s 1.7
 - Aggregated APIs allows services running anywhere in the cluster to extend the Master API
 - Metrics Servers are registered in the main API through the Kubernetes API Aggregation interface
- **Implementations can be switched-out/added** but the API stays the same
- The custom metrics API allows requesting arbitrary metrics
 - Custom metrics API implementations are specific to the respective backing monitoring system
- Implementations:
 - **Prometheus Adapter** – an implementation of the custom metrics API that attempts to support arbitrary metrics following a set label and naming scheme
 - **Microsoft Azure Adapter** – an implementation of the custom metrics API that allows you to retrieve arbitrary metrics from Azure Monitor
 - **Datadog Cluster Agent** – implementation of the external metrics provider, using Datadog as a backend for the metrics
 - **Kube Metrics Adapter** – general purpose metrics adapter for Kubernetes that can collect and serve custom and external metrics for Horizontal Pod Autoscaling
 - Provides the ability to scrape pods directly or from Prometheus through user defined queries
 - Also capable of serving external metrics from a number of sources including AWS' SQS and ZMON monitoring

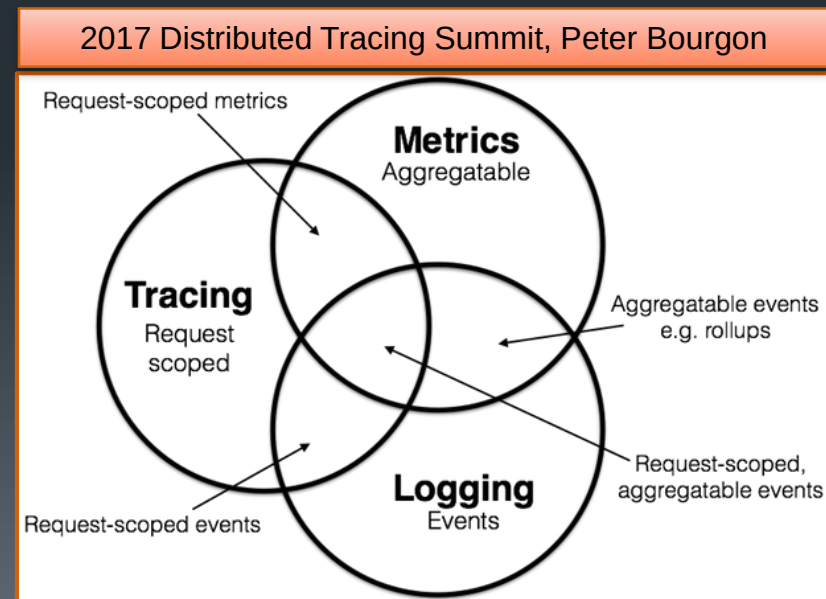


Conflating Metrics, Tracing and Logging

179

Copyright 2013-2020, RX-M LLC

- Are metrics, traces and logs distinct concepts ?
 - You can log traces and metrics ...
 - You can plot data from traces and logs ...
 - You can corollate events from log data
- Depending on the system under consideration these concepts can get muddy
- Many tools support metrics, tracing and/or logging (!) making the water more opaque
 - You can flow traces, logs and metrics into Kafka topics
 - You can store traces, logs and metric data in Elasticsearch
 - You can plot log, trace and metric data in Kibana
- The concepts are related yet distinguishing them yields value
 - The Cloud Native industry is actively working to standardize and develop all of these areas independently
 - While many tools do (and should) support log and metric capture/operations, distinguishing the data streams enables simplified operations specialized to the task of processing them separately



Node Problem Detector

180

Copyright 2013-2020, RX-M LLC

- Collects node problems from daemons and reports them to the apiserver as **NodeCondition** and **Event**
 - Kubernetes will not take any action
 - In the future, a remedy system could be introduced
- Supports **well known kernel issues**
 - Included kernel monitor daemon is extensible by adding regex to the json config file
 - Examples:
 - Infrastructure daemon issues: ntp service down
 - Hardware issues: bad cpu, memory or disk
 - Kernel issues: kernel deadlock, corrupted file system
 - Container runtime issues: unresponsive runtime daemon
- Current limitations:
 - Only supports file-based kernel log
 - Does not support log tools like journald
 - Only works on Ubuntu and Debian
 - Assumption on kernel log format
- **Enabled as an addon** by default when deploying K8s with kube-up.sh in GCE
 - Can be manually deployed from spec
 - Put it under the addon directory on the master node: /etc/kubernetes/addons/node-problem-detector

```
# node-problem-detector.yaml

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-problem-detector-v0.1
  namespace: kube-system
  labels:
    k8s-app: node-problem-detector
    version: v0.1
spec:
  selector:
    matchLabels:
      k8s-app: node-problem-detector
      version: v0.1
  template:
    metadata:
      labels:
        k8s-app: node-problem-detector
        version: v0.1
    spec:
      hostNetwork: true
      containers:
        - name: node-problem-detector
          image: k8s.gcr.io/node-problem-detector:v0.1
          securityContext:
            privileged: true
      volumes:
        - name: log
          hostPath:
            path: /var/log/
```

Determine Pod Failure

- Termination messages provide a way for containers to write information about fatal events to a location where it can be easily retrieved
 - Surfaced by tools like dashboards and monitoring software
 - Written to the general logs
- **terminationMessagePath** – Kubernetes retrieves termination messages from the termination message file specified in field of a Container
 - Default value of `/dev/termination-log`
 - Contents of the file populate the Container's status message on success and failure
- **terminationMessagePolicy**
 - **File** – uses contents of `terminationMessagePath`
 - **FallbackToLogsOnError** – use the last chunk of container output if the file is empty when a container exits with an error
 - Limited to 2048 bytes or 80 lines (whichever is smaller)

```
apiVersion: v1
kind: Pod
metadata:
  name: msg-path-demo
spec:
  containers:
  - name: msg-path-demo-container
    image: debian
    terminationMessagePath: "/tmp/my-log"
```

Debugging: Pods

182

Copyright 2013-2020, RX-M LLC

- Check the current state of the pod and **recent events with describe or events**
 - Are they all running? Look for recent restarts
- **Pending** – a pod can not be scheduled onto a node
 - Insufficient resources (CPU, memory)
 - Check Node capacities:
 - `$ kubectl get nodes -o json | jq '.items[] | {name: .metadata.name, cap: .status.capacity}'`
 - Resource quota may be in effect
 - Using hostPort limits pod placement, use a Service
 - Pod might specify a label selector that doesn't match any nodes
- **Waiting** – scheduled but cannot run on given Node
 - Most common cause is failure to pull an image, check:
 - Correct image name
 - Image exists in repository
 - Run `docker image pull` to test if Docker can pull the image
- **Crashing** or unhealthy pods
 - Inspect logs of a viable container: `$ kubectl logs <pod-name> <container-name>`
 - Use `--previous (-p)` for a crashed container
 - Run commands inside container with `exec`
 - Or attach to a Pod's namespace(s) with a container that has debugging tools

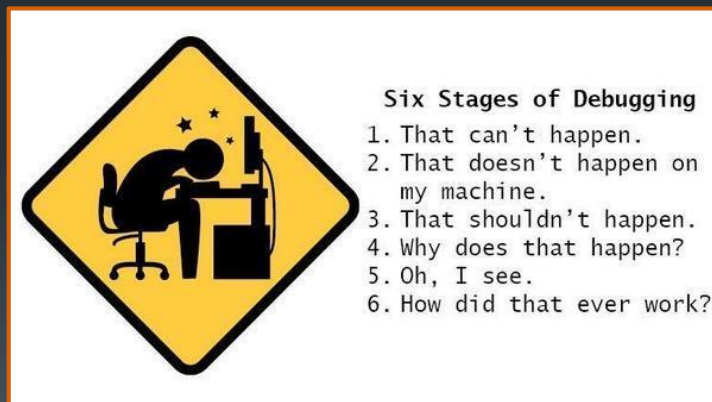
Status	Meaning
Init:x/y	The Pod has x Init Containers, and y have completed so far
Init:Error	An Init Container has failed to execute
Init:CrashLoopBackOff	An Init Container has failed repeatedly
Pending	The Pod has not yet begun executing Init Containers
PodInitializing or Running	The Pod has already finished executing Init Containers

Debugging: Services

183

Copyright 2013-2020, RX-M LLC

- Can you query the Service with DNS?
 - From a pod:
 - From the same namespace:
 - `$ nslookup mysvc`
 - From separate namespaces:
 - `$ nslookup mysvc.default`
 - With a fully-qualified name:
 - `$ nslookup mysvc.default.svc.cluster.local`
 - From a Node in the cluster:
 - `$ nslookup mysvc.default.svc.cluster.local 10.96.0.10`
 - If you are able to do a fully-qualified name lookup but not a relative one, you need to check that the `/etc/resolv.conf` file is correct
 - Does *any* service work? The Kubernetes master service should always work:
 - `nslookup kubernetes.default`
- IPs
 - Can you access the Service VIP by IP?
 - `$ curl 10.11.12.13:80`
 - Does the Service have any Endpoints?
 - `$ kubectl get endpoints mysvc`
- kube-proxy
 - Is kube-proxy running?
 - `$ ps auxw | grep kube-proxy`
 - Is it writing iptables rules?
 - `$ sudo iptables -L -t nat`
 - There should be 1 rule in KUBE-SERVICES, 1 or 2 rules per endpoint in KUBE-SVC-(hash)
- Does the Service actually exist?
 - (yes, this is a forgotten step sometimes!)



Cluster Troubleshooting

184

Copyright 2013-2020, RX-M LLC

- Make sure all your Nodes are registered correctly
 - If functional, use `kubectl get nodes` and look for Nodes not reporting a Ready state
- Cluster logs vary by deployment method
 - Kubeadm puts control plane components into pods
 - Use `kubectl get logs` like any other pod
 - Other deployment methods require SSH to hosts
 - Masters
 - `/var/log/kube-apiserver.log`
 - `/var/log/kube-scheduler.log`
 - `/var/log/kube-controller-manager.log`
 - Worker nodes
 - `/var/log/kubelet.log` (will also be on the Masters when deploying with kubeadm)
 - `/var/log/kube-proxy.log`
- Best practices mitigate many issues
 - Use auto-restart features of IaaS provider
 - Use IaaS reliable storage for cluster components too!
 - Snapshot frequently – AWS S3 SLA is 11 9s, EBS is 99.95%
 - Multiple independent clusters
 - Avoid making risky changes to all clusters all at once



End-to-End (E2E) Testing Kubernetes clusters

185

Copyright 2013-2020, RX-M LLC



- A technique used to test whether the flow of an application right from start to finish is behaving as expected
- The primary objectives of Kubernetes e2e tests are:
 - To ensure a consistent and reliable behavior of the Kubernetes code base
 - To catch hard-to-test bugs before users do when unit and integration tests are insufficient
- Primary use: help Kubernetes contributors test their changes on Kubernetes code
- E2E Testing tools
 - kubernetes
 - kind (Kubernetes in Docker)
- Create clusters appropriate for testing code using:
 - Code from the Kubernetes Repo
 - Pre-built node images
 - User-built node images

```
ubuntu@ip-172-31-6-52:~/go/bin$ sudo kind create cluster --name k8s
Creating cluster "k8s" ...
 ✓ Ensuring node image (kindest/node:v1.17.0) [11s]
 ✓ Preparing nodes [0s]
 ✓ Writing configuration [0s]
 ✓ Starting control-plane [57s]
 ✓ Installing CNI [11s]
 ✓ Installing StorageClass [0s]
Set kubectl context to "kind-k8s"
You can now use your cluster with:

kubectl cluster-info --context kind-k8s

Not sure what to do next? ☺ Check out https://kind.sigs.k8s.io/docs/user/quick-start/
ubuntu@ip-172-31-6-52:~/go/bin$ kubectl cluster-info --context kind-k8s
Kubernetes master is running at https://127.0.0.1:32769
KubeDNS is running at https://127.0.0.1:32769/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

```
ubuntu@ip-172-31-6-52:~/go/bin$ kubectl config set-context kind-k8s
Context "kind-k8s" modified.
ubuntu@ip-172-31-6-52:~/go/bin$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
k8s-control-plane   Ready    master   53s   v1.17.0
ubuntu@ip-172-31-6-52:~/go/bin$
```

Summary

- Observability is critical in distributed systems like Kubernetes
- The three pillars of observability:
 - Logging
 - Capturing logs begins with the container runtime and requires planning to ensure logs are saved beyond the life of a given container
 - Metrics
 - cAdvisor, built into kubelet is a composable container aware monitoring tool
 - Heapster has been used for metrics aggregation but is being replaced by the metrics server
 - Tracing
 - Tracing can be implemented in many ways, as custom application logging code, pluggable Libraries that allow developers to code tracing abstractly or with telemetry reported by proxies

Lab 8

- Working with the metrics server and HPAs

Lab 8 Documentation

References

188

Copyright 2013-2020, RX-M LLC

- Metrics Server
 - <https://kubernetes.io/docs/tasks/debug-application-cluster/resource-metrics-pipeline/>
- Kubelet TLS Bootstrap
 - <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet-tls-bootstrapping/>
 - <https://kubernetes.io/docs/reference/access-authn-authz/bootstrap-tokens/>
- HPA
 - <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
 - <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>

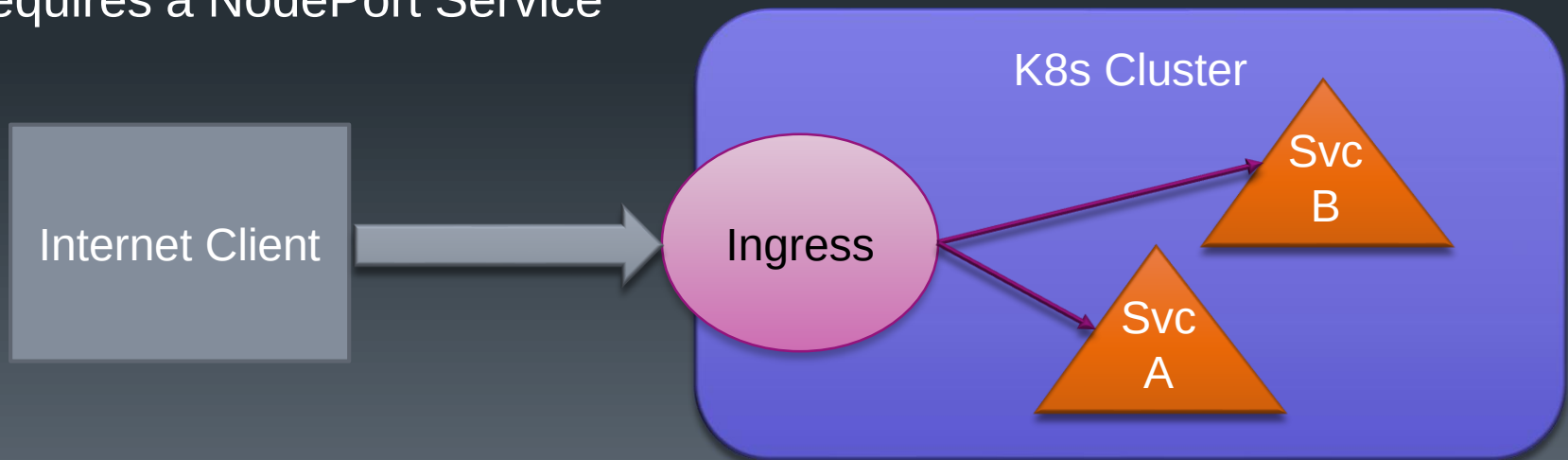
9. Ingress

Objectives

- Describe the architecture of Ingress in Kubernetes
- Understand common Ingress requirements
- Explore sample ingress RBAC roles and rules
- List the available ingress controllers
- Explain the characteristics of TLS termination

Kubernetes Ingress

- Added in Kubernetes v1.1
- Exposes HTTP and HTTPS routes to clients outside the cluster
- Maps these routes to services within the cluster
- Traffic routing is controlled by rules within Ingress resources
- Ingress load balance traffic and terminate SSL
- An Ingress does not expose arbitrary ports or protocols
- Exposing services other than HTTP and HTTPS outside the cluster requires a NodePort Service



Available Ingress Controllers

192

Copyright 2013-2020, RX-M LLC

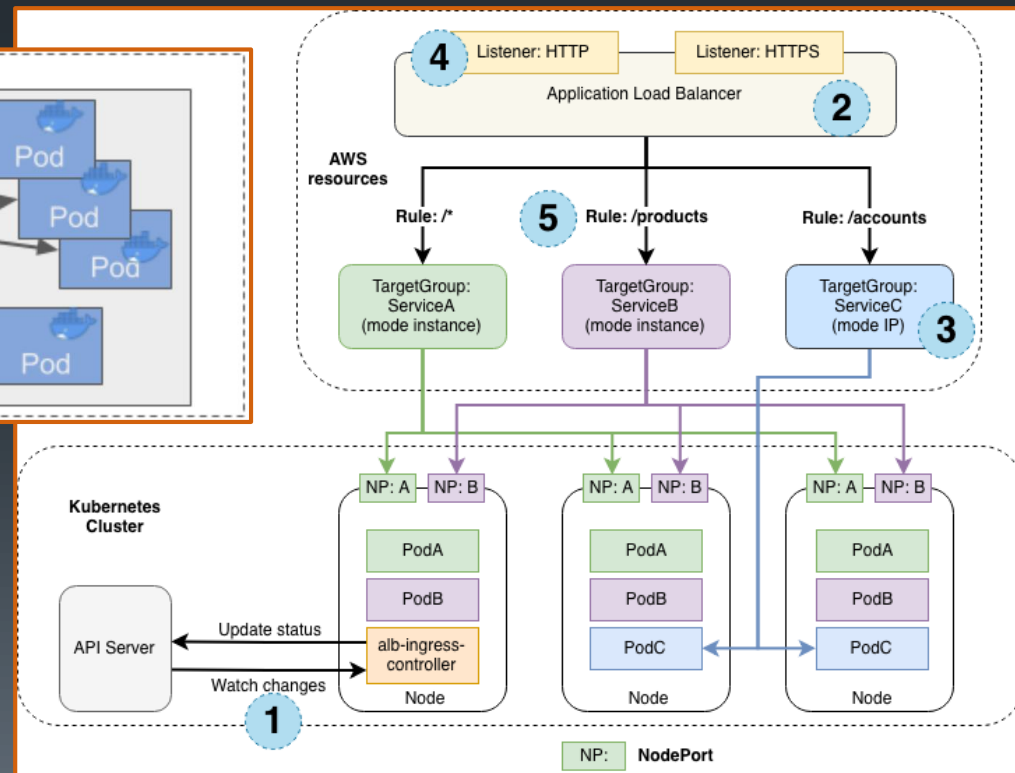
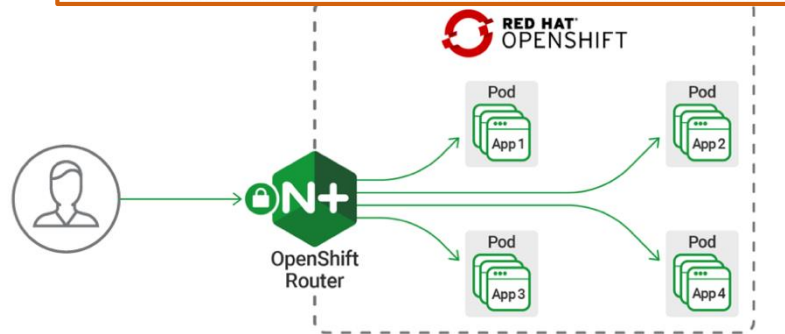
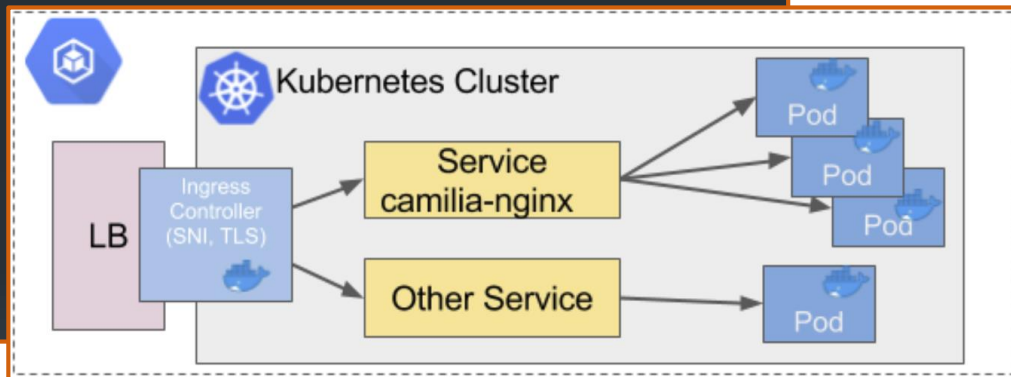
- Kubernetes as a project currently supports and maintains
 - **GCE** – Google compute engine ingress controller
 - **Nginx** – Open source Nginx ingress controller
- Additional controllers
 - **Ambassador** - an Envoy based ingress controller (commercial support from Datawire)
 - **Voyager** - HAProxy based ingress controller (commercial support from AppsCode Inc.)
 - **Contour** - an Envoy based ingress controller (commercial support from Heptio, now VMware)
 - **ADC** - Citrix hardware (MPX), virtualized (VPX) and free containerized (CPX) ADC Ingress Controller for baremetal and cloud deployments (commercial support from Citrix)
 - **F5 BIG-IP** Controller for Kubernetes - (commercial support from F5 Networks)
 - **Gloo** - an open-source ingress controller based on Envoy (enterprise support from solo.io)
 - **jcmoraisjr/haproxy-ingress** - HAProxy based ingress controller (commercial support from HAProxy Technologies)
 - **Istio** - an open source Envoy based service mesh
 - **Kong** Ingress Controller for Kubernetes - (commercial support from Kong)
 - **NGINX** Ingress Controller for Kubernetes - (commercial support from NGINX, Inc., now part of F5)
 - **Traefik** - ingress controller (commercial support by Containous)

A wide range of solutions and features

193

Copyright 2013-2020, RX-M LLC

- Nearly every Kubernetes solution offers some form of provided ingress controller
 - RedHat Openshift
 - Router/Nginx+
 - AWS ALB Ingress controller
 - Triggers the creation of an ALB whenever an Ingress resource is created
 - GKE ingress controller
 - Creates a Google Cloud Platform HTTP(S) load balancer whenever an Ingress is created
 - Helm charts
 - helm install stable/nginx-ingress
 - Used by AKS and PKS



Ingress

194

Copyright 2013-2020, RX-M LLC

- Collections of **rules that allow inbound connections** to reach cluster services
- An Ingress resource supports:
 - Exposing services:
 - Via custom URLs
 - ex: service A at the URL /serviceA and service B at the URL /serviceB
 - Via multiple host names
 - ex: groupa.example.com for one group of services and groupb.example.com for another group
 - Configuring SSL termination for each exposed host name
- Ingress spec contains a list of rules matched against all incoming requests
 - Currently the Ingress resource **only supports http rules**, each rule contains:
 - Host
 - List of paths
 - Backends associated with the paths
 - Includes a service:port combination
- An **Ingress controller is required** to satisfy an Ingress
 - Simply creating the Ingress resource will have no effect
 - Using annotation it is possible to run multiple Ingress controllers at the same time

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: nginx-ingress
  annotations:
    kubernetes.io/ingress.class: "nginx"
spec:
  rules:
  - host: host.example.com
    http:
      paths:
      - path: /somepath
        backend:
          serviceName: somesvc
          servicePort: 80
```

Ingress Types 1

- Single Service Ingress
 - Specify a default backend with no rules
- Fanout
 - Routes traffic from a single IP address to more than one Service, based on the HTTP URI being requested
 - Keeps the number of load balancers to a minimum

```
# Simple
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: single-ingress
spec:
  backend:
    serviceName: singlesvc
    servicePort: 80
```

```
# Fanout
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: fanout-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: dev.bar.com
    http:
      paths:
      - path: /dev
        backend:
          serviceName: service-one
          servicePort: 80
      - path: /bar
        backend:
          serviceName: service-two
          servicePort: 8080
```


Ingress Types 2

- Name based virtual hosting
 - Supports routing HTTP traffic to multiple host names at the same IP address
 - The load balancer routes requests based on the host header
 - In the example, any traffic without a hostname defined in the request is routed to service-three

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: name-virtual-host-ingress
spec:
  rules:
    - host: first.example.com
      http:
        paths:
          - backend:
              serviceName: service-one
              servicePort: 80
    - host: second.foo.com
      http:
        paths:
          - backend:
              serviceName: service-two
              servicePort: 80
    - http:
        paths:
          - backend:
              serviceName: service-three
              servicePort: 80
```

TLS Termination

- Ingress controllers can serve connection using TLS by **specifying a secret** that contains a private key and certificate
- Currently Ingress supports a single TLS port, 443, and assumes TLS termination
- If the TLS configuration section in an Ingress specifies different hosts, they will be **multiplexed on the same port** according to the hostname specified through the SNI TLS extension
- TLS secrets must contain keys named **tls.crt** and **tls.key** that contain the certificate and private key
 - In the example, TLS secret is from a certificate embedded in a K8s Secret that contains a CN for `ssl.example.com`

```
apiVersion: v1
kind: Secret
metadata:
  name: tlssecret
  namespace: default
type: kubernetes.io/tls
data:
  tls.crt: <base64 encoded cert>
  tls.key: <base64 encoded key>

---
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  tls:
  - hosts:
    - ssl.example.com
    secretName: tlssecret
  rules:
  - host: ssl.example.com
    http:
      paths:
      - path: /
        backend:
          serviceName: tls-service
          servicePort: 80
```

Ingress Controllers

198

Copyright 2013-2020, RX-M LLC

- Monitor Ingress resources via the Kubernetes API and update the configuration of a load balancer in case of any changes
 - Acts as reverse proxy for the Ingress rules found at api-server's /ingresses endpoint
- Types:
 - **GLBC** – is a GCE L7 load balancer controller that manages external loadbalancers configured through the Kubernetes Ingress API
 - **nginx** – deployed as a Deployment/Pod & **uses a ConfigMap** to store its configuration
 - Non-official: Træfik, others...

```
apiVersion: apps/v1beta1
kind: Deployment
metadata:
  name: nginx-ingress-dep
  namespace: kube-system
  labels:
    app: nginx-ingress
spec:
  replicas: 1
  template:
    metadata:
      labels:
        app: nginx-ingress
    spec:
      containers:
        - image: gcr.io/google_containers/nginx-ingress-controller:0.8.3
          name: nginx-ingress
          ports:
            - containerPort: 80
              hostPort: 80
            - containerPort: 443
              hostPort: 443
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-ingress-svc
  namespace: kube-system
  labels:
    app: nginx-ingress
spec:
  ports:
    - name: http
      port: 80
      targetPort: 80
    - name: https
      port: 443
      targetPort: 443
  selector:
    app: nginx-ingress
```

<https://github.com/kubernetes/ingress/tree/master/examples>
contains a catalog of examples on how to run, configure and
scale Ingress

Ingress Cluster Role Rules

199

Copyright 2013-2020, RX-M LLC

- Ingress controllers require various types of access to the Kubernetes API

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1beta1
metadata:
  name: nginx-ingress
rules:
- apiGroups:
  - ""
  resources:
  - services
  - endpoints
  verbs:
  - get
  - list
  - watch
- apiGroups:
  - ""
  resources:
  - secrets
  verbs:
  - get
  - list
  - watch
```

```
- apiGroups:
  - ""
  resources:
  - configmaps
  verbs:
  - get
  - list
  - watch
  - update
  - create
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - list
- apiGroups:
  - ""
  resources:
  - events
  verbs:
  - create
  - patch
```

```
- apiGroups:
  - extensions
  resources:
  - ingresses
  verbs:
  - list
  - watch
  - get
- apiGroups:
  - "extensions"
  resources:
  - ingresses/status
  verbs:
  - update
```

Summary

- Ingress controllers implement inbound load balancing tasks for a Kubernetes cluster
- Ingress resources define discrete mappings from ingress routes to Kubernetes cluster IP services

Lab 9

- Working with ingress controllers

Lab 9 Documentation

References

202

Copyright 2013-2020, RX-M LLC

- Creating Namespaces:
 - <https://kubernetes.io/docs/tasks/administer-cluster/namespaces-walkthrough/#create-new-namespaces>
- Service Accounts:
 - <https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/>
- Roles & Role Bindings
 - <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
 - <https://kubernetes.io/docs/reference/access-authn-authz/rbac/#api-overview>
- Distributing Secrets
 - <https://kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/>
- Ingress Controllers
 - <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>
 - <https://kubernetes.io/docs/concepts/services-networking/ingress/>
- Deploying Ingress (feat. MiniKube, but still applies to regular clusters)
 - <https://kubernetes.io/docs/tasks/access-application-cluster/ingress-minikube/#enable-the-ingress-controller>
- Creating a Service
 - <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Creating a Deployment
 - <https://kubernetes.io/docs/concepts/services-networking/service/>

Day 4

- 10. Networking
- 11. etcd
- 12. Exam Prep I

10. Kubernetes Networking

Objectives

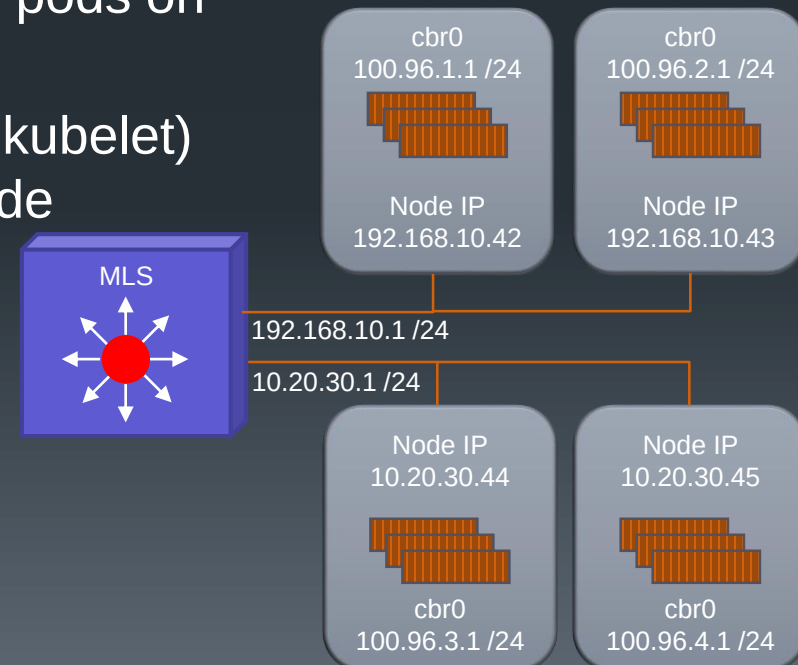
- Explain SDN solutions at:
 - L2
 - L3
 - L4
- Examine the range of SDN options for container systems
- Discuss the integration and operation of CNI plugins in Kubernetes
- Learn about policy-based networking

K8s Network Model

206

Copyright 2013-2020, RX-M LLC

- Each pod has its own IP address
 - **Host IPs** are problematic as container counts grow [**too few IPs**]
 - Ports must be used to expose services on containers and allow external communication
 - The complexity of running multiple services that may or may not know about each other (and their custom ports), and managing the port space becomes challenging
 - **Container IPs** can be overkill [**too many IPs**]
 - **Pod IPs** represent a happy medium [**just right IPs**]
- Pods on a node can communicate with all pods on all nodes without NAT
- Agents on a node (e.g. system daemons, kubelet) can communicate with all pods on that node
- Pods in the host network of a node can communicate with all pods on all nodes without NAT



CNI Plugins

207

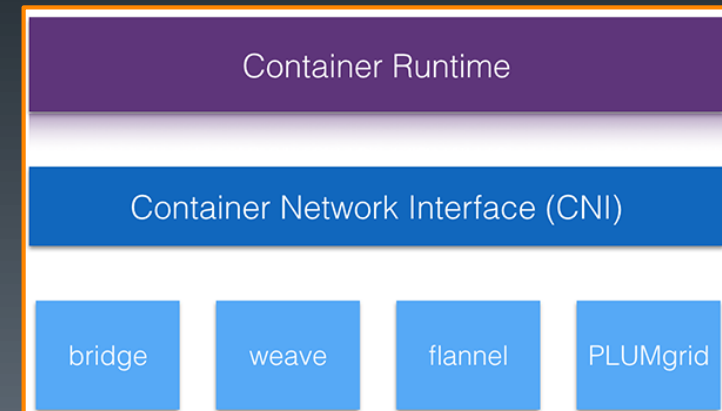
Copyright 2013-2020, RX-M LLC

■ Container Network Interface (CNI)

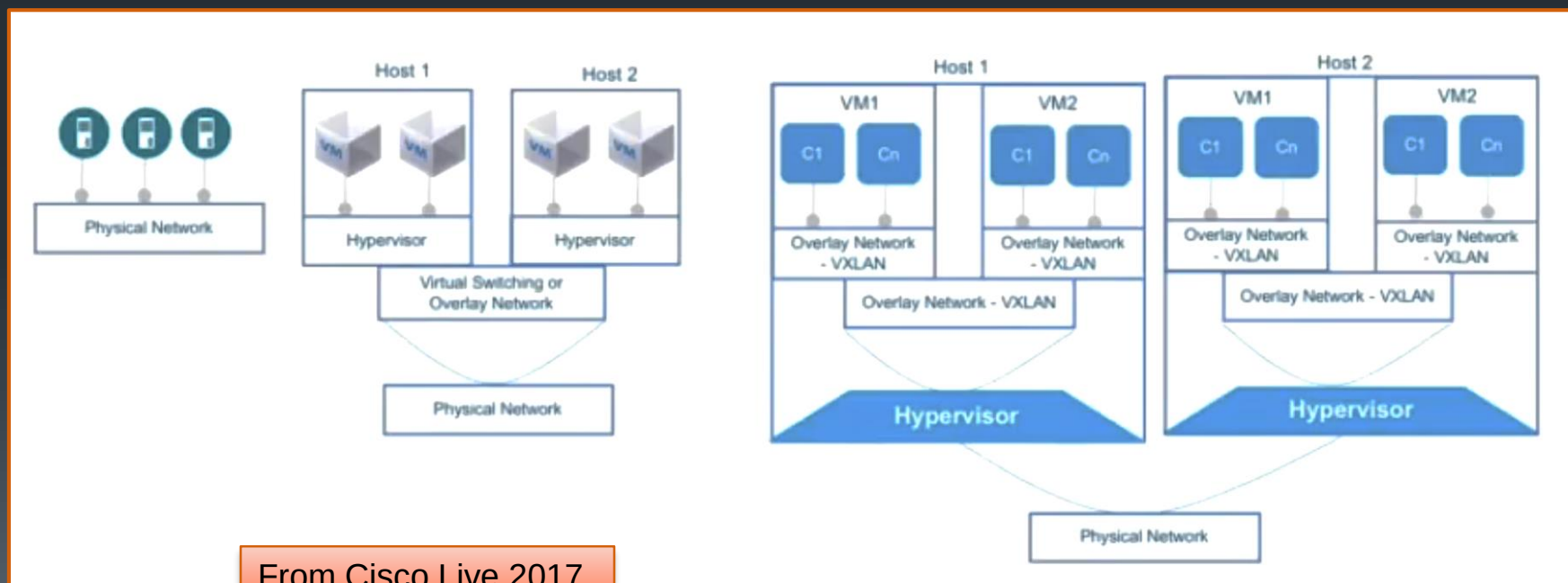
- A specification and libraries for writing plugins to configure network interfaces in Linux containers
- Concerned only with **network connectivity of containers** and removing allocated resources when the container is deleted
- CNI has a wide range of support and the specification is simple to implement
- Used by:
 - Kubernetes
 - Kubernetes Network plugins are in alpha
 - Mesos
 - Rocket
 - Cloud Foundry
- Downside: some docker features (inspect) will not know about the CNI networking info (IP addresses, etc.)

■ Docker's Container Network Model (CNM)

- **Not supported directly by Kubernetes** but not explicitly disallowed
- You can configure docker for multihost networking without Kubernetes involvement and it will just work
- Issues:
 - You will need to ensure that Kubernetes resolves DNS if you use DNS
 - Docker names containers, Kubernetes names Pods



The Evolution of SDN



Container Network Integration

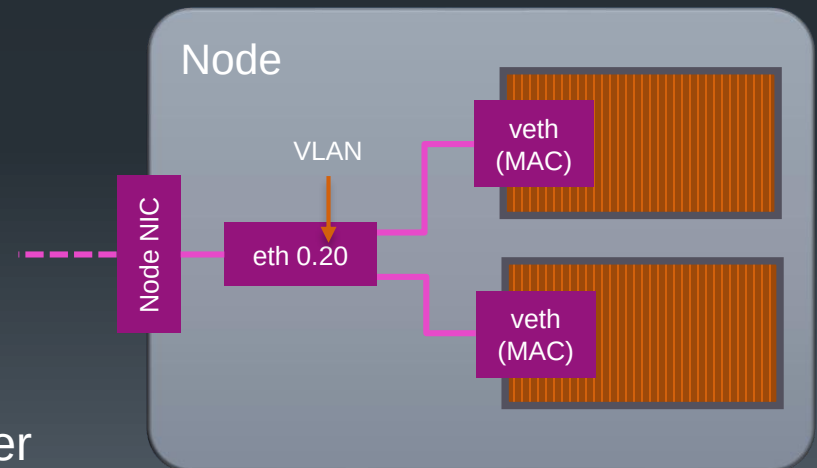
- Typical networking solutions for container based systems involve one or more layers of the networking stack
- L2
 - VLAN integration
 - e.g. MACVLAN, IPVLAN
- L3
 - Routing
 - e.g. Host routes, MLS, Routers
- L4
 - Tunneling
 - e.g. VXLAN

L2 Example: MACVLAN

210

Copyright 2013-2020, RX-M LLC

- Uses unnumbered host interfaces
- Bonds container MACs to the outside
- Disallows internal cross container traffic
- Can bond to subinterfaces for VLAN tagging
- `docker network create -d macvlan --subnet=192.168.50.0/24 \ --gateway=192.168.50.1 -o parent=eth0.20 macvlan20`
- `docker container run --net=macvlan20 ...`
- Pure L2 solution
- Pros
 - Simple and fast
 - No port mapping
 - No bridge forwarding
- Cons
 - Less flexible
 - External process/device required for inter container communications

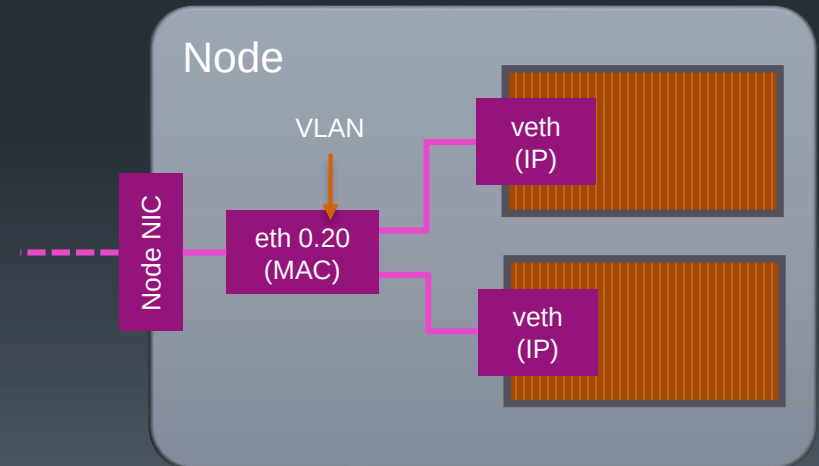


L2 Example: IPVLAN

211

Copyright 2013-2020, RX-M LLC

- Like MACVLAN however **all containers share a single MAC**
- The kernel delivers packets based on IP address
 - L2 networking outside the host based on shared MAC
 - L3 networking inside the host based on unique IP
- All cross container traffic must still transit an external process/device
- Two modes
 - **L2 mode** – container If behaves like an L2 device
 - TX processing **down to L2 happens on the net stack of the container** and packets are switched after that into the host device (default-ns) and queued for xmit
 - **The routing table of the container is used** in this mode
 - RX processing is simple and all multicast, broadcast and unicast belonging to the address(es) are delivered to the container
 - **L3 mode** – container behaves like a L3 device
 - TX processing **down to L3 happens on the stack of the container** and packets are switched to the host (default-ns) for the L2 processing
 - **The routing table of the host (default-ns) is used** in this mode
 - RX processing is similar to the L2 mode except that **only Unicast packets are delivered** to the container, the host handles all other packets

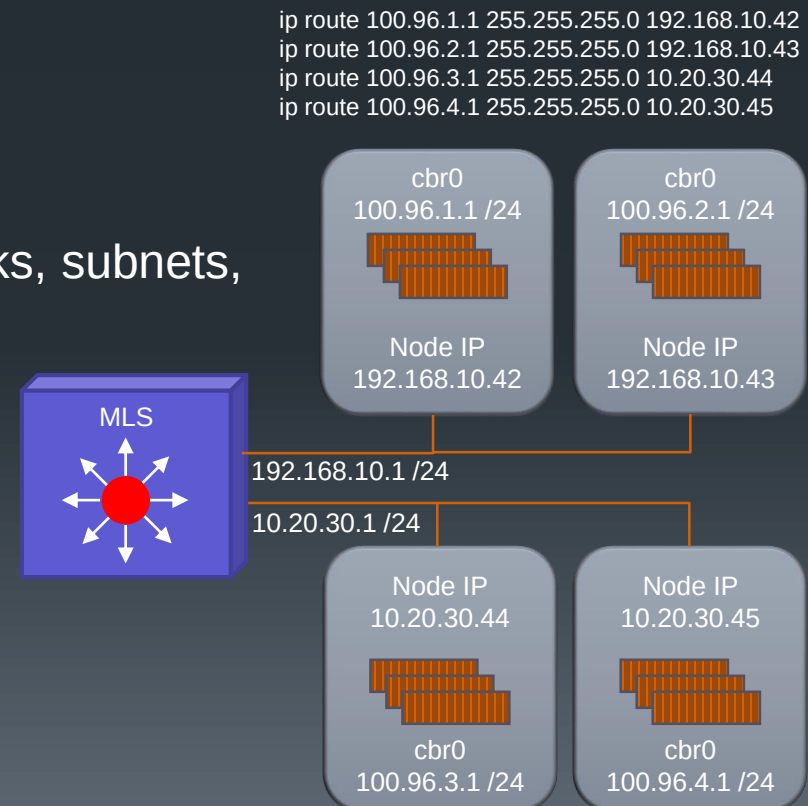


L3 Example: Bridge Routing

212

Copyright 2013-2020, RX-M LLC

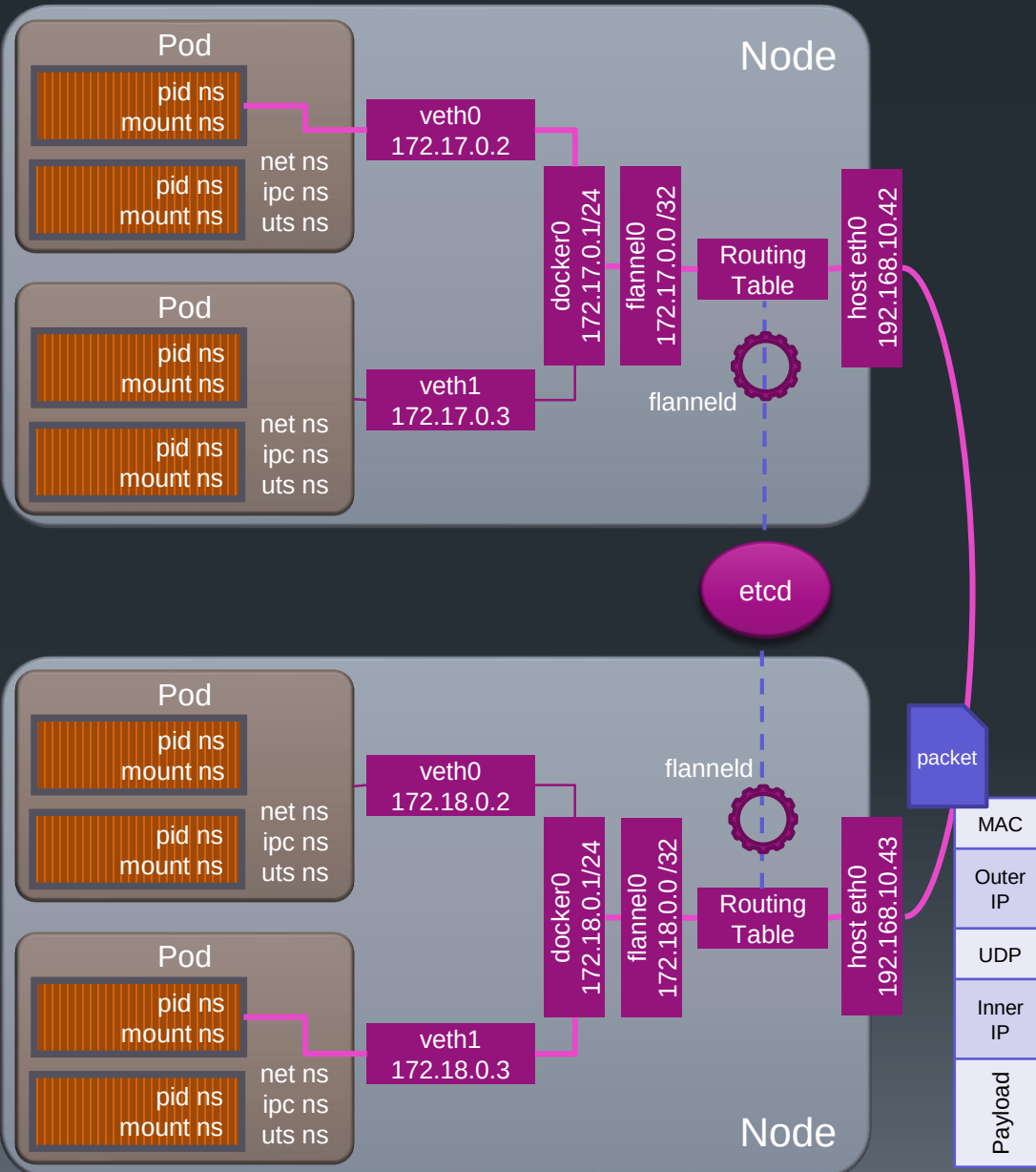
- In a routed solution **each container bridge** has a unique subnet
 - Docker's `docker0` bridge is an example of Linux bridge
 - Kubelet can create and manage bridges in **kubenet** mode (`cbr0`)
- **Hosts route traffic directly or through a central router**
 - Often an MLS is used
- **Pros**
 - High performance
 - Easy configuration
 - Integration with cloud-provider VPC networks, subnets, security groups, etc.
- **Cons**
 - Routes advertised everywhere
 - External process required for policy-based networking enforcement
 - Limits on cloud-provider VPC networks
 - IP translation overhead



L4 Example: VXLAN

213

Copyright 2013-2020, RX-M LLC



- Uses VLAN-like encapsulation to encapsulate layer 2 Ethernet frames within layer 4 UDP datagrams
- Pros
 - Agnostic to underlying networks
 - Resilient to topology changes
 - Provides logical isolation
 - Agent can enforce network policies (in many cases)
- Cons
 - Encapsulation/de-encapsulation slows down traffic
 - VM guest traffic may already be encapsulated, leading to double-encapsulation when container-to-container traffic is between guests on separate hosts
 - More difficult to monitor

Kubernetes Networking Options

- No K8s networking
 - `--network-plugin=noop`
 - Host based or Docker supported SDN
- K8s kubenet
 - `--network-plugin=kubenet`
 - Only works on Linux
 - Like docker0 bridge but uses a linux bridge called `cbr0`
- K8s CNI plugin (alpha)
 - `--network-plugin=cni`
 - Pod network setup and teardown handled by a CNI plugin
 - Uses `--cni-bin-dir` and `--cni-conf-dir`

CNI integration

- Kubernetes kubelets create Pods, which requires the kubelet to configure pod networking
 - The CNI plugin is selected by passing Kubelet the `--network-plugin=cni` command-line option
 - Kubelet reads a file from `--cni-conf-dir` (default `/etc/cni/net.d`) and uses the CNI configuration from that file to set up each pod's network
 - The CNI configuration file must match the CNI specification
 - Any required CNI plugins referenced by the configuration must be present in `--cni-bin-dir` (default `/opt/cni/bin`)
 - Kubernetes requires the standard CNI Io plugin, at minimum version 0.2.0
 - HostPort does not presently work with CNI networking plugins
 - Pod hostPort attributes are ignored presently when using CNI
 - <https://github.com/kubernetes/kubernetes/issues/31307>

CNI

- Container Network Interface – github.com/containernetworking/cni
- Open Source project consisting of:
 - A specification
 - Libraries in Go for writing plugins to configure network interfaces in Linux containers
 - Sample supported plugins
- CNI concerns itself only with
 - Network connectivity of containers
 - Removing allocated resources when the container is deleted
- A CNCF project as of Spring 2017
- Diverges from the more complex CNM (Container Networking Model) used by Docker
 - containerd subproject go-cni adds CNI support to containerd

Container runtimes

- rkt - container engine
- Kubernetes - a system to simplify container operations
- OpenShift - Kubernetes with additional enterprise features
- Cloud Foundry - a platform for cloud applications
- Apache Mesos - a distributed systems kernel
- Amazon ECS - a highly scalable, high performance container management service
- Singularity - container platform optimized for HPC, EPC, and AI
- OpenSVC - orchestrator for legacy and containerized application stacks

3rd party plugins

- Project Calico - a layer 3 virtual network
- Weave - a multi-host Docker network
- Contiv Networking - policy networking for various use cases
- SR-IOV
- Cilium - BPF & XDP for containers
- Infoblox - enterprise IP address management for containers
- Multus - a Multi plugin
- Romana - Layer 3 CNI plugin supporting network policy for Kubernetes
- CNI-Genie - generic CNI network plugin
- Nuage CNI - Nuage Networks SDN plugin for network policy kubernetes support
- Silk - a CNI plugin designed for Cloud Foundry
- Linen - a CNI plugin designed for overlay networks with Open vSwitch and fit in SDN/OpenFlow network environment
- Vhostuser - a Dataplane network plugin - Supports OVS-DPDK & VPP
- Amazon ECS CNI Plugins - a collection of CNI Plugins to configure containers with Amazon EC2 elastic network interfaces (ENIs)
- Bonding CNI - a Link aggregating plugin to address failover and high availability network
- ovn-kubernetes - an container network plugin built on Open vSwitch (OVS) and Open Virtual Networking (OVN) with support for both Linux and Windows
- Juniper Contrail / TungstenFabric - Provides overlay SDN solution, delivering multicloud networking, hybrid cloud networking, simultaneous overlay-underlay support, network policy enforcement, network isolation, service chaining and flexible load balancing
- Knitter - a CNI plugin supporting multiple networking for Kubernetes
- DANM - a CNI-compliant networking solution for TelCo workloads running on Kubernetes
- VMware NSX – a CNI plugin that enables automated NSX L2/L3 networking and L4/L7 Load Balancing; network isolation at the pod, node, and cluster level; and zero-trust security policy for your Kubernetes cluster.

CNI Operations

- Network Add and Delete methods with supporting JSON
 - `cniVersion` (string) - ensures backwards/forwards compatibility
 - `name` (string) - “network” name
 - `type` (string) - plugin name
 - `args` (dictionary) - optional
 - `ipam` (dictionary) - specify the IPAM plugin, if any
 - `dns` - static DNS settings
 - Nameservers
 - Domain
 - Search
 - options

```
{  
  "cniVersion": "0.3.1",  
  "name": "dbnet",  
  "type": "bridge",  
  "bridge": "cni0",  
  "ipam": {  
    "type": "host-local",  
    "subnet": "10.1.0.0/16",  
    "gateway": "10.1.0.1"  
  },  
  "dns": {  
    "nameservers": [ "10.1.0.1" ]  
  }  
}
```

SDN Solutions

218

Copyright 2013-2020, RX-M LLC

- **ACI** – APIC Release 3.0(1)+ supports Kubernetes 1.7-1.8 on bare-metal servers
 - Enterprise support from **Cisco**, uses ACI CNI plugin and Open vSwitch daemonsets
- **Antrea** - Open vSwitch (OVS) based CNI solution for Kubernetes with network policy support
 - A Linux Foundation project backed by **VMware**
- **AOS** - uses commercial and white-box systems, supported by **Apstra**
- **AWS VPC** - Integrated AWS Virtual Private Cloud (VPC) networking for Kubernetes clusters
- **Big Cloud Fabric** – SDN fabric solution based on industry-standard white-/brite-box hardware
 - Enterprise support from **Big Switch Networks**
- **Calico** – uses BGP to enable real container ips
 - Enterprise support from **Tigera**
- **Canal** combines Flannel L4 SDN with Calico policy-based management features
 - Enterprise support from **Tigera**
- **Cilium** – generates Berkeley Packet Filter (BPF) programs to provide networking, security, and load balancing
- **Contiv** – supports routed (L3/BGP), bridged (L2/VLAN), overlay (VXLAN) and ACI
 - Enterprise support from **Cisco**
- **DANM** - a networking solution for telco workloads running in a Kubernetes cluster, backed by **Nokia**
- **Docker Overlay**– uses VXLAN
 - Enterprise support from **Mirantis**
- **Flannel** –automatically assigns subnets to Docker Daemon bridges and tunnels between nodes via VXLAN
 - Created by CoreOS as part of Tectonic platform, now supported by **Tigera** through Canal
- **NSX-T** – network virtualization for a multi-cloud and multi-hypervisor environment
 - Uses Geneve (rather than VXLAN) [Generic Network Virtualization Encapsulation], enterprise support from **VMware**
- **Nuage** – policy based Open vSwitch SDN
 - Enterprise support from **Nuage Networks** (a Nokia company)
- **OpenVSwitch** – somewhat more mature but complicated way to build an overlay network endorsed by several key networking vendors
 - A Linux Foundation project, created by Nicira (now part of **VMware**) [ovn-kubernetes]
- **Romana** – layer 3 networking solution for pod networks
- **Tungsten Fabric** – Linux Foundation's Tungsten Fabric (formerly OpenContrail from Juniper Networks) multi-cloud
- **Weave** – provides networking (VXLAN) and network policy; does not require an external database
 - Enterprise support from **Weaveworks**

- Integrating Kubernetes with NSX-T involves three components
 - **NSX Container Plugin (NCP)**
 - Runs as a Pod in the Kubernetes cluster
 - Watches for changes on Kubernetes Objects (namespaces, network policies, services etc.)
 - Creates networking constructs in NSX-T based on the object add/change events
 - **NSX CNI Plugin**
 - A small executable installed on all Kubernetes Nodes
 - Kubelet will instantiate/call the CNI Plugin to handle the Pod network attachment
 - **NSX Kube-Proxy**
 - A daemon running on the Kubernetes Nodes in a container image
 - Run as a Kubernetes Daemon-Set
 - Uses OpenVSwitch (OVS) load-balancing features
 - **Replaces the native Kube-Proxy**, which uses IPTables
- Integration with underlying VMware IaaS NSX eliminates double encapsulation

Taken from a Spring 2017 blog by Yves Fauser, a Technical Product Manager in VMware's Network and Security Business Unit

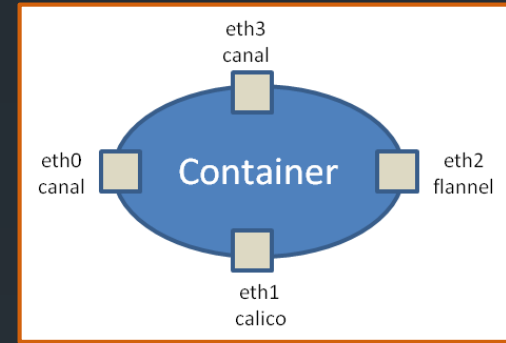
Multi CNI

220

Copyright 2013-2020, RX-M LLC

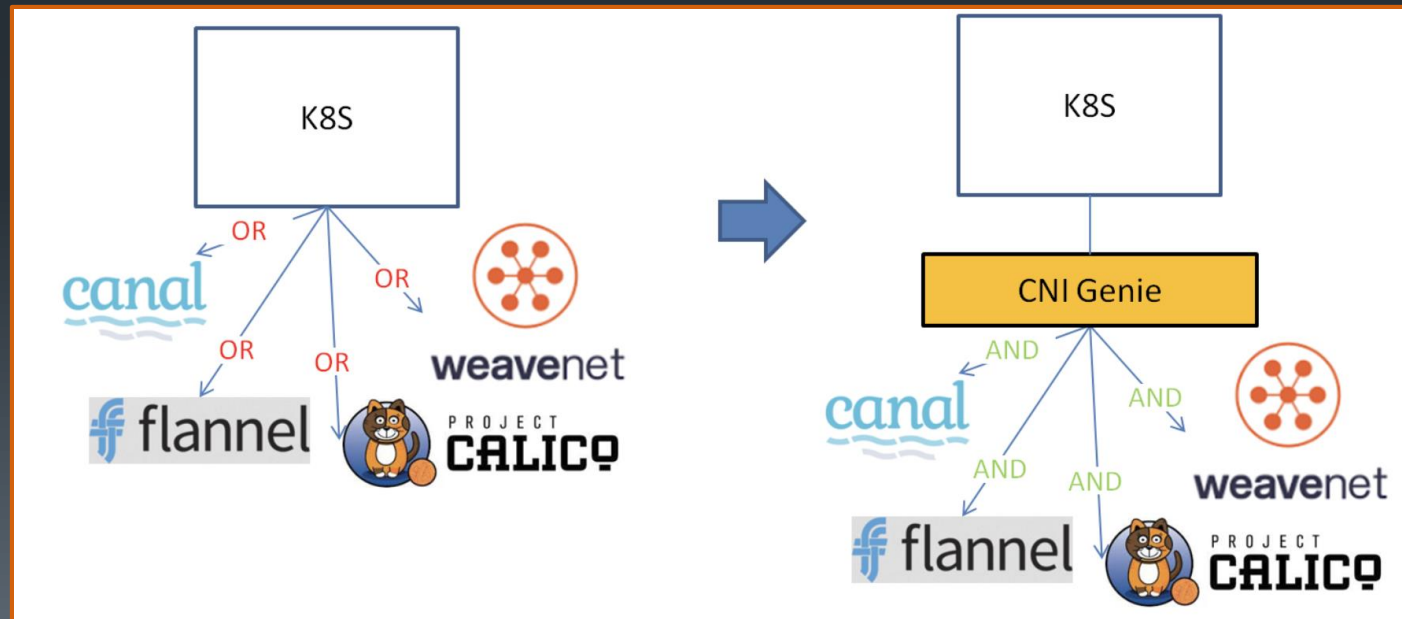
■ CNI Gene

- Multiple CNI plugins are available simultaneously behind the Genie CNI interface
- Enables orchestrators to connect to a given CNI implementation determined at run time
- Current support for:
 - Calico
 - Canal
 - Romana
 - Weave
- Individual pods/containers can be connected to multiple network implementations simultaneously
- Currently tested with Kubernetes



■ Multus

- A similar Multi CNI feature using CRD based network objects in Kubernetes



Network Policies

- Specification of how groups of pods are **allowed to communicate with each other** and other network endpoints
 - Use labels to select pods and define rules which specify what traffic is allowed to the selected pods
 - A pod will reject any connections that are not allowed by a NetworkPolicy
- **Implemented by a network plugin**
 - Must use a provider which supports NetworkPolicy
 - Simply creating the resource without a controller to implement it will have no effect
- **Spec**
 - **podSelector** – defines the destination pods for the policy
 - e.g. pods with the label `role=db`
 - **policyTypes** – indicates whether the given policy applies to ingress traffic, egress traffic, or both
 - **ingress** – whitelist allows traffic which matches **both** the **from** and **ports** sections
 - **ipBlock** – describes a particular CIDR that is allowed
 - **except** – slice of CIDRs that should not be included within an IP Block
 - **egress** – each rule allows traffic which matches both the **to** and **ports**
- Create a “default” *isolation policy* for a Namespace by creating a NetworkPolicy that selects all pods but does not allow any traffic
 - By default, pods accept traffic from any source

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mynetpolicy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - ipBlock:
            cidr: 172.17.0.0/16
            except:
              - 172.17.1.0/24
        - namespaceSelector:
            matchLabels:
              team: project
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 6379
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.0.0/24
      ports:
        - protocol: TCP
          port: 5978

```



Summary

- SDN solutions enable communications across virtual machines and containers
- SDN options range from L2 to L3 to L4
 - VLANs
 - Routing
 - Overlay/Tunneling
- Third parties provide a range of Container networking functions that (properly integrated) the kube-proxy can work with

Lab 10

- Network Policy

Lab 10 Documentation References

- Network Policy

- <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- <https://kubernetes.io/blog/2017/10/enforcing-network-policies-in-kubernetes/>

- Network Policy Spec

- <https://kubernetes.io/docs/concepts/services-networking/network-policies/#the-networkpolicy-resource>

11. etcd

Objectives

- Examine etcd cluster architecture
- Understand how to properly spec and size an etcd cluster
- Learn how to setup and bootstrap a multinode etcd cluster
- Explore day two operations
 - Adding/removing nodes
 - Cluster backup/restore
 - Troubleshooting
- Describe etcd best practices

Etcdd defined

- An open source, highly consistent, distributed, key/value store
 - /etc directory distributed
 - /etc is traditionally where Unix systems store their configuration files
 - An open source clone of Google's Chubby [Burrows, OSDI '06]
 - Designed to reliably store infrequently updated data and provide reliable watch queries
- etcd stores data in a multiversion persistent key-value store
 - Preserves the previous version of a key-value pair when the value is updated
 - Existing key-value data is immutable
 - Operations do not update existing state, they append new state versions
 - All past versions of keys are still accessible and watchable after modification
 - To prevent the data store from growing indefinitely the store may be compacted to shed older versions of superseded data



Etcd versions

228

Copyright 2013-2020, RX-M LLC

- 2019-05-02 v3.3.13 – current version
- 2018-02-01 v3.3.0
 - Adds support for dbs larger then 8GB (8GB is still the recommended maximum)
 - Support for certificate revocation
- 2018-01-30 >> RedHat Acquires CoreOS <<
- 2017-06-19 v3.2.0
 - JWT authentication
 - Multi-tenancy through new namespacing client/proxy providing isolated keyspaces
 - Scaling improvements with gRPC proxy now supporting a million events per second
- 2017-01-20 v3.1.0
 - Leadership in now transferred when stopping the leader
 - Experimental gRPC proxy (accepts many client watches and creates one etcd backend watch, increasing watch scalability while reducing load on backend)
 - Many bug fixes
- 2017-02-17 v2.3.8 latest
 - Bug fix final release of the v2 branch
- 2016-06-30 v3.0.0
 - Server API uses gRPC instead of HTTP/JSON as in etcd2
 - etcd2 JSON endpoints and cluster protocol are still provided in etcd3
 - Key space moved from hierarchical JSON to binary k/v with an MVCC (multi-version concurrency control) transaction model
 - Performance improvement over v2 is 2 x or more
- 2016-03-18 v2.3.0
 - Profiling
 - Experimental v3 support
- 2015-09-10 v2.2.0
 - Zero downtime rolling upgrades
- 2015-06-21 v2.1.0
 - Authentication support
- 2015-01-28 v2.0.0
 - First GA release
- 2014-05-20 v0.4.0
 - Bug fixes
- 2014-02-07 v0.3.0
 - Windows support
- 2013-12-27 v0.2.0
 - New v2 API
- 2013-08-11 v0.1.0
 - Initial release by Core OS



Open Source Projects for Linux Containers

Getting Etcd

- The etcd [releases page on github](#) packages various releases:
 - [OSX](#) (Darwin)
 - [Linux](#) (Intel, ARM and PowerPC)
 - [Windows](#)
 - Source
- Releases can be verified with the release signing key at:
 - coreos.com/security/app-signing-key
- Docker containers are hosted on gcr.io:
 - gcr.io/coreos/etcd-development/etcd

```
docker run \  
-p 2379:2379 \  
-p 2380:2380 \  
--mount type=bind,source=/tmp/etcd-data.tmp,destination=/etcd-data \  
--name etcd-gcr-v3.3.8 \  
gcr.io/etcd-development/etcd:v3.3.8 \  
/usr/local/bin/etcd \  
--name s1 \  
--data-dir /etcd-data \  
--listen-client-urls http://0.0.0.0:2379 \  
--advertise-client-urls http://0.0.0.0:2379 \  
--listen-peer-urls http://0.0.0.0:2380 \  
--initial-advertise-peer-urls http://0.0.0.0:2380 \  
--initial-cluster s1=http://0.0.0.0:2380 \  
--initial-cluster-token tkn \  
--initial-cluster-state new
```

The screenshot shows the GitHub repository page for `coreos/etcd`. The 'Releases' tab is selected, displaying the latest release, `v3.3.8`, which was released 28 days ago by `gyuho`. The release is marked as 'Latest release' and 'Verified'. Below the release information, a list of 'Assets' is provided, including binaries for Darwin, Linux, and Windows, as well as source code archives. The assets listed are:

- `etcd-v3.3.8-darwin-amd64.zip`
- `etcd-v3.3.8-darwin-amd64.zip.asc`
- `etcd-v3.3.8-linux-aarch64.aci`
- `etcd-v3.3.8-linux-aarch64.aci.asc`
- `etcd-v3.3.8-linux-amd64.aci`
- `etcd-v3.3.8-linux-amd64.aci.asc`
- `etcd-v3.3.8-linux-amd64.tar.gz`
- `etcd-v3.3.8-linux-amd64.tar.gz.asc`
- `etcd-v3.3.8-linux-arm64.tar.gz`
- `etcd-v3.3.8-linux-arm64.tar.gz.asc`
- `etcd-v3.3.8-linux-ppc64le.tar.gz`
- `etcd-v3.3.8-linux-ppc64le.tar.gz.asc`
- `etcd-v3.3.8-windows-amd64.zip`
- `etcd-v3.3.8-windows-amd64.zip.asc`
- `v3.3.8.tar.gz`
- `v3.3.8.tar.gz.asc`
- `v3.3.8.zip`
- `v3.3.8.zip.asc`
- `Source code (zip)`
- `Source code (tar.gz)`

etcdctl

230

Copyright 2013-2020, RX-M LLC

- etcdctl is a command line tool for interacting with etcd servers
- By default etcdctl talks to the etcd server with the v2 API for backward compatibility
- etcdctl can speak to etcd using the v3 API by setting the API version to 3
 - `export ETCDCTL_API=3`
- To get help on a specific command use the `--help` switch
 - `$ etcdctl get --help`

```
user@ubuntu:~$ export ETCDCTL_API=3
user@ubuntu:~$ etcdctl version
etcdctl version: 3.3.8
API version: 3.3
user@ubuntu:~$ etcdctl put car mustang
OK
user@ubuntu:~$ etcdctl get car
car
mustang
user@ubuntu:~$
```

```
user@ubuntu:~$ etcd-v3.3.8-linux-amd64/etcdctl
NAME:
  etcdctl - A simple command line client for etcd.

WARNING:
  Environment variable ETCDCTL_API is not set; defaults to etcdctl v2.
  Set environment variable ETCDCTL_API=3 to use v3 API or ETCDCTL_API=2 to use v2 API.

USAGE:
  etcdctl [global options] command [command options] [arguments...]

VERSION:
  3.3.8

COMMANDS:
  backup          backup an etcd directory
  cluster-health  check the health of the etcd cluster
  mk             make a new key with a given value
  mkdir          make a new directory
  rm            remove a key or a directory
  rmdir         removes the key if it is an empty directory or a key-value pair
  get           retrieve the value of a key
  ls           retrieve a directory
  set          set the value of a key
  setdir      create a new directory or update an existing directory TTL
  update      update an existing key with a given value
  updatedir   update an existing directory
  watch       watch a key for changes
  exec-watch  watch a key for changes and exec an executable
  member      member add, remove and list subcommands
  user        user add, grant and revoke subcommands
  role        role add, grant and revoke subcommands
  auth        overall auth controls
  help, h     Shows a list of commands or help for one command
```

Configuring etcd

231

Copyright 2013-2020, RX-M LLC

- etcd is configurable through:
 - A configuration file
 - Command-line flags
 - Environment variables
- Configuration files:
 - The etcd configuration file is YAML formatted and composed of command-line flag keys with argument values
 - The configuration file path is specified on the command line using the `--config-file` flag
- Options precedence is as follows:
 - First: configuration file values
 - Second: command line arguments
 - Third: environment variables
 - Fourth: defaults
- Environment Variables
 - Environment variable are named for the flag they implement with dashes changed to underbars
 - For example: the `--snapshot-count` command line switch can be set with the `SNAPSHOT_COUNT` environment variable
- Official etcd ports are:
 - 2379 for client requests
 - 2380 for peer communication
- etcd ports can be set to accept TLS traffic, non-TLS traffic, or both

```
# Number of committed transactions to trigger a snapshot to disk.
snapshot-count: 10000

# Time (in milliseconds) of a heartbeat interval.
heartbeat-interval: 100

# Time (in milliseconds) for an election to timeout.
election-timeout: 1000
```

Basic command line args

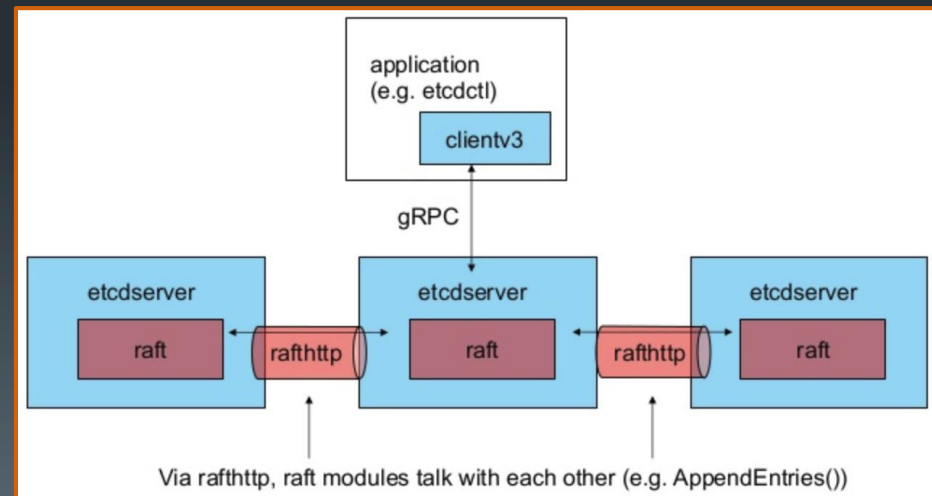
- **--name**
 - Sets server name
 - default: "default"
- **--data-dir**
 - Sets WAL and SNAP data dir
 - default: "\${name}.etcd"
- **--listen-peer-urls**
 - URLs to listen on for peer traffic
 - can be http or https
 - Must be IP based with the exception of localhost, 0.0.0.0 specifies all interfaces
 - Ports can be added if the default port is not appropriate
 - Multiple URLs may be used to specify multiple listening addresses/ports
 - default: "http://localhost:2380"
- **--listen-client-urls**
 - Like peer URLs but for client traffic
 - default: "http://localhost:2379"
- **--advertise-client-urls**
 - List of this member's client URLs to advertise to the rest of the cluster (can contain domain names)
 - default: "http://localhost:2379"
- **--initial-advertise-peer-urls**
 - List of this member's peer URLs to advertise to the rest of the cluster
 - Must be routable to all cluster members and can contain domain names
 - default: "http://localhost:2380"
- **--initial-cluster**
 - Initial cluster configuration for bootstrapping
 - default: "default=http://localhost:2380"
 - Typically like:
--initial-cluster
nodea=http://10.0.1.10:2380,nodeb=http://10.0.1.11:2380,nodec=http://10.0.1.12:2380
- **--initial-cluster-token**
 - Initial cluster token for the etcd cluster during bootstrap
 - default: "etcd-cluster"
- **--initial-cluster-state**
 - Can be "new" or "existing"
 - Set to new for all members present during initial bootstrapping
 - If this option is set to existing, etcd will attempt to join the existing cluster
 - default: "new"

Architecture

233

Copyright 2013-2020, RX-M LLC

- **Transaction log** based
 - In etcd the transaction log is known as a **WAL (write ahead log)**
 - Each etcd update is written to the transaction log
 - Each etcd node independently maintains and updates its own log
- **Snapshot** operations
 - Snapshots capture the current state of all data
- **gRPC API**
 - Etcd exposes a gRPC API to clients
- **Raft consensus group**
 - Raft is a consensus algorithm designed to be easy to implement correctly
 - <https://raft.github.io/>
 - A majority of nodes must commit a transaction for it to survive
- **Leader based cluster**
 - **Only the leader node writes** to the other nodes
 - Any node can support reads
 - Non-leaders can accept writes but pass them to the leader for processing
 - If the leader fails the remaining nodes can elect a new leader

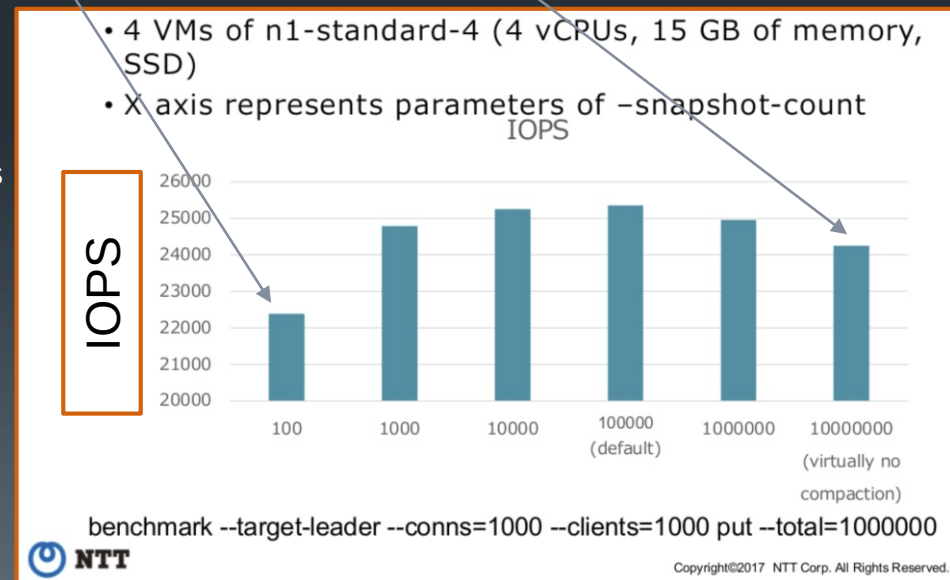


Snapshots

234

Copyright 2013-2020, RX-M LLC

- If systems had enough space to allow the WAL to grow perpetually, system operation would be trivial, appending new data perpetually
 - This is not usually possible, disk and memory are limited, requiring the log to be “compacted” periodically
- **Compaction in etcd involves snapshotting the current state and disposing of the WAL**
- There are tradeoffs between frequent and in-frequent compaction
 - **Frequent compaction:**
 - Hurts runtime performance (snapshots are expensive to create)
 - Speeds recovery times (recovering a cluster involves restoring a snapshot and applying any additional transactions from the WAL [small WAL == fast recovery])
 - **Infrequent compaction:**
 - Speeds runtime performance (less compaction overhead)
 - Hurts recovery times (many changes need to be applied to recover the current state [large WAL == slow recovery])
- etcd nodes can be tuned for efficient snapshotting
 - Snapshot count
 - Because creating snapshots is expensive snapshots are only created by default after 100,000 log entries
 - If memory/disk usage is high, lowering snapshot threshold will reduce log size and thus memory consumption
 - `--snapshot-count=10000`
 - Max Snapshots
 - The maximum number of snapshots maintained by etcd defaults to 5 and can be set
 - `--max-snapshots=3`
 - 0 means unlimited



etcd disk structure

235

Copyright 2013-2020, RX-M LLC

- etcd stores all state in the etcd data directory
 - By default: `./default.etcd/member`
- The data has two sub-directories:
 - `/wal`
 - (write ahead) log files are stored here
 - Files space is pre allocated to avoid write delays
 - Files contain sequential transactions applied to the data store
 - Each transaction creates a new database version
 - Can be set using `--wal-dir`
- `/snap`
 - snapshots are stored here
 - A copy of all keys and values at a given version number

```
user@ubuntu:~$ docker run -d --name etcd gcr.io/etcd-development/etcd
38e368995aa840c0b772366f466b2bfba3f12384d9056537e78e1b61855e2ba7
user@ubuntu:~$ docker exec -it 38e3 sh
/ # ls -l
total 56
drwxr-xr-x  2 root    root    4096 Jan  9  2018 bin
drwx----- 3 root    root    4096 Jul 13 23:08 default.etcd
drwxr-xr-x  5 root    root    340 Jul 13 23:08 dev
drwxr-xr-x  1 root    root    4096 Jul 13 23:08 etc
drwxr-xr-x  2 root    root    4096 Jan  9  2018 home
drwxr-xr-x  5 root    root    4096 Jan  9  2018 lib
drwxr-xr-x  5 root    root    4096 Jan  9  2018 media
drwxr-xr-x  2 root    root    4096 Jan  9  2018 mnt
dr-xr-xr-x 230 root    root      0 Jul 13 23:08 proc
drwx----- 1 root    root    4096 Jul 13 23:09 root
drwxr-xr-x  2 root    root    4096 Jan  9  2018 run
drwxr-xr-x  2 root    root    4096 Jan  9  2018/sbin
drwxr-xr-x  2 root    root    4096 Jan  9  2018/srv
dr-xr-xr-x 13 root    root      0 Jul 13 23:08 sys
drwxrwxrwt  2 root    root    4096 Jan  9  2018 tmp
drwxr-xr-x  1 root    root    4096 Jan  9  2018 usr
drwxr-xr-x  1 root    root    4096 Jun 15 16:57 var
/ # cd default.etcd/
/default.etcd # ls -l
total 4
drwx----- 4 root    root    4096 Jul 13 23:08 member
/default.etcd # cd member/
/default.etcd/member # ls -l
total 8
drwx----- 2 root    root    4096 Jul 13 23:08 snap
drwx----- 2 root    root    4096 Jul 13 23:08 wal
/default.etcd/member # ls -l wal
total 125000
-rw-----  1 root    root    64000000 Jul 13 23:08 0.tmp
-rw-----  1 root    root    64000000 Jul 13 23:08 0000000000000000-0000000000000000.wal
/default.etcd/member # ls -l snap
total 20
-rw-----  1 root    root    20480 Jul 13 23:08 db
/default.etcd/member #
```


etcd cluster sizing

- An etcd cluster needs a majority of nodes (a quorum) to agree on updates to the cluster state
 - For a cluster with n members, quorum is $(n/2)+1$
 - If the cluster is in a state where it can't tolerate any more failures, adding a node before removing nodes is dangerous because if the new node fails to register with the cluster (e.g., the address is misconfigured), quorum will be permanently lost
- An etcd cluster should have no more than seven nodes
 - Google Chubby lock service (similar to etcd) is widely deployed within Google and suggests running five nodes
 - A 5-member etcd cluster can tolerate two member failures, which is enough in most cases
 - You can perform maintenance on one node and still tolerate another node to failing
- Larger clusters provide better fault tolerance but write performance suffers because data must be replicated across more machines

The diagram shows a table with three columns: Cluster Size, Majority, and Failure Tolerance. Arrows point from the table to labels for different environments: Local (points to size 1), UAT/Staging (points to size 3), Prod (points to size 5), and Paranoid (points to size 9).

Cluster Size	Majority	Failure Tolerance
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3
8	5	3
9	5	4

AZ distribution

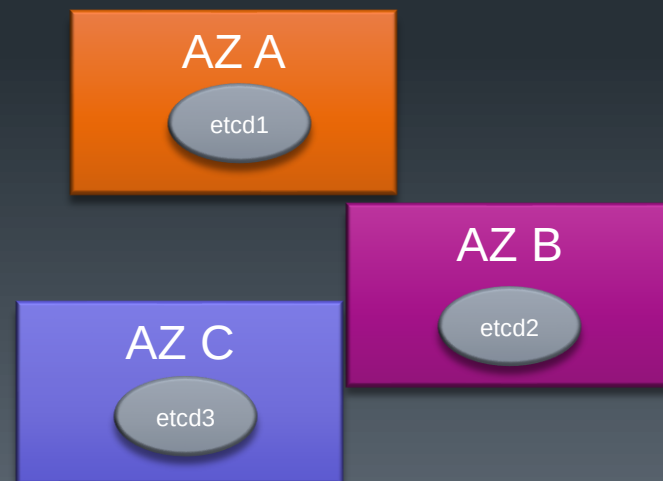
237

Copyright 2013-2020, RX-M LLC

- Bad!
 - Even number of zones
 - If the wrong zone goes down you lose quorum



-
- Good!
 - Odd number of zones
 - Any zone can go down and you retain quorum



etcd system sizing

238

Copyright 2013-2020, RX-M LLC

- RedHat standard etcd performance benchmark system:

- 8 vCPU
- 16GB RAM
- 50GB SSD

- Etcd works well with a modern low latency storage system and a few GB of memory in many use cases

- Etcd v3 uses memory mapped files enlisting OS optimized paging logic to move data to disk

- etcd is very sensitive to disk write latency

- Minimum disk performance of 50 random write IOPS (e.g., 7200 RPM disk)
- For heavily loaded clusters, > 500 rand write IOPS (local SSD or high performance virtualized block device)
- Use disk test tools to measure actual IOPS
 - The most common I/O utilities on Linux are Sysbench, FIO, and IOzone

```
ubuntu@ip-172-31-7-33:~$ free
              total        used        free      shared  buff/cache   available
Mem:           4045064      132272      1326676        6032       2586116       3622448
Swap:              0              0              0
ubuntu@ip-172-31-7-33:~$ lscpu | grep 'CPU(s):'
CPU(s):                2
NUMA node0 CPU(s):      0,1
ubuntu@ip-172-31-7-33:~$ fio --name=randwrite --ioengine=libaio --iodepth=1 --rw=randwrite --bs=4k
--direct=0 --size=4GB --numjobs=2 --runtime=240 --group_reporting
randwrite: (g=0): rw=randwrite, bs=4K-4K/4K-4K/4K-4K, ioengine=libaio, iodepth=1
...
fio-2.2.10
Starting 2 processes
randwrite: Laying out IO file(s) (1 file(s) / 4096MB)
randwrite: Laying out IO file(s) (1 file(s) / 4096MB)
Jobs: 1 (f=0): [E(1),w(1)] [36.6% done] [0KB/14092KB/0KB /s] [0/3523/0 iops] [eta 06m:57s]
randwrite: (groupid=0, jobs=2): err= 0: pid=8929: Thu Jul 12 03:22:22 2018
   write: io=3065.4MB, bw=13078KB/s, iops=3269, runt=240010msec
    slat (usec): min=2, max=92007, avg=608.61, stdev=3700.35
    clat (usec): min=0, max=3332, avg= 0.91, stdev= 8.13
      lat (usec): min=4, max=92014, avg=610.01, stdev=3700.95
    clat percentiles (usec):
       | 1.00th=[  0], 5.00th=[  0], 10.00th=[  0], 20.00th=[  0],
       | 30.00th=[  1], 40.00th=[  1], 50.00th=[  1], 60.00th=[  1],
       | 70.00th=[  1], 80.00th=[  1], 90.00th=[  1], 95.00th=[  1],
       | 99.00th=[  3], 99.50th=[  4], 99.90th=[  6], 99.95th=[ 10],
       | 99.99th=[ 17]
    bw (KB /s): min= 3968, max=516562, per=49.25%, avg=6441.06, stdev=20822.51
    lat (usec) : 2=95.91%, 4=3.24%, 10=0.80%, 20=0.05%, 50=0.01%
    lat (usec) : 100=0.01%, 750=0.01%
    lat (msec) : 2=0.01%, 4=0.01%
    cpu          : usr=0.38%, sys=1.19%, ctx=41669, maj
    IO depths    : 1=100.0%, 2=0.0%, 4=0.0%, 8=0.0%, 16
    submit      : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 3
    complete    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 3
    issued      : total=r=0/w=784732/d=0, short=r=0/w=
    latency     : target=0, window=0, percentile=100.0
Run status group 0 (all jobs):
   WRITE: io=3065.4MB, aggrb=13078KB/s, minb=13078KB/s
msec
Disk stats (read/write):
   xvda: ios=13/604380, merge=0/170646, ticks=20/1472468, in_queue=1472584, util=98.48%
ubuntu@ip-172-31-7-33:~$
```

FIO TEST (sudo apt-get install fio)
AWS t2.medium instance
(4GB RAM, 2 vCPU, 30GB gp2 disk [SSD])
FIO disk test: 3,269 rand write IOPS
Tested using 2x RAM size to reduce
caching with 2 jobs (sim #s for 1-8 jobs)
Divide by # of jobs and 10 to est etcd iops
 $3269 / 2 / 10 == 163.4$

Sample hardware configurations

239

Copyright 2013-2020, RX-M LLC

- etcd has a relatively small memory footprint however
 - 8GB is appropriate for most moderate use cases
 - Large deployments with thousands of watchers and millions of keys will need 16GB to 64GB
- etcd memory uses:
 - caching key-value data
 - tracking watchers

K8s Nodes	Cloud	Type	vCPUs	Memory (GiB)	Storage (GB)	Network Performance	Price/ Hour
50	Up to 100 clients, up to 200 requests per second, up to 100MB of data						\$0.10
	AWS	m5.large	2	8	EBS	Up to 10 Gigabit	
	GCP	n1-standard-2	2	7.5	PD	Up to 10 Gigabit	
250	Up to 500 clients, up to 1,000 requests per second, up to 500MB of data						\$0.20
	AWS	m5.xlarge	4	16	EBS	Up to 10 Gigabit	
	GCP	n1-standard-4	4	15	PD	Up to 10 Gigabit	
1000	Up to 1,500 clients, up to 10,000 requests per second, up to 1GB of data						\$0.40
	AWS	m5.2xlarge	8	32	EBS	Up to 10 Gigabit	
	GCP	n1-standard-8	8	30	PD	Up to 10 Gigabit	
3000	Over 1,500 clients, over 10,000 requests per second, over 1GB of data						\$0.80
	AWS	m5.4xlarge	16	64	EBS	Up to 10 Gigabit	
	GCP	n1-standard-16	16	60	PD	Up to 10 Gigabit	

Creating a Cluster

240

Copyright 2013-2020, RX-M LLC

- In etcd cluster initialization **each member must know at least one other member** in the cluster
 - If the IPs of the cluster members are not known ahead of time the cluster can be bootstrapped with a discovery service
- There are 3 common bootstrap methods
 - **Static** – hand configured
 - **etcd Discovery** – discovery service provided on the internet by RedHat
 - **DNS Discovery** – normal DNS lookups used
- Adding/Removing members is done at runtime on running clusters
- Every etcd cluster has an **ClusterID** and every member in an etcd cluster has a **MemberID**
- The initial cluster token setting allows you to create multiple etcd clusters with same configuration without conflicts
 - **etcd generates unique cluster IDs and member IDs using the initial cluster token**

```
$ etcd --name infra0 --initial-advertise-peer-urls http://10.0.1.10:2380 \  
--listen-peer-urls http://10.0.1.10:2380 \  
--listen-client-urls http://10.0.1.10:2379,http://127.0.0.1:2379 \  
--advertise-client-urls http://10.0.1.10:2379 \  
--initial-cluster-token etcd-cluster-1 \  
--initial-cluster infra0=http://10.0.1.10:2380,infra1=http://10.0.1.11:2380,infra2=http://10.0.1.12:2380 \  
--initial-cluster-state new
```

Adding/Removing nodes

- Adding a member is a two step process:
 - Add the new member to the cluster via the API or etcdctl member add
 - `$ etcdctl member add infra3 http://10.0.1.13:2380`
 - Start the new member with the new cluster configuration
- To remove a member:
 - Discover the member's ID:

```
$ etcdctl member list
6e3bd23ae5f1eae0: name=node2 peerURLs=http://localhost:23802 clientURLs=http://127.0.0.1:23792
924e2e83e93f2560: name=node3 peerURLs=http://localhost:23803 clientURLs=http://127.0.0.1:23793
a8266ecf031671f3: name=node1 peerURLs=http://localhost:23801 clientURLs=http://127.0.0.1:23791
```
 - To remove the member ID **a8266ecf031671f3**:
 - `$ etcdctl member remove a8266ecf031671f3`
 - The target member will stop and print out the removal in the log
 - It is safe to remove the leader, however the cluster will be inactive while a new leader is elected
 - This duration is normally the period of election timeout plus the voting process

Migrating Nodes

- To migrate a member:

- Discover the member's ID:

```
$ etcdctl member list
```

```
6e3bd23ae5f1eae0: name=node2 peerURLs=http://localhost:23802 clientURLs=http://127.0.0.1:23792
```

```
924e2e83e93f2560: name=node3 peerURLs=http://localhost:23803 clientURLs=http://127.0.0.1:23793
```

```
a8266ecf031671f3: name=node1 peerURLs=http://localhost:23801 clientURLs=http://127.0.0.1:23791
```

- Stop the member process
 - Copy the data directory of the now-idle member to the new machine
 - Update the peer URLs for the replaced member to reflect the new machine
- ```
$ etcdctl member update a8266ecf031671f3 http://10.0.1.10:2380
```
- Start etcd on the new machine, using the same configuration and the copy of the data directory

- etcd supports encrypted communication through TLS
- TLS can be used between peers and clients
- To start a cluster with TLS each cluster member should have:
  - **ca-client.crt**: the CA (certificate authority) trusted by the server to sign client certs
    - Only used to auth clients if the `--client-cert-auth` switch is set, else any client can connect
  - **node-client.crt**: the server public key certificate signed by a CA for use with clients
  - **node-client.key**: the server private key for use with clients
  - **ca-peer.crt**: the CA trusted by the server to sign peer certs
    - Only used to auth peers if the `--peer-client-cert-auth` switch is set, else any peer can connect
  - **node-peer.crt**: the server public key certificate signed by a CA for use with peers
  - **node-peer.key**: private key associated with the node-peer.crt for use with peers

```
$ etcd --name infra0 --initial-advertise-peer-urls https://10.0.1.10:2380 \
--listen-peer-urls https://10.0.1.10:2380 \
--listen-client-urls https://10.0.1.10:2379,https://127.0.0.1:2379 \
--advertise-client-urls https://10.0.1.10:2379 \
--initial-cluster-token etcd-cluster-1 \
--initial-cluster infra0=https://10.0.1.10:2380,infra1=https://10.0.1.11:2380,infra2=https://10.0.1.12:2380 \
--initial-cluster-state new \
--client-cert-auth --trusted-ca-file=/path/to/ca-client.crt \
--cert-file=/path/to/infra0-client.crt --key-file=/path/to/infra0-client.key \
--peer-client-cert-auth --peer-trusted-ca-file=ca-peer.crt \
--peer-cert-file=/path/to/infra0-peer.crt --peer-key-file=/path/to/infra0-peer.key
```



# Best Practices

244

Copyright 2013-2020, RX-M LLC

- Always build odd number of node clusters
  - 1 for experimentation
  - 3 for important systems
  - 5 for critical systems
  - 7 for paranoid systems
- Properly size clusters
- Never reuse initial cluster tokens
  - Duplicate tokens with duplicate configurations will create clusters with duplicate IDs, making cross node associate possible (and failed clusters to go with it)
- Avoid adding a node to a cluster that cannot tolerate a failure
  - A config error could cause the cluster to permanently lose quorum (!)
- Consider running etcd on isolated nodes
  - Improves performance predictability
- Prefer locally attached SSD storage for etcd nodes
  - Etcd performance is closely linked to disk write performance
- Prefer a dedicated disk for the WAL directory
  - Reserves all of the disk's IOPS for WAL writes
- Members with lost or corrupt data should be removed from the cluster
  - The data (wal and snap) directory can be emptied and then the node can be added back as a new node
- Do not advertise localhost URLs
  - They cause proxies to loop, routing to themselves

# Summary

- etcd is a highly consistent key/value store
- Standard cloud system sizing and storage guidelines can help build an appropriate etcd cluster
- Etcd support file based, environment variable base and command line based configuration
- Running clusters can be upgraded, increased and reduced in size
- etcdctl is the etcd administration and testing client

# Lab 11

- Etcd cluster operations

# Lab 11 Documentation References

- etcd in Kubernetes
  - <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/>
- Working with etcdctl
  - <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/#replacing-a-failed-etcd-member>
- Backing up etcd:
  - <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/#backing-up-an-etcd-cluster>

## 12. Exam Prep I

# Objectives

- Review the CKA curriculum
  - Ensure that you understand all of the concepts listed
  - Ask your instructor to cover anything and everything you are not yet comfortable with
  - Discuss the weightings of each topic area and prepare/focus accordingly
- Run practice drills to:
  - Improve your task speed
  - Run into problems now so that you don't on the test
  - Identify knowledge gaps now so that you can shore them up before the test

# CKA Curriculum

# Things to consider

- As you review the curriculum:
  - Ask questions about anything you do not understand or are unfamiliar with
  - The instructor may do demos, assign additional labs or provide challenge drills to help round out knowledge gaps but only you can tell the instructor where your gaps are
- Take notice of the weightings
  - Not all topics are covered equally in the test, being fuzzy in a 5% area may be ok, but being fuzzy in a 19% area could cost you the test



## 5% - Scheduling

- Use label selectors to schedule Pods.
- Understand the role of DaemonSets.
- Understand how resource limits can affect Pod scheduling.
- Understand how to run multiple schedulers and how to configure Pods to use them.
- Manually schedule a pod without a scheduler.
- Display scheduler events.
- Know how to configure the Kubernetes scheduler.

## 5% - Logging/Monitoring

- Understand how to monitor all cluster components.
- Understand how to monitor applications.
- Manage cluster component logs.
- Manage application logs.

## 8% - Application Lifecycle Management

- Understand Deployments and how to perform rolling updates and rollbacks.
- Know various ways to configure applications.
- Know how to scale applications.
- Understand the primitives necessary to create a self-healing application.

## 11% - Cluster Maintenance

- Understand Kubernetes cluster upgrade process.
- Facilitate operating system upgrades.
- Implement backup and restore methodologies.

## 12% - Security

- Know how to configure authentication and authorization.
- Understand Kubernetes security primitives.
- Know to configure network policies.
- Create and manage TLS certificates for cluster components.
- Work with images securely.
- Define security contexts.
- Secure persistent key value store.

## 7% - Storage

- Understand persistent volumes and know how to create them.
- Understand access modes for volumes.
- Understand persistent volume claims primitive.
- Understand Kubernetes storage objects.
- Know how to configure applications with persistent storage.

# Curriculum Review II

253

Copyright 2013-2020, RX-M LLC

## 11% - Networking

- Understand the networking configuration on the cluster nodes.
- Understand Pod networking concepts.
- Understand service networking.
- Deploy and configure network load balancer.
- Know how to use Ingress rules.
- Know how to configure and use the cluster DNS.
- Understand CNI.

## 12% - Installation, Configuration & Validation

- Design a Kubernetes cluster.
- Install Kubernetes masters and nodes.
- Configure secure cluster communications.
- Configure a Highly-Available Kubernetes cluster.
- Know where to get the Kubernetes release binaries.
- Provision underlying infrastructure to deploy a Kubernetes cluster.
- Choose a network solution.
- Choose your Kubernetes infrastructure configuration.
- Run end-to-end tests on your cluster.
- Analyse end-to-end tests results.
- Run Node end-to-end tests.
- Install and use kubeadm to install, configure, and manage Kubernetes clusters

## 10% - Troubleshooting

- Troubleshoot application failure.
- Troubleshoot control plane failure.
- Troubleshoot worker node failure.
- Troubleshoot networking.

## 19% - Core Concepts

- Understand the Kubernetes API primitives.
- Understand the Kubernetes cluster architecture.
- Understand Services and other network primitives.

# Test taking strategy

# General Tips

255

Copyright 2013-2020, RX-M LLC

- Get to the end as fast as possible
  - Skip questions you do not immediately know the answer to or cannot answer quickly
    - You can go back and solve them when the other questions are done
    - Passing is 75% - you don't have to answer everything!
- Don't write YAML, make K8s do that for you
  - Use as many imperative commands as you can!
  - `$ kubectl create deploy mydep --image=nginx \`  
`--dry-run -o yaml > mydep.yaml`
  - `$ kubectl expose deploy hello-world \`  
`--type=nodePort --name=my-service`
  - `$ kubectl create secret generic db-secret \`  
`--from-literal=username=produser \`  
`--from-literal=password=Y4nys7f11`
  - `$ kubectl create configmap special-config \`  
`--from-literal=special.how=very \`  
`--from-literal=special.type=charm`
  - `$ kubectl create ns development`
  - `$ kubectl create quota test \`  
`--hard=count/services.nodeports=4,count/pods=3,count/secrets=4 \`  
`-n mynamespace`

# Sample Strategy

256

Copyright 2013-2020, RX-M LLC

- Test breakdown
  - 24 problems
  - 3 hours exam duration
  - 7.5 minutes per question
- Difficulty range varies
  - 12 ish problems are easy
  - 8 ish problems are medium
  - 4 ish problems are hard
- Sample Strategy
  - Complete 10 easy in hour 1
  - Complete 8 medium hour 2
  - Work on 6 hard in hour 3
- Points
  - Verify that your output is saved as requested (grading is semi-automated)
  - Save time with `kubectl run`, `kubectl create` and `kubectl edit`
  - The exam will test you on what is documented not what is not

Drills

# Kubectl Mastery

# kubectl

- kubectl is the one and only tool you will use in almost all aspects of the test
- Knowing kubectl inside out will pay huge dividends when taking the test
  - Not to mention making you a faster and more efficient k8s operator

https://kubernetes.io/docs/reference/kubect!/cheatsheet/ 259

kubernetes Documentation Blog Partners Community Case Studies English ^ v1.14 ^

# Reference

HOME SETUP CONCEPTS TASKS TUTORIALS REFERENCE CONTRIBUTE Search

## Reference

- Standardized Glossary
- ▶ Kubernetes Issues and Security
- ▶ Using the Kubernetes API
- ▶ Accessing the API
- ▶ API Reference
- ▶ Federation API
- ▶ Setup tools reference
- ▶ Command line tools reference
- ▼ kubect! CLI
  - Overview of kubect!
  - JSONPath Support
  - kubect!
  - kubect! Cheat Sheet**
  - kubect! Commands

## kubect! Cheat Sheet

See also: [Kubect! Overview](#) and [JsonPath Guide](#).

This page is an overview of the **kubect!1** command.

- [Kubect! Autocomplete](#)
- [Kubect! Context and Configuration](#)
- [Apply](#)
- [Creating Objects](#)
- [Viewing, Finding Resources](#)
- [Updating Resources](#)
- [Patching Resources](#)
- [Editing Resources](#)
- [Scaling Resources](#)
- [Deleting Resources](#)
- [Interacting with running Pods](#)
- [Interacting with Nodes and Cluster](#)
- [What's next](#)

This page is available to you while taking the test!



# kubectl Imperative Commands Reference

260

Copyright 2013-2020, RX-M LLC



## Managing Kubernetes Objects Using Imperative Commands

Kubernetes objects can quickly be created, updated, and deleted directly using imperative commands built into the `kubectl` command-line tool. This document explains how those commands are organized and how to use them to manage live objects.

- [Before you begin](#)
- [Trade-offs](#)
- [How to create objects](#)
- [How to update objects](#)
- [How to delete objects](#)
- [How to view an object](#)
- Using `set` commands to modify objects before creation
- Using `--edit` to modify objects before creation
- [What's next](#)

This page is also available to you while taking the test!

<https://kubernetes.io/docs/tasks/manage-kubernetes-objects/imperative-command/>

# kubectl

261

Copyright 2013-2020, RX-M LLC

- Autocomplete
  - `$ source <(kubectl completion bash)`
- Context
  - Display the current-context
    - `$ kubectl config current-context`
  - Set the default context to "my-cluster-name"
    - `$ kubectl config use-context my-cluster-name`
- Sorting objects
  - List Events sorted by timestamp
    - `$ kubectl get events --sort-by=.metadata.creationTimestamp`
  - List pods sorted by restart count
    - `$ kubectl get pods --sort-by=.status.containerStatuses[0].restartCount`
  - List PVs by timestamp
    - `$ kubectl get pv --sort-by=.spec.capacity.storage`
- Get more information from fewer commands
  - Show more information using wide output
    - `$ kubectl get pods -o wide`
- Updating Resources
  - `$ kubectl set image deployment website nginx=nginx:1.9.1 --record`
  - `$ kubectl edit deployment website`
  - History of updates
    - `$ kubectl rollout history deployment website`
  - Rollback
    - `$ kubectl rollout undo deployment website --to-revision=2`



# API Versions

263

Copyright 2013-2020, RX-M LLC

- All K8s manifests require an apiVersion value
- kubectl api-versions shows you the versions supported by a given k8s api-server

```
user@ubuntu:~$ kubectl api-versions
admissionregistration.k8s.io/v1beta1
apiextensions.k8s.io/v1beta1
apiregistration.k8s.io/v1
apiregistration.k8s.io/v1beta1
apps/v1
apps/v1beta1
apps/v1beta2
authentication.k8s.io/v1
authentication.k8s.io/v1beta1
authorization.k8s.io/v1
authorization.k8s.io/v1beta1
autoscaling/v1
autoscaling/v2beta1
autoscaling/v2beta2
batch/v1
batch/v1beta1
certificates.k8s.io/v1beta1
coordination.k8s.io/v1beta1
events.k8s.io/v1beta1
extensions/v1beta1
networking.k8s.io/v1
policy/v1beta1
rbac.authorization.k8s.io/v1
rbac.authorization.k8s.io/v1beta1
scheduling.k8s.io/v1beta1
storage.k8s.io/v1
storage.k8s.io/v1beta1
v1
user@ubuntu:~$
```

# More kubectl

264

Copyright 2013-2020, RX-M LLC

- Logs
  - `$ kubectl logs my-pod -c my-container`
  - Dump pod container logs for a previous instantiation of a container
    - `$ kubectl logs my-pod -c my-container --previous`
- Interacting with Nodes
  - Mark Node as unschedulable
    - `$ kubectl cordon nodeName`
  - Drain Node
    - `$ kubectl drain nodeName`
  - Mark Node as schedulable
    - `$ kubectl uncordon nodeName`
- Cluster Information
  - Display addresses of the Master and system services
    - `$ kubectl cluster-info`
  - Dump current cluster state to stdout
    - `$ kubectl cluster-info dump`
  - Dump current cluster stat to a path
    - `$ kubectl cluster-info dump --output-directory=/path/to/cluster-state`
  - Display Health of Master components
    - `$ kubectl get componentstatuses`

# Kubectl Drills

## Drill 1

1. Create a pod imperatively with the `redis:latest` image and label it `app=redis` with a single replica
2. List pods with the label `app=redis`
3. Get logs for all pods with the label `app=redis`

## Drill 2

1. Set the worker node to unavailable and reschedule all the pods running on it

# Kubectl Drills

266

Copyright 2013-2020, RX-M LLC

1. Create a pod imperatively with the `redis:latest` image and label it `app=redis` with a single replica

```
kubectl run --generator=run-pod/v1 redis \
 --image=redis -l app=redis
```

2. List pods with the label `app=redis`

```
kubectl get pod -l app=redis
```

3. Get logs for all pods with the label `app=redis`

```
kubectl logs -l app=redis
```

1. Set the worker node to unavailable and reschedule all the pods running on it

```
kubectl drain nodeb (may need --ignore-daemonsets flag)
```

# Kubecttl Events Drills

- Retrieve the events from the kube-system namespace
- View events generated by the Replicaset controller and Default scheduler from the whole cluster
- Retrieve events generated by all kubelets and sort them by timestamp



# Kubectl Events Drills

- Retrieve the events, if any, from the kube-system namespace

```
kubectl get events -n kube-system
```

- View events generated by the Replicaset controller and Default scheduler from the whole cluster

```
kubectl get events --all-namespaces --field-selector
source!=kubelet -o wide
```

- Retrieve events generated by all kubelets and sort them by timestamp

```
kubectl get events --all-namespaces --field-selector
source=kubelet --sort-by --sort-
by=.metadata.creationTimestamp
```

# Pods

# Init Containers

270

Copyright 2013-2020, RX-M LLC

- **Init Containers run before App Containers**
  - Contain utilities that are not desirable to include in the app container for security reasons
  - Execute custom code for setup that is not present in an app image
  - Place values into a config file or run a template tool to dynamically generate a config file for the main app container
- Easy way to block or delay the startup of containers until some set of preconditions are met
- Init Containers are **started in order**
  - **After** network and volumes are initialized
  - Each Init Container **must exit successfully** before the next is started
  - Retried according to the Pod restartPolicy
    - If Always Init Containers use OnFailure
- A Pod is not Ready until all Init Containers have succeeded
- Ports on an Init Container are not aggregated under a service

```
spec:
 initContainers:
 - name: "init-chown-data"
 image: "busybox:latest"
 # 65534 is the nobody user that prometheus uses.
 command: ["chown", "-R", "65534:65534", "/data"]
 securityContext:
 capabilities:
 add: ["CAP_CHOWN"]
 drop: ["ALL"]
 volumeMounts:
 - name: storage-volume
 mountPath: /data

 containers:
 - name: prometheus-server
 securityContext:
 runAsUser: 65534
 image: prom/prometheus:latest
 args:
 - --config.file=/etc/config/prometheus.yml
 - --storage.tsdb.path=/data
 volumeMounts:
 - name: storage-volume
 mountPath: /data
 securityContext:
 capabilities:
 drop: ["ALL"]
 volumes:
 - name: storage-volume
 persistentVolumeClaim:
 name: storage-volume
 ...
```

# Init Containers

- Create a Pod, in that pod the primary container should check for the existence of a file under `/tmp/file.txt`
  - If the file does not exist, the primary container should exit
  - Use an `initContainer` to create the file and set it on the path expected by the primary container

# Init Containers

272

Copyright 2013-2020, RX-M LLC

- Create a Pod, in that pod the primary container should check for the existence of a file under /tmp/file.txt
  - If the file does not exist, the primary container should exit
  - Use an initContainer to create the file and set it on the path expected by the primary container
- For basic pod yaml:
- `kubectl run \`  
`--generator=run-pod/v1 \`  
`init --image=nginx \`  
`--dry-run -o yaml`
- Edit the file, adding the `initContainers` array, commands, args, volume, and volume mount

```
apiVersion: v1
kind: Pod
metadata:
 creationTimestamp: null
 labels:
 run: init
 name: init
spec:
 initContainers:
 - image: busybox
 name: init-c
 command: ["sh", "-c", "touch /tmp/file.txt"]
 volumeMounts:
 - mountPath: "/tmp"
 name: mydir
 containers:
 - image: nginx
 name: main-c
 command: ["/bin/bash", "-c"]
 args:
 - "[-f '/tmp/file.txt'] && echo 1 || nginx"
 volumeMounts:
 - mountPath: "/tmp"
 name: mydir
 dnsPolicy: ClusterFirst
 restartPolicy: Never
 volumes:
 - name: mydir
 emptyDir: {}
```

# Assigning Pods to Nodes

- **NodeAffinity** – constrain which nodes your pod is eligible to schedule on, based on labels on the node
- **podAffinity** and **podAntiAffinity** – based on labels on pods that are already running on the node
- Two types:
  - **requiredDuringSchedulingIgnoredDuringExecution**: Specifies rules that *must* be met for a pod to schedule
  - **preferredDuringSchedulingIgnoredDuringExecution**: Specifies *preferences* that the scheduler will try to enforce but will not guarantee
- **Tolerations** – specify taints that pods can ignore
  - **tolerations**:
    - key: "key"
    - operator: "Equal"
    - value: "value"
    - effect: "NoSchedule"

```
spec:
 affinity:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: kubernetes.io/e2e-az-name
 operator: In
 values:
 - e2e-az1
 - e2e-az2
 ...
 template:
 metadata:
 labels:
 app: cassandra01
 spec:
 tolerations:
 - effect: NoSchedule
 key: storage
 operator: Equal
 value: cassandra
 nodeSelector:
 storage: cassandra
 affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values:
 - cassandra01
 topologyKey: kubernetes.io/hostname
 preferredDuringSchedulingIgnoredDuringExecution:
 - weight: 1
 podAffinityTerm:
 labelSelector:
 matchExpressions:
 - key: app
 operator: In
 values:
 - cassandra01
 topologyKey: failure-domain.beta.kubernetes.io/zone
```

# Assigning Pods to Nodes 2

- **nodeName** - request to schedule this pod onto a specific node
- **nodeSelector** – for the pod to be eligible to run on a node, the node must have each of the indicated key-value pairs as labels
  - It can have additional labels as well
- **schedulerName** – assign a scheduler to place a pod

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
spec:
 nodeName: nodea
 containers:
 - name: nginx

...
spec:
 nodeSelector:
 disktype: ssd
 containers:
 - name: nginx
 image: nginx

...
spec:
 schedulerName: default-scheduler
 containers:
 - name: pod-with-default-annotation-container
 image: k8s.gcr.io/pause:2.0
```

# Pod Scheduling Drill

1. Label both of your Nodes with a unique label
  - `kubectl label node master node=master`
  - `kubectl label node worker node=worker`
2. Use the simplest method to schedule a single pod on each Node



# Pod Scheduling Drill

1. Label both of your Nodes with a unique label
  - `kubectl label node ubuntu node=master`
  - `kubectl label node nodeb node=worker`
2. Use the simplest method to schedule a single pod on each Node

```
kubectl run --generator=run-pod/v1 \
 mypod --image=nginx --dry-run -o yaml
```

- Copy yaml output, pasting 2x into file (`vim mypods.yaml`)
- Place "---" between pods
- Set nodeSelector to master and worker
- Change the name of each pod
- `kubectl apply -f mypods.yaml`

Or:

```
kubectl run --generator=run-pod/v1 \
 mypod --image=nginx --overrides='{ "apiVersion": "v1",
"spec": { "nodeSelector": { "node": "master" } } }'
```

And:

```
kubectl run --generator=run-pod/v1 \
 mypod --image=nginx --overrides='{ "apiVersion": "v1",
"spec": { "nodeSelector": { "node": "worker" } } }'
```

```
apiVersion: v1
kind: Pod
metadata:
 creationTimestamp: null
 labels:
 run: mypod
 name: mypod
spec:
 nodeSelector:
 node: master
 containers:
 - image: nginx
 name: mypod
 resources: {}
 dnsPolicy: ClusterFirst
 restartPolicy: Always
status: {}

apiVersion: v1
kind: Pod
metadata:
 creationTimestamp: null
 labels:
 run: mypod1
 name: mypod1
spec:
 nodeSelector:
 node: worker
 containers:
 - image: nginx
 name: mypod1
 resources: {}
 dnsPolicy: ClusterFirst
 restartPolicy: Always
status: {}
```

# Controllers

# Controller examples

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: web
spec:
 replicas: 3
 selector:
 matchLabels:
 app: nginx
 updateStrategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 template:
 metadata:
 labels:
 app: nginx
 spec:
 containers:
 - name: nginx
 image: nginx
 ports:
 - containerPort: 80
 volumeMounts:
 - name: www
 mountPath: /usr/share/nginx/html
 serviceName: nginx
 volumeClaimTemplates:
 - metadata:
 name: www
 spec:
 accessModes: ["ReadWriteOnce"]
 storageClassName: slow
 resources:
 requests:
 storage: 5Gi

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
 name: logger
spec:
 replicas: 3
 selector:
 matchLabels:
 app: logger
 strategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 template:
 metadata:
 labels:
 app: logger
 spec:
 containers:
 - name: logger
 image: logagg:latest
 ports:
 - containerPort: 80

```

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: monitor
spec:
 #####
 selector:
 matchLabels:
 app: monitor
 updateStrategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 template:
 metadata:
 labels:
 app: monitor
 spec:
 containers:
 - name: monitor-container
 image: google/cadvisor
 ports:
 - containerPort: 8080

```

- Note the differences/similarities
  - Leverage one type to help with creating others
    - `$ kubectl run mydep --image=nginx \`  
`--dry-run -o yaml > mydep.yaml`
- How are these updated?
- Rolled back?
- How do we track the change?

# Deployment Drill

1. Create a Deployment with nginx version 1.14 with 3 replicas
2. Update the app to a new version 1.15.2
3. Rollback the update to the previous version
4. Note: make sure that you record each step

# Deployment Drill

1. Create a Deployment with nginx version 1.14 with 3 replicas

```
kubectl create deploy mydep --image=nginx:1.14
kubectl scale deploy mydep --replicas=3
```

2. Update the app to a new version 1.15.2

```
kubectl set image deploy/mydep mydep=nginx:1.15.2 --record
```

3. Rollback the update to the previous version

```
kubectl rollout undo deploy/mydep
```

4. Note: make sure that you record each step

# Jobs Drill

1. Create a Job that:
  1. Specifies 45 completions with 2 pods running in parallel
  2. Each pod should output the string "Hello" and not get restarted if it exits successfully

# Jobs Drill

1. Create a Job that:
  1. Specifies 45 completions with 2 pods running in parallel
  2. Each pod should output the string "Hello" and not get restarted if it exits successfully

```
apiVersion: batch/v1
kind: Job
metadata:
 name: myjob
spec:
 parallelism: 2
 completions: 45
 template:
 metadata:
 name: myjob
 spec:
 containers:
 - name: myjob
 image: busybox
 command: ["sh", "-c", "echo Hello"]
 restartPolicy: OnFailure
```

# Secrets and ConfigMaps



# Creating ConfigMaps

- ConfigMaps can be created from:
  - Directories
  - Files
  - Literal values
- **--from-file**
  - If a directory, combines the contents of the directory into a single ConfigMap object
  - If a file, creates the ConfigMap from the contents of the file
    - **--from-file=<my-key-name>=<path-to-file>**
      - Defines a key other than the file name to use as the data key
- **--from-literal=**
  - Define a literal value from the command line
- Multiple **--from-file** or **--from-literal** arguments can be passed, combining files from different paths into a single ConfigMap object

```
$ cat app.conf
key=value1
property=config
another.property=configValue

$ cat ui.conf
color.one=blue
color.two=white
font.headings=sans-serif
font.body=serif

$ kubectl create configmap config \
--from-file=/home/user/configmap/

configmap/config created

$ kubectl describe configmap config

Name: config
Namespace: default
Labels: <none>
Annotations: <none>

Data
====
app.conf:

key=value1
property=config
another.property=configValue

ui.conf:

color.one=blue
color.two=white
font.headings=sans-serif
font.body=serif

Events: <none>
```

# Mounting Secrets/ConfigMaps

- When a pod is created Secret/ConfigMap existence is not checked
- If the kubelet can not fetch the Secret/ConfigMap:
  - It will report a pod event
  - The kubelet will periodically retry
  - Once fetched the kubelet will create and mount the volume
  - None of the pod's containers will start until all the pod's volumes are mounted
- **Files**
  - You can specify the permission mode bits a Secret/ConfigMap or files of part of a Secret/ConfigMap will have
    - Otherwise 0644 is used by default
  - When a Secret/ConfigMap being consumed in a volume is updated, projected keys are **eventually updated** as well
- An **imagePullSecret** is a way to pass a secret that contains a Docker (or other) image registry password to Kubelet so it can pull a private image on behalf of your Pod

```
apiVersion: v1
kind: Pod
metadata:
 name: secret-file-pod
 labels:
 secretpod: db-client
spec:
 volumes:
 - name: secret-volume
 secret:
 secretName: prod-db-secret
 defaultMode: 400
 - name: config-file
 configMap:
 name: config
 containers:
 - name: secret-file-container
 image: redis
 imagePullSecrets:
 - name: myregistrykey
 volumeMounts:
 - name: secret-volume
 readOnly: true
 mountPath: "/etc/secret-volume"
 - name: config-file
 mountPath: "/etc/app/config"
```

# ConfigMap Drill

- Create a ConfigMap that contains this data:

```
droid=r2
shade=light
chastise=rebelscum
```

- Create and test a pod that mounts the ConfigMap as files with the path `/tmp/cmdata`
- The Pod should cat one of the files and exit

# ConfigMap Drill

287

Copyright 2013-2020, RX-M LLC

- Create a ConfigMap that contains this data:

```
droid=r2
shade=light
chastise=rebelscum
```

```
kubectl create cm mycm \
--from-literal=droid=r2 \
--from-literal=shade=light \
--from-literal=chastise=rebelscum
```

- Create and test a pod that mounts the ConfigMap as files with the path /tmp/cmdata
- The Pod should cat one of the files and exit when it runs
  - `kubectl run \`  
  `--generator=run-pod/v1 \`  
  `cmpod --image=busybox \`  
  `--dry-run \`  
  `-o yaml > cmpod.yaml`
  - Edit the file, adding the CM

```
apiVersion: v1
kind: Pod
metadata:
 creationTimestamp: null
 labels:
 run: cmpod
 name: cmpod
spec:
 containers:
 - image: busybox
 name: cmpod
 command: ["sh", "-c", "cat /tmp/cmdata/chastise"]
 volumeMounts:
 - mountPath: "/tmp/cmdata"
 name: data
 dnsPolicy: ClusterFirst
 restartPolicy: Never
 volumes:
 - name: data
 configMap:
 name: mycm
```

# Secrets

- Users can create secrets
  - The system also creates some secrets
- Can only be referenced by pods in the **same namespace** where created
- Each item is **base64 encoded**
- Limited to 1MB in size
  - Discourages large secrets which would exhaust apiserver and kubelet memory
- Imperatively:
  - `$ kubectl create secret \`  
`generic db-secret \`  
`--from-literal=username=admin \`  
`--from-literal=password=1f2d1e2e67df`

```
$ echo -n "admin" | base64
YWRtaW4=
```

```
$ echo -n "1f2d1e2e67df" | base64
MWYyZDFlMmU2N2Rm
```

```
apiVersion: v1
kind: Secret
metadata:
 name: db-secret
type: Opaque
data:
 password: MWYyZDFlMmU2N2Rm
 username: YWRtaW4=
```

# Secrets/ConfigMaps as Environment Variables

289

Copyright 2013-2020, RX-M LLC

- Secrets appear as normal environment variables containing the **base-64 decoded values** of the secret data
- Add an environment variable **to each container for each key** you wish to consume
  - The environment variable that consumes the key should populate the name and key in `env[].valueFrom.secretKeyRef` | `configMapKeyRef`
  - Modify your image or command so that the program sees the values in the environment variables
- Keys that are considered invalid environment variable names will be skipped
  - The pod will be allowed to start
  - An `InvalidVariableNames` event will contain the list of invalid keys that were skipped
- Use **envFrom** to define all of a ConfigMap's data as container environment variables

```
apiVersion: v1
kind: Pod
metadata:
 name: env-pod
spec:
 containers:
 - name: env-container
 image: redis
 env:
 - name: SECRET_USERNAME
 valueFrom:
 secretKeyRef:
 name: prod-db-secret
 key: username
 - name: SECRET_PASSWORD
 valueFrom:
 secretKeyRef:
 name: prod-db-secret
 key: password
 - name: CONFIG_VAR
 valueFrom:
 configMapKeyRef:
 name: config
 key: property
 envFrom:
 - configMapRef:
 name: multi-config
```

# Secrets Drill

1. Create a secret that sets a username and password *from literals*
2. Create a Pod that uses the secret as environment variables
3. Check your work by exec-ing into the pod, getting the values of the env vars

# Secrets Drill

1. Create a secret that sets a username and password *from literals*

```
kubectl create secret generic mysec \
--from-literal=username=user \
--from-literal=password=pass
```

2. Create a Pod that uses the secret as environment variables

```
kubectl run --generator=run-pod/v1 \
--env=USERNAME=user,PASSWORD=pass \
secpod --image=nginx --dry-run \
-o yaml > secpod.yaml
```

Edit the yaml, or omit the --env= flag and use envFrom in the yaml

3. Check your work by exec-ing into the pod, getting the values of the env vars

```
kubectl exec -it secpod -- /bin/bash
$ echo $USERNAME $PASSWORD
```

```
apiVersion: v1
kind: Pod
metadata:
 creationTimestamp: null
 labels:
 run: secpod
 name: secpod
spec:
 containers:
 - image: nginx
 name: secpod
 env:
 - name: USERNAME
 valueFrom:
 secretKeyRef:
 name: mysec
 key: username
 - name: PASSWORD
 valueFrom:
 secretKeyRef:
 name: mysec
 key: password
 --- OR ---
 envFrom:
 - secretRef:
 name: mysec
 --- \OR ---
 resources: {}
 dnsPolicy: ClusterFirst
 restartPolicy: Always
status: {}
```



# Day 4

## 12. Exam Prep II

## 13. Exam Prep II

# Objectives

- Final review of the CKA curriculum
  - Ensure that you understand all of the concepts listed
  - Ask your instructor to cover anything and everything you are not yet comfortable with
- Understand the mechanical aspects of the test
  - How the test works
  - What the environment is like
  - What you can and can not do
- Run more practice drills to:
  - Improve your task speed
  - Run into problems now so that you don't on the test
  - Identify knowledge gaps now so that you can shore them up before the test

# CKA Curriculum final review

## 5% - Scheduling

- Use label selectors to schedule Pods.
- Understand the role of DaemonSets.
- Understand how resource limits can affect Pod scheduling.
- Understand how to run multiple schedulers and how to configure Pods to use them.
- Manually schedule a pod without a scheduler.
- Display scheduler events.
- Know how to configure the Kubernetes scheduler.

## 5% - Logging/Monitoring

- Understand how to monitor all cluster components.
- Understand how to monitor applications.
- Manage cluster component logs.
- Manage application logs.

## 8% - Application Lifecycle Management

- Understand Deployments and how to perform rolling updates and rollbacks.
- Know various ways to configure applications.
- Know how to scale applications.
- Understand the primitives necessary to create a self-healing application.

## 11% - Cluster Maintenance

- Understand Kubernetes cluster upgrade process.
- Facilitate operating system upgrades.
- Implement backup and restore methodologies.

## 12% - Security

- Know how to configure authentication and authorization.
- Understand Kubernetes security primitives.
- Know to configure network policies.
- Create and manage TLS certificates for cluster components.
- Work with images securely.
- Define security contexts.
- Secure persistent key value store.

## 7% - Storage

- Understand persistent volumes and know how to create them.
- Understand access modes for volumes.
- Understand persistent volume claims primitive.
- Understand Kubernetes storage objects.
- Know how to configure applications with persistent storage.

# Curriculum Review II

297

Copyright 2013-2020, RX-M LLC

## 11% - Networking

- Understand the networking configuration on the cluster nodes.
- Understand Pod networking concepts.
- Understand service networking.
- Deploy and configure network load balancer.
- Know how to use Ingress rules.
- Know how to configure and use the cluster DNS.
- Understand CNI.

## 12% - Installation, Configuration & Validation

- Design a Kubernetes cluster.
- Install Kubernetes masters and nodes.
- Configure secure cluster communications.
- Configure a Highly-Available Kubernetes cluster.
- Know where to get the Kubernetes release binaries.
- Provision underlying infrastructure to deploy a Kubernetes cluster.
- Choose a network solution.
- Choose your Kubernetes infrastructure configuration.
- Run end-to-end tests on your cluster.
- Analyse end-to-end tests results.
- Run Node end-to-end tests.
- Install and use kubeadm to install, configure, and manage Kubernetes clusters

## 10% - Troubleshooting

- Troubleshoot application failure.
- Troubleshoot control plane failure.
- Troubleshoot worker node failure.
- Troubleshoot networking.

## 19% - Core Concepts

- Understand the Kubernetes API primitives.
- Understand the Kubernetes cluster architecture.
- Understand Services and other network primitives.

# Test Mechanics and Rules

# Test Environment

- The test is proctored by an **online proctor**
- You will need to:
  - **Clear all windows** from your desktop
    - Except the browser window with the test session
  - **Enable your microphone and camera**
  - **Allow screen sharing**
- The proctor will monitor your screen, camera and microphone
- You must not talk, cover your mouth or look away from the test session excessively
- If you need to take a break you will need to ask the proctor
  - They may or may not stop the clock
- If connectivity or technical problem on the proctor side occurs they will extend your test time to compensate



# Guidelines

300

Copyright 2013-2020, RX-M LLC

- Exam tips in terminal: `$ man lf_exam`
- Root access in the exam: `$ sudo -i`
- Rebooting of k8s servers **IS** permitted
- Stopping or tampering with the **gateone** process **will END YOUR EXAM**
- You can use various browsers but **Chrome** is most tested and recommended
- You may only have **two browser tabs** open
  - **1 for the test session**
  - **1 for reference**
    - Allowed reference sites
      - <https://kubernetes.io/docs/>
      - <https://kubernetes.io/blog/>
      - <https://github.com/kubernetes>
    - No other tabs/sites may be used

The allowed sites may contain links that point to external sites, it is the responsibility of the candidate not to click on any links that cause them to navigate to a domain that is not allowed

# Test Session Info

301

Copyright 2013-2020, RX-M LLC

- Do not block ports 8080/tcp, 4505/tcp or 4506/tcp
- Do not use Ctrl+W
  - This closes the current Chrome tab (!)
  - Use Ctrl+Alt+W
- Copy/Paste
  - Ctrl+C & and Ctrl+V are not supported
  - copying large amounts of text is not supported
  - To copy a few lines of text use:
    - Linux: select text, middle button for paste (or left/right simultaneously if you have no middle button)
    - Mac: ⌘+C to copy and ⌘+V to paste
    - Windows: Ctrl+Insert copy/Shift+Insert paste
- Only a single terminal is available for the exam
  - Terminal multiplexers can be used
    - GNU Screen
    - tmux
- Font size in the terminal
  - Ctrl and '+'
  - Ctrl and '-'
- The exam uses 6 Kubernetes clusters
- At the start of each question, you'll be provided with a `kubectl config use-context <xxx>` command to use to connect to the appropriate cluster
- The clusters comprising the exam environment are currently running **Kubernetes 1.17**

# T-mux Terminal Multiplexer

302

Copyright 2013-2020, RX-M LLC

- Allows a user to spawn multiple shell sessions from within a single terminal
  - Child shell instances are spawned inside **panes**
  - Each pane hosts a **new, separate shell session**
    - Environment variables are not carried over
    - Launching a persistent process will only lock the pane it was run in
  - Available to or pre-installed on most Linux distributions

## Usage:

- Launch with **tmux** in a terminal
- Use **ctrl b** to create and interact with panes
  - ctrl b** + **"** – **create** a horizontal pane below
  - ctrl b** + **%** – **create** a vertical pane to the side
  - ctrl b** + **arrow key** – **Move** between panes
  - ctrl b** + **pgUp / pgDn** – **Enable scrolling** mode
    - Use **arrow keys** to scroll up and down
    - Press **esc** to exit scrolling mode
  - Holding **ctrl b** + **arrow key press** – **Resize** current pane
  - ctrl b** + **alt 1...5** – **auto-arrange** panes
- Type **exit** inside a pane to **close** it
  - Exiting all shell panes will exit tmux

```
user@ubuntu: ~
File Edit View Search Terminal Help
user@ubuntu:~$ kubectl cordon ubuntu
node/ubuntu cordoned
user@ubuntu:~$ kubectl run --generator=run-pod/v1 --image redis redis
pod/redis created
user@ubuntu:~$
```

| Every 2.0s: kubectl get pods,deploy |       |         |          |     | Every 2.0s: kubectl get nodes |                          |        |      |         |
|-------------------------------------|-------|---------|----------|-----|-------------------------------|--------------------------|--------|------|---------|
| NAME                                | READY | STATUS  | RESTARTS | AGE | NAME                          | STATUS                   | ROLES  | AGE  | VERSION |
| pod/redis                           | 0/1   | Pending | 0        | 12s | ubuntu                        | Ready,SchedulingDisabled | master | 151d | v1.14.0 |

[0] 0: bash\* "ubuntu" 12:29 27-Mar-19

## Use cases:

- Monitoring machine resources in real time
  - \$ watch kubectl get pod
- Confirming program activity
- Making more terminals in CLI-only environments (like the CKA Exam!)

Drills

# Namespaces & RBAC

# Namespaces

- Intended for environments with users spread across multiple teams or projects
- Provide scope for names of objects
  - Naming need only be unique within a namespace
- Cannot be nested
- An single instance of a Kubernetes resource can only exist in a single namespace
  - Most, but not all, API objects are namespaced
    - To see what is/is not namespaced:  
\$ kubectl api-resources --namespaced=true|false
- Divide cluster resources equally between teams/projects/namespaces via quotas and limits
- kubectl requests can be scoped by a namespace with the --namespace (-n) flag, ex:  
\$ kubectl -n storage get pods
  - Permanently save the namespace for subsequent commands by setting it as part of the context:  
\$ kubectl config set-context \  
\$(kubectl config current-context) \  
--namespace=storage

```
$ kubectl get namespaces
```

| NAME            | STATUS | AGE |
|-----------------|--------|-----|
| default         | Active | 16h |
| kube-node-lease | Active | 16h |
| kube-public     | Active | 16h |
| kube-system     | Active | 16h |

```
$ kubectl create namespace marketing
```

```
namespace/marketing created
```

```
$ kubectl get ns
```

| NAME            | STATUS | AGE  |
|-----------------|--------|------|
| default         | Active | 16h  |
| kube-node-lease | Active | 16h  |
| kube-public     | Active | 16h  |
| kube-system     | Active | 16h  |
| marketing       | Active | 110s |

# Quotas & LimitRanges

305

Copyright 2013-2020, RX-M LLC

- Quotas provide constraints that **limit aggregate resource consumption** for namespaces in 2 ways:
  - Quantity of objects** by type
    - Counts of objects are charged against a given quota when the object exists in etcd (whether or not is actually deployed)
    - The 1.9 release added support to quota all standard namespaced resource types in the format: `count/<resource>.<api-group>`
  - Total amount of resources:**
    - CPU
    - Memory
    - Persistent and ephemeral storage
- Quotas may be scoped** to only apply to a resource that matches the scope enumeration
  - Ex. `NotBestEffort` – matches pods that are not "best effort" quality of service
- Static and independent of cluster capacity**
  - Manual adjustments will be required in autoscaling clusters
- LimitRange configures default requests and limits** enforced for a given namespace
  - When container(s) in a podspec do not specify their own values for requests and limits, the values held by the LimitRange setting for a namespace are applied

Imperative command equivalent example:

```
kubectl create quota myquota \
--hard=count/pods=20,count/service.loadbalancer=2
```

```
$ kubectl describe ns marketing
```

```
Name: marketing
Labels: <none>
Annotations: <none>
Status: Active
```

## Resource Quotas

| Name:           | pod-count |
|-----------------|-----------|
| Resource        | Used Hard |
| limits.cpu      | 0 1500m   |
| limits.memory   | 0 2Gi     |
| pods            | 2 5       |
| requests.cpu    | 0 1       |
| requests.memory | 0 1Gi     |

## Resource Limits

| Type      | Resource | Min | Max | Default Request | Default Limit | Max Limit/Request Ratio |
|-----------|----------|-----|-----|-----------------|---------------|-------------------------|
| Container | cpu      | -   | -   | 250m            | 500m          | -                       |
| Container | memory   | -   | -   | 128Mi           | 256Mi         | -                       |

# RBAC Roles

- RBAC API declares two top-level types:
  - **Role** and **ClusterRole**
    - Role contains rules that represent a set of permissions on API objects
      - Purely additive, no “deny” rules
    - Defined within a namespace (Role) or cluster wide (ClusterRole)
  - Roles and ClusterRoles define:
    - **apiGroups** – filter to specific API group(s)
      - Empty ("" ) means all API groups
    - **resources** – API objects that exist in the specified API group
    - **verbs** – actions allowed against the listed resources

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 namespace: default
 name: svc-reader
rules:
- apiGroups: [""]
 resources: ["services"]
 verbs: ["get", "watch", "list"]
```

---

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: system:coredns
rules:
- apiGroups:
 - ""
 resources:
 - endpoints
 - services
 - pods
 - namespaces
 verbs:
 - list
 - watch
```

Imperative command equivalent example:

```
kubectl create role svc-reader \
--verb=get,list,watch \
--resource=service -n default
```

# RBAC Bindings

- RBAC API declares two top-level types:
  - **RoleBinding** and **ClusterRoleBinding**
  - Grants permissions defined in a Role/ClusterRole to a user or set of users, a group, or ServiceAccount
  - RoleBinding may only reference a Role in the same namespace
    - May also reference a ClusterRole to grant the permissions to namespaced resources
    - Allows administrators to define a set of **common roles for the entire cluster**, then reuse them within multiple namespaces
      - If the ClusterRole grants permissions to list endpoints then the user would have the ability to list endpoints in their assigned namespace
  - ClusterRoleBinding may be used to grant permission at cluster level in all namespaces

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: read-svc
 namespace: default
subjects:
 - kind: User
 name: usera
roleRef:
 kind: Role
 name: svc-reader
 apiGroup: rbac.authorization.k8s.io

```

---

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: system:coredns
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: system:coredns
subjects:
 - kind: ServiceAccount
 name: coredns
 namespace: kube-system

```

Imperative command equivalent example:

```

kubectl create rolebinding read-svc \
--role=svc-reader --user=usera \
-n default

```



# Namespaces Drill

1. Create a namespace named `myns`
2. Apply a quota:
  - `PersistentVolumeClaims: 1`
  - `nodePorts: 2`
  - `Pods: 5`
3. Create a Role in your new namespace that allows the verbs "get" and "list" on Pods
4. Bind that role to a user named Paul

# Namespaces Drill

309

Copyright 2013-2020, RX-M LLC

## 1. Create a namespace named myns

```
kubectl create ns myns
```

## 2. Apply a quota:

- Persistentvolumeclaims: 1
- nodePorts: 2
- Pods: 5

```
kubectl create quota myquota \
--hard=count/pvc=1,count/service.nodeport=2,count/pods=5 \
--namespace=myns
```

## 3. Create a Role in your new namespace that allows the verbs "get" and "list" on Pods

```
kubectl create role myrole --verb=get,list --resource=pod -n myns
```

## 4. Bind that role to a user named Paul

```
kubectl create rolebinding myrb --role=myrole --user=paul -n myns
```

# Network Policy Drill

- Run a server pod in the default namespace

```
kubectl run server --generator=run-pod/v1 --image rxmllc/hostinfo --port 80 -l app=info
```

- Run a client pod in another namespace called drill

```
kubectl create ns drill -l purpose=test
```

```
kubectl label ns drill purpose=test
```

```
kubectl run client -n drill --generator=run-pod/v1 --image busybox --command -- tail -f /dev/null
```

- Adjust your cluster so the client pod in the drill namespace can communicate with the server pod while a default-deny network policy is in place. Do not delete the network policy:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny
spec:
 podSelector: {}
 policyTypes:
 - Ingress
```

# Network Policy Drill

1. Adjust your cluster so the client pod in the drill namespace can communicate with the server pod while a default-deny network policy is in place. Do not delete the network policy:

Create the following permissive network policy:

nano np-client.yaml:

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
 name: access-hostinfo
spec:
 podSelector:
 matchLabels:
 app: info
 ingress:
 - from:
 - namespaceSelector:
 matchLabels:
 purpose: "test"
```

```
kubectl apply -f np-client.yaml
```

```
kubectl exec -it client -- wget -qO - http://<server pod IP>:9898
```

# Kubelet

# Provide PodSpec to Kubelet

- Other than a **PodSpec** from the apiserver, there are three ways that a pod manifest can be provided to the Kubelet:
  - File:** files found on the **staticPodPath** / **--pod-manifest-path** passed on the command line
    - Rechecked every 20 seconds (configurable with **fileCheckFrequency** / **--file-check-frequency**)
    - Example: `$ sudo kubelet --pod-manifest-path=./kubelet/pod.yaml`
  - HTTP endpoint:** HTTP endpoint passed as a parameter on the command line: **staticPodURL** / **--manifest-url**
    - Rechecked every 20 seconds (configurable with **httpCheckFrequency** / **--http-check-frequency**)
    - Example: `$ sudo kubelet --manifest-url=https://raw.githubusercontent.com/kubernetes/kubernetes/master/examples/pod`
  - HTTP server:** The kubelet can also listen for HTTP and respond to a simple API to which PodSpecs can be posted (**--enable-server** defaults to true) to submit a new manifest (underspec'd currently)
    - Ex. `curl --insecure https://localhost:10250/pods`

```
user@nodea:~$ sudo $HOME/k8s/_output/bin/kubelet --manifest-url=https://raw.githubusercontent.com/kubernetes/kubernetes/master/examples/pod
I0912 20:14:13.405789 100524 feature_gate.go:144] feature gates: map[]
W0912 20:14:13.406216 100524 server.go:496] No API client: no api servers specifiedI0912 20:14:13.406338 100524 client.go:72] Connecting to docker on
unix:///var/run/docker.sock
I0912 20:14:13.406387 100524 client.go:92] Start docker client with request timeout=2m0s
W0912 20:14:13.408627 100524 cni.go:189] Unable to update cni config: No networks found in /etc/cni/net.d
I0912 20:14:13.416342 100524 manager.go:143] cAdvisor running in container: "/user.slice"
W0912 20:14:13.498900 100524 manager.go:151] unable to connect to Rkt api service: rkt: cannot tcp Dial rkt api service: dial tcp [::1]:15441: getsockopt: connection refused
I0912 20:14:13.532217 100524 fs.go:117] Filesystem partitions: map[/dev/sda1:{mountpoint:/var/lib/docker/aufs major:8 minor:1 fsType:ext4 blockSize:0}]
...
```

# Kubelet Troubleshooting

- Start with the basics
  - Node state: ready / not ready
    - `kubectl get node`
  - Node state: details about pressure, max pods, taints
    - `kubectl describe node`
- systemd status:
  - `systemctl status kubelet`
- Swap fail?
  - `sudo cat /proc/swaps`
- Inspect configs for potential problems:
  - `sudo cat /etc/systemd/system/kubelet.service.d/10-kubeadm.conf`
  - `sudo cat /var/lib/kubelet/kubeadm-flags.env`
  - `sudo cat /var/lib/kubelet/config.yaml`
- Look for invalid/malformed overrides:
  - `sudo cat /etc/default/kubelet`

# Kubelet Logs



Logs: journalctl -u kubelet

Aug 20 06:24:27 worker-31vq kubelet[1148]: I0820 06:24:27.856920 1148 kubelet\_node\_status.go:79] Attempting to register node worker-31vq

Aug 20 06:24:27 worker-31vq kubelet[1148]: E0820 06:24:27.858891 1148 kubelet\_node\_status.go:103] Unable to register node "worker-31vq" with API server: Post https://192.168.225.236:6443/api/v1/nodes: dial tcp 192.168.225.236:6443: connect: connection refused

Aug 20 06:25:00 worker-31vq kubelet[1148]: I0820 06:25:00.797362 1148 kubelet\_node\_status.go:82] Successfully registered node worker-31vq

Aug 19 16:43:49 lfs458-worker-31vq kubelet[5337]: F0819 16:43:49.468977 5337 server.go:190] failed to load Kubelet config file /var/lib/kubelet/config.yaml, error failed to read kubelet config file "/var/lib/kubelet/config.yaml", error: open /var/lib/kubelet/config.yaml: no such file or directory

Aug 20 06:24:27 ubuntu kubelet[1148]: E0820 06:24:27.531685 1148 reflector.go:205] k8s.io/kubernetes/pkg/kubelet/config/apiserver.go:47: Failed to list \*v1.Pod: Get https://192.168.225.236:6443/api/v1/pods?fieldSelector=spec.nodeName%3Dubuntu&limit=500&resourceVersion=0: dial tcp 192.168.225.236:6443: connect: connection refused

Aug 20 06:24:28 ubuntu kubelet[1148]: E0820 06:24:28.533678 1148 reflector.go:205] k8s.io/kubernetes/pkg/kubelet/kubelet.go:464: Failed to list \*v1.Node: Get https://192.168.225.236:6443/api/v1/nodes?fieldSelector=metadata.name%3Dubuntu&limit=500&resourceVersion=0: dial tcp 192.168.225.236:6443: connect: connection refused

Aug 20 06:24:28 ubuntu kubelet[1148]: E0820 06:24:28.534714 1148 reflector.go:205] k8s.io/kubernetes/pkg/kubelet/kubelet.go:455: Failed to list \*v1.Service: Get https://192.168.225.236:6443/api/v1/services?limit=500&resourceVersion=0: dial tcp 192.168.225.236:6443: connect: connection refused



# Kubelet Drill

1. Create a PodSpec that runs a single container
  - Deploy the pod w/o using the api-server

# Kubelet Drill

1. Create a PodSpec that runs a single container
  - Deploy the pod w/o using the api-server

```
sudo cat /var/lib/kubelet/config.yaml |grep -i manifest
```

```
kubect1 run --generator=run-pod/v1 kubeletpod --image=nginx \
--dry-run=true -o yaml
```

- copy the file to /etc/kubernetes/manifests (or the manifest path)

# Storage

# Persistent Volumes

319

Copyright 2013-2020, RX-M LLC

```
apiVersion: v1
kind: PersistentVolume
metadata:
 name: fiftygigpv
spec:
 capacity:
 storage: 50Gi
 accessModes:
 - ReadWriteOnce
 persistentVolumeReclaimPolicy: Recycle
 storageClassName: bronze
 hostPath:
 path: /tmp/hostpathvol
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: fiftygigpvc
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 50Gi
 storageClassName: bronze
```

```
apiVersion: v1
kind: Pod
metadata:
 name: frontendpod
spec:
 containers:
 - name: myfrontend
 image: nginx
 volumeMounts:
 - mountPath: "/var/www/html"
 name: abstractedstorage
 volumes:
 - name: abstractedstorage
 persistentVolumeClaim:
 claimName: fiftygigpvc
```

- accessModes
  - ReadWriteOnce
  - ReadOnlyMany
  - ReadWriteMany
- persistentVolumeReclaimPolicy
  - Retained
  - Recycled
  - Deleted
- capacity.storage = resources.requests

# Persistent Storage Drill

1. Statically provision a PV, create a matching PVC and a Pod that uses the PVC as a volume mount

# Persistent Storage Drill

1. Statically provision a PV, create a matching PVC and a Pod that uses the PVC as a volume mount

```
$ mkdir /home/user/pv
```

```
$ kubectl run --generator=run-pod/v1 \
kubetepod --image=nginx \
--dry-run=true -o yaml
```

```
$ kubectl exec -it persistentpod bash
```

```
root@persistentpod:/# touch /var/www/html/file
```

```
root@persistentpod:/# exit
```

```
$ ls -l pv
```

```
-rw-r--r-- 1 root root 0 Aug 17 12:16 file
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
 name: hppv
spec:
 capacity:
 storage: 100Mi
 accessModes:
 - ReadWriteOnce
 persistentVolumeReclaimPolicy: Retain
 hostPath:
 path: /home/user/pv
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: hppvc
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 100Mi
```

```
apiVersion: v1
kind: Pod
metadata:
 name: persistentpod
spec:
 containers:
 - name: persistentcontainer
 image: nginx
 volumeMounts:
 - mountPath: "/var/www/html"
 name: persistence
 volumes:
 - name: persistence
 persistentVolumeClaim:
 claimName: hppvc
```

# Shared Storage Drill

1. Create a pod with the following attributes:
  - a. The pod must run two busybox containers
    - i. Configure both busybox containers to run the `tail -f /dev/null` command
  - b. The pod must have a shared host volume which will be removed by Kubernetes when the pod is deleted
  - c. Create a file in the volume called “message”; make sure that this file is created before either of the two main containers run
  - d. Mount the volume at `/vola` in one container and at `/data` in the other
  - e. Exec into both containers to verify the “message” file is accessible on the appropriate path in both containers

# Shared Storage Drill

1. Create a pod with the following attributes:
  - a. The pod must run two busybox containers
    - i. Configure both busybox containers to run the “tail -f /dev/null” command
  - b. The pod must have a shared host volume which will be removed by Kubernetes when the pod is deleted
  - c. Create a file in the volume called “message”; make sure that this file is created before either of the two main containers run
  - d. Mount the volume at “/vola” in one container and at “/data” in the other
  - e. Exec into both containers to verify the “message” file is accessible on the appropriate path in both containers

```
kubect1 run --generator=run-pod/v1 \
kubect1 run --image=nginx \
--dry-run=true -o yaml
```

```
apiVersion: v1
kind: Pod
metadata:
 name: drillpod
spec:
 restartPolicy: Always
 volumes:
 - name: shared-vol
 emptyDir: {}
 initContainers:
 - name: msg-creator
 image: busybox
 command: ["sh", "-c"]
 args: ["touch /share/message"]
 volumeMounts:
 - mountPath: /share
 name: shared-vol
 containers:
 - name: msg-one
 image: busybox
 command: ["tail", "-f", "/dev/null"]
 volumeMounts:
 - mountPath: /vola
 name: shared-vol
 - name: msg-two
 image: busybox
 command: ["tail", "-f", "/dev/null"]
 volumeMounts:
 - mountPath: /data
 name: shared-vol
```



# Cluster Debugging

# Component Statuses

- Check the key control plane services

```
user@ubuntu:~$ kubectl get componentstatuses
```

| NAME               | STATUS  | MESSAGE            | ERROR |
|--------------------|---------|--------------------|-------|
| controller-manager | Healthy | ok                 |       |
| scheduler          | Healthy | ok                 |       |
| etcd-0             | Healthy | {"health": "true"} |       |

```
user@ubuntu:~$
```

# Cluster Logs

- Cluster logs vary by deployment method
  - Kubeadm puts control plane components into pods
    - Use `kubectl get logs` like any other pod
    - Other deployment methods require SSH to hosts
  - Worker nodes
    - kubelet
      - `journalctl -u kubelet`
        - Will also be on the Masters when deploying with kubeadm
    - kube-proxy
      - Containerized:
        - `$ sudo docker container ls |grep kube-proxy`
        - Get kube-proxy's container ID
        - `$ sudo docker container logs 4ed10eba3c68`
      - Systemd:
        - `$ journalctl -u kube-proxy`
        - `/var/log/kube-proxy.log`



# Cluster Component Deployments

327

Copyright 2013-2020, RX-M LLC

- Kubeadm

- Control plane components run as pods in the cluster
- Names suffixed by the name of the host

```
user@ubuntu:~$ kubectl get pods -n kube-system | grep $(hostname)
etcd-ubuntu 1/1 Running 0 18h
kube-apiserver-ubuntu 1/1 Running 0 18h
kube-controller-manager-ubuntu 1/1 Running 0 18h
kube-scheduler-ubuntu 1/1 Running 0 18h
user@ubuntu:~$
```

- K8s components can be deployed as systemd services

- Kubelets should be run as systemd services on all worker nodes
- Usually prefixed by "kube"
  - `systemctl list-units --type=service | grep kube`

```
user@ubuntu:~$ systemctl list-units --type=service | grep kube
kubelet.service loaded active running kubelet: The Kubernetes Node Agent
user@ubuntu:~$
```

- Older clusters may run the kube-apiserver, kube-controller-manager, and kube-scheduler as systemd services
  - Systemd unit files are no longer part of Kubernetes core repo
- Know how to manipulate systemd services
  - `systemctl start | stop | restart <service>`
  - `systemctl enable <service>` - Enable a service to start automatically (persist after reboot)
  - `systemctl daemon-reload` - After changing or adding systemd unit or drop-in files

# Etc

# Etcd Drill

1. Backup etcd by copying the member/snap/db file
2. Backup an etcd cluster using etcdctl
3. Save the backups to /tmp/cka

# Etcctl Drill

1. Backup etcd by copying the member/snap/db file

```
sudo cp /var/lib/etcd/member/snap/db /tmp/cka/
```

2. Backup an etcd cluster using etcdctl

```
kubectl -n kube-system exec etcd-labsys -- \
/bin/sh -c "ETCDCTL_API=3 etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot save /var/lib/etcd/backup.db"
```

3. Save the backups to /tmp/cka

```
sudo cp /var/lib/etcd/backup.db /tmp/cka
```

# Cluster Management



# Add Node Drill

## 1. Add a worker node with kubeadm

- Install docker:
  - `wget -O - https://get.docker.com | sh`
- Add the Kubernetes Apt Repo:
  - `curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -`
  - `echo "deb http://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee -a /etc/apt/sources.list.d/kubernetes.list`
- Use the quickest way to join a node to your cluster

# Add Node Drill

## 1. Add a worker node with kubeadm

- Install docker:
  - `wget -O - https://get.docker.com | sh`
- Add the Kubernetes Apt Repo:
  - `curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -`
  - `echo "deb http://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee -a /etc/apt/sources.list.d/kubernetes.list`
- Use the quickest way to join a node to your cluster
  - On Master
    - `kubeadm token create --print-join-command`
  - On worker
    - `kubeadm join master:6443 \`  
`--token 1ab2c3.v7zjj0oj39ypq0x8 \`  
`--discovery-token-ca-cert-hash sha256:...`

# Kubelet Service Drill

1. Identify the following settings for one of your Kubernetes nodes
2. Make the cka\_kubelet service persists through reboots
3. Enable server certificate rotation on the Kubelet

# Kubelet Service Drill

## 1. Identify the following settings for one of your Kubernetes nodes

- Kubelet Service Status
- Path to Kubelet Systemd Service
- API Server IP Address
- Kubelet Static Pod Manifest Path

```
sudo systemctl status kubelet
sudo systemctl status kubelet | grep "Loaded:"
sudo cat /etc/kubernetes/kubelet.conf | grep "server:"
sudo cat /var/kubelet/config.yaml | grep "static"
```

## 2. Make the cka\_kubelet service persists through reboots

```
sudo systemctl enable cka_kubelet
```

## 3. Enable server certificate rotation on the Kubelet

Add the flag --rotate-server-certificates to the end of that file:

```
sudo nano /etc/systemd/system/kubelet.service.d/10-kubeadm.conf
```

Or: append --rotate-server-certificates to the end of the kubelet systemd service ExecStart

```
sudo nano /lib/systemd/system/kubelet.service
```

Then reload the systemd service:

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

# Kubelet Config Drill

1. Create a kubelet.conf file that connects to your cluster as a node named "new\_worker"
2. Set up the new kubelet.conf to use certificates found /tmp/

# Kubelet Config Drill

1. Create a kubelet.conf file that connects to your cluster as a node named "new worker"

```
sudo cat /etc/kubernetes/kubelet.conf
sudo cp /etc/kubernetes/kubelet.conf /tmp/cka_kubelet.conf
nano /tmp/cka_kubelet.conf
```

Change all references to `system:node:ubuntu` to `system:node:new_worker`

```
apiVersion: v1
clusters:
- cluster:
 certificate-authority-data: ...
 server: https://<Cluster API Server>:6443
 name: kubernetes
contexts:
- context:
 cluster: kubernetes
 user: system:node:new_worker
 name: system:node:new_worker@kubernetes
current-context: system:node:new_worker@kubernetes
kind: Config
preferences: {}
users:
- name: system:node:new_worker
 user:
 client-certificate: /var/lib/kubelet/pki/kubelet-client-current.pem
 client-key: /var/lib/kubelet/pki/kubelet-client-current.pem
```

# TLS Bootstrap Drill

338

Copyright 2013-2020, RX-M LLC

- Use Kubectl to generate a bootstrap-kubeconfig file under /tmp/bootstrap-kubeconfig.conf
  - Find out your current connection information
  - Your CA certificate is found under /etc/kubernetes/pki/ca.crt
  - Try to find a bootstrap token in your cluster
    - If you do not have one, use kubeadm to create a token

# TLS Bootstrap Drill

Use Kubectl to create a bootstrap-kubeconfig file under /tmp/bootstrap-kubeconfig.conf

- Find out your current connection information

```
kubectl config view
```

- Your CA certificate is found under /etc/kubernetes/pki/ca.crt

```
kubectl config set-cluster bootstrap \
--kubeconfig=/tmp/bootstrap-kubeconfig \
--server='https://172.31.6.52:6443' \
--certificate-authority=/etc/kubernetes/pki/ca.crt
```

- Try to find a bootstrap token in your cluster

```
kubectl get secrets -n kube-system | grep bootstrap
```

- Use base64 to decode the token-id and token-secret

```
kubectl get secret -n kube-system $BOOTSTRAPSECRET -o jsonpath='{.data.token-id}' | base64 --
decode
kubectl get secret -n kube-system $BOOTSTRAPSECRET -o jsonpath='{.data.token-secret}' | base64
--decode
```

- If you do not have one, use kubeadm to create a token

```
kubeadm token create
```



# TLS Bootstrap Drill (cont.)

- Associate the token with a user:

```
kubectl config set-credentials kubelet-bootstrap \
--kubeconfig=/tmp/bootstrap-kubeconfig \
--token=ppr919.z8k9u58ng23hqe5m
```

- Associate a user with a cluster using a context:

```
kubectl config set-context bootstrap \
--kubeconfig=/tmp/bootstrap-kubeconfig \
--user=kubelet-bootstrap \
--cluster=bootstrap
```

- Create the file by switching the context:

```
kubectl config use-context bootstrap \
--kubeconfig=/tmp/bootstrap-kubeconfig
```

# The End

Many thanks for attending!

# References

- CKA Curriculum
  - <https://github.com/cncf/curriculum>
- CKA practice test environment
  - <https://github.com/arush-sal/cka-practice-environment>
- CKAD drills:
  - <https://github.com/dgkanatsios/CKAD-exercises>